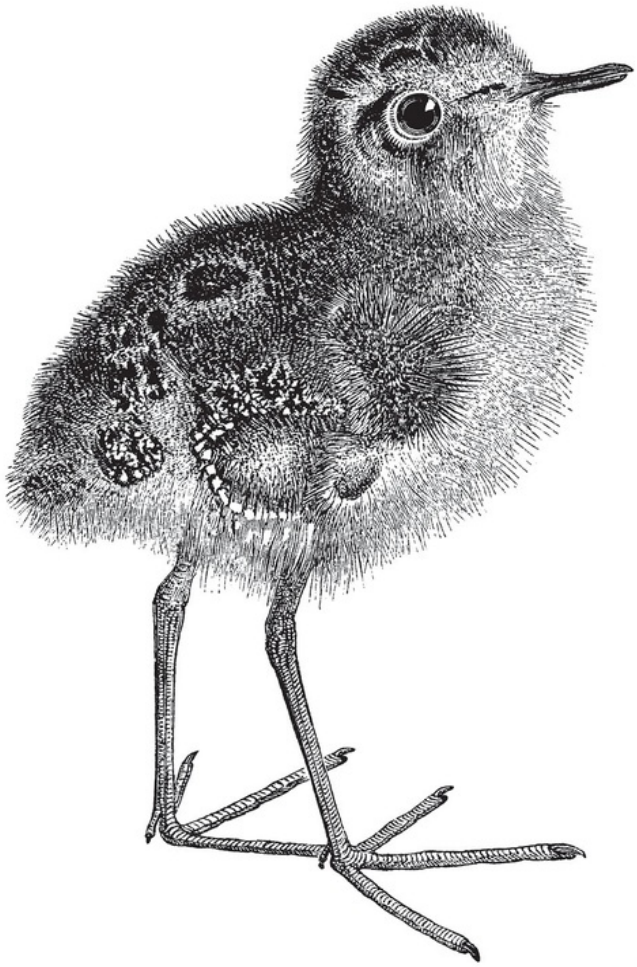
O'REILLY®

Fluent C

Principles, Practices, and Patterns

Early Release

RAW & UNEDITED



Christopher Preschern

Fluent C

Principles, Practices, and Patterns

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Christopher Preschern

Fluent C: Principles, Practices, and Patterns

by Christopher Preschern

Copyright © 2022 Christopher Preschern. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*https://oreilly.com*). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

Acquisitions Editor: Suzanne McQuade

Developmental Editor: Corbin Collins

Production Editor: Jonathon Owen

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

Revision History for the Early Release

- 2022-05-02: First Release

See https://oreilly.com/catalog/errata.csp?isbn=9781492097334 for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Fluent C: Principles, Practices, and Patterns*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-49209-733-4

[LSI]

Preface

You picked up this book to move your programming skills one step forward. That is good, because you'll definitely benefit from the hands-on knowledge provided in this book. If you have a lot of experience programming in C, you'll learn details good design decisions and about their benefits and drawbacks. If you are fairly new to C programming, you'll find guidance about design decisions and you'll see how these decisions are applied bit by bit to running code examples for building larger scale programs.

The book answers questions such as how to structure a C program, how to cope with error handling, or how to design flexible interfaces. As you learn more about C programming, questions often pop up, such as the following:

- Should I return any error information I have?
- Should I use the global variable `errno` to do that?
- Should I have few functions with many parameters or the other way around?
- How do I build a flexible interface?
- How can I build basic things like an iterator?

For object-oriented languages, most of these questions are answered to a great extent by the Gang of Four book *Design Patterns: Elements of Reusable Object-Oriented Software* by Erich Gamma, Richard Helm, Ralph Johnson und John Vlissides (Prentice Hall, 1997). Design patterns provide a programmer with best practices on how objects should interact and on which object owns which other kinds of objects. Also, design patterns show, how such objects can be grouped together.

However, for procedural programming languages like C, most of these design patterns cannot be implemented in the way described by the Gang of

Four. There are no native object-oriented mechanisms in C. It is possible to emulate inheritance or polymorphism in the C programming language, but that might not be the first choice, because such emulation makes things unfamiliar for programmers who are used to programming C and are not used to programming with object-oriented languages like C++ and using concepts like inheritance and polymorphism. Such programmers may want to stick to their native C programming style that they are used to. However, with the native C programming style, not all object-oriented design patterns guidance is usable, or at least the specific implementation of the idea presented in a design pattern is not provided for non-object-oriented programming languages.

And that is where we stand: we want to program in C, but we cannot directly use most of the knowledge documented in design patterns. This book shows how to bridge this gap and implement hands-on design knowledge for the C programming language.

Why I Wrote This Book

Let me tell you why the knowledge gathered in this book turned out to be very important for me and why such knowledge is hard to find.

In school I learned C programming as my first programming language. Just like every new C programmer, I wondered why arrays start with index 0, and I first rather randomly tried out how to place the operators * and & in order to finally get the C pointer magic working.

At university I learned how the C syntax actually works and how it translates to bits and bytes on the hardware. With that knowledge I was able to write small programs that worked very well. However, I still had troubles understanding why longer code looked the way it did and I surely wouldn't have come up with solutions like the following:

```
typedef struct INTERNAL_DRIVER_STRUCT* DRIVER_HANDLE;
typedef void (*DriverSend_FP)(char byte);
typedef char (*DriverReceive_FP)();
typedef void (*DriverIOCTL_FP)(int ioctl, void* context);
```

```
struct DriverFunctions
{
    DriverSend_FP fpSend;
    DriverReceive_FP fpReceive;
    DriverIOCTL_FP fpIOCTL;
};

DRIVER_HANDLE driverCreate(void* initArg, struct DriverFunctions
f);
void driverDestroy(DRIVER_HANDLE h);
void sendByte(DRIVER_HANDLE h, char byte);
char receiveByte(DRIVER_HANDLE h);
void driverIOCTL(DRIVER_HANDLE h, int ioctl, void* context);
```

Looking at code like that prompted many questions:

- Why have function pointers in the `struct`?
- Why do the functions need that `DRIVER_HANDLE`?
- What is an IOCTL and why would I not have separate functions instead?
- Why have explicit create and destroy functions?

These questions came up as I began writing industrial applications. I regularly came across situations where I realized I did not have the C programming knowledge, for example, to decide how to implement an iterator or to decide how to cope with error handling in my functions. I realized that although I knew C syntax, I had no clue how to apply it. I tried to achieve something, but just managed to do that in a clumsy way or not at all. What I needed were best-practices on how to achieve specific tasks with the C programming language. For example, I needed to know things like the following:

- How can I acquire and release resources in an easy way?
- Is it a good idea to use `goto` for error handling?

- Should I design my interface to be flexible or should I simply change it when the need arises?
- Should I use an `assert` statement or should I return an error code?
- How is an iterator implemented in C?

It was very interesting for me to realize that while my experienced work colleagues had many different answers for these questions, nobody could point me to anything that documented these design decisions and their benefits and drawbacks.

So next I questioned the internet and yet again I was surprised: It was very hard to find sound answers to these questions even though the C programming language has been around for decades. I found out that while there is much literature on the C programming language basics and its syntax, there's not much on advanced C programming topics or how to write beautiful C code that holds up to industrial applications.

And that is exactly where this book comes in. This book teaches you how to advance your programming skills from writing basic C programs to writing larger-scale C programs that consider error handling and that are flexible regarding certain future changes in requirements and design. This book uses the concept of design patterns to provide you bit by bit with design decisions and their benefits and drawbacks. These design patterns are applied to running code examples that teach you how code, like that mentioned earlier, evolves and why it ends up looking the way it does.

The presented patterns can be applied to any C programming domains. As I come from the domain of embedded programming in a multi-threaded real-time environment, some of the patterns are biased towards that domain. Anyways, you'll see that the general idea of the patterns can be applied to other C programming domains and even beyond the scope of C programming.

Patterns Basics

The design guidance in this book is provided in the form of patterns. The idea of presenting knowledge and best-practices in the form of patterns comes from the architect Christopher Alexander (*The Timeless Way of Building* by Christopher Alexander, Oxford University Press, 1979). He uses small pieces of well-proven solutions to tackle a huge problem in his domain: how to design and construct cities. The approach of applying patterns was adopted by the software development domain, where pattern conferences like the conference on Pattern Languages of Programs (PLoP) are held to extend the body of knowledge of patterns. In particular the book *Design Patterns: Elements of Reusable Object-Oriented Software* by the "Gang of Four" (Prentice Hall, 1997) had a significant impact and made the concept of design patterns well known to software developers.

But what exactly is a pattern? There are many definitions out there, and if you are deeply interested in the topic, then the book *Pattern Oriented Software Architecture: On Patterns and Pattern Languages* by Frank Buschmann et al. (Wiley, 2007) can provide you with accurate descriptions and details. For the purposes of this book, a pattern provides a well-proven solution to a real-life problem. The patterns presented in this book have the structure shown in Table P-1.

Table P-1. How patterns are broken down

i
n
t
h
i
s
b
o
o
k

Pattern section	Description
Name	This is the name of the pattern, which should be easy to remember. The aim is that this name will be used by the programmers in their everyday language (as is the case with the Gang of Four patterns, where you hear programmers say: "… and the Abstract Factory creates the object"). Pattern names are capitalized in this book.
Context	The context section sets the scene for the pattern. It tells you under which circumstances this pattern can be applied.
Problem	The problem section gives you information about the issue you want to tackle. It starts with the major problem statement written in bold font type and then adds details on why the problem is hard to solve. In other pattern formats these details go into a separate section called "forces").
Solution	This section provides guidance on how to tackle the problem. It starts with stating the main idea of the solution written in bold font type and continues with details about the solution. It also provides a code example in order to provide very concrete guidance.
Consequences	This section lists the benefits and drawbacks of applying the described solution. When applying a pattern, you should always check whether the consequences that arise are OK with you.
Known Uses	The known uses give you evidence that the proposed solution is good and actually works in real-life applications. They also show you concrete examples to help you better understand how to apply the pattern.

A major benefit of presenting design guidance in the form of patterns is that these patterns can be applied one after another. If you have a huge design problem, it's hard to find the one guidance document and the one solution that addresses exactly that problem. Instead, you can think of your huge and very specific problem as a sum of many smaller and more generic problems, and you can tackle these problems bit by bit by applying one pattern after the other. You simply check the problem descriptions of the patterns and apply the one that fits your problem and that has consequences you can live with. These consequences might lead to another problem that you can then address by applying another pattern. That way you incrementally design your code instead of trying to come up with a big-bang up-front design before even writing the first line of code.

How to Read This Book

You should already know C programming basics. You should know the C syntax and how it works – for example, this book won't teach you what a pointer is or how to use it. This book delivers hints and guidance on advanced topics.

The chapters in this book are self-standing. You can read them in an arbitrary order and you can simply pick out the topics you are interested in. You'll find an overview of all patterns in the next section and from there you can jump to the patterns you are interested in. So if you exactly know what you are looking for, you can start right there.

If you are not looking for one particular pattern, but instead want to get an overview of possible design options for a specific topic, read through Part I of the book. Each chapter in there focuses on a particular topic (e.g. error handling or interface design) and presents patterns related to that topic. In addition, a running example is presented for each of the topics that shows with code examples how the topic-related patterns can be applied bit by bit.

Part II of this book shows two larger running examples that apply many of the patterns from Part I. Here you can learn how to build up some larger piece of software bit by bit through the application of patterns.

Overview of the Patterns

You'll find an overview of all patterns presented in this book in Table P-2, Table P-3, Table P-4, Table P-5, Table P-6, Table P-7, Table P-8, Table P-9, and Table P-10. The tables show a short form of the patterns that only contains a brief description of the core problem, followed by the keyword "Therefore", followed by the core solution.

Table P-2. Patterns on Error Handling

Pattern Name	Summary

“Function Split”	The function has several responsibilities and that makes the function hard to read and hard to maintain. Therefore, split it up. Take a part of a function that seems useful on its own, create a new function with that, and call that function.
“Guard Clause”	The function is hard to read and hard to maintain because it mixes pre-condition checks with the main functionality of the function. Therefore, check whether you have mandatory pre-conditions and immediately return from the function if these pre-conditions are not met.
“Samurai Principle”	When returning error information, you assume that the caller checks for this information. However, the caller can simply omit this check and the error might go unnoticed. Therefore, return from a function victorious or not at all. If there is a situation for which you know that an error cannot be handled, then abort the program.
“Goto Error Handling”	Code gets difficult to read and to maintain if it acquires and cleans up multiple resources at different places within a function. Therefore, have all resource cleanup and error handling at the end of the function. If a resource cannot be acquired, use the goto statement to jump to the resource cleanup code.
“Cleanup Record”	It is difficult to make a piece of code easy to read and to maintain if this code acquires and cleans up multiple resources, in particular if those resources depend on one another. Therefore, call resource acquisition functions as long as they succeed and store which functions require cleanup. Call the cleanup functions depending on these stored values.
“Object-Based Error Handling”	Having multiple responsibilities in one function, such as resource acquisition, resource cleanup and usage of that resource, make that code difficult to implement, difficult to read, difficult to maintain and difficult to test. Therefore, put initialization and cleanup into separate functions similar to the concept of constructors and destructors in object-oriented programming.

Table P-3. Patterns on Returning Error I

n
f
o
r
m
a
t
i
o
n

Pattern Name	Summary
“Return Error Codes”	You want to have a mechanism to transport error information to the caller, so that the caller can react to it. You want the mechanism to be simple to use, and the caller should be able to clearly distinguish between different error situations that could occur. Therefore, use the Return Value of a function to transport error information. Return a value that represents a specific kind of error. You as the callee and the caller must have a mutual understanding of what the value means.
“Return Relevant Errors”	On the one hand, the caller should be able to react to errors; on the other hand the more error information you return, the more your code and the code of your caller has to deal with error handling, which makes the code longer. Longer code is harder to read and maintain and brings in the risk of additional bugs. Therefore, only transport error information to the caller if that information is relevant to the caller. Error information is only relevant to the caller if the caller can react to that information.
“Special Return Values”	You want to transport error information, but it’s not an option to explicitly Return Error Codes, because that implies that you cannot use the Return Value of the function to return other data, and you’d have to transport that data via Out-Parameters, which would make calling your function more difficult. Therefore, use the Return Value of your function to transport the data computed by the function. Reserve one or more special values to be returned if an error occurs.
“Log Errors”	You want to make sure that in case of an error you can easily find out its cause. However, you don’t want your error handling code to become complicated because of that. Therefore, use different channels to transport error information that is relevant for the calling code and error information

	that is relevant for the developer. For example, write debug error information into a log file and don’t return the detailed debug error information to the caller.

Table P-4. Pattern on Memory Management

n
t

Pattern Name	Summary
“Stack First”	Deciding the storage-class and memory section (stack, heap, …) for variables is a decision every programmer has to make often. It gets exhausting if for each and every variable, the pros and cons of all possible alternatives have to be considered in detail. Therefore, simply put your variables by default on the stack to profit from automatic cleanup of stack variables.
“Eternal Memory”	Holding large amounts of data and transporting it between function calls is difficult, because you have to make sure that the memory for the data is large enough and that the lifetime extends across your function calls. Therefore, put your data into memory that is available throughout the whole lifetime of your program.
“Screw Freeing”	Having dynamic memory is required if you need large amounts of memory and memory where you don’t know the required size beforehand. However, handling cleanup of dynamic memory is a hassle and is the source of many programming errors. Therefore, allocate dynamic memory and let the operating system cope with deallocation by the end of your program.
“Dedicated Ownership”	The great power of using dynamic memory comes with the great responsibility of having to properly clean that memory up. In larger programs, it becomes difficult to make sure that all dynamic memory is cleaned up properly. Therefore, right at the time when you implement memory allocation, clearly define and document where it’s going to be cleaned up and who is going to do that.
“Allocation Wrapper”	Each allocation of dynamic memory might fail, so you should check allocations in your code to react accordingly. That is cumbersome because you have many places for such checks in your code. Therefore, wrap the allocation and deallocation calls and implement error handling or additional memory management organization in these wrapper functions.
“Pointer Check”	Programming errors that lead to accessing an invalid pointer cause uncontrolled program behavior, and such errors are difficult to debug. However, because your code works a lot with pointers, there is a good chance that you introduced such programming errors. Therefore, explicitly invalidate uninitialized or freed pointers and always check pointers for validity before accessing them.
“Memory Pool”	Frequently allocating and deallocating objects from the heap leads to memory

fragmentation. Therefore, hold a large piece of memory throughout the whole lifetime of your program. At runtime, retrieve fixed-size chunks of that memory pool instead of directly allocating new memory from the heap.

Table P-5. Pattern on Returning Data from

C

F
u
n
c
ti
o
n
s

Pattern Name	Summary
“Return Value”	The function parts you want to split are not independent from one another. As usual in procedural programming, some part delivers a result that is then needed by some other part. The function parts that you want to split need to share some data. Therefore, simply use the one C mechanism intended to retrieve information about the result of a function call: the Return Value. The mechanism to return data in C copies the function result and provides the caller access to this copy.
“Out-Parameters”	C only supports returning a single type from a function call and that makes it complicated to return multiple pieces of information. Therefore, return all the data with one single function call by emulating by-reference arguments with pointers.
“Aggregate Instance”	C only supports returning a single type from a function call and that makes it complicated to return multiple pieces of information. Therefore, put all data that is related together into a newly defined type. Define this Aggregate Instance to contain all the related data that you want to share. Define it in the interface of your component to let the caller directly access all the data stored in the instance.
“Immutable Instance”	You want to provide information held in large pieces of immutable data from your component to a caller. Therefore, have an instance (for example, a `struct`) containing the data to share in static memory. Provide this data to users who want to access it and make sure that they cannot modify it.
“Caller-Owned Buffer”	You want to provide complex or large data of known size to the caller and that data is not immutable - it changes at runtime. Therefore, require the caller to

	provide a buffer and its size to the function that returns the complex, large data. In the function implementation, copy the required data into the buffer if the buffer size is large enough.
“Callee Allocates”	You want to provide complex or large data of unknown size to the caller, and that data is not immutable (it changes at runtime). Therefore, allocate a buffer with the required size inside the function that provides the complex, large data. Copy the required data into the buffer and return a pointer to that buffer.

Table P-6. Patterns on Data Life time and d

Ownership

Pattern Name	Summary
“Stateless Software-Module”	You want to provide logically related functionality to your caller and you and make that functionality for the caller as easy as possible to use. Therefore, keep your functions simple and don’t build up state information in your implementation. Put all related functions into one header file and provide the caller this interface to your software-module.
“Software-Module with Global State”	You want to structure your logically related code that requires common state information and you want to make that functionality for the caller as easy as possible to use. Therefore, have one global instance to let your related functions share common resources. Put all functions that operate on that instance into one header file and provide the caller this interface to your software-module.
“Caller-Owned Instance”	You want to provide multiple callers access to functionality with functions that depend on one another and the interaction of the caller with your functions builds up state information. Therefore, require the caller to pass an instance, which is used to store resource and state information, along to your functions. Provide explicit functions to create and destroy these instances, so that the caller can determine their lifetime.
“Shared Instance”	You want to provide multiple callers access to functionality with functions that depend on one another and the interaction of the caller with your functions builds up state information, which your callers want to share. Therefore, require the caller to pass an instance, which is used to store resource and state information, along to your functions. Use the same instance for multiple callers and keep the ownership of that instance in your software-module.

Table P-7. Pattern Transson Flexible APIs

Pattern Name	Summary

“Header Files”	You want some functionality that you implement to be accessible for code from other implementation files, but you want to hide your implementation details from the caller. Therefore, provide function declarations in your API for any functionality you want to provide to your user. Hide any internal functions, internal data, and your function definitions (the implementations) in your implementation file and don’t provide this implementation file to the user.
“Handle”	You have to share state information or operate on shared resources in your function implementations, but you don’t want your caller to see or even access all that state information and shared resources. Therefore, have a function to create the context on which the caller operates and return an abstract pointer to internal data for that context. Require the caller to pass that pointer to all your functions which can then use the internal data to store state information and resources.
“Dynamic Interface”	It should be possible to call implementations with slightly deviating behaviors, but it should not be necessary to duplicate any code, not even the control logic implementation and interface declaration. Therefore, define a common interface for the deviating functionalities in your API and require the caller to provide a callback function for that functionality which you then call in your function implementation.
“Function Control”	You want to call implementations with slightly deviating behaviors, but you don’t want to duplicate any code, not even the control logic implementation or the interface declaration. Therefore, apply data-based abstraction. Add a parameter to your function that passes meta-information about the function call and that specifies the actual functionality to be performed.

Table P-8. Pattern Sonification Interface

c
e
s

Pattern Name	Summary
“Index Access”	You want to make it possible for the user to iterate elements in your data structure in a convenient way, and it should be possible to change internals of the data structure without resulting in changes to the user’s code. Therefore, provide a function that takes an index to address the element in your underlying data structure and return the content of this element. The user calls this function in a loop to iterate over all elements.
“Cursor Iterator”	You want to provide an iteration interface to your user which is robust in case the elements change during the iteration and which enables you to change the underlying data structure at a later point in time without requiring any changes to the user’s code. Therefore, create an iterator instance that points to an element in the underlying data structure. An iteration function takes this iterator instance as argument, retrieves the element the iterator currently points to, and modifies the iteration instance to point to the next element. The user then iteratively calls this function to retrieve one element at a time.
“Callback Iterator”	You want to provide a robust iteration interface which does not even require the user to implement a loop in the code for iterating over all elements and and which enables you to change the underlying data structure at a later point in time without requiring any changes to the user’s code. Therefore, use your existing data structure specific operations to iterate over all your elements within your implementation and call some provided user-function on each element during this iteration. This user-function gets the element content as a parameter and can then perform its operations on this element. The user just calls one function to trigger the iteration and the whole iteration takes place inside your implementation.

Table P-9. Patterns on Organizing Files

in Modular Programs

Pattern Name	Summary
“Include Guard”	It’s easy to include a header file multiple times, but including one and the same header file leads to compile errors if types or certain macros are part of it, because during compilation they get redefined. Therefore, protect the content of your header files against multiple inclusion so that the developer using the header files does not have to care whether it is included multiple times. Use an interlocked `#ifdef` statement or a `#pragma once` statement to achieve that.
“Software-Module Directories”	Splitting code into different files increases the number of files in your codebase. Having all files in one single directory makes it difficult to keep an overview of all the files, in particular for large codebases. Therefore, put header files and implementation files that belong to a tightly coupled functionality into one directory. Name that directory after the functionality that is provided via the header files.
“Global Include Directory”	To include files from other software modules, you have to use relative paths like *../othersoftwaremodule/file.h*. You have to know the exact location of the other header file. Therefore, have one global directory in your codebase that

	contains all software-module APIs. Add this directory to the global include paths in your toolchain.
“Self-Contained Component”	From the directory structure it is not possible to see the dependencies in the code. Any software-module can simply include the header files from any other software-module, so it’s impossible to check dependencies in the code via the compiler. Therefore, identify software-modules that contain similar functionality and that should be deployed together. Put these software-modules into a common directory and have a designated subdirectory for their header files that are relevant for the caller.
“API Copy”	You want to develop, version, and deploy the parts of your codebase independently from one another. However, to do that, you need clearly defined interfaces between the code parts and to be able to separate that code into different repositories. Therefore, to use the functionality of another component, copy its API. Build that other component separately and copy the build artifacts and its public header files. Put these files into a directory inside your component and configure that directory as a global include path.

Table P-10. Patterns to Escape #ifd

e
f
H
e
l
l

Pattern Name	Summary
“Avoid Variants”	Using different functions for each platform makes the code harder to read and harder to write. The programmer is required to initially understand, to correctly use, and to test these multiple functions in order to achieve one single functionality across multiple platforms. Therefore, use standardized functions, which are available on all platforms. If there are no standardized functions, consider not implementing the functionality.
“Isolate Primitives”	Having code variants organized with `#ifdef` statements makes the code unreadable. It is very difficult to follow the program flow, because it is implemented multiple times for multiple platforms. Therefore, isolate your code variants. In your implementation file, put the code handling the variants into separate functions and call these functions from your main program logic, which then only contains platform independent code.
“Atomic Primitives”	The function that contains the variants and is called by the main program is still hard to comprehend, because all the complex `#ifdef` code was simply put into this function in order to get rid of it in the main program. Therefore, make your primitives atomic. Only handle exactly one kind of variant per function. If you handle multiple kinds of variants, for example, operating system variants and hardware variants, then have separate functions for that.
“Abstraction Layer”	You want to use the functionality which handles platform variants at several places in your codebase, but you do not want to duplicate the code of that functionality. Therefore, provide an API for each functionality that requires platform specific code. Define only platform independent functions in the header file and put all platform specific `#ifdef` code into the implementation file. The caller of your functions only includes your header file and does not have to include any platform specific files.
“Split Implementation Variants”	The platform specific implementations still contain `#ifdef` statements to distinguish between code variants. That makes it difficult to see and to select which part of the code should be built for which platform. Therefore, put each

variant implementation into a separate implementation file and select per file what you want to compile for which platform.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

`Constant width bold`

Shows commands or other text that should be typed literally by the user.

`Constant width italic`

Shows text that should be replaced with user-supplied values or by values determined by context.

> **NOTE**
>
> This element signifies a general note.

> **WARNING**
>
> This element indicates a warning or caution.

Using Code Examples

The code examples in this book show short code snippets which focus on the core idea to showcase the patterns and their application. The code snippets by themselves won't compile, because to keep it simple several things like for example include files are omitted. If you are interested in getting the full code which does compile, you can download it from GitHub. *https://github.com/christopher-preschern/fluent-c*.

If you have a technical question or a problem using the code examples, please send email to *bookquestions@oreilly.com*.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Book Title* by Some Author (O'Reilly). Copyright 2012 Some Copyright Holder, 978-0-596-xxxx-x."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at *permissions@oreilly.com*.

The patterns in this book all present existing code examples which apply these patterns. The following list shows the references to these code examples:

- The game NetHack (*https://www.nethack.org/v361/download-src.html*)
- OpenWrt Project (*https://github.com/openwrt*)

- OpenSSL library (*https://github.com/openssl/openssl*)
- Wireshark network sniffer (*https://gitlab.com/wireshark/wireshark.git*)
- Portland Pattern repository (*https://wiki.c2.com/?PortlandPatternRepository*)
- Git version control system (*https://github.com/git/git*)
- Apache Portable Runtime (*https://apr.apache.org/download.cgi*)
- Apache Webserver (*https://svn.apache.org/viewvc/httpd/httpd/trunk/*)
- B&R Automation Runtime operating system (Proprietary and undisclosed code of the company B&R Industrial Automation GmbH)
- B&R Visual Components automation system visualization editor (Proprietary and undisclosed code of the company B&R Industrial Automation GmbH)
- NetDRMS data management system (*https://jsoc.stanford.edu/jsocwiki/DRMSSetup*)
- MATLAB programming and numeric computing platform (*https://mathworks.com/help/*)
- GLib library (*https://github.com/GNOME/glib*)
- GoAccess real-time web analyzer (*https://github.com/allinurl/goaccess*)
- Cloudy physical calculation software (*https://viewvc.nublado.org/?root=cloudy*)
- GCC compiler (*https://github.com/gcc-mirror/gcc*)
- MySQL database system (*https://dev.mysql.com/downloads/*)
- Andriod ION manager (*https://ionicframework.com/docs/developing/android*)
- Windows API (*https://docs.microsoft.com*)

- Apple's Cocoa API (*https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/CocoaFundamentals/Introduction/Introduction.html*)
- VxWorks real-time operating system (*https://www.windriver.com/resource/vxworks-product-overview*)
- sam text editor (*https://doc.cat-v.org/plan_9/4th_edition/papers/sam/*)
- C standard library functions: glibc implementation (*https://sourceware.org/git/glibc.git*)
- Subversion project (*https://subversion.apache.org/download.cgi*)
- Netdata real-time performance monitoring and visualization system (*https://github.com/netdata/netdata*)
- Nmap network tool (*https://github.com/nmap/nmap*)
- OpenZFS file system (*https://github.com/openzfs/zfs*)
- RIOT Operating system (*https://github.com/topics/riot-os*)
- Radare reverse engineering framework (*https://github.com/radare/radare*)
- Education First digital learning products (*https://www.ef.com*)
- VIM text editor (*https://github.com/vim/vim*)
- GNUplot graphing utility (*https://sourceforge.net/projects/gnuplot/files/gnuplot/*)
- SQLite database engine (*https://sqlite.org/src/doc/trunk/README.md*)
- gzip data compression program (*https://www.gnu.org/software/gzip/*)
- lighttpd webserver (*https://github.com/lighttpd*)
- U-Boot bootloader (*https://github.com/u-boot/u-boot*)

O’Reilly Online Learning

> **NOTE**
>
> For more than 40 years, *O’Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O’Reilly’s online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O’Reilly and 200+ other publishers. For more information, visit *https://oreilly.com*.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O’Reilly Media, Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

800-998-9938 (in the United States or Canada)

707-829-0515 (international or local)

707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at *https://oreilly.com/catalog/errata.csp?isbn=9781492097334*.

Email *bookquestions@oreilly.com* to comment or ask technical questions about this book.

For news and information about our books and courses, visit *https://oreilly.com*.

Find us on LinkedIn: *https://www.linkedin.com/company/oreilly-media*

Follow us on Twitter: *https://twitter.com/oreillymedia*

Watch us on YouTube: *https://www.youtube.com/oreillymedia*

Acknowledgments

I want to thank my wife Silke who by now even knows what patterns are :-) and I want to thank my daughter Ylvi. They both make my life happier and they both make sure that I don't end up sitting in front of my computer working all the time, but that I instead enjoy life.

That book would not have come to life without the help of many pattern enthusiasts. I want to thank all the participants of Writers' Workshops at the European Conference on Pattern Languages of Programs for providing me with feedback on the patterns. In particular I want to thank the following people, who provided me with very helpful feedback during the so-called shepherding process of that conference: Jari Rauhamäki, Tobias Rauter, Andrea Höller, James Coplien, Uwe Zdun, Thomas Raser, Eden Burton, Claudius Link, Valentino Vranić. Special thanks also to my work colleagues, in particular to Thomas Havlovec, who made sure that I got the C programming details in my patterns right.

The content of this book is based on the following papers that were accepted at the European Conference on Pattern Languages of Programs and that were published with ACM. These papers can be accessed for free at the website *https://www.preschern.com*.

- "A Pattern Story about C Programming" (*https://doi.org/10.1145/3489449.3489978*)

- “Patterns for Organizing Files in Modular C Programs” (*https://doi.org/10.1145/3424771.3424772*)
- “Patterns to Escape the #ifdef Hell” (*https://doi.org/10.1145/3361149.3361151*)
- “Patterns for Returning Error Information in C” (*https://doi.org/10.1145/3361149.3361152*)
- “Patterns for Returning Data from C Functions” (*https://doi.org/10.1145/3361149.3361188*)
- “C Patterns on Data Lifetime and Ownership” (*https://doi.org/10.1145/3361149.3361187*)
- “Patterns for C Iterator Interfaces” (*https://doi.org/10.1145/3147704.3147714*)
- “API Patterns in C” (*https://doi.org/10.1145/3011784.3011791*)
- “Idioms for error handling in C” (*https://doi.org/10.1145/2855321.2855377*)

Part I. C Patterns

Patterns make your life easier. They take the burden of having to cope with each and every design decision from you. Patterns explain to you well proven solutions and in this first part of the book, you'll find such well proven solutions and the consequences that arise when applying these solutions. Each of the following chapters focuses on a particular topic for C programming, presents patterns to that topic and shows their application to a running example.

Chapter 1. Error Handling

> **A NOTE FOR EARLY RELEASE READERS**
>
> With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.
>
> This will be the 1st chapter of the final book. Please note that the GitHub repo will be made active later on.
>
> If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

Error handling is a big part of writing software and when done poorly, the software becomes difficult to extend and to maintain. Programming languages like C++ or Java provide "Exceptions" and "Destructors" that make error handling easier. Such mechanisms are not natively available for C and literature on good error handling in C is well scattered over the Internet.

This chapter provides collected knowledge on good error handling in the form of C error handling patterns and a running example that applies the patterns. The patterns provide good practice design decisions and elaborate on when to apply them and which consequences they bring. For a programmer, these patterns remove the burden of making many fine grained decisions. Instead, a programmer can rely on the knowledge presented in these patterns and use them as a starting point to write good code.

Figure 1-1 shows an overview of the patterns presented in this chapter and their relationships and Table 1-1 provides a summary of the patterns.

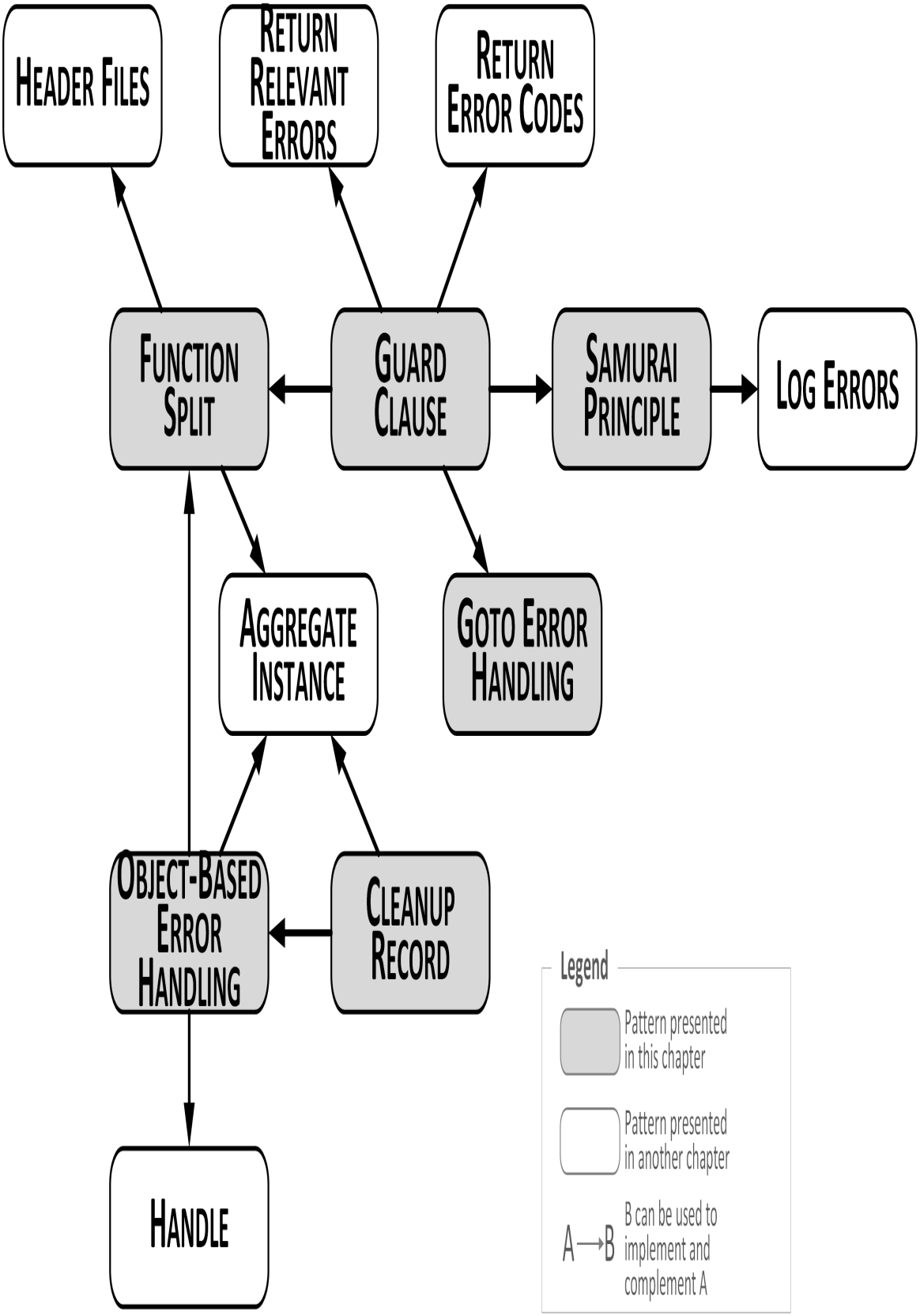

Header Files
Return Relevant Errors
Return Error Codes
Function Split
Guard Clause
Samurai Principle
Log Errors
Aggregate Instance
Goto Error Handling
Object-Based Error Handling
Cleanup Record
Handle
Legend
Pattern presented in this chapter
Pattern presented in another chapter
A → B
B can be used to implement and complement A

Figure 1-1. Overview of patterns on error handling

Table 1-1. Patterns on error handling

Pattern Name	Summary

	Function Split	The function has several responsibilities and that makes the function hard to read and hard to maintain. Therefore, split it up. Take a part of a function that seems useful on its own, create a new function with that, and call that function.
	Guard Clause	The function is hard to read and hard to maintain because it mixes pre-condition checks with the main functionality of the function. Therefore, check whether you have mandatory pre-conditions and immediately return from the function if these pre-conditions are not met.
	Samurai Principle	When returning error information, you assume that the caller checks for this information. However, the caller can simply omit this check and the error might go unnoticed. Therefore, return from a function victorious or not at all. If there is a situation for which you know that an error cannot be handled, then abort the program.
	Goto Error Handling	Code gets difficult to read and to maintain if it acquires and cleans up multiple resources at different places within a function. Therefore, have all resource cleanup and error handling at the end of the function. If a resource cannot be acquired, use the goto statement to jump to the resource cleanup code.
	Cleanup Record	It is difficult to make a piece of code easy to read and to maintain if this code acquires and cleans up multiple resources, in particular if those resources depend on one another. Therefore, call resource acquisition functions as long as they succeed and store which functions require cleanup. Call the cleanup functions depending on these stored values.
	Object-Based Error Handling	Having multiple responsibilities in one function, such as resource acquisition, resource cleanup and usage of that resource, make that code difficult to implement, difficult to read, difficult to maintain and difficult to test. Therefore, put initialization and cleanup into separate functions similar to the concept of constructors and destructors in object-oriented programming.

Running Example

You want to implement a function that parses a file for certain keywords and that returns the information which of the keywords was found.

The standard way to indicate an error situation in C is to provide this information via the return value of a function. To provide additional error information, legacy C functions often set the `errno` variable (see *errno.h*) to a specific error code. The caller can then check `errno` to get information about the error.

However, in the following code, you simply use return values and you don't use `errno`, because you don't need very detailed error information. You come up with the following initial piece of code:

```
int parseFile(char* file_name)
{
  int return_value = ERROR;
  FILE* file_pointer = 0;
  char* buffer = 0;

  if(file_name!=NULL)
  {
    if(file_pointer=fopen(file_name, "r"))
    {
      if(buffer=malloc(BUFFER_SIZE))
      {
        /* parse file content*/
        return_value = NO_KEYWORD_FOUND;
        while(fgets(buffer, BUFFER_SIZE, file_pointer)!=NULL)
        {
          if(strcmp("KEYWORD_ONE\n", buffer)==0)
          {
            return_value = KEYWORD_ONE_FOUND_FIRST;
            break;
          }
          if(strcmp("KEYWORD_TWO\n", buffer)==0)
          {
            return_value = KEYWORD_TWO_FOUND_FIRST;
            break;
          }
        }
        free(buffer);
      }
```

```
        fclose(file_pointer);
      }
    }
    return return_value;
}
```

In the code you have to check the return values of the function calls to know whether an error occurred, so you end up with deeply nested `if`-statements in your code. That has the following problems:

- The function is long and mixes error-handling, initialization, cleanup, and functional code. This makes it difficult to maintain the code.
- The main code that reads and interprets the file data is deeply nested inside the `if` clauses and that makes it difficult to follow the program logic.
- The cleanup functions are far separated from their initialization functions and that makes it easy to forget some cleanup. This is particularly true if the function contains multiple return statements.

To make things better, you first perform a Function Split.

Function Split

Context

You have a function that performs multiple actions. For example, it allocates a resource, uses this resource, and cleans it up.

Problem

The function has several responsibilities and that makes the function hard to read and hard to maintain.

Such a function could be responsible for allocating resources, operating on these resources, and cleaning up these resources. In particular error

handling of failed resource allocation makes such a function hard to read, because quite often that ends up in nested `if` statements.

Coping with allocation, cleanup, and usage of multiple resources in one single function makes it easy to forget cleanup of a resource, in particular if the code is changed later on. For example, if a return statement is added in the middle of the code, then it is easy to forget cleaning up the resources that were already allocated at that point in the function.

Solution

Split it up. Take a part of a function that seems useful on its own, create a new function with that, and call that function.

To find out which part of the function to isolate, simply check whether you can give it its own meaningful name and whether the split isolates responsibilities. That could, for example, result in one function containing just functional code and one containing just error handling code.

A good indicator for a function to be split is if it contains cleanup of the same resource at multiple places in the function. In such a case, it is a lot better to split the code into one function that allocates and cleans up the resources and one function that uses these resources. The called function that uses the resources can then easily have multiple return statements without the need to clean up the resources before each return statement, because that is done in the other function as show in the following code:

```
void someFunction()
{
  char* buffer = malloc(LARGE_SIZE);
  if(buffer)
  {
    mainFunctionality(buffer);
  }
  free(buffer);
}

void mainFunctionality()
{
```

```
    // implementation goes here
}
```

Now, you have two function instead of one. That means of course, that the calling function is not self-contained anymore and depends on the other. You have to define where to put that other function. The first step is surely to put it right in the same file as the calling function, but if the two functions are not closely coupled, you can consider putting the called function into a separate implementation file and to include a Header File declaration of that function.

Consequences

You improved the code, because two short functions are easier to read and maintain compared to one long function. For example, the code is easier to read, because the cleanup functions are closer to the functions that need cleanup and it is easier to read, because the resource allocation and cleanup does not mix with the main program logic.

The called function can now easily contain several return statements, because it does not have to care about cleanup of the resources before each return statement. That cleanup is done at one single point by the calling function.

If many resources are used by the called function, also all these resources have to be passed to that function. Having a lot of function parameters makes the code hard to read. That can be improved by having an Aggregate Instance in such a case.

Known Uses

- Pretty much each C code contains parts that apply this pattern and contains parts that do not apply this pattern and that are thus difficult to maintain. According to the book *Clean Code: A Handbook of Agile Software Craftsmanship* by Robert C. Martin (Prentice Hall, 2008) each function should only have exactly one responsibility (single

responsibility principle) and thus resource handling and other program logic should always be split into different functions.

- This pattern is called Function Wrapper in the Portland pattern repository.
- The criteria when and where to split the function are described in *Refactoring: Improving the Design of Existing Code* by Martin Fowler (Addison-Wesley, 1999) as the Extract Method pattern.
- The game NetHack applies this pattern in its function `read_config_file` in which resources are handled and in which the function `parse_conf_file` is called that then works on the resources.
- The OpenWrt code uses this pattern at several places for buffer handling. For example, the code responsible for MD5 calculation allocates a buffer, passes this buffer to another function that works on that buffer and then cleans that buffer up.

Applied to Running Example

Your code already looks a lot better. Instead of one huge function you now have two large functions with distinct responsibilities. One function is responsible for retrieving and releasing resources and the other is responsible for searching for the keywords like shown in the following code:

```
int searchFileForKeywords(char* buffer, FILE* file_pointer)
{
  while(fgets(buffer, BUFFER_SIZE, file_pointer)!=NULL)
  {
    if(strcmp("KEYWORD_ONE\n", buffer)==0)
    {
      return KEYWORD_ONE_FOUND_FIRST;
    }
    if(strcmp("KEYWORD_TWO\n", buffer)==0)
    {
      return KEYWORD_TWO_FOUND_FIRST;
    }
```

```
  }
  return NO_KEYWORD_FOUND;
}

int parseFile(char* file_name)
{
  int return_value = ERROR;
  FILE* file_pointer = 0;
  char* buffer = 0;

  if(file_name!=NULL)
  {
    if(file_pointer=fopen(file_name, "r"))
    {
      if(buffer=malloc(BUFFER_SIZE))
      {
        return_value = searchFileForKeywords(buffer,
file_pointer);
        free(buffer);
      }
      fclose(file_pointer);
    }
  }
  return return_value;
}
```

The depth of the `if` cascade decreased, but the function `parseFile` still contains three `if` statements and that is way too much. You can make that function easier by implementing a Guard Clause.

Guard Clause

Context

You have a function that performs some functionality that can only be successfully performed under certain conditions.

Problem

The function is hard to read and hard to maintain because it mixes pre-condition checks with the main functionality of the function.

Allocating resources always also requires their cleanup. If you allocate a resource and then later on realize that another pre-condition of the function was not met, then that resource also has to be cleaned up. However, all that resource handling would not have been necessary, because the pre-conditions were not met anyway.

It is difficult to follow the program flow if there are several pre-condition checks scattered across the function, in particular if these checks are implemented in nested `if` statements. That is, because in case of many such checks, the function becomes very long, which by itself is a code smell.

Solution

Check whether you have mandatory pre-conditions and immediately return from the function if these pre-conditions are not met.

For example, check for the validity of input parameters or check whether the program is in a state that allows execution of the rest of the function. Carefully think about which kind of pre-conditions for calling your function you want to set. On the one hand, it makes life easier for you to be very strict on what you allow as function input, but on the other hand it would make life easier for the caller of your function, if you are more liberal regarding possible inputs (as described by Postel's law).

If you have many pre-condition checks, you can call a separate function for performing these checks. In any case, perform the checks before any resource allocation has been done, because then it is very easy to return from a function as no cleanup of resources has to be done.

Clearly describe the pre-conditions for your function in the function's interface. The best place to document that behavior in the header file where the function is declared.

If it is important for the caller to know which pre-condition was not met, you can provide the caller with error information. For example you can Return Error Codes, but make sure to only Return Relevant Errors. The following code shows an example without returning error information:

someFile.h

```
/* This function operates on the 'user_input', which must not be
NULL */
void someFunction(char* user_input);
```

someFile.c

```
void someFunction(char* user_input)
{
  if(user_input == NULL)
  {
    return;
  }
  operateOnData(user_input);
}
```

Consequences

Immediately returning in case the preconditions are not met makes the code easier to read compared to nested `if` constructs. It is made very clear in the code, that the function execution is not continued if the preconditions are not met. That makes the preconditions very well separated from the rest of the code.

However, some coding guidelines forbid returning in the middle of a function. For example, for code that has to be formally proved, return statements are usually only allowed at the very end of the function. In such a case, a Cleanup Record can be kept.

Known Uses

- The Guard Clause is described in the Portland pattern repository.
- The article *Error Detection* by Klaus Renzel (Proceedings 2nd EuroPLoP conference) describes the very similar Error Detection pattern that suggests to introduce pre-condition and post-condition checks.

- The NetHack game uses this pattern at several places in its code, for example, in the `placebc` function. That function puts a chain to the NetHack hero as punishment that reduces the hero's movement speed. The function immediately returns if no chain objects are available.
- The OpenSSL code uses this pattern. For example, the `SSL_new` function immediately returns in case of invalid input parameters.
- The Wireshark code `capture_stats`, which is responsible for gathering statistics when sniffing network packets, first checks its input parameters for validity and immediately returns in case of invalid parameters.

Applied to Running Example

The following code shows how the `parseFile` function applies a Guard Clause to check preconditions of the function:

```
int parseFile(char* file_name)
{
  int return_value = ERROR;
  FILE* file_pointer = 0;
  char* buffer = 0;

  if(file_name==NULL) ❶
  {
    return ERROR;
  }
  if(file_pointer=fopen(file_name, "r"))
  {
    if(buffer=malloc(BUFFER_SIZE))
    {
      return_value = searchFileForKeywords(buffer, file_pointer);
      free(buffer);
    }
    fclose(file_pointer);
  }
  return return_value;
}
```

❶ If invalid parameters are provided, we immediately return and no cleanup is required, because no resources were acquired yet.

The code Returns Error Codes to implement the Guard Clause. That means that the caller has to check for the error codes to know if the provided parameter was invalid. An invalid parameter usually indicates a programming error and checking for programming errors and propagating this information within the code is not a good idea. In such a case, it is easier to simply apply the Samurai Principle.

Samurai Principle

Context

You have some code with complicated error handling for errors of which some are very severe. Your system does not perform safety-critical actions and high availability is not very important.

Problem

When returning error information, you assume that the caller checks for this information. However, the caller can simply omit this check and the error might go unnoticed.

In C it is not mandatory to check return values of the called functions and your caller can simply ignore the return value of a function. If the error that occurs in your function is severe and cannot gracefully be handled by the caller, you don't want your caller to decide whether and how the error should be handled. Instead you'd want to make sure that definitely an action is taken.

If the caller handles some error situation, quite often the program will still crash or some error will still occur. The error might simply show up somewhere else - maybe somewhere in the caller's caller code who might not handle error situations properly. In such a case, handling the error disguises the error and that makes it much harder to debug the error in order to find out the root cause.

Some errors in your code might only occur very rarely. To Return Error Codes for such situation and to handle them in the caller's code makes that code less readable, because it distracts from the main program logic and the actual purpose of the caller's code. The caller might have to write many lines of code to handle very rarely occurring situations.

Returning such error information also poses the problem of how to do that. Using the Return Value or Out-Parameters of the function to do that makes the function's signature more complicated and makes the code more difficult to understand. Because of that, you don't want to have additional parameters for your function only to transport error information.

Solution

Return from a function victorious or not at all (samurai principle). If there is a situation for which you know that an error cannot be handled, then abort the program.

Don't use Out-Parameters or the Return Value to transport error information. In case an error occurs, simply let the program crash. Abort the program in a structured way by using the `assert` statement. Additionally you can provide debug information with the `assert` statement by having an assertion context as shown in the following code:

```
void someFunction()
{
  assert(checkPreconditions() && "Preconditions are not met");
  mainFunctionality();
}
```

This piece of code checks for the condition in the `assert` statement and if it is not true, the `assert` statement including the string on the right will be printed to `stderr` and the program will be aborted.

Aborting the program in a less structured way by not checking for NULL pointers and by accessing such pointers, would be OK. Simply make sure that the program crashes at the point where the error occurs.

Quite often, the Guard Clauses are good candidates for aborting the program in case of errors. For example, if you know that a coding error occurred (if the caller provided you a NULL pointer), abort the program and log debug information instead of returning error information to the caller.

The caller has to be well aware of the behavior of your function, so you have to document in the functions API in which cases the function aborts the program. For example, the function documentation has to state whether the program crashes if the function is provided a NULL pointer as parameter.

Of course, the Samurai Principle is not appropriate for all errors and not appropriate for all application domains. You wouldn't want to let the program crash in case of some unexpected user input. However, in case of a programming error, it can be appropriate to fail fast and let the program crash. That makes it as simple as possible for the programmers to find the error.

Still, to the user such a crash need not be necessarily shown. If your program is just some non-critical part of a larger application, then you might still want your program to crash, but in the context of the overall application your program might fail silent to not disturb the rest of the application and to not disturb the user.

Consequences

The error cannot go unnoticed, because it is handled right at the point where it shows up. The caller is not burdened with having to check for this error, so the caller code becomes more simple. However, now the caller cannot choose how to react to the error.

In some cases aborting the application is OK, because a fast crash is better than unpredictable behavior later on. Still, you have to consider how such an error should be presented to the user. Maybe the user will see some abort statement on the screen. However, for embedded applications that use sensors and actors to interact with the environment, you have to take more

care and have to consider which influence an aborting program has on the environment and whether that is acceptable. In many such cases, the application might have to be more robust and simply aborting the application will not be acceptable.

To abort the program and to Log Errors right at the point where the error shows up makes it easier to find and fix the error, because the error is not disguised. Thus, in the long term, by applying this pattern you end up with more robust and bug-free software.

Known Uses

- A similar pattern that suggests to add a debug information string to an assert statement is called Assertion Context and is described in the book *Patterns in C* by Adam Tornhill (Leanpub, 2014).
- The Wireshark network sniffer applies this pattern all over its code. For example the function `register_capture_dissector` uses `assert` to check that the registration of a dissector is unique.
- The source code of the Git project uses `assert` statements. For example, the functions for storing SHA1 hash values uses `assert` to check whether the path to the file where the hash value should be stored is correct.
- The OpenWrt code responsible for handling big numbers uses `assert` statements to check preconditions in its functions.
- A similar pattern with the name Let It Crash is presented by Pekka Alho and Jari Rauhamäki in the article *Patterns for Light-Weight Fault Tolerance and Decoupled Design in Distributed Control Systems* (*https://researchportal.tuni.fi/files/1840436/alho_rauhamaki_patterns_for_light_weight_fault_tolerance.pdf*). The pattern targets distributed control systems and suggests to let single fail-safe processes crash and to let them restart quickly.

- The C stdlib function `strcpy` does not check for valid user input. If you provide the function with a NULL pointer, it crashes.

Applied to Running Example

The `parseFile` function now looks a lot better. Instead of returning an Error Code, you now have a simple `assert` statement. That makes the following code shorter and the caller of the code does not have the burden to check against the Return Value:

```
int parseFile(char* file_name)
{
  int return_value = ERROR;
  FILE* file_pointer = 0;
  char* buffer = 0;

  assert(file_name!=NULL && "Invalid filename");
  if(file_pointer=fopen(file_name, "r"))
  {
    if(buffer=malloc(BUFFER_SIZE))
    {
      return_value = searchFileForKeywords(buffer, file_pointer);
      free(buffer);
    }
    fclose(file_pointer);
  }
  return return_value;
}
```

While the `if` statements that require no resource cleanup are eliminated, the code still contains nested `if` statements for everything that requires cleanup. Also, you don't yet handle the error situation if the `malloc` call fails. All that can be improved by having Goto Error Handling.

Goto Error Handling

Context

You have a function that acquires and cleans up multiple resources. Maybe you already tried to reduce the complexity by applying Guard Clause, Function Split, or Samurai Principle, but you still have a deeply nested `if`-construct in the code, in particular because of resource acquisition and cleanup.

Problem

Code gets difficult to read and to maintain if it acquires and cleans up multiple resources at different places within a function.

Such code becomes difficult, because usually each resource acquisition can fail and each resource cleanup can just be called if the resource was successfully acquired. To implement that, a lot of `if` statements are required and when implemented poorly, nested `if` statements in a single function make the code hard to read and maintain.

Because you have to cleanup the resources, returning in the middle of the function when something goes wrong is not a good option, because all resources already acquired have to be cleaned up before each return statement. So you end up with multiple points in the code where the same resource is being cleaned up, but you don't want to have duplicated error handling and cleanup.

Solution

Have all resource cleanup and error handling at the end of the function. If a resource cannot be acquired, use the `goto` statement to jump to the resource cleanup code.

Acquire the resources in the order you need them and at the end of your function clean the resources up in the reverse order. For the resource cleanup, have a separate label to which you can jump for each cleanup function. Simply jump to the label if an error occurs or if a resource cannot be acquired, but don't jump multiple times and only jump forward as it is done in the following code:

```
void someFunction()
{
  if(!allocateResource1())
  {
    goto cleanup1;
  }
  if(!allocateResource2())
  {
    goto cleanup2;
  }
  mainFunctionality();
cleanup2:
  cleanupResource2();
cleanup1:
  cleanupResource1();
}
```

If your coding standard forbids the usage of `goto` statements, you can emulate it with a `do{ ... }while(0);` loop around your code and on error use `break` to jump to the end of the loop where you put your error handling. However, that workaround is usually a bad idea, because if `goto` is not allowed by your coding standard, then you should also not be emulating it just to continue programming in your own style. You could use a Cleanup Record as an alternative to `goto`.

In any case, the usage of `goto` might simply be an indicator that your function is already too complex and splitting the function, for example with Object-Based Error Handling, might be a better idea.

GOTO: GOOD OR EVIL?

There are many discussions whether the usage of `goto` is good or bad. The most famous article against the use of `goto` is written by Edsger W. Dijkstra (*https://dl.acm.org/doi/10.1145/362929.362947*) who argues that it obscures the program flow. That is true if `goto` is being used to jump back and forth in a program, but `goto` in C cannot be as badly abused as in the programming languages Dijkstra wrote about (in C you can use `goto` just to jump within a function).

Consequences

The function has one single point of return and the main program flow is well separated from the error handling and resource cleanup. No nested `if` statements are required anymore to achieve that, but not everybody is used to and likes reading `goto` statements.

If you use `goto` statements, you have to be careful, because it is tempting to use these statements for other things than error and cleanup handling and that definitely makes the code unreadable. Also you have to be extra careful to have the correct cleanup functions at the correct labels. It is a common pitfall to accidentally put cleanup functions to the wrong label.

Known Uses

- The Linux kernel code uses mostly `goto`-based error handling. For example, the book *Linux Device Drivers* by Alessandro Rubini and Jonathan Corbet (O'Reilly Media, 2001) describes `goto`-based error handling for programming Linux device drivers.
- The *The CERT C Coding Standard* by Robert C. Seacord (Addison-Wesley Professional, 2014) suggests the use of `goto` for error handling.
- The `goto`-emulation using a do-while loop is described in the Portland Pattern Repository as the Trivial Do-While-Loop pattern.
- The OpenSSL code uses the `goto` statement. For example, the functions that handle X509 certificates use `goto` to jump forward in the functions to a central error handler.
- The Wireshark code uses `goto` statements to jump from its `main` function to a central error handler at the end of that function.

Applied to Running Example

Even though `goto` statements are now used (and quite a few people highly disapprove of that), the error handling is a better compared to the previous

code example. In the following code there are no nested `if` statements and the cleanup code is well separated from the main program flow:

```
int parseFile(char* file_name)
{
  int return_value = ERROR;
  FILE* file_pointer = 0;
  char* buffer = 0;

  assert(file_name!=NULL && "Invalid filename");
  if(!(file_pointer=fopen(file_name, "r")))
  {
    goto error_fileopen;
  }
  if(!(buffer=malloc(BUFFER_SIZE)))
  {
    goto error_malloc;
  }
  return_value = searchFileForKeywords(buffer, file_pointer);
  free(buffer);
error_malloc:
  fclose(file_pointer);
error_fileopen:
  return return_value;
}
```

Now let's say you don't like `goto` statements or your coding guidelines forbid them, but still you have to cleanup your resources. There are alternatives. You can for example simply have a Cleanup Record instead.

Cleanup Record

Context

You have a function that acquires and cleans up multiple resources. Maybe you already tried to reduce the complexity by applying Guard Clause, Function Split, or Samurai Principle, but you still have a deeply nested `if`-construct in the code, in particular because of resource acquisition and cleanup.

Problem

It is difficult to make a piece of code easy to read and to maintain if this code acquires and cleans up multiple resources, in particular if those resources depend on one another.

That is difficult, because usually each resource acquisition can fail and each resource cleanup can just be called if the resource was successfully acquired. To implement that, a lot of `if` statements are required and when implemented poorly, nested `if` statements in a single function make the code hard to read and maintain.

Because you have to cleanup the resources, returning in the middle of the function when something goes wrong is not a good option, because all resources already acquired have to be cleaned up before each return statement. So you end up with multiple points in the code where the same resource is being cleaned up, but you don't want to have duplicated error handling and cleanup.

Solution

Call resource acquisition functions as long as they succeed and store which functions require cleanup. Call the cleanup functions depending on these stored values.

In C, lazy evaluation of `if`-statements can be used to achieve that. Simply call a sequence of functions inside a single `if`-statement as long as these functions succeed. For each function call, store the acquired resource in a variable. Have the code operating on the resources in the body of the `if`-statement and have all resource cleanup after the `if`-statement only if the resource was successfully acquired. The following code shows an example for that:

```
void someFunction()
{
  if((r1=allocateResource1()) && (r2=allocateResource2()))
  {
    mainFunctionality();
```

```
    }
    if(r1) ❶
    {
      cleanupResource1();
    }
    if(r2) ❶
    {
      cleanupResource2();
    }
}
```

❶ To make the code easier to read, you can alternatively put these checks inside the cleanup functions. That is a good approach if you have to provide the resource variable to the cleanup function anyway.

Consequences

You now have no nested `if` statements anymore and still you have one central point at the end of the function for resource cleanup. That makes the code a lot easier to read, because the main program flow is not obscured by error handling anymore.

Also, the function is easy to read, because it has one single exit point. However, the fact that you have to have many variables for keeping track of which resources were successfully allocated makes the code more complicated. Maybe an Aggregate Instance can help to structure the resource variables.

If many resources are being acquired also many functions are being called in the one single `if`-statement. That make that `if`-statement very hard to read and even harder to debug. Because of that, If many resources are being acquired, it is a much better solution to have Object-Based Error Handling.

Another reason for having Object-Based Error Handling instead is that the preceding code is still complicated, because it has one single function that contains the main functionality as well as resource allocation and cleanup. So the one single function has multiple responsibilities.

Known Uses

- In the Portland pattern repository, a similar solution where each of the called functions registers a cleanup handler to a callback list is presented. For cleanup, all functions from the callback list are called.
- The OpenSSL function `dh_key2buf` uses lazy evaluation in an `if`-statement to keep track of allocated bytes that are then cleaned up later on.
- The function `cap_open_socket` of the Wireshark network sniffer uses lazy evaluation of an `if` statement and stores the resources allocated in this `if` statement in variables. At cleanup, these variables are then checked and if the resource allocation was successful, the resource is being cleaned up.
- The `nvram_commit` function of the OpenWrt source code allocates its resources inside an `if` statement and stores these resources to a variables right inside that `if` statement.

Applied to Running Example

Now you got rid of the `goto` statements. Instead of them and instead of the nested `if` statements you now have one single `if` statement. That brings the advantage that without the use of `goto` statements in the following code the error handling is well separated from the main program flow:

```
int parseFile(char* file_name)
{
  int return_value = ERROR;
  FILE* file_pointer = 0;
  char* buffer = 0;

  assert(file_name!=NULL && "Invalid filename");
  if((file_pointer=fopen(file_name, "r")) &&
     (buffer=malloc(BUFFER_SIZE)))
  {
    return_value = searchFileForKeywords(buffer, file_pointer);
  }
  if(file_pointer)
  {
    fclose(file_pointer);
```

```
    }
    if(buffer)
    {
      free(buffer);
    }
    return return_value;
}
```

Still, the code does not look nice. This one function has a lot of responsibilities: Resource allocation, Resource deallocation, file handling, error handling. These responsibilities should be split into different functions with Object-Based Error Handling.

Object-Based Error Handling

Context

You have a function that acquires and cleans up multiple resources. Maybe you already tried to reduce the complexity by applying Guard Clause, Function Split, or Samurai Principle, but you still have a deeply nested `if`-construct in the code, in particular because of resource acquisition and cleanup. Maybe you already got rid of nested `if` statements by having Goto Error Handling or a Cleanup Record.

Problem

Having multiple responsibilities in one function, such as resource acquisition, resource cleanup and usage of that resource, make that code difficult to implement, difficult to read, difficult to maintain and difficult to test.

All that becomes difficult, because usually each resource acquisition can fail and each resource cleanup can just be called if the resource was successfully acquired. To implement that, a lot of `if` statements are required and when implemented poorly, nested `if` statements in a single function make the code hard to read and maintain.

Because you have to cleanup the resources, returning in the middle of the function when something goes wrong is not a good option, because all resources already acquired have to be cleaned up before each return statement. So you end up with multiple points in the code where the same resource is being cleaned up, but you don't want to have duplicated error handling and cleanup code.

Even if you already have a Cleanup Record or Goto Error Handling, the function is still hard to read, because the function mixes different responsibilities. The function is responsible for resource acquisition, error handling, resource cleanup and the operations for which the resources are required. However, a function should only have one single responsibility.

Solution

Put initialization and cleanup into separate functions similar to the concept of constructors and destructors in object-oriented programming.

In your main function, simply call one function that acquires all resources, one function that operates in these resources, and one function that cleans up the resources.

If the acquired resources are not global, then you have to pass the resources along the functions. When having multiple resources, you can pass an Aggregate Instance containing all resources along the functions. Instead, if you want to hide the actual resources from the caller, you can use a Handle for passing the resource information between the functions.

If resource allocation fails, store this information in a variable (for example, a NULL pointer if memory allocation fails). When using or cleaning up the resources, first check whether that resource is valid. Perform that check not in your main function, but rather in the called functions, because that makes your main function a lot more readable:

```
void someFunction()
{
  allocateResources();
```

```
  mainFunctionality();
  cleanupResources();
}
```

Consequences

The function is now easy to read. While it requires allocation and cleanup of multiple resources as well as the operations on these resources, these different tasks are still well separated into separate functions.

Having object-like instances that you pass along functions is known as “object-based” programming style. This style makes procedural programming more similar to object-oriented programming and thus code written in such a style is also more familiar to programmers who are used to object-orientation.

In the main function, there is no reason anymore for having multiple return statements, because there are no more nested `if` statements for the logic of resource allocation and cleanup. However, of course you did not eliminate the logic regarding resource allocation and cleanup. All this logic is still present in the separated functions, but it is not mixed with the operation on the resources anymore.

Instead of having one single function, you now have multiple functions. While that could have a negative impact on performance, usually that does not matter a lot. The performance impact is minor and for most applications it is not relevant.

Known Uses

- This form of cleanup is used in object-oriented programming where constructors and destructors are implicitly called.
- The OpenSSL code uses this pattern. For example, the allocation and cleanup of buffers is realized with the functions `BUF_MEM_new` and `BUF_MEM_free` that are called across the code to cover buffer handling.

- The `show_help` function of the OpenWrt source code shows help information in a context menu. The function calls an initialization function to create a `struct`, then operates on that `struct` and then calls a function to clean up that `struct`.
- The function `cmd__windows_named_pipe` of the Git project uses a Handle to create a pipe, then operates on that pipe and then calls a separate function to cleanup the pipe.

Applied to Running Example

You finally end up with the following code, where the `parseFile` function calls other functions to create and cleanup a parser instance:

```
typedef struct
{
  FILE* file_pointer;
  char* buffer;
}FileParser;

int parseFile(char* file_name)
{
  int return_value;
  FileParser* parser = createParser(file_name);
  return_value = searchFileForKeywords(parser);
  cleanupParser(parser);
  return return_value;
}

int searchFileForKeywords(FileParser* parser)
{
  if(parser == NULL)
  {
    return ERROR;
  }
  while(fgets(parser->buffer, BUFFER_SIZE, parser-
>file_pointer)!=NULL)
  {
    if(strcmp("KEYWORD_ONE\n", parser->buffer)==0)
    {
      return KEYWORD_ONE_FOUND_FIRST;
    }
    if(strcmp("KEYWORD_TWO\n", parser->buffer)==0)
```

```
    {
      return KEYWORD_TWO_FOUND_FIRST;
    }
  }
  return NO_KEYWORD_FOUND;
}

FileParser* createParser(char* file_name)
{
  assert(file_name!=NULL && "Invalid filename");
  FileParser* parser = malloc(sizeof(FileParser));
  if(parser)
  {
    parser->file_pointer=fopen(file_name, "r");
    parser->buffer = malloc(BUFFER_SIZE);
    if(!parser->file_pointer || !parser->buffer)
    {
      cleanupParser(parser);
      return NULL;
    }
  }
  return parser;
}

void cleanupParser(FileParser* parser)
{
  if(parser)
  {
    if(parser->buffer)
    {
      free(parser->buffer);
    }
    if(parser->file_pointer)
    {
      fclose(parser->file_pointer);
    }
    free(parser);
  }
}
```

In the code, there is no more `if` cascade in the main program flow and that makes the `parseFile` function a lot easier to read, debug, and maintain. The main function does not cope with resource allocation, resource deallocation, or error handling details anymore. Instead those details are all put into separate functions, so each function has one single responsibility.

Have a look at the beauty of this final code example compared to the first code example. The applied patterns helped step by step to make the code easier to read and maintain. Step by step the nested `if` cascade was removed and step by step the way of how to handle errors was improved.

Summary

This chapter showed you how to perform error handling in C. Function Split tells you to split your functions into smaller parts to make error handling of these parts easier. A Guard Clause for your functions checks pre-conditions of your function and returns immediately if they are not met and that leaves less error handling obligations for the rest of that function. Instead of returning from the function, you could also abort the program adhering to the Samurai Principle. When it comes to more complex error handling - in particular in combination with acquiring and releasing resources, you have several options. Goto Error Handling makes it possible to jump forward in your function to some error handling section. Instead of jumping, Cleanup Record stores the info, which resources require cleanup and performs that by the end of the function. A way of resource acquisition that is closer to object-oriented programming is Object-Based Error Handling, which uses separate initialization and cleanup functions similar to the concept of constructors and destructors.

With these error handling patterns in your repertoire, you now have the skill to write small programs that handle error situations in a way that the code stays maintainable.

Further Reading

- The Portland Pattern Repository (*http://c2.com/cgi/wiki*) provides many patterns and discussions on error handling as well as other topics. Most of the error handling patterns target exception handling or how to use assertions, but also some C patterns are presented.

- A comprehensive overview of error handling in general is provided in the master's thesis *Error Handling in Structured and Object-Oriented Programming Languages* by Thomas Aglassinger (University of Oulu, 1999). This thesis describes how different kinds of errors arise, it discusses error handling mechanisms of the programming languages C, Basic, Java, and Eiffel, and it provides best practices for error handling in these languages, such as reversing the cleanup order of resources compared to the order of their allocation. The thesis also mentions several third party solutions in the form of C libraries providing enhanced error handling features for C, like exception handling by using the commands `setjmp` and `longjmp`.
- 15 object-oriented patterns on error handling tailored for business information systems are presented in the article *Error Handling for Business Information Systems* by Klaus Renzel (*http://www.objectarchitects.de/arcus/cookbook/exhandling*) and most of the patterns can be applied for non-object-oriented domains as well. The presented patterns cover error detection, error logging, and error handling.
- Implementations including C code snippets for some GoF design patterns are presented in the book *Patterns in C* by Adam Tornhill (Leanpub, 2014). The book further provides best practices in the form of C patterns, some of them covering error handling.
- A collection of patterns for error logging and error handling is presented in the articles *Patterns for Generation, Handling and Management of Errors* and *More Patterns for the Generation, Handling and Management of Errors* by Andy Longshawand Eoin Woods (*http://www.eoinwoods.info/writing/*). Most of the patterns target exception-based error handling.

Outlook

The next chapter shows you how to handle errors when looking at larger programs that transport error information across interfaces to other functions. The patterns tell you which kind of error information to transport and how to do that.

Chapter 2. Returning Error Information

> **A NOTE FOR EARLY RELEASE READERS**
>
> With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.
>
> This will be the 2nd chapter of the final book. Please note that the GitHub repo will be made active later on.
>
> If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

Error handling is a major concern for every program - not just inside one single function or inside the code of one programmer, but also across interfaces. For every larger program, programmers have to decide how to react to errors arising in their own code, how to react to errors arising in 3rd party code, how to pass this error information along in the code, and how to present this error information to the user.

Most object-oriented programming languages come with the handy mechanism of exceptions to provide the programmer with an additional channel for transporting error information, but C does not natively provide such a mechanism. There are ways to emulate exception handling or even inheritance among exceptions in C as for example described in the book *Object oriented programming with ANSI-C* by Axel-Tobias Schreiner (Lulu, 2011), but for C programmers working on legacy C code or for C programmers who want to stick to the native C style they are used to,

introducing such exception mechanisms is not the way to go. Instead such C programmers need guidance on how to use the mechanisms for error handling already natively present in C.

This chapter provides such guidance on how error information can be transported between functions and across interfaces. Figure 2-1 shows an overview of the patterns presented in this chapter and their relationships, and Table 2-1 provides a summary of the patterns.

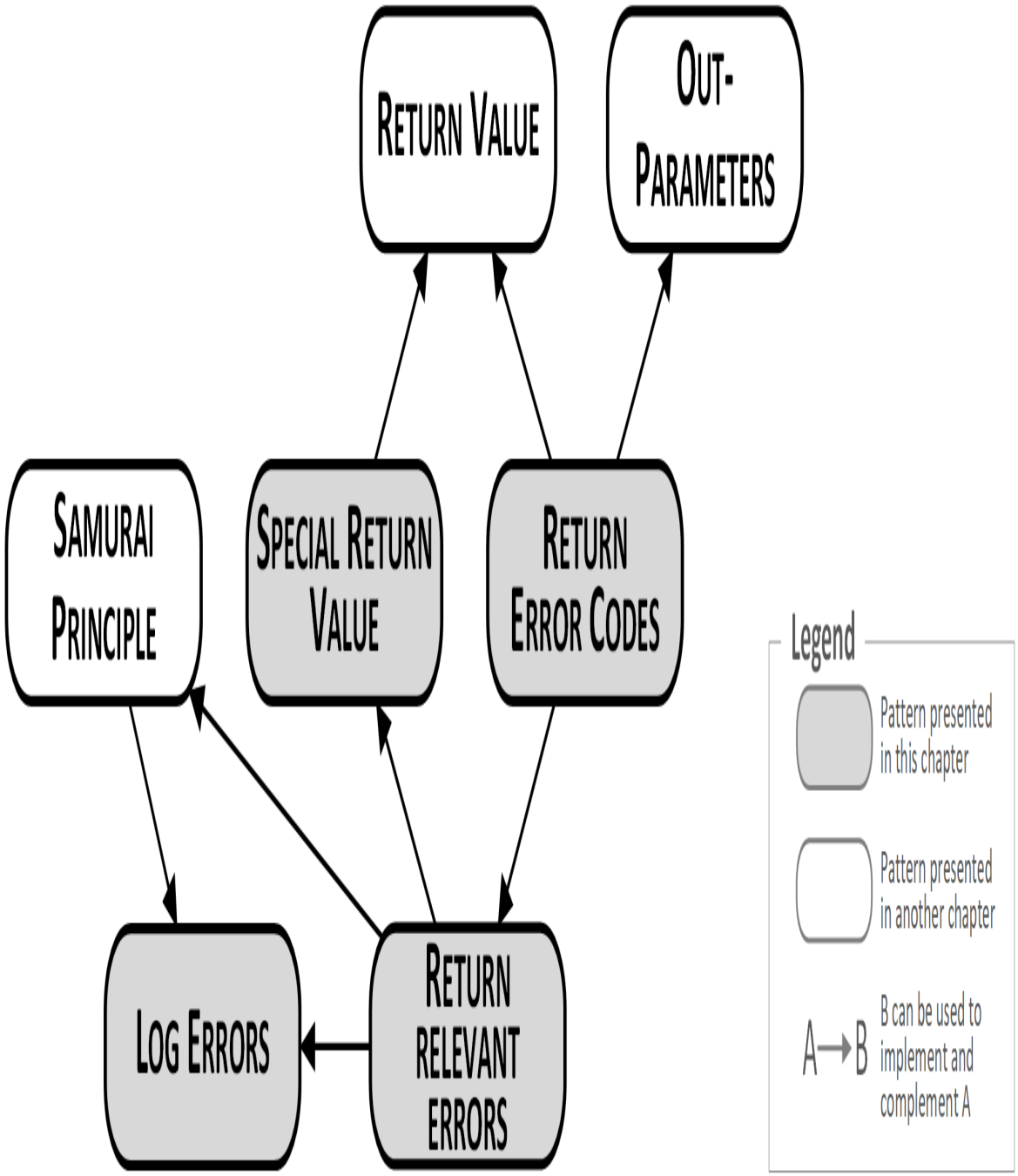


Figure 2-1. Overview of patterns on returning error information

Table 2-1. Patterns on returning error i

n
f
o
r
m
a
t
i
o
n

	Pattern Name	Summary
	Return Error Codes	You want to have a mechanism to transport error information to the caller, so that the caller can react to it. You want the mechanism to be simple to use, and the caller should be able to clearly distinguish between different error situations that could occur. Therefore, use the Return Value of a function to transport error information. Return a value that represents a specific kind of error. You as the callee and the caller must have a mutual understanding of what the value means.
	Return Relevant Errors	On the one hand, the caller should be able to react to errors; on the other hand the more error information you return, the more your code and the code of your caller has to deal with error handling, which makes the code longer. Longer code is harder to read and maintain and brings in the risk of additional bugs. Therefore, only transport error information to the caller if that information is relevant to the caller. Error information is only relevant to the caller if the caller can react to that information.
	Special Return Value	You want to transport error information, but it's not an option to explicitly Return Error Codes, because that implies that you cannot use the Return Value of the function to return other data, and you'd have to transport that data via Out-Parameters, which would make calling your function more difficult. Therefore, use the Return Value of your function to transport the data computed by

	the function. Reserve one or more special values to be returned if an error occurs.
Log Errors	You want to make sure that in case of an error you can easily find out its cause. However, you don't want your error handling code to become complicated because of that. Therefore, use different channels to transport error information that is relevant for the calling code and error information that is relevant for the developer. For example, write debug error information into a log file and don't return the detailed debug error information to the caller.

Running Example

You want to implement a software-module that provides functionality to store string-values for keys identified via strings. In other words you want to implement some functionality similar to the Windows registry. To keep things simple, the following code will not contain hierarchical relationships between the keys and only functions to create registry elements will be discussed:

Registry API

```
/* Handle for registry keys */
typedef struct Key* RegKey;

/* Create a new registry key identified via the provided
'key_name' */
RegKey createKey(char* key_name);

/* Store the provided 'value' to the provided 'key' */
void storeValue(RegKey key, char* value);

/* Make the key available for being read (by other
   functions that are not part of this code example) */
void releaseKey(RegKey key);
```

Registry implementation

```
#define STRING_SIZE 100
#define MAX_KEYS 40

struct Key
{
  char key_name[STRING_SIZE];
  char key_value[STRING_SIZE];
};

/* file-global array holding all registry keys */
static struct Key* key_list[MAX_KEYS];

RegKey createKey(char* key_name)
{
  RegKey newKey = calloc(1, sizeof(struct Key));
  strcpy(newKey->key_name, key_name);
  return newKey;
}

void storeValue(RegKey key, char* value)
{
  strcpy(key->key_value, value);
}

void releaseKey(RegKey key)
{
  int i;
  for(i=0; i<MAX_KEYS; i++)
  {
    if(key_list[i] == NULL)
    {
      key_list[i] = key;
    }
  }
}
```

With the preceding code, you are not sure how you should provide your caller with error information in case of internal errors or, for example, in case of invalid function input parameter values. Your caller does not really know whether the calls succeeded or whether something failed and ends up with the following code:

Caller's code

```
RegKey my_key = createKey("myKey");
storeValue(my_key, "A");
releaseKey(my_key);
```

The caller's code is very short and easy to read, but the caller does not know whether any error occurred and is not able to react to errors. To give the caller that possibility you next want to introduce error handling in your code and you want to provide your caller with error information. The first idea that comes to your mind is to let the caller know about any errors showing up in your software-module. To do that, you Return Error Codes.

Return Error Codes

Context

You implement a software-module that performs some error handling and you want to transport error information to your caller.

Problem

You want to have a mechanism to transport error information to the caller, so that the caller can react to it. You want the mechanism to be simple to use, and the caller should be able to clearly distinguish between different error situations that could occur.

In the old days of C, error information was transported by an error code with the global `errno` variable. The global `errno` variable had to be reset by the caller, then a function had to be called, and the function indicated occurring errors by setting the global `errno` variable, which the caller had to check after the function call.

However compared to using `errno`, you rather want a way to transport error information that makes it easier for the caller to check for errors. The caller should see from the function signature at best how the error information will be transported and which kind of error information to expect.

Also, the mechanism to transport error information should be safe to use in a multi-threaded environment and only the called function should have the possibility to influence the transported error information. In other words, it should be possible to use the mechanism and still have a reentrant function.

Solution

Use the Return Value of a function to transport error information. Return a value that represents a specific kind of error. You as the callee and the caller must have a mutual understanding of what the value means.

Usually, the returned value is a numeric identifier. The caller can check the function return value against the error identifiers and can react accordingly. If the function has to return other function results, provide them to the caller in the form of Out-Parameters.

Define the numeric error identifiers in your API as an `enum` or by using `#define`. If there are many error codes or if your software-module consists of more than one header file, you could have a separate header file that just contains the error codes and that is included by your other header files.

Give the error identifiers a meaningful name and document their meaning with comments. Make sure to name your error codes in a consistent way across your APIs.

The following code shows an example for using error codes:

Caller's code using error codes

```
ErrorCode status = func();
if (status == MAJOR_ERROR)
{
  /* abort program */
}
else if (status == MINOR_ERROR)
{
  /* handle error */
}
```

```
else if (status == OK)
{
  /* continue normal execution */
}
```

Callee API providing error codes

```
typedef enum
{
  MINOR_ERROR,
  MAJOR_ERROR,
  OK
} ErrorCode;

ErrorCode func();
```

Callee implementation providing error codes

```
typedef enum
{
  MINOR_ERROR,
  MAJOR_ERROR,
  OK
} ErrorCode;

ErrorCode func();
```

Consequences

You now have a way to transport error information that makes it very easy for the caller to check for occurring errors. Compared to `errno`, the caller does not have to set and check the error information in steps additionally to the function call, but instead the caller can directly check against the return value of the function call.

Returning error codes can safely be used in multi-threaded environments. Callers can be sure that only the called function and no other side-channels influence the returned error.

The function signature makes it very clear how the error information is transported. This is made clear for the caller and also clear for the compiler or for static code analysis tools, which can check whether the caller checked the function return value and whether the caller checked against all errors that could occur.

As the function now provides different results in different error situations, these results have to be tested. Compared to a function without any error handling, more extensive testing has to be done. Also the caller is burdened with having to check these error situations and that might blow up the size of the caller's code.

C only provides one single return value that is now used to transport error information. Thus the return value cannot be used for transporting other function results anymore. That means that other function results now have to be transported as Out-Parameters, which have the drawback that an additional parameter is required for the function and that from the function signature, it is not clear that that additional parameter is not used as function input, but is used to transport results computed by the function.

Known Uses

- Microsoft uses `HRESULT` to return error information. An `HRESULT` is a unique error code. Making the error code unique has the advantage that the error information can be transported across many functions while still making it possible to find out where the error originated. But making the error code unique brings in additional effort for assigning error numbers and for keeping track of who is allowed to use which error numbers. Another specialty of `HRESULT` is that it encodes specific information, like for example the severity of an error, into the error code by using some bits dedicated to transport this information.
- The code of the Apache Portable Runtime defines the type `apr_status_t` to return error information. Any function that returns error information via this way returns `APR_SUCCESS` on

success or any other value to indicate errors. Other values are uniquely defined error codes specified via `#define` statements.

- The OpenSSL code defines error codes in several header files (*dsaerr.h*, *kdferr.h*, …). As an example, the error codes `KDF_R_MISSING_PARAMETER` or `KDF_R_MISSING_SALT` inform the caller in detail about missing or wrong input parameters. The error codes in each of the files are just defined for a specific set of functions that belong to that file and the error code values are not unique across the whole OpenSSL code.
- Having an Error Code is described in the Portland Pattern Repository as a pattern sketch that also describes the idea of returning error information by explicitly using the function's return value.

Applied to Running Example

Now you provide your caller with information in case of errors in your code. In the following code you check for things that could go wrong and you provide that information to the caller:

Registry API

```
/* Error codes returned by this registry */
typedef enum
{
  OK,
  OUT_OF_MEMORY,
  INVALID_KEY,
  INVALID_STRING,
  STRING_TOO_LONG,
  CANNOT_ADD_KEY
}RegError;

/* Handle for registry keys */
typedef struct Key* RegKey;

/* Create a new registry key identified via the provided
'key_name'.
   Returns OK if no problem occurs, INVALID_KEY if the 'key'
   parameter is NULL, INVALID_STRING if 'key_name' is NULL,
```

```
   STRING_TOO_LONG if 'key_name' is too long, or OUT_OF_MEMORY
   if no memory resources are available. */
RegError createKey(char* key_name, RegKey* key);

/* Store the provided 'value' to the provided 'key'.
   Returns OK if no problem occurs, INVALID_KEY if the 'key'
   parameter is NULL, INVALID_STRING if 'value' is NULL, or
   STRING_TOO_LONG if 'value' is too long. */
RegError storeValue(RegKey key, char* value);

/* Make the key available for being read. Returns OK if no
   problem occurs, INVALID_KEY if 'key' is NULL, or
CANNOT_ADD_KEY
   if the registry is full and no more keys can be released. */
RegError releaseKey(RegKey key);
```

Registry implementation

```
#define STRING_SIZE 100
#define MAX_KEYS 40

struct Key
{
  char key_name[STRING_SIZE];
  char key_value[STRING_SIZE];
};

/* file-global array holding all registry keys */
static struct Key* key_list[MAX_KEYS];

RegError createKey(char* key_name, RegKey* key)
{
  if(key == NULL)
  {
    return INVALID_KEY;
  }

  if(key_name == NULL)
  {
    return INVALID_STRING;
  }

  if(STRING_SIZE <= strlen(key_name))
  {
    return STRING_TOO_LONG;
```

```
  }

  RegKey newKey = calloc(1, sizeof(struct Key));
  if(newKey == NULL)
  {
    return OUT_OF_MEMORY;
  }

  strcpy(newKey->key_name, key_name);
  *key = newKey;
  return OK;
}

RegError storeValue(RegKey key, char* value)
{
  if(key == NULL)
  {
    return INVALID_KEY;
  }

  if(value == NULL)
  {
    return INVALID_STRING;
  }

  if(STRING_SIZE <= strlen(value))
  {
    return STRING_TOO_LONG;
  }

  strcpy(key->key_value, value);
  return OK;
}

RegError releaseKey(RegKey key)
{
  int i;
  if(key == NULL)
  {
    return INVALID_KEY;
  }

  for(i=0; i<MAX_KEYS; i++)
    if(key_list[i] == NULL)
    {
      key_list[i] = key;
      return OK;
    }
```

```
  return CANNOT_ADD_KEY;
}
```

Now the caller can react to the provided error information and can, for example, provide the user of the application with detailed information about what went wrong:

Caller's code

```
  RegError err;
  RegKey my_key;

  err = createKey("myKey", &my_key);
  if(err == INVALID_KEY || err == INVALID_STRING)
  {
    printf("Internal application error\n");
  }
  if(err == STRING_TOO_LONG)
  {
    printf("Provided registry key name too long\n");
  }
  if(err == OUT_OF_MEMORY)
  {
    printf("Insufficient resources to create key\n");
  }

  err = storeValue(my_key, "A");
  if(err == INVALID_KEY || err == INVALID_STRING)
  {
    printf("Internal application error\n");
  }
  if(err == STRING_TOO_LONG)
  {
    printf("Provided registry value to long to be stored to this
key\n");
  }

  err = releaseKey(my_key);
  if(err == INVALID_KEY)
  {
    printf("Internal application error\n");
  }
  if(err == CANNOT_ADD_KEY)
  {
    printf("Key cannot be released, because the registry is
```

```
full\n");
  }
```

Now the caller can react to errors, but the code for the registry software-module as well as the code for the caller more than doubled in size. The caller code could be cleaned up a litte by having a separate function for mapping the error code to error texts, but still the majority of that code would cope with error handling.

You can see that error handling did not come for free. A lot of effort was put into implementing error handling. That can also be seen in the registry API. The comments for the functions became a lot longer, because they have to describe which error situations can occur. Also the caller has to put a lot of effort into thinking about what to do if a specific error occurs.

With providing such detailed error information to the caller, you burden the caller with reacting on these errors and with thinking about which errors are relevant to handle and which are irrelevant to the caller. Thus, special care has to be taken to on the one hand, provide the caller with the necessary error information, but to also on the other hand not flood the caller with unnecessary information.

Next you want to make these considerations in your code and you only want to provide error information that is actually useful to the caller. Thus, you only Return Relevant Errors.

Return Relevant Errors

Context

You implement a software-module that performs some error handling and you want to transport error information to your caller.

Problem

On the one hand, the caller should have be able to react to errors; on the other hand the more error information you return, the more your code and the code of your caller has to deal with error handling, which makes the code longer. Longer code is harder to read and maintain and brings in the risk of additional bugs.

In order to transport error information to your caller, detecting the error and returning the information are not your only tasks. Additionally you have to document in your API which errors are returned. If you don't do that, then your caller would not know which errors to expect and handle. Documenting error behavior is work that has to be done. The more errors there are, the more documentation work has to be done.

Returning very detailed, implementation-specific error information and adding additional error information later on in your code if the implementation changes implies that with such an implementation change you have to semantically change your interface that documents the returned error information. Such changes might not be desirable for your existing callers, because they would have to adapt their code to additionally react to the newly introduced error information.

Providing more detailed error information is also not always a good thing for the caller either. Each error information transported to the caller means additional work for the caller. The caller has to decide whether the error information is relevant and how to handle it.

Solution

Only transport error information to the caller, if that information is relevant to the caller. Error information is only relevant to the caller if the caller can react to that information.

If the caller cannot react to the error information, then it would be unnecessary to provide the caller the opportunity (or the burden) to do so.

There are several ways to only return relevant error information. One extreme way is to simply not return any error information at all. For

example, when having some function `cleanupMemory (void* handle)` that cleans up some memory, then there is no need to transport information whether the cleanup succeeded, because the caller cannot react in the code on such a cleanup error (retrying to call a cleanup function is in most cases no solution). Thus the function simply does not return any error information. To make sure that errors within the function do not go unnoticed, aborting the program in case of error (Samurai Principle) might even be an option.

Or imagine the only reason why you transport the error to the caller is that the caller can then log this error. In that case, do not transport the error to the caller, but instead simpy Log Errors yourself in order to make life for the caller easier.

If you do already Return Error Codes, then only the error information that is relevant to the caller should be transported. Other errors that occur can be summarized as one internal error code. Also, detailed error codes from the functions you call need not necessarily all be returned by your function. Maybe they can be summarized as one internal error code as shown in the following code:

Caller's code

```
ErrorCode status = func();
if (status == MAJOR_ERROR || status == UNKNOWN_ERROR)
{
  /* abort program */
}
else if (status == MINOR_ERROR)
{
  /* handle error */
}
else if (status == OK)
{
  /* continue normal execution*/
}
```

API

```
typedef enum
{
  MINOR_ERROR,
  MAJOR_ERROR,
  UNKNOWN_ERROR,
  OK
} ErrorCode;

ErrorCode func();
```

Implementation

```
ErrorCode func()
{
  if (minorErrorOccurs())
  {
    return MINOR_ERROR;
  }
  else if (majorErrorOccurs())
  {
    return MAJOR_ERROR;
  }
  else if(internalError1Occurs() || internalError2Occurs())
  {
    return UNKNOWN_ERROR; ❶
  }
  else
  {
    return OK;
  }
}
```

❶ You return the same error information if `internalError1Occurs` or `internalError2Occurs`, because it is irrelevant for the caller which of the two implementation-specific errors occurs. The caller would react to both errors in the same way (in the preceding example the reaction is to abort the program).

If more detailed error information is needed for debugging purposes, you could Log Errors. If you realize that there are not many error situations after only returning relevant errors, then instead of error codes, it might be a

better solution to simply have Special Return Values to transport the error information.

Consequences

Not returning detailed information about which kind of internal errors occurred is a relief for the caller. The caller is not burdened with thinking about how to handle all possible internal errors that occur and it is more likely that the caller does actually react to all different kind of errors that are transported, because all of the transported errors are relevent for the caller. Also testers can be happy, because now that fewer error information is returned by the functions, fewer error situations have to be tested.

If the caller uses very strict compilers or static code analysis tools that verify whether the caller does check for all possible return values, the caller does not have to explicitly handle unrelevant errors (for example, a switch statement with many fallthroughs and one error handling for all internal errors). Instead, the caller only handles one internal error code or if you abort the program on errors, the caller does not have to handle any of such errors.

Not returning the detailed error information makes it impossible to the caller to show this error information to the user or to save this error information for debugging purposes for the developer. However, for such debugging information it would be better to Log Errors directly in the software-module where they occur and not burden the caller with doing that.

If you don’t transport all information about errors occurring in your function, but instead you only transport information that you think is relevant to the caller, then there is the chance that you get it wrong. You might forget some information that is necessary for the caller and maybe that leads to a change request for adding this information. If you Return Error Codes, additional error codes can easily be added to your function, but when using Special Return Values, it might not be so easy to add error information later on.

Known Uses

- For security-relevant code it is very common to only return relevant information in case of errors. For example, if a function to authenticate a user returns detailed information whether authentication did not work, because the username is invalid, or because the password is invalid, then the caller could use this function for checking which usernames are already taken. To avoid opening such information side-channels, it is common to only transport the binary information about whether authentication worked or not. For example, the function `rbacAuthenticateUserPassword` used to authenticate users in the B&R Automation Runtime operating system has the return type `bool` and returns `true` if the authentication worked or returns `false` if it did not work. No detailed information about why the authentication did not work is returned.
- The function `FlushWinFile` of the game NetHack flushes a file to the disk calling the Macintosh function `FSWrite`, which does return error codes. However, the NetHack wrapper explicitly ignores the error code and `FlushWinFile` is of return type `void`, because the code using that function cannot react accordingly if an error occurs. Thus, the error information is not passed along.
- The OpenSSL function `EVP_CIPHER_do_all` initializes cipher suites with the internal function `OPENSSL_init_crypto`, which does return error information. However, this error information is ignored by the `EVP_CIPHER_do_all` function that is of return type `void`.

Applied to Running Example

When you only Return Relevant Errors, your registry code looks like the following. To keep things simple, only the `createKey` function is shown here:

Implementation of the function createKey

```
RegError createKey(char* key_name, RegKey* key)
{
  if(key == NULL || key_name == NULL)
  {
    return INVALID_PARAMETER; ❶
  }

  if(STRING_SIZE <= strlen(key_name))
  {
    return STRING_TOO_LONG;
  }

  RegKey newKey = calloc(1, sizeof(struct Key));
  if(newKey == NULL)
  {
    return OUT_OF_MEMORY;
  }

  strcpy(newKey->key_name, key_name);
  *key = newKey;
  return OK;
}
```

❶ Instead of returning `INVALID_KEY` or `INVALID_STRING`, you now return `INVALID_PARAMETER` for all these error cases.

Now the caller cannot handle specific invalid parameters differently which also means the caller does not have to think about how to handle these error situations differently. The caller code becomes simpler, because now there is one error situation less to be handled.

That is good, because what would the caller do if the function returns `INVALID_KEY` or `INVALID_STRING`? It wouldn't make any sense for the caller to try calling the function again. In both cases the caller could just accept that calling the function did not work and the caller could report that to the user or could abort the program. As there would be no reason for the caller to react differently on the two errors, you now took the caller the burden to think about two different error situations. Now the caller only has to think about one error situation and has to react accordingly.

To make things even easier, you next apply the Samurai Principle. Instead of returing all these error codes, you handle some of the errors by aborting

the program:

Declaration of the function createKey

```
/* Create a new registry key identified via the provided
'key_name'
   (must not be NULL, max. STRING_SIZE characters). Stores a
handle
   to the key in the provided 'key' parameter (must not be NULL).
   Returns OK on success, or OUT_OF_MEMORY in case of
insufficient memory. */
RegError createKey(char* key_name, RegKey* key);
```

Implementation of the function createKey

```
RegError createKey(char* key_name, RegKey* key)
{
  assert(key != NULL && key_name != NULL); ❶
  assert(STRING_SIZE > strlen(key_name)); ❶

  RegKey newKey = calloc(1, sizeof(struct Key));
  if(newKey == NULL)
  {
    return OUT_OF_MEMORY;
  }

  strcpy(newKey->key_name, key_name);
  *key = newKey;
  return OK;
}
```

❶ Instead of returning an `INVALID_PARAMETER` or `STRING_TOO_LONG` you now abort the program if one of the provided parameters is not what you expect them to be.

Aborting in case of too long strings at first seems a bit drastic. However, similar to NULL pointers, a too long string is invalid input for your function. If your registry does not get its string input from a user via a GUI, but instead gets a fixed input from the caller's code, then also for too long strings this code only aborts in case of programming errors and that is perfectly fine behavior.

Next, you realize that the `createKey` function only returns two different error codes: `OUT_OF_MEMORY` and `OK`. Your code can be made much more beautiful by simply transporting this kind of error information with Special Return Values.

Special Return Values

Context

You have a function that computes some result and you want to transport error information to your caller if an error occurs when executing the function. You only want to Return Relevant Errors.

Problem

You want to transport error information, but it's not an option to explicitly Return Error Codes, because that implies that you cannot use the Return Value of the function to return other data, and you'd have to transport that data via Out-Parameters, which would make calling your function more difficult.

Returning no error information at all is also not an option to you. You want to provide your caller with some error information and you want your caller to be able to react to these errors. There is not a lot of error information that you want to transport to your caller. It might just be the binary information about whether the function call worked or about whether it did not work. To Return Error Codes for such simple information would be an overkill.

You cannot apply the Samurai Principle and abort the program, because the errors occurring in your function are not severe or because you want to make it possible for the caller to decide how the errors should be handled, because maybe the caller can handle the errors gracefully.

Solution

Use the Return Value of your function to transport the data computed by the function. Reserve one or more special values to be returned if an error occurs.

If, for example, your function returns a pointer, then you could use as reserved special value the NULL pointer to indicate that some error occurred. The NULL pointer is by definition no valid pointer, so you can be sure that this special value is not confused with a valid pointer calculated by your function as a result. The following code shows how to return error information when using pointers:

Caller's code

```
if (pointer != NULL)
{
  /* continue */
}
else
{
  /* handle error */
}
```

Callee implementation

```
void* func()
{
  if(somethingGoesWrong())
  {
     return NULL;
  }
  else
  {
    return some_pointer;
  }
}
```

You have to make sure to document in the API which returned special value has which meaning. In some cases a common convention settles which special values indicate errors. For example, UNIX functions usually

indicate an error by returning negative integer values. Still, even in such cases the meaning of the specific return values have to be documented.

You have to make sure that the special value that indicates error information really is a value that cannot occur in case of no error. For example, if a function returns a temperature value in degree Celsius as an integer value, then it would not be a good idea to stay with the UNIX convention where any negative value indicates an error. Instead, it would be better to use, for example, the value -300 to indicate an error, because it is physically impossible that a temperature takes a value below -273 degree Celsius.

Consequences

The function can now return error information via the Return Value even though the Return Value is used to transport the computation result of the function. No additional Out-Parameters have to be used only for having a way to transport error information.

Sometimes you don't have many special values to encode error information. For example, for pointers there is only the NULL pointer to indicate error information. That leads to the situation that it is only possible to indicate to the caller whether everything worked well or whether anything went wrong. This has the drawback that you cannot transport detailed error information. However, this also has the benefit that you are not tempted to transport unnecessary error information. In many cases it is sufficient to only transport the information that anything went wrong and the caller cannot react to more detailed information anyway.

If, at a later point in time, you realize that you have to transport more detailed error information, then perhaps that is not possible anymore because you have no more unused special values left. You'd have to change the whole function signature and instead Return Error Codes to transport that additional error information. Changing the function signature might not always be an option, because your API might have to stay compatible for existing callers. In such cases, it might be better to Return Error Codes right

at the beginning in order to be able to react to such changes while maintaining compatibility.

Sometimes programmers assume that it is clear which returned values indicate errors. For example, to some programmers it might be clear that a NULL pointer indicates an error. For some other programmers it might be clear that -1 indicates an error. This brings in the dangerous situation that the programmers assume that it is clear to everybody which values indicate errors. However, these are just assumptions. In any case it should be well-documented in the API which values indicate errors, but sometimes programmers forget to do that wrongly assuming that that is absolutely clear.

Known Uses

- The C stdlib function `fopen` returns a `FILE` handle if no error occurs. In case of an error, like for example, if you don't have the permission to open the file, the function returns NULL. Additionally, the function sets the `errno` variable to transport more specific information about the error.
- The `getobj` function of the game NetHack returns the pointer to some object if no error occurs and returns NULL if an error occurs. To indicate the special case that there is no object to return, the function returns the pointer to a global object called `zeroobj` that is a object of the return type defined for the function and that is also known to the caller. The caller can then check whether the returned pointer is the same as the pointer to the global object and can thus distinguish between a pointer to any valid object and a pointer to the `zeroobj` that carries some special meaning.
- The C stdlib function `getchar` reads a character from `stdin`. The function has return type `int` which allows transporting much more information than simple characters. If no more characters are available, the function returns `EOF` which is usually defined as -1. As characters

cannot take negative integer representations, `EOF` can clearly be distinguished from regular function results and can thus be used to indicate the special situation when no more characters are available.

- Most UNIX or POSIX function use negative numbers to transport error information. For example, the POSIX function `write` returns the number of written bytes or -1 on error.

Applied to Running Example

With Special Return Values, your code looks like the following. To keep it simple, only the `createKey` function is shown:

Declaration of the function createKey

```
/* Create a new registry key identified via the provided
'key_name' (must not be NULL,
   max. STRING_SIZE characters). Returns a handle to the key or
NULL on error. */
RegKey createKey(char* key_name);
```

Implementation of the function createKey

```
RegKey createKey(char* key_name)
{
  assert(key_name != NULL);
  assert(STRING_SIZE > strlen(key_name));

  RegKey newKey = calloc(1, sizeof(struct Key));
  if(newKey == NULL)
  {
    return NULL;
  }

  strcpy(newKey->key_name, key_name);
  return newKey;
}
```

The `createKey` function is much simpler now. It does not Return Error Codes anymore, but instead it directly returns the handle and no Out-

Parameter is needed to transport this information. Because of that, also the API documentation for the function becomes much simpler, because there is no need to describe the additional parameter and there is no need to lengthly describe how the function result will be transported to the caller.

Things also are much simpler for your caller. The caller does not have to provide a handle as an Out-Parameter anymore, but instead the caller directly retrieves this handle via the Return Value, which makes the caller's code a lot more readable and thus easier to maintain.

However, now you have the problem, that compared to the detailed error information that you can transport if you Return Error Codes, now the only error information that comes out of the function is whether it worked or whether it did not work. The internal details about the error are thrown away and if you need these details later on, for example as debug information, there is no way to get it. To address that issue, you can Log Errors.

Log Errors

Context

You have a function in which you handle errors. You only want to Return Relevant Errors to your caller for reacting on it in the code, but you want to keep detailed error information for later debugging.

Problem

You want to make sure that in case of an error you can easily find out its cause. However, you don't want your error handling code to become complicated because of that.

One way would be to transport very detailed error information, such as error information indicating programming errors, directly to the caller. To do that you can Return Error Codes to the caller who then displays the detailed error codes to the user. The user might get back to you (for

example, via some service hotline) to ask what that error code means and how to fix the problem. Then you'd have your detailed error information to debug the code and you could figure out what went wrong.

However, such an approach has the major drawback that the caller, who does not care at all about that error information, has to transport the error information to the user only for the sake of having a way that this error information is provided to you. Also, the user does not really care about such detailed error information.

In addition, to Return Error Codes has the drawback that you have to use the Return Value of the function to transport error information and you have to use additional Out-Parameters to transport the actual function results. In some cases, instead, you can transport error information via Special Return Values, but that is not always possible. You don't want to have additional parameters for your function only to transport error information, because it makes your caller's code more complicated.

Solution

Use different channels to transport error information that is relevant for the calling code and error information that is relevant for the developer. For example, write debug error information into a log file and don't return the detailed debug error information to the caller.

If an error occurs, the user of the program has to provide you with the logged debug information so that you can easily find out the cause of the error. For example, the user has to send you a log file via e-mail.

Alternatively, you could, at the interface between you and your caller, log the error and additionally Return Relevant Errors to the caller. For example, the caller could be informed that some internal error occurred, but the caller does not see which detailed kind of error occurred. Thus, the caller could still handle the error in the code without requiring knowledge on how to handle very detailed errors and you'd still not be losing valuable debug information.

For such valuable debug information, you should log information about programming errors and you should log unexpected errors. For such errors it is valuable to store information about where the error occurred, for example, the source code file name and the line number, or the backtrace. The C language comes with special macros to get information about the current line number (`__LINE__`), the current function (`__func__`) or the current file (`__FILE__`). The following code uses the `__func__` macro for logging:

```
void someFunction()
{
  if(something_goes_wrong)
  {
     logInFile("something went wrong", ERROR_CODE, __func__);
  }
}
```

To get more detailed logging, you could even trace your function calls and log their return information. That makes it easier to reverse-engineer error situations with these logs, but of course that logging also introduces computational overhead. For tracing return values of your function calls, you can use the following code:

```
#define RETURN(x)           \
do {                        \
 logInFile(__func__, x);    \
 return x;                  \
} while (0)

int soneFunction()
{
  RETURN(-1);
}
```

The log information can, as indicated in the preceding code, be stored in files. You'll have to care about special situations like not having enough memory to store the file or a crashing program while writing to the file. Handling such situations is not an easy task, but it is very important to have a robust code for your logging meachnism, because later on you'll rely on

the log files for debug purposes. If the data in these files is not correct, then you might be misled when hunting down coding errors.

MULTI-LINE MACROS

By having a `do/while` loop around the statements in a macro you can avoid problems like shown in the following code.

```
#define MACRO(x) \
x=1;             \
x=2;             \

if(x==0)
  MACRO(x)
```

The code does not use curly braces around its `if` body and when reading the code you might think that the thing in the macro is only executed in case `x==0`. But actually when the macro expands, you end up with the following code

```
if(x==0)
  x=1;
x=2;
```

The last line of that code is not inside the body of the `if` statement and that is not what was intended. To avoid problems like this one, it is a best practice to have a `do/while` loop around the statements in a macro.

Consequences

You can obtain debug information without requiring your caller to handle or to transport this information. That makes life for the caller a lot easier, because the caller does not have to handle or transport the unrequired detailed error information. Instead, you transport the detailed error information yourself.

Maybe in some cases, you just want to log some error or situation that occurred, but that is completely irrelevant to the caller. Thus, you don't even have to transport any error information to the caller. For example, if you abort the program if the error occurs, the caller does not at all have to react to the error and still you can make sure to not lose valuable debug information if in such a case, you Log Errors. So there are no additional parameters to your function required in order to transport error information and that makes calling your function a lot easier and it helps the caller to keep the code clean.

Still, you don't lose this valuable error information and can still use it for debug purposes to hunt down programming errors. To not loose this debug information, you transport it via a different channel, for example, via log files. However, you have to think about how to get to these log files. You could ask the users to send you the logfile via e-mail or, more advanced, you could implement some automatic bug report mechanism. Still, with both of these approaches you cannot be 100% sure that the log information really gets back to you. If the users do not want that, they could prevent it.

Known Uses

- The Apache webserver code uses the function `ap_log_error` that writes errors related to requests or connections to an error log. Such a log entry contains information about the filename and line of code where the error occurred and it contains a custom string provided to the function by the caller. The log information is stored in a `error_log` file on the server.
- The B&R Automation Runtime operating system provides a logging system that allows programmers to transport logging information to the user via calling the function `eventLogWrite` from anywhere in the code. This makes it possible to transport information to the user without having to transport this information across the whole calling stack up to some central logging component.

- The pattern Assertion Context from the book *Patterns in C* by Adam Tornhill (Leanpub, 2014) suggests to abort the program in case of errors and to additionally log information about the reason or about the position of the crash by adding a string statement inside the `assert` call. If the `assert` fails, then the link of code containing the assert statement will be printed and that then includes the added string.

Applied to Running Example

After applying the patterns, you'll get to the following final code for your registry software-module. This code provides the caller with relevant error information, but does not require the caller to handle any internal error situations:

Registry API

```
/* max. size of string parameters (including NULL-termination) */
#define STRING_SIZE 100

/* Error codes returned by this registry */
typedef enum
{
  OK,
  CANNOT_ADD_KEY
}RegError;

/* Handle for registry keys */
typedef struct Key* RegKey;

/* Create a new registry key identified via the provided
'key_name'
   (must not be NULL, max. STRING_SIZE characters).  Returns a
handle
   to the key or NULL on error. */
RegKey createKey(char* key_name);

/* Store the provided 'value' (must not be NULL, max. STRING_SIZE
characters) to
   the 'key' (MUST NOT BE NULL) */
void storeValue(RegKey key, char* value);

/* Make the 'key' (must not be NULL) available for being read.
```

```
   Returns OK if no problem occurs or CANNOT_ADD_KEY if the
   registry is full and no more keys can be released. */
RegError releaseKey(RegKey key);
```

Registry implementation

```
#define MAX_KEYS 40

struct Key
{
  char key_name[STRING_SIZE];
  char key_value[STRING_SIZE];
};

/* macro to log debug info and to assert */
#define logAssert(X)                          \
    if(!(X))                                  \
    {                                         \
      printf("Error at line %i", __LINE__); \
      assert(false);                          \
    }

/* file-global array holding all registry keys */
static struct Key* key_list[MAX_KEYS];

RegKey createKey(char* key_name)
{
  logAssert(key_name != NULL)
  logAssert(STRING_SIZE > strlen(key_name))

  RegKey newKey = calloc(1, sizeof(struct Key));
  if(newKey == NULL)
  {
    return NULL;
  }

  strcpy(newKey->key_name, key_name);
  return newKey;
}

void storeValue(RegKey key, char* value)
{
  logAssert(key != NULL && value != NULL)
  logAssert(STRING_SIZE > strlen(value))
```

```
  strcpy(key->key_value, value);
}

RegError releaseKey(RegKey key)
{
  logAssert(key != NULL)

  int i;
  for(i=0; i<MAX_KEYS; i++)
    if(key_list[i] == NULL)
    {
      key_list[i] = key;
      return OK;
    }

  return CANNOT_ADD_KEY;
}
```

That code is shorter compared to the earlier code of the running example for these reasons.

- The code does not check for programming errors but aborts the program in case of programming errors. Invalid parameters like NULL pointers are not gracefully handled in the code, but instead the API documents that the handles must not be NULL.
- The code only returns errors that are relevant for the caller. For example, the `createKey` function does not Return Error Codes, but instead simply returns a handle and NULL in case of error, because the caller does not need more detailed error information.

Although to code is shorter, the API comments grew. The comments now specify more clearly how the functions behave in case of errors. That helped that apart from your code, also the caller's code became simpler, because now the caller is not burdened anymore with that many decisions on how to react to different kinds of error information:

Caller's code

```
RegKey my_key = createKey("myKey");
if(my_key == NULL)
{
```

```
  printf("Cannot create key\n");
}

storeValue(my_key, "A");

RegError err = releaseKey(my_key);
if(err == CANNOT_ADD_KEY)
{
  printf("Key cannot be released, because the registry is
full\n");
}
```

That is shorter compared to the earlier code of the running example, because:

- The return value of functions that abort in case of error do not have to be checked
- Functions where no detailed error information is required directly return the requested item. For example, `createKey()` now returns a handle and the caller does not have to provide an Out-Parameter anymore.
- Error codes that indicate a programming error, like for example an invalid provided parameter, are not returned anymore and thus do not have to be checked by the caller anymore.

The final code of the running example showed that it is important to think about which kind of error should be handled in the code and how these errors should be handled. Simply returning all kinds of errors and requiring the caller to cope with all these errors is not always the best solution, because maybe the caller is not interested with that detailed error information or maybe the caller does not want to react to the error in the application. Maybe the error is severe enough so that already at the point where the error occurs it can be decided to abort the program. Such measures make the code simpler and have to be considered when designing the API of a software-component.

Summary

This chapter showed you how to handle errors across different functions and different parts of your software. The pattern Return Error Codes provides the caller with numeric codes representing an occurring error. Return Relevant Errors only transports error information to the caller, if the caller can react to these errors in the code and Special Return Value is one way to do that. Log Errors provides an additional channel to transport error information that is not intended for the caller, but for the user or for debugging purposes.

The patterns equip you with some more tools on how to tackle error situations and can guide your first steps when implementing a larger piece of code.

Further Reading

- The master's thesis *Error Handling in Structured and Object-Oriented Programming Languages* by Thomas Aglassinger (University of Oulu, 1999) provides a comprehensive overview of error handling in general and describes error handling best practices including code examples for several programming languages including C.
- The Portland Pattern Repository (*http://c2.com/cgi/wiki*) provides many patterns and discussions on error handling as well as other topics. Most of the error handling patterns target exception handling, but also some C idioms are presented.
- The articles *Patterns for Generation, Handling and Management of Errors* and *More Patterns for the Generation, Handling and Management of Errors* by Andy Longshaw and Eoin Woods (*http://www.eoinwoods.info/writing/*) present patterns for error logging and error handling with a focus on exception-based error handling.

Outlook

The next chapter gives guidance on how to cope with dynamic memory. In order to transport more complex data between your functions and in order to organize larger data and its lifetime throughout your application, you'll need to deal with dynamic memory and you'll need advice on how to do that.

Chapter 3. Memory Management

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 3rd chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

Each program stores some values in the memory to use them later on in the program. That functionality is so common for programs, that modern programming languages make it as easy as possible to do that. The C++ programming language as well as other object-oriented programming languages provide constructors and destructors, which make it very easy to have a defined place and time to allocate and clean up memory. The Java programming language even comes with a garbage collector, which makes sure that memory that is not used anymore by the program is made available to others.

Compared to that, programming in C is special in the way that the programmer has to manually manage the memory. The programmer has to decide whether to put variables on the stack, on the heap, or in static memory. Also, the programmer has to make sure that heap variables are

manually cleaned up afterwards and there is no mechanism like a destructor or a garbage collector, which would make such tasks much easier.

Guidance on how to perform such tasks is well scattered over the Internet and that makes it quite hard to answer questions like the following: “Should that variable go on the stack or on the heap?” To answer that as well as other questions, this chapter presents patterns on how to handle memory in C programms. The patterns provide guidance on when to use the stack, when to use the heap and when and how to clean heap memory up. To make the core idea of the patterns easier to grasp, the patterns are applied to a running code example throughout the chapter.

Figure 3-1 shows an overview of the patterns presented in this chapter and their relationships and Table 3-1 provides a summary of the patterns.

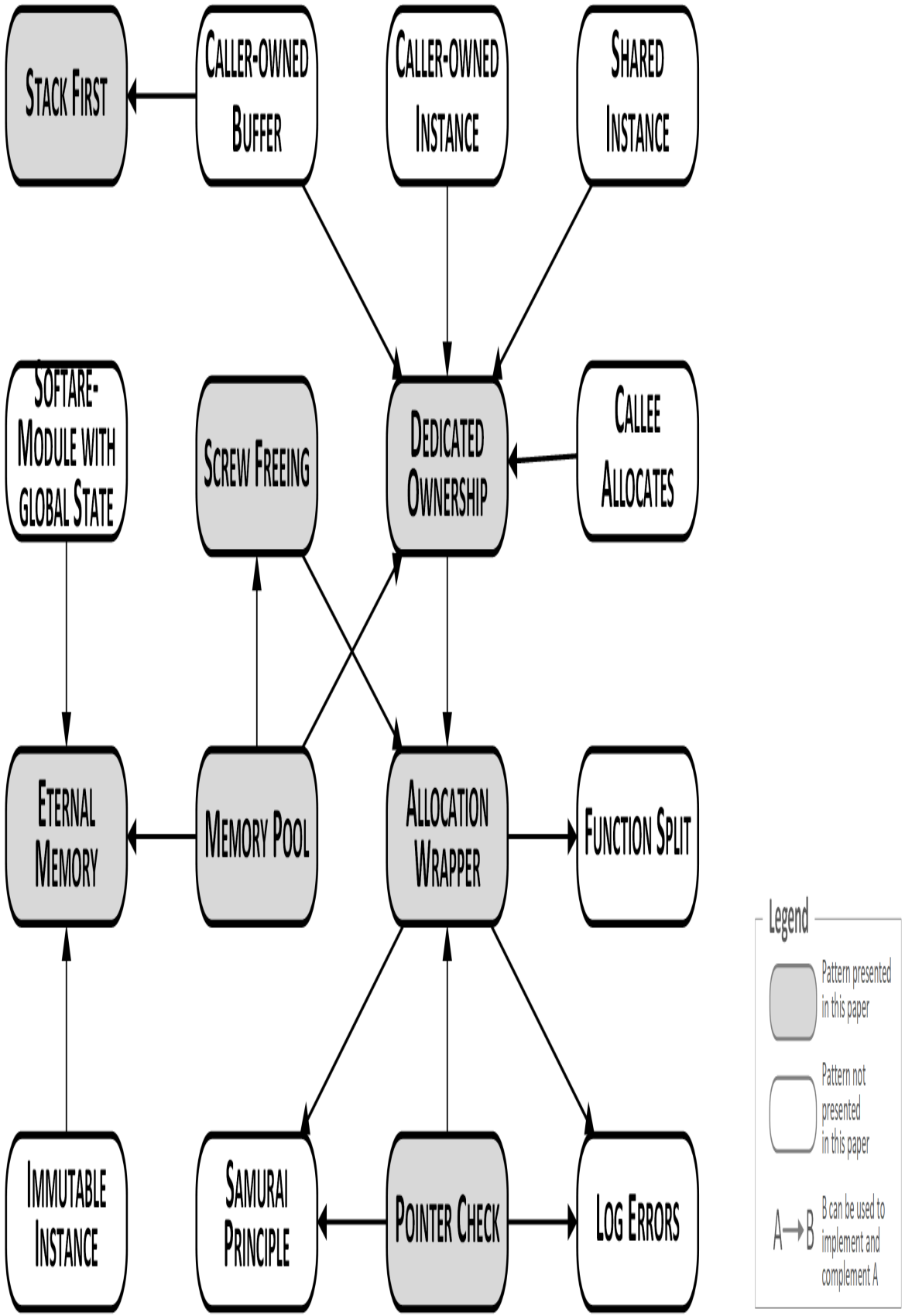

Stack First
Caller-owned Buffer
Caller-owned Instance
Shared Instance
Softare-Module with global State
Screw Freeing
Dedicated Ownership
Callee Allocates
Eternal Memory
Memory Pool
Allocation Wrapper
Function Split
Immutable Instance
Samurai Principle
Pointer Check
Log Errors
Legend
Pattern presented in this paper
Pattern not presented in this paper
A→B
B can be used to implement and complement A

Figure 3-1. Overview of patterns on memory management

Table 3-1. Patterns on memory management

n
t

	Pattern Name	Summary
	Stack First	Deciding the storage-class and memory section (stack, heap, …) for variables is a decision every programmer has to make often. It gets exhausting if for each and every variable, the pros and cons of all possible alternatives have to be considered in detail. Therefore, simply put your variables by default on the stack to profit from automatic cleanup of stack variables.
	Eternal Memory	Holding large amounts of data and transporting it between function calls is difficult, because you have to make sure that the memory for the data is large enough and that the lifetime extends across your function calls. Therefore, put your data into memory that is available throughout the whole lifetime of your program.
	Screw Freeing	Having dynamic memory is required if you need large amounts of memory and memory where you don't know the required size beforehand. However, handling cleanup of dynamic memory is a hassle and is the source of many programming errors. Therefore, allocate dynamic memory and let the operating system cope with deallocation by the end of your program.
	Dedicated Ownership	The great power of using dynamic memory comes with the great responsibility of having to properly clean that memory up. In larger programs, it becomes difficult to make sure that all dynamic memory is cleaned up properly. Therefore, right at the time when you implement memory allocation, clearly define and document where it's going to be cleaned up and who is going to do that.
	Allocation Wrapper	Each allocation of dynamic memory might fail, so you should check allocations in your code to react accordingly. That is cumbersome because you have many places for such checks in your code. Therefore, wrap the allocation and deallocation calls and implement error handling or additional memory management organization in these wrapper functions.

	Pointer Check	Programming errors that lead to accessing an invalid pointer cause uncontrolled program behavior, and such errors are difficult to debug. However, because your code works a lot with pointers, there is a good chance that you introduced such programming errors. Therefore, explicitly invalidate uninitialized or freed pointers and always check pointers for validity before accessing them.
	Memory Pool	Frequently allocating and deallocating objects from the heap leads to memory fragmentation. Therefore, hold a large piece of memory throughout the whole lifetime of your program. At runtime, retrieve fixed-size chunks of that memory pool instead of directly allocating new memory from the heap.

Data Storage and Problems with Dynamic Memory

In C you have several options where to put your data:

- You can put the data on the stack. The stack is a fixed-size memory reserved for each thread (allocated when creating the thread). In such a thread, when calling a function, a block on the top of the stack is reserved for the function parameters and automatic variables used by that function. After the function call, that memory is automatically cleaned up. To put data on the stack, simply declare variables in the functions where they are used. These variables can be accessed as long as they don't run out of scope (when the function block ends):

```
void main()
{
  int my_data;
}
```

- You can put data into static memory. The static memory is a fixed-size memory allocated at compile-time. To use the static memory, simply place the `static` keyword in front of your variable declaration. Such

variables don't run out of scope and are available throughout the whole lifetime of your program. The same holds true for global variables, even without the `static` keyword:

```
int my_global_data;
static int my_fileglobal_data;
void main()
{
  static int my_local_data;
}
```

- If your data is of fixed size and immutable, you can simply store that data directly in the static memory where the code is stored. Quite often, fixed string values are stored like that. Such data is available throughout the whole lifetime of your program (even though in the example below the pointer to that data runs out of scope):

```
void main()
{
  char* my_string = "Hello World";
}
```

- You can allocate dynamic memory on the heap to store the data. The heap is a global memory pool available for all processes on the system and it is up to the programmer to allocate and deallocate from that pool at any time:

```
void main()
{
  void* my_data = malloc(1000);
  /* work with the allocated 1000 byte memory */
  free(my_data);
}
```

Allocating dynamic memory is the starting point where things can easily go wrong and tackling the problems that can arise are the main focus of this chapter. Using dynamic memory in C programs comes with many problems that have to be solved or at least considered. The following shows the major problems with dynamic memory:

- Memory that is allocated has to be freed at some point later on. When not doing that for all memory you allocated, you'll consume more memory than you need and you have a so-called memory leak. If that happens frequently and your applications runs for a long time, you'll end up having no more memory.
- Freeing more than once is a problem and can lead to undefined program behavior and that is something really bad. Worst case, nothing goes wrong in the actual code line where you made the mistake, but at some random point later in time, you program might crash. Such errors are a hassle to debug.
- Trying to access freed memory is a problem as well. It happens easily that you free some memory and then later on make a mistake and dereference a pointer to that memory (a so-called dangling pointer). Again, that leads to error situations that are a hassle to debug. Best case, the program would simply crash. Worst case, it would not crash and the memory already belongs to somebody else and errors related to using that memory are a security risk and might show up as some kind of hard to understand error later during program execution.
- You as programmer have to cope with lifetime and ownership of allocated data. You have to know who cleans up which data when and that can be particularly tricky in C. In C++ it would be possible to simply allocate data for objects in the constructor and free them in the destructor and with that you'd nicely have defined lifetime and ownership. However, that is not possible in C, because we don't have such a thing as a destructor. We are not notified when a pointer runs out of scope and the memory should be cleaned up.
- Working with heap memory takes more time compared to working with memory from the stack or with static memory. The allocation of heap memory has to be protected against race conditions, because other processes use the same pool of memory. That makes allocation slower. Also, accessing the heap memory is slower, because usually in

comparison, the stack memory is accessed more often and thus more likely already reside in the cache or in CPU registers.

- A huge issue with heap memory is that it becomes fragmented, which is depicted in Figure 3-2. When allocating memory blocks A, B, and C and later on freeing memory block B, then your overall free heap memory is not consecutive anymore. If you want to allocate a large memory block D then, you won't get that memory, although there'd be enough total memory available. However, as that available memory is not consecutive, your `malloc` call will fail. Fragmentation is a huge issue on systems that run for a long time (like embedded real-time systems).

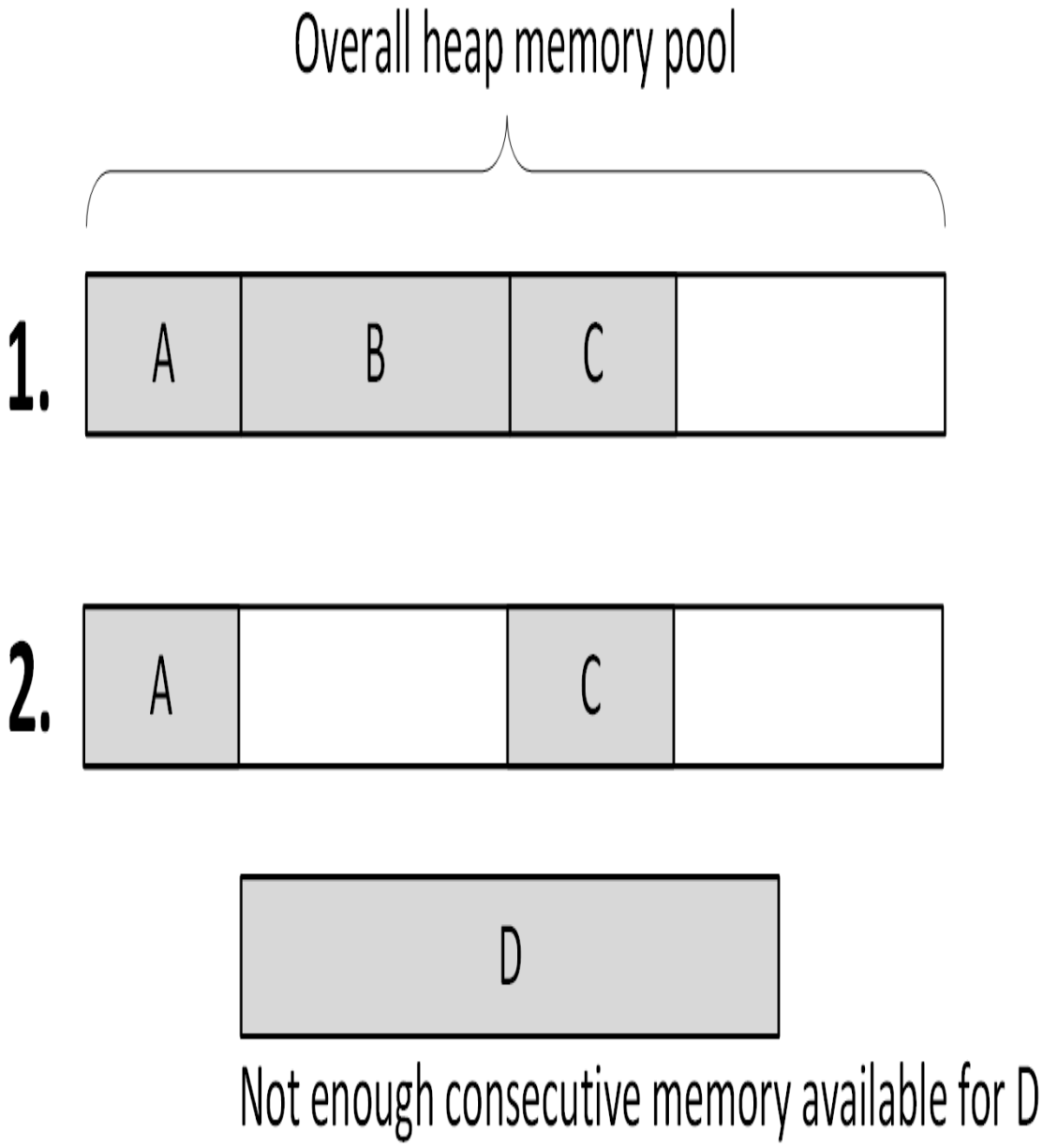


Figure 3-2. Memory fragmentation

Tackling the described issues is not easy. The patterns in the following sections describe bit by bit how to either avoid dynamic allocation or how to live with it in an acceptable way.

Running Example

You want to implement a simple program that encrypts some text with the Caesar cipher. The Caesar cipher simply replaces each letter with another letter that is some fixed number of positions down the the alphabet. For example, if the fixed number of positions is 3, then the letter A would be replaced by letter D. You start to implement a function that performs the Caesar encryption:

```
/* Performs a Caesar encryption with the fixed key 3.
   The parameter 'text' must contain a text with only capital
letters. The parameter
   'length' must contain the length of the text excluding NULL
termination. */
void caesar(char* text, int length)
{
  for(int i=0; i<length; i++)
  {
    text[i] = text[i]+3;

    /* if we shifted beyond the alphabetic characters, we restart
at the beginning */
    if(text[i] > 'Z')
    {
      text[i] = text[i] - 'Z' + 'A' - 1;
    }
  }
}
```

Now you simply want to check whether your function works and you need to feed it with some text for that. Your function takes a pointer to a string. But where should you store that string? Should you allocate it dynamically or should you work with memory from the stack? You realize the most simple solution is to use the Stack First.

Stack First

Context

You want to store data and access it at a later point in your program. You know it’s maximum size beforehand and the data is not very large in size.

Problem

Deciding the storage-class and memory section (stack, heap, …) for variables is a decision every programmer has to make often. It gets exhausting if for each and every variable, the pros and cons of all possible alternatives have to be considered in detail.

For storing data in your C program, you have a myriad of possibilities of which the most common ones are storage on the stack, in static memory, or in dynamic memory. Each of these possibilities has its own specific benefits and drawbacks and the decision of where to store the variable is very important. It affects the lifetime of the variable and it determines whether the variable is cleaned up automatically or whether you have to manually clean it up.

Making the decision also affects the required effort and discipline for you as a programmer. You want to make your life as easy as possible, so if you have no special requirements for storing the data, you want to take the kind of memory were you have the least possible effort with allocation, deallocation, and bugfixes due to potential programming errors.

Solution

Simply put your variables by default on the stack to profit from automatic cleanup of stack variables.

All variables declared inside a code block are by default so-called *automatic variables* that are put on the stack and that are automatically cleaned up once the code block ends (when the variable runs out of scope). It could be made explicit that a variable is declared as *automatic variable* by putting the `auto` storage-class specifier before it, but that is rarely done, because it is the default anyway.

You can pass the memory from the stack along to other functions (Caller-owned Buffer), but make sure to not return the address of such a variable. The variable runs out of scope at the end of the function and is automatically cleaned up. Returning the address of such a variable would

lead to a dangling pointer and accessing it would quite likely result in a crash of the program.

The following code shows a very simply example with variables on the stack:

```
void someCode()
{
  /* This variable is an automatic variable that is put on the
stack and
     that will run out of scope at the end of the function */
  int my_variable;

  {
    /* This variable is an automatic variable that is put on the
stack and
       that will run out of scope right after this code block -
after the first '}' */
    int my_array[10]; ❶
  }
}
```

❶ The array is of fixed size. It is very common to only put data of fixed size known at compile time on the stack, but it is also possible to decide the size of stack variables during runtime either using functions like `alloca()` (which is not part of the C standard and which causes stack overflows if you allocate to much) or using variable length arrays (regular arrays whose size is specified by a variable), which are introduced with the C99 standard.

Consequences

Storing the data on the stack makes it very easy to access that data. Compared to dynamically allocated memory, there is no need to work with pointers. That makes it possible to eliminate the risk of programming errors related to dangling pointers. Also, there is no heap fragmentation and memory cleanup is easier. The variables are *automatic variables* which means they are automatically cleaned up. There is no need to manually free the memory and that eliminates the risk of memory leaks or of accidentally freeing memory multiple times. In general, most of the hard-to-debug errors

related to incorrect memory usage can be eliminated when simply putting variables on the stack.

The data on the stack can be allocated and accessed very quickly compared to dynamic memory. For the allocation there is no need to go through complex data structures that manage the available memory and there is no need to ensure mutual exclusion from other threads, because each thread has its own stack. The stack data can usually be accessed quickly, because that memory is used often and usually you have it in the cache memory.

However, a drawback of using the stack is that it is limited. Compared to the heap memory, it is very small (maybe just few kB). If you put too much data on the stack, you cause a stack overflow, which usually results in a crashing program. The problem is, that you don't know how much stack memory you have left. Depending on how much stack memory is already used by the functions that called you, you might just have very little left. You have to make sure that the data you put on the stack is not too large and you have to know its size in advance.

Programming errors related to buffers on the stack can be major security issues. If you produce a buffer overflow on the stack, then attackers can easily exploit that to overwrite some other data on the stack. If attackes manage to overwrite the address your code returns to after processing the function, then the attackers can execute any code they want.

Also, having the data on the stack will not suit all your needs. If you have to return large data to the caller, then you cannot simply return the address of some array on the stack, because that variable will be cleaned up once you return from your function. For returning large data, other approaches have to be used.

Known Uses

- Nearly every C program stores something on the stack. In most programs, you'll find storage on the stack as default, because it is the easiest.

- The `auto` storage-class specifier of C, which specifies that the variable is an *automatic variable* and that it goes on the stack, is the default storage-class specifier (and is usually omitted in the code, because it is the default anyway).
- The book *Small Memory Software: Patterns for Systems With Limited Memory* by James Noble and Charles Weir (Addison-Wesley, 2000) describes in its Memory Allocation pattern, that among the choices of where to put the memory, you should go for the simplest one, which is the stack for C programmers.

Applied to Running Example

Well, that was simple. You now put the memory that you need for storing the text on the stack and you provide that memory to your Caesar's cipher function:

```
#define MAX_TEXT_SIZE 64

void encryptCaesarText()
{
  char text[MAX_TEXT_SIZE];
  strlcpy(text, "PLAINTEXT", MAX_TEXT_SIZE);
  caesar(text, strnlen(text, MAX_TEXT_SIZE));
  printf("Encrypted text: %s\n", text);
}
```

That was a very easy solution. You did not have to cope with dynamic memory allocation. There is no need to clean up the memory, because once the `text` runs out of scope, it is automatically cleaned up.

Next, you want to encrypt a larger text. That's not easily possible with your current solution, because the memory resides on the stack and you usually don't have a lot of stack memory. Depending on your platform, it could just be a few kB. Still, you want to make it possible to also encrypt larger texts. To avoid coping with dynamic memory, you decide to give Eternal Memory a try.

Eternal Memory

Context

You have large amounts of data with fixed size that you need for a longer time in your program.

Problem

Holding large amounts of data and transporting it between function calls is difficult, because you have to make sure that the memory for the data is large enough and that the lifetime extends across your function calls.

Putting the data on the stack is not a solution, because it does not allow to pass large data between functions - although the stack would be handy, because it would do all the memory cleanup work for you. The alternative of manually allocating the memory at each place in the program where you need it and deallocating it as soon as it is not required anymore, would work, but it is cumbersome and error-prone. In particular, keeping an overview of the lifetime of each data and knowing where and when the data is being freed is a complicated task.

If you operate in an environment where you must be sure that there is memory available, then neither using memory from the stack nor using dynamic memory is an option, because both could run out of memory and you cannot easily know beforehand.

Solution

Put your data into memory that is available throughout the whole lifetime of your program.

The most common way to do that, is to use the static memory. Either mark your variable with the `static` storage-class specifier, or if you want that variable to have larger scope, declare it outside any function. That memory

is then allocated at startup of your program and is available all through your program's lifetime. The following code gives an example for that:

```
#define ARRAY_SIZE 10 ❶

int global_array[ARRAY_SIZE]; /* variable in static memory with
global scope */
static int file_global_array[ARRAY_SIZE]; /* variable in static
memory with
                                                scope limited to
this file */

void someCode()
{
  static int local_array[ARRAY_SIZE]; /* variable in static
memory with
                                            scope limited to this
function */
}
```

❶ Using constants to specify the size of the memory makes it easier to change the size afterwards if your application grows and requires more memory.

Alternatively to having static variables, you could on program startup call some initialization function that allocates the memory and by the end of your program call some deinitialization function that deallocates that memory. That way you'd also have the memory available all through the lifetime of your program, but you'd have to cope with allocation and deallocation yourself.

No matter whether you allocate the memory on your own or whether you use static memory, you have to be careful when accessing this memory. As it is not on the stack, you don't have a separate copy of that memory per thread. In case of multi-threading, you have to use synchronization mechanisms when accessing that memory. Also, there is a fixed size of that data. Compared to dynamic memory, that size does not grow at runtime and in your code you have to check if the accessed memory is within that size.

Consequences

You don’t have to break your head about lifetime and the right place for manually deallocating memory. The rules are simple: let the memory live throughout your whole program lifetime. Using static memory even takes the whole burden of allocation and deallocation from you.

You can now store large amounts of data in that memory and even pass it along to other functions. Compared to using Stack First, you can now even provide data to the caller’s of your function.

However, you have to know at compile time or latest at startup time how much memory you need, because you already allocate it at program startup. For memory of unknown size or for memory that will be expanded during runtime, Eternal Memory is not the best choice and instead, heap memory should be used.

With Eternal Memory, starting the program will take longer, because all the memory has to be allocated at that time. But that pays off once you have that memory, because there is no allocation necessary during runtime anymore.

Allocating and accessing static memory does not need any complex data structures maintained by your operating system or runtime environment for managing the heap. Thus, the memory is more efficiently used. Another huge advantage of Eternal Memory is that you don’t fragment the heap, because you don’t allocate and deallocate memory all the time. But not doing that has the drawback that you block memory that you, depending on your application, might not need all the time.

One issue with Eternal Memory is that you don’t have a copy of it for each of your threads (if you use static variables). So you have to make sure that the memory is not accessed by multiple threads as the same time. Although, in the special case of an Immutable Instance that would not be much of an issue.

Known Uses

- The game NetHack uses static variables to store data that is required during the whole lifetime of the game. For example, the information about artifacts found in the game is stored in the static array `artifact_names`.
- The code of the network sniffer Wireshark uses a static buffer in its function `cf_open_error_message` for storing error message information. In general, many programs use static memory or memory allocated at program startup for their error logging functionality. That is, because in case of errors, you want to be sure that at least that part works and does not run out of memory.
- The OpenSSL code uses the static array `OSSL_STORE_str_reasons` to hold error information about error situations that can occur when working with certificates.

Applied to Running Example

Your code pretty much stayed the same. The only thing that changed is that you add the `static` keyword before the variable declaration of `text` and you increased the size of the text:

```
#define MAX_TEXT_SIZE 1024

void encryptCaesarText()
{
  static char text[MAX_TEXT_SIZE];
  strlcpy(text, "LARGETEXTTHATCOULDBETHOUSANDCHARACTERSLONG",
MAX_TEXT_SIZE);
  caesar(text, strnlen(text, MAX_TEXT_SIZE));
  printf("Encrypted text: %s\n", text);
}
```

Now, your text is not stored on the stack, but instead it resides in the static memory. When doing that you should remember that that also means the variable only exists once and remains its value (even when entering the function multiple times). That could be an issue for multi-threaded systems,

because then you'd have to make sure of mutual exclusion when accessing the variable.

You currently don't have a multi-threaded system. However, the requirements for your system change: now you want to make it possible to read the text from a file, to encrypt it and to show the encrypted text. You don't know how long the text will be and it could be quite long. So you decide to use dynamic allocation as shown in the following code:

```
void encryptCaesarText()
{
  /* open file */
  FILE * f = fopen ("my-file.txt", "r");

  /* get file length */
  fseek (f, 0, SEEK_END);
  int size = ftell (f);

  /* allocate buffer */
  char* text = malloc(size);

  ...
}
```

But how should that code continue? You allocated the text on the heap. But how would you clean that memory up? As a very first step, you realize that cleaning up that memory could be done by somebody completely else: the operating system. So you think for yourself: Screw Freeing.

Screw Freeing

Context

You want to store some data in your program and that data is large and maybe you don't even know its size beforehand. The size of that data does not change often during runtime and the data is needed nearly throughout the whole lifetime of the program. Your program is short-lived (does not run over a very long time).

Problem

Having dynamic memory is required if you need large amounts of memory and memory where you don't know the required size beforehand. However, handling cleanup of dynamic memory is a hassle and is the source of many programming errors.

There are many situations in which you cannot put the data on the stack or in static memory. These kinds of memory are not a good choice if you have very large data for which you might not even know the size beforehand. So you end up with having to use dynamic memory and with having to cope with allocating it.

Now the question comes up, how to clean that data up. Cleaning it up is a major source of programming errors. You could accidentally free the memory too early causing a dangling pointer. You could accidentally free the same memory twice. Both of these programming errors can lead to undefined program behavior, like for example a program crash at some later point in time. Such errors are very difficult to debug and C programmers spend way too much of their time with troubleshooting such situations.

All kinds of memory come with some kind of automatic cleanup. The stack memory is automatically cleaned up when returning from a function. The static memory and the heap memory are automatically cleaned up on program termination.

Solution

Allocate dynamic memory and let the operating system cope with deallocation by the end of your program.

When your program ends and the operating system cleans up your process, the operating system also cleans up any memory that you allocated and didn't deallocate. Take advantage of that and let the operating system do the whole job of keeping track which memory still needs cleanup and actually cleaning it up as done in the following code:

```
void someCode()
{
  char* memory = malloc(1024);
  ...
  /* do something with the memory */
  ...
  /* don't care about freeing the memory */
}
```

The approach looks very brutal at first sight. You deliberately create memory leaks. However, that's the style of coding you'd as well use in other programming languages that have a garbage collector. You could even include some garbage collector library in C to use that style of coding with the benefit of automatic memory cleanup (and the drawback of less predictive timing behavior).

Deliberately having memory leaks might be an option for some applications. In particular those that don't run for very long time and that don't allocate very often. But for other applications it will not be an option and you'll need Dedicated Ownership of memory and also cope with it's deallocation. An easy way to clean the memory up if you earlier Screwed Freeing is to use an Allocation Wrapper and to then have one single function that by the end of your program cleans up all the allocated memory.

Consequences

The obvious advantage here is that you can benefit from using dynamic memory without having to cope with freeing the memory. That makes life for a programmer a lot easier. Also, you don't waste any processing time on freeing memory and the can in particular speed up the shutdown procedure of your program.

However, that comes at the cost of other running processes who might need the memory that you do not release. Maybe even you cannot allocate any new memory yourself, because there is not much left and you didn't free the memory that you could have freed. In particular, if you allocate very often, that becomes a major issue and it will not be a good solution for you to

Screw Freeing. Instead, you'd rather Dedicate Ownership and also free the memory.

With the pattern you deliberately create memory leaks and you do accept it. While that might be OK with you, it might not be OK with other people calling your functions. If you write some library that can be used by others, having memory leaks in that code will not be an option. Also, if you yourself want to stay very clean in some other part of the code and, for example, use some memory debugging tool like *valgrind* to detect memory leaks, you'd have problems with interpreting the results of the tool if some other part of your program is messy and does not free its memory.

Known Uses

- The Wireshark function `pcap_free_datalinks` does under certain circumstances not free all memory. The reason is that part of the Wireshark code might have been built with a different compiler and different C runtime libraries. Freeing memory that was allocated by such code might crash. Therefore, the memory is explicitly not freed at all.
- The device drivers of the Automation Runtime operating system of the company B&R usually don't have any functionality for deinitializing them. All memory they allocated is never freed. That is, because these drivers are never unloaded at runtime. If a different driver shall be used, the whole system reboots. That makes explicitly freeing the memory unnecessary.
- The code of the NetDRMS data management system, which is used to store images of the sun for scientific processing, does explicitly not properly free all memory in error situations. For example, the function `EmptyDir` does not clean up all memory and other resources related to accessing files if an error occurs, because such an error would lead to a more severe error and program abort anyway.

- Any C code with uses some garbage collection library applies this pattern (and conquers its drawbacks of memory leaks with the garbage collection).

Applied to Running Example

In your code, you simply omit using any `free` function call. Also, you restructured the code, to have the file access functionality in separate functions:

```
/* Returns the length of the file with the provided 'filename' */
int getFileLength(char* filename)
{
  FILE * f = fopen (filename, "r");
  fseek (f, 0, SEEK_END);
  int file_length = ftell (f);
  fclose(f);
  return file_length;
}

/* Stores the content of the file with the provided 'filename'
into the provided
   'buffer' (which has to be least of size 'file_length'). The
file must only contain
   capital letters with no newline in between (that's what our
caesar function accepts
   as input) */
void readFileContent(char* filename, char* buffer, int
file_length)
{
  FILE * f = fopen (filename, "r");
  fseek (f, 0, SEEK_SET);
  int read_elements = fread (buffer, 1, file_length, f);
  buffer[read_elements] = '\0';
  fclose (f);
}

void encryptCaesarFile()
{
  char* text;
  int size = getFileLength("my-file.txt");
  if(size>0)
  {
    text = malloc(size);
```

```
        readFileContent("my-file.txt", text, size);
        caesar(text, strnlen(text, size));
        printf("Encrypted text: %s\n", text);
        /* you don't free the memory here */
    }
}
```

You do allocate the memory, but you don't call `free` to deallocate it. Instead, you let the pointers to the memory run out of scope and you have a memory leak there. However, that is not important to you, because right afterwards, your program ends anyway and the operating system cleans up the memory.

That approach seems quite savage, but in some few cases that is completely acceptable. If you need the memory all through the lifetime of your program or if your program is very short-lived and you are very sure that your code is not going to evolve or going to be reused somewhere else, then simply not having to cope with cleaning the memory up can be a solution that makes life very simple for you. Still, you have to be very careful that your program does not evolve and become long-lived or that more and more usage of the heap memory comes up (for example, by using dynamic data structures). In that case, you'd definitely have to find another approach.

And that is exactly what you'll do next. You want to encrypt more than one single file. You want to encrypt all files from the current directory. You quickly realize that you have to allocate more often and that not deallocating any of the memory in the meantime is not an option anymore, because you'd use up a lot of memory and that could be a problem to your program or to other programs.

The question comes up of where in the code your memory should be deallocated. Who is responsible for doing that? You definitely need Dedicated Ownership.

Dedicated Ownership

Context

You have large data of beforehand unknown size in your program and you use dynamic memory to store it. You need that memory not for the whole lifetime of the program. You have to allocate memory of different size often and you cannot afford to simply Screw Freeing.

Problem

The great power of using dynamic memory comes with the great responsibility of having to properly clean that memory up. In larger programs, it becomes difficult to make sure that all dynamic memory is cleaned up properly.

There are many pitfalls when cleaning dynamic memory up. You might clean it up too soon and somebody else afterwards still wants to access that memory (dangling pointer). Or you might accidentally free the memory too often. Both of these situations lead to unexpected program behavior - like a crash of the program at some later point in time. Such errors are extremely difficult to debug.

Yet, you do have to clean the memory up, because over time, you'd use up to much memory when only allocating new one without freeing it. Then your program or other processes would run out of memory.

Solution

Right at the time when you implement memory allocation, clearly define and document where it's going to be cleaned up and who is going to do that.

It should be clearly documented in the code who owns the memory and how long it's going to be valid. Best case, even before writing your first `malloc`, you should have said to yourself where that code shall be freed and you should have written some comments in the function declarations to make clear whether memory buffers are passed along by that function and if so, who is responsible for cleaning it up.

In other programming languages, like C++, you have the possibility to use code constructs for documenting that. Pointer constructs like `unique_ptr` or `shared_ptr` make it possible to see from the function declarations who is responsible for cleaning the memory up. As there are no such constructs in C, extra care has to be taken to document these responsibility in the form of code comments.

If possible, make one and the same function responsible for allocation and deallocation, just like it is the case with Object-Based Error Handling where you have exactly one point in the code for calling constructor- and destructor-like functions for allocation and deallocation. If the responsibility for allocation and deallocation is spread across the code and if ownership of memory is transferred, it gets complicated. In some cases, that will be necessary, but if possible, avoid it and keep things simple like in the following code:

```
/* Allocates and returns a buffer that has to be freed by the
caller */
char* functionA()
{
  char* memory = malloc(1024); ❶
  /* fill memory */
  return memory;
}

void functionB()
{
  char* memory = functionA();
  /* work with the memory */
  free(memory);

  char* other_memory = malloc(1024);
  /* work with other memory */
  free(other_memory); ❷
}
```

❶ The Callee Allocates some memory.
❷ The caller is responsible for cleaning the memory up.

Other patterns that describe more specific situations related to memory ownership are the Caller-Owned Buffer or the Caller-Owned Instance

where the caller is responsible for allocating and deallocating memory.

Consequences

Finally, you can allocate memory and properly handle it's cleanup. That gives you flexibility. You can temporarily use large amounts of memory from the heap and at a later point in time let other's use that memory.

But of course that benefit comes at some additional cost. You have to cope with cleaning up the memory. That makes your work of programming harder, it takes some time to free the memory, and even when having Dedicated Ownership, memory-related programming errors can occur and lead to hard to debug situations. Explicitly documenting where memory will be cleaned up already helps to prevent some of these errors and in general makes the code easier to understand and maintain. To further avoid memory-related programming errors, you can in addition use an Allocation Wrapper and Pointer Check.

With the allocation and deallocation of dynamic memory, the problems of heap fragmentation and increased time for allocating and accessing the memory comes up. For some applications that might not be an issue at all, but for other applications these topics are very serious. In that case, a Memory Pool can help.

Known Uses

- The book *Extreme C* by Kamran Amini (Packt Publishing Limited, 2019) suggests that the function that allocated memory should also responsible for freeing it and that the function or object that owns the memory should be documented as comments. Of course that concept also holds true, if you have wrapper functions. Then the function that calls the allocation wrapper should be the one that calls the cleanup wrapper.
- The implementation of the function `mexFunction` of the numeric computing environment MATLAB clearly documents which memory

it owns and will free.

- The NetHack game explicitly documents for the caller's of the functions if they have to free some memory. For example, the function `nh_compose_ascii_screenshot` allocates and returns a string that has to be freed by the caller.
- The Wireshark dissector for "Community ID flow hashes" clearly documents for its functions who is responsible for freeing memory. For example, the function `communityid_calc` allocates some memory and requires the caller do free it.

Applied to Running Example

The functionality of the function `encryptCaesarFile` did not change. The only thing you changed is that you now also call `free` to deallocate the memory and you now clearly document in the code comments who is responsible for cleaning up which memory. Also, you implemented the function `encryptDirectoryContent` that encrypts all files in the current working directory:

```
/* For the provided file 'filename', this function reads text
from the file and prints
   the Caesar-encrypted text. This function is responsible for
allocating and
   deallocating the required buffers for storing the file content
*/
void encryptCaesarFile(char* filename)
{
  char* text;
  int size = getFileLength(filename);
  if(size>0)
  {
    text = malloc(size);
    readFileContent(filename, text, size);
    caesar(text, strnlen(text, size));
    printf("Encrypted text: %s\n", text);
    free(text);
  }
}
```

```
/* For all files in the current directory, this function reads
text from the file and
   prints the Caesar-encrypted text. */
void encryptDirectoryContent()
{
  struct dirent *directory_entry;
  DIR *directory = opendir(".");
  while ((directory_entry = readdir(directory)) != NULL)
  {
    encryptCaesarFile(directory_entry->d_name);
  }
  closedir(directory);
}
```

The preceding code prints the Caesar-encrypted content of all files of the current directory. Note, that the code only works on UNIX systems and that for reasons of simplicity no specific error handling is implemented for the case that the files in the directory don't have the expected content.

The memory is now also cleaned up when it is not required anymore. Note, that not all the memory that the program requires during its runtime is allocated at the same time. At most the memory required for one of the files is allocated at any time throughout running the program. That makes the memory footprint of the program significantly smaller - in particular, if the directory contains many files.

The preceding code does not cope with error handling. For example, what happens if no more memory is available? The code would simply crash. You want to have some kind of error handling for such situations, but checking the pointers returned from `malloc` at each and every point where you allocate memory can be cumbersome. What you need is an Allocation Wrapper.

Allocation Wrapper

Context

You allocate dynamic memory at several places in your code and you want to react to error situations.

Problem

Each allocation of dynamic memory might fail, so you should check allocations in your code to react accordingly. That is cumbersome because you have many places for such checks in your code.

The `malloc` function returns NULL if the requested memory is not available. On the one hand, not checking the return value of `malloc` would cause your program to crash if no memory is available and you access a NULL pointer. On the other hand, checking the return value at each and every place where you allocate, makes your code more complicated and thus harder to read and maintain.

If you distribute such checks across your code base and later on want to change your behavior in case of allocation errors, then you'd have to touch code at many different places. Also, simply adding an error check to existing functions violates the single responsibility principle, which says that one function should only be responsible for one single thing (and not for multiple things like allocation, program logic, and error handling).

Also, if you want to change the way of allocation later on, maybe to explicitly initialize all allocated memory, then having many calls to allocation functions distributed all over your code makes that very hard.

Solution

Wrap the allocation and deallocation calls and implement error handling or additional memory management organization in these wrapper functions.

Implement a wrapper function for the `malloc` and `free` calls and for memory allocation and deallocation only call these wrapper functions. In the wrapper function, you can implement error handling at one central

point. For example, you can check the allocated pointer (see Pointer Check) and in case of error abort the program as shown in the following code:

```
void* checkedMalloc(size_t size)
{
  void* pointer = malloc(size);
  assert(pointer);
  return pointer;
}

void someFunction()
{
  char* memory = checkedMalloc(1024);
  /* work with the memory */
  free(memory);
}
```

Alternatively to aborting the program, you can Log Errors. For logging the debug information, using a macro instead of a wrapper function can make life even easier. You could then without any effort for the caller log the file name, the function name, or the code line number where the error occured. With that information, it is very easy for the programmer to pinpoint the part of the code where the error occurred. Also, having a macro instead of a wrapper function saves you the additional function call of the wrapper function (but in most cases that doesn't matter or the compiler would inline the function anyway). With macros for allocation and deallocation you could even build a constructure-like syntax as shown in the following code:

```
#define NEW(object, type)                    \
do {                                         \
  object = malloc(sizeof(type));             \
  if(!object)                                \
  {                                          \
    printf("Malloc Error: %s\n", __func__);  \
    assert(false);                           \
  }                                          \
} while (0)

#define DELETE(object) free(object)


typedef struct{
```

```
  int x;
  int y;
}MyStruct;

void someFunction()
{
  MyStruct* myObject;
  NEW(myObject, MyStruct);
  /* work with the object */
  DELETE(myObject);
}
```

Alternatively or additional to handling error situations in the wrapper functions, you could also do other things. For example, you could keep track of which memory your program allocated and store that information along with the code file and code line number in a list (for that you'd also need a wrapper for `free`, like in the preceding example). That way you can easily print debug information if you want to see which memory is currently allocated (and which of it you might have forgotten to free). But if you are looking for such information, you could also simply use some memory debugging tool like *valgrind*. Still, by keeping track of which memory you allocated, you could also implement some function to free all your memory - that might be an option to make your program cleaner if you earlier Screwed Freeing.

Having everything at one place will not always be a solution for you. Maybe there are non-critical parts of your application for which you do not want the whole application to abort if an allocation error occurs there. Then, maybe having multiple different Allocation Wrappers could work for you. One wrapper could still assert on error and can be used for the critical allocations that are mandatory for your application to work. Another wrapper for the non-critical part of your application could on error Return Error Codes to make it possible to gracefully handle that error situation.

Consequences

Error handling and other memory handling is now at one central place. At the places in the code where you need to allocate memory, you now simply

call the wrapper and there is no need to explicitly handle errors at that point in the code. But that just works for some kinds of error handling. It works very well if you abort the program in case of errors, but if you react to errors by continuing the program with some degraded functionality, then you still have to return some error information from the wrapper and react to it. For that, the Allocation Wrapper does not make life easier. However, also in such a situation, there could still be some logging functionality implemented in the wrapper to improve the situation for you.

Also for testing, the wrapper function brings advantages, because you have one central point for changing the behavior of your memory allocation function and you could mock the wrapper (replace the wrapper calls with some other test-function) while still leaving other calls to `malloc` (maybe from 3rd party code) untouched.

Separating the error handling part with a wrapper function from the calling code is good practice, because then the caller is not tempted to implement error handling directly inside the code that handles other programing logic. Having several things done in one single function (program logic and error handling) would violate the single responsibility principle.

Having allocation error handling at one single place makes it easier for you, if you later on want to change the error handling behavior or the memory allocation behavior. If you decide that you want to log additional information, there is just one place in the code you'd have to touch. If you decide to later on not directly call `malloc` but to use a Memory Pool instead, then that is a lot easier when having the wrapper.

Known Uses

- The book *C Interfaces and Implementations* by David R. Hanson (Addison-Wesley, 1996) uses a wrapper function for allocating memory in an implementation for a Memory Pool. The wrappers simply call `assert` to abort the program in case of errors.

- GLib provides the functions `g_malloc` and `g_free` amoung other memory-related functions. The benefit of using `g_malloc` is that in case of error, it aborts the program (Samurai Principle). Because of that, there is no need for the caller to check the return value of each and every function call for allocating memory.
- The GoAccess real-time web log analyzer implements the function `xmalloc` to wrap `malloc` calls with some error handling.

Applied to Running Example

Now, instead of directly calling `malloc` and `free` everywhere in your code, you use wrapper functions:

```
/* Allocates memory and asserts if no memory is available */
void* mallocWrapper( size)
{
  void* pointer = malloc(size);
  assert(pointer); ❶
  return pointer;
}

/* Deallocates the memory of the provided 'pointer' */
void freeWrapper(void *pointer)
{
  free(pointer);
}

/* For the provided file 'filename', this function reads text
from the file and prints
   the Caesar-encrypted text. This function is responsible for
allocating and
   deallocating the required buffers for storing the file content
*/
void encryptCaesarFile(char* filename)
{
  char* text;
  int size = getFileLength(filename);
  if(size>0)
  {
    text = mallocWrapper(size);
    readFileContent(filename, text, size);
    caesar(text, strnlen(text, size));
```

```
    printf("Encrypted text: %s\n", text);
    freeWrapper(text);
  }
}
```

❶ If the allocation fails, you adhere to the Samurai Principle and abort the program. For some applications like yours, that is a valid option. If there is no way for you to gracefully handle the error, then directly aborting the program is the right and consequent choice.

With the Allocation Wrapper you have the advantage that you now have a central point for handling allocation errors. There is no need to write some lines of code for checking the pointer after each allocation in your code. You also have a wrapper for freeing the code, because that might in future might come in handy if you, for example, decide to keep track of which memory is currently allocated by your application.

After the allocation you now check whether the retrieved pointer is valid. After that, you don't check the pointer for validity anymore and also across function boundaries, you trust that the pointers you received are valid. That is fine, as long as no programming errors sneak in, but if you accidentally access invalid pointers, the situation becomes difficult and hard to debug. To improve your code and to be on the safe side, you decide to use a Pointer Check.

Pointer Check

Context

Your program contains many places where you allocate and deallocate memory and many places where you access that memory or other resources with pointers.

Problem

Programming errors that lead to accessing an invalid pointer cause uncontrolled program behavior, and such errors are difficult to debug. However, because your code works a lot with pointers, there is a good chance that you introduced such programming errors.

C programming requires a lot of struggling with pointers and the more places in the code you have that work with pointers, the more places in the code you have where you could introduce programming errors. Using a pointer that was already freed or using an uninitialized pointer would lead to error situations that are hard to debug.

Any such error situation is very severe. It leads to uncontrolled program behavior and (if you are lucky) to a program crash. If you are less lucky you end up with some error that occurs at a later point in time during program execution and that takes you a week to debug and pinpoint. You want your program to be more robust against such errors. You want to make such errors less severe and you want to make it easier to find the cause of such error situations if they occur in your running program.

Solution

Explicitly invalidate uninitialized or freed pointers and always check pointers for validity before accessing them.

Right at the variable declaration, set pointer variables explicitly to NULL. Also, right after calling `free`, set them explicitly to NULL. If you use an Allocation Wrapper that uses a macro for wrapping the `free` function, you could directly set the pointer to NULL inside the macro to avoid having additional lines of code for invalidating the pointer at each deallocation.

Have some wrapper function or a macro that checks a pointer for NULL and in case of a NULL pointer aborts the program and Logs Errors to have some debug information. If aborting the program is not an option for you, then instead you could in case of NULL pointers not perform the pointer access and try to handle the error graceful so that your program can continue with reduced functionality as shown in the following code:

```
void someFunction()
{
  char data;
  char* pointer = NULL; /* explicitly invalidate the
uninitialized pointer */
  pointer = malloc(1024);
  ...
  if (pointer != NULL) /* check pointer validity before accessing
it */
  {
    data = *pointer;
  }
  ...
  free(pointer);
  pointer = NULL; /* explicitly invalidate the pointer to freed
memory */
}
```

Consequences

Your code is a bit more protected against pointer-related programming errors and each such error that can be identified and does not lead to undefined program behavior might save you hours and days of debugging effort.

However, that does not come for free. Your code became longer and more complicated. The strategy you apply here is like having belt and suspenders. You do some extra work to be more safe. You have additional checks for each pointer access. That makes the code harder to read. For checking the pointer validity before accessing it, you'll at least have one additional line of code. If you do not abort the program, but instead continue with degraded functionality, then your program even becomes much more difficult to read, maintain, and test.

If you accidentally call `free` on a pointer multiple times, then your second call would not lead to an error situation, because after the first call you invalidated the pointer and then calling `free` on a NULL pointer does no harm. Still, you could Log Errors like that to make it possible to pinpoint the root cause for that error.

But even after all that, you are not fully protected against any kind of pointer-related errors. For example, you could forget to free some memory and produce a memory leak. Or you could access a pointer that you did not properly initialize, but at least you'd detect that and could react accordingly. A possible drawback here is that if you decide to gracefully degrade your program and continue, you might obscure error situations that are then later on hard to find.

Known Uses

- The implementation for C++ smart pointers invalidates the wrapped raw pointer when releasing the smart pointer.
- "Cloudy" is a program for physical calculations (spectral synthesis). It contains some code for interpolation of data (Gaunt factor). That program checks pointers for validity before accessing them and explicitly sets the pointers to NULL after calling `free`.
- The libCPP of the GNU Compiler Collection (GCC) invalidates the pointers after freeing them. For example, the pointers in the implementation file *macro.c* do that.
- The function `HB_GARBAGE_FUNC` of the MySQL database management system sets the pointer `ph` to NULL to avoid accidentally accessing it or freeing it multiple times later on.

Applied to Running Example

You now have the following code:

```
/* For the provided file 'filename', this function reads text
from the file and prints
   the Caesar-encrypted text. This function is responsible for
allocating and
   deallocating the required buffers for storing the file content
*/
void encryptCaesarFile(char* filename)
{
```

```
  char* text = NULL; ❶
  int size = getFileLength(filename);
  if(size>0)
  {
    text = mallocWrapper(size);
    if(text != NULL) ❷
    {
      readFileContent(filename, text, size);
      caesar(text, strnlen(text, size));
      printf("Encrypted text: %s\n", text);
    }
    freeWrapper(text);
    text = NULL; ❶
  }
}
```

❶ At places where the pointer is not valid, you explicitly set it to NULL - just to be on the safe side.

❷ Before accessing the pointer `text`, you check whether it is valid. If it is not valid, you don't use the pointer (you dereference it nowhere).

LINUX OVERCOMMIT

Beware that having a valid memory pointer does not always mean that you can safely access that memory. Modern Linux systems work with the *overcommit* principle. That principle provides virtual memory the to program which allocates, but that virtual memory has no direct correspondance to physical memory. Whether the required physical memory is available is checked once you access that memory. In case not enough physical memory is then available, the Linux kernel shuts down applications which consume a lot of memory (and that might be your application). Overcommit brings the advantage that it becomes less important to check whether allocation worked (because it usually does not fail) and it has the advantage that you can allocate a lot of memory to be on the safe side, even if you just need a little bit. But overcommit also comes with the big disadvantage that even with a valid pointer you can never be sure that your memory access works and does not lead to some crash. Another disadvantage is that you might become lazy with checking allocation return values and with figuring out and allocating just the amout of memory that you actually need.

Next, you also want to show the Caesar encrypted file name along with the encrypted text. You decide against directly allocating the required memory from the heap, because you are afraid of memory fragmentation when repeatedly allocating small memory chunks (for the file names) and large

memory chunks (for the file content). Instead of directly allocating dynamic memory, you implement a Memory Pool.

Memory Pool

Context

You frequently allocate and deallocate dynamic memory from the heap in your program for elements of roughly the same size. You don't know at compile time or startup time where and when exactly in your program these elements are needed.

Problem

Frequently allocating and deallocating objects from the heap leads to memory fragmentation.

When allocating objects, in particular of strongly varying size, and in the meantime deallocating some of them, the heap memory becomes fragmented. Even if the allocations from your code are roughly the same size, they might be mixed with allocations from other programs running in parallel and you'd end up with allocations of strongly varying size and fragmentation.

The `malloc` function only succeeds, if there is enough free consecutive memory available. That means that even if there is enough free memory available, the `malloc` function might fail if the memory is fragmented and no consecutive chunk of memory of the required size is available. That means the memory is not very well utilized.

Fragmentation is a serious issue for long running systems, like for most embedded systems. If a system runs for some years and allocates and deallocates many small chunks, then quite surely it will not be possible to allocate a larger chunk of memory anymore. That means that you definitely have to tackle the fragmentation issue for such systems if you don't accept that the system has to be rebooted from time to time.

Another issue when using dynamic memory, in particular, in combination with embedded systems, is that the allocation of memory from the heap takes some time. Other process try to use the same heap and thus the allocation has to be interlocked and its required time becomes very hard to predict.

Solution

Hold a large piece of memory throughout the whole lifetime of your program. At runtime, retrieve fixed-size chunks of that memory pool instead of directly allocating new memory from the heap.

The memory pool can either be placed in static memory or it can be allocated from the heap at program startup and freed at the end of the program. Allocation from the heap has the advantage, that, if needed, additional memory could be allocated to increase the size of the memory pool.

Implement functions for retrieving and releasing memory chunks of fixed pre-configured size from that pool. All of you code that needs memory of that size can use these functions (instead of `malloc` and `free`) for acquiring and releasing dynamic memory as shown in the following code:

```
#define MAX_ELEMENTS 20;
#define ELEMENT_SIZE 255;

typedef struct
{
  bool occupied;
  char memory[ELEMENT_SIZE];
}PoolElement;

static PoolElement memory_pool[MAX_ELEMENTS];

/* Returns memory of at least the provided 'size' or NULL
   if no memory chunk from the pool is available */
void* poolTake(size_t size)
{
  if(size <= ELEMENT_SIZE)
  {
    for(int i=0; i<MAX_ELEMENTS; i++)
```

```
    {
      if(memory_pool[i].occupied == false)
      {
        memory_pool[i].occupied = true;
        return &(memory_pool[i].memory);
      }
    }
  }
  return NULL;
}

/* Gives the memory chunk ('pointer') back to the pool */
void poolRelease(void* pointer)
{
  for(int i=0; i<MAX_ELEMENTS; i++)
  {
    if(&(memory_pool[i].memory) == pointer)
    {
      memory_pool[i].occupied = false;
      return;
    }
  }
}
```

The preceding code shows a simple implementation of a Memory Pool and there would be many ways to improve that implementation. For example, free memory slots could be stored in a list to speed up taking such a slot. Also, mutex or semaphores could be used to make sure that it works in multi-threaded environments.

For the Memory Pool, you have to know which kind of data will be stored, because you have to know the size of the memory chunks before runtime. You could as well use these chunks to store smaller data, but then you'd waste some of the memory.

Alternatively to having fixed-size memory chunks, you could even implement a Memory Pool that allows retrieving variable-size memory chunks. With that alternative solution while you'd have performance gains, you'd still end up with the same fragmentation problem that you have with the heap memory.

Consequences

You tackled fragmentation. With the pool of fixed size memory chunks, you can be sure that at least as soon as you release one chunk, another one will be available. However, you have to know which kinds of elements to store in the pool and which size they have beforehand. If you decide to also store smaller elements in it, you waste memory.

When using a pool of variable size, you don't waste memory for smaller elements, but your memory in the pool gets fragmented. This fragmentation situation is still a bit better compared to directly using the heap, because you are the only user of that memory (other processes don't use the same memory). Also, you don't fragment the memory used by other processes, but the fragmentation problem is still there.

No matter whether you use variable sized or fixed sized chunks in your pool, you have performance benefits. Getting memory from the pool is faster compared to allocating it from the heap, because no mutual exclusion from other processes trying to get memory is required. Also, accessing the memory from the pool might be a bit faster, because all the memory in the pool that your program uses lies closely next to each other and that minimizes time overhead due to paging mechanisms from the operating system. However, initially creating the pool takes some time and that will increase the startup time for your program.

Within your pool, you release the memory in order to reuse it somewhere else in your program. However, your program all the time holds the total pool memory and that memory will not be available to others. If you don't need all of that memory, you waste it from an overall system perspective.

If the pool is of initially fixed size, then you might at runtime have no more pool memory chunks available, even if there would be enough memory available in the heap. If the pool can at runtime increase it's size, then you have the drawback that the time for retrieving memory from the pool can be unpredictably increased if for retrieving a memory chunk the pool size has to be increased.

Beware of Memory Pools in security- or safety-critical domains. The pool makes your code more difficult to test and it makes it more difficult for code analysis tools to find bugs related to accessing that memory. For example, it is difficult for tools to detect if you by mistake access memory outside the boundaries of an acquired memory chunk of that pool. Your process also owns the other memory chunks of the pool that are located directly before and after that chunk you intend to access and that makes it hard for code analysis tools to realize that accessing data across the boundary of a Memory Pool chunk is something not intended. Actually, the OpenSSL Heartbleed bug could have been prevented by code analysis if the affected code was not using a Memory Pool (*https://dwheeler.com/essays/heartbleed.html*).

Known Uses

- UNIX systems use a pool of fixed size for their process objects.
- The book *C Interfaces and Implementations* by David R. Hanson (Addison-Wesley, 1996) shows an example for a memory pool implementation.
- The Memory Pool pattern is also described in the book *Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems* by Bruce P. Douglass (Addison-Wesley, 2002) and in the book *Small Memory Software: Patterns for Systems With Limited Memory* by James Noble and Charles Weir (Addison-Wesley, 2000).
- The ION memory manager of Andriod implements memory pools in its file *ion_system_heap.c*. On release of memory parts, the caller has the option to actually free that part of the memory if it is security-critical.

Applied to Running Example

To keep things easy, you decide to implement a Memory Pool with fixed maximum memory chunk size. You do not have to cope with multi-

threading and simultaneous access to that pool from multiple threads, so you can simply use the exact implementation from the Memory Pool pattern.

You end up with the following final code for your Caesar encryption:

```
#define ELEMENT_SIZE 255
#define MAX_ELEMENTS 10

typedef struct
{
  bool occupied;
  char memory[ELEMENT_SIZE];
}PoolElement;

static PoolElement memory_pool[MAX_ELEMENTS];

void* poolTake( size)
{
  if(size <= ELEMENT_SIZE)
  {
    for(int i=0; i<MAX_ELEMENTS; i++)
    {
      if(memory_pool[i].occupied == false)
      {
        memory_pool[i].occupied = true;
        return &(memory_pool[i].memory);
      }
    }
  }
  return NULL;
}

void poolRelease(void* pointer)
{
  for(int i=0; i<MAX_ELEMENTS; i++)
  {
    if(&(memory_pool[i].memory) == pointer)
    {
      memory_pool[i].occupied = false;
      return;
    }
  }
}

#define MAX_FILENAME_SIZE ELEMENT_SIZE
```

```
/* Prints the Caesar-encrypted 'filename'.This function is
responsible for allocating
   and deallocating the required buffers for storing the file
content.
   Notes: The file name must be all capital letters and we accept
that the '.' of
   the file name will also be shifted by the Caesar encryption.
*/
void encryptCaesarFilename(char* filename)
{
  char* buffer = poolTake(MAX_FILENAME_SIZE);
  strlcpy(buffer, filename, MAX_FILENAME_SIZE);
  caesar(buffer, strnlen(buffer, MAX_FILENAME_SIZE));
  printf("\nEncrypted file name: %s ", buffer);
  poolRelease(buffer);
}

/* For all files in the current directory, this function reads
text from the file and
   prints the Caesar-encrypted text. */
void encryptDirectoryContent()
{
  struct dirent *directory_entry;
  DIR *directory = opendir(".");
  while ((directory_entry = readdir(directory)) != NULL)
  {
    encryptCaesarFilename(directory_entry->d_name);
    encryptCaesarFile(directory_entry->d_name);
  }
  closedir(directory);
}
```

With this final version of your code, you can now perform your Caesar encryption without stumbling across the common pitfalls of dynamic memory handling in C. You make sure that the memory pointers you use are valid, you assert if no memory is available, and you even avoid fragmentation outside of your predefined memory area.

Looking at the code, you realize that it became very complicated. You simply want to work with some dynamic memory and you had to implement dozens of lines of code for that. Keep in mind that most of that code can be reused for any other allocation in your code base. Still, applying one pattern after the other did not come for free. With each pattern

you added some additional complexity. Keep in mind that it is not the aim to apply as many patterns as possible. It is the aim to apply those patterns that solve your problems. If, for example, fragmentation is not a big issue for you, then please don't use a custom Memory Pool. If you can keep things simpler, then do that and, for example, directly allocate and deallocate the memory using `malloc` or `free`. Or even better, if you have the option, then don't use dynamic memory at all.

Summary

This chapter presented patterns on handling memory in C programs. The Stack First pattern tells to put variables on the stack if possible. Eternal Memory is about using memory with lifetime as long as your program in order for not having to cope with complicated dynamic allocation and freeing. Screw Freeing also makes freeing the memory easier to the programmer by suggesting to simply not cope with it. Dedicated Ownership on the other hand tells to define where memory is freed by whom. The Allocation Wrapper provides a central point for handling allocation errors and provides a central point for invalidating pointers and that makes it possible to implement a Pointer Check when dereferencing variables. If fragmentation or long allocation times become an issue, a Memory Pool helps out.

With these patterns, the burden of making a lot of detailed design decisions on which memory to use and when to clean it up, is taken from the programmer. Instead, the programmer can simply rely on the guidance from these patterns and can easily tackle memory management in C programs.

Further Reading

Compared to other advanced C programming topics, there is quite a lot of literature out there on the topic of memory management. Most of that literature focuses on the basis of the syntax for allocating and freeing memory, but the following books also provide some advanced guidance:

- The book *Small Memory Software: Patterns for Systems With Limited Memory* by James Noble and Charles Weir (Addison-Wesley, 2000) contains a lot of well-elaborated patterns on memory management. For example, the patterns describe the different strategies for allocating memory (at start-up or during runtime) and also describe strategies such as memory pools or garbage collectors. All patterns also provide code examples for multiple programming languages.
- The book *Hands-on Design Patterns with C++* by Fedor G. Pikus (Packt Publishing Limited, 2019) is as it's name says not tailored for C, but the memory management concepts between C and C++ are the same, so there is also relevant guidance on how to manage memory in C in this book. It contains a chapter that focuses on memory ownership and tells how to use C++ mechanisms (like smart pointers) to make very clear who owns which memory.
- The book *Extreme C* by Kamran Amini (Packt Publishing Limited, 2019) covers many C programming topics, like the compilation process, toolchains, unit-testing, concurrency, intra-process communication, and also the basic C syntax. There is also a chapter on heap and stack memory in that book and it describes platform-specific details on how these memories are represented in the code-, data-, stack-, or heap-segment.
- The book *Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems* by Bruce P. Douglass (Addison-Wesley, 2002) contains patterns for real-time systems. Some of the patterns address allocation and cleanup of memory.

Outlook

The next chapter gives guidance on how to transport information in general across interface boundaries. The chapter presents patterns that elaborte on the kinds of mechanisms that C provides for transporting information

between functions and the chapter elaboarates on which of these mechnisms should be used.

Chapter 4. Returning Data from C Functions

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 4th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

Returning data from a function call is a task you are faced with when writing any kind of code that is longer than 10 lines and that you intend to be maintainable. Returning data is a simple task - you simply have to pass the data you want to share between two functions - in C you only have the possibility to directly return some value or to return data via emulated "by-reference" parameters. There are not many choices and there is not much guidance to give - right? Wrong! Even the simple task of returning data from C functions is already tricky and there are many ways you can take of how to structure your program and of how to structure your function parameters.

Especially in C, where you have to manage the memory allocation and deallocation on your own, passing complex data between functions becomes tricky, because there is no destructor or garbage collector, which helps you clean up the data. You have to ask yourself: shall the data be put

on the stack, or shall it be allocated? Who should allocate - the caller or the callee?

This chapter provides best practices on how to share data between functions. These patterns help C programming beginners to understand techniques for returning data in C and they help advanced C programmers to better understand why these different techniques are applied.

Figure 4-1 shows an overview of the patterns presented in this chapter and their relationships and Table 4-1 provides a summary of the patterns.

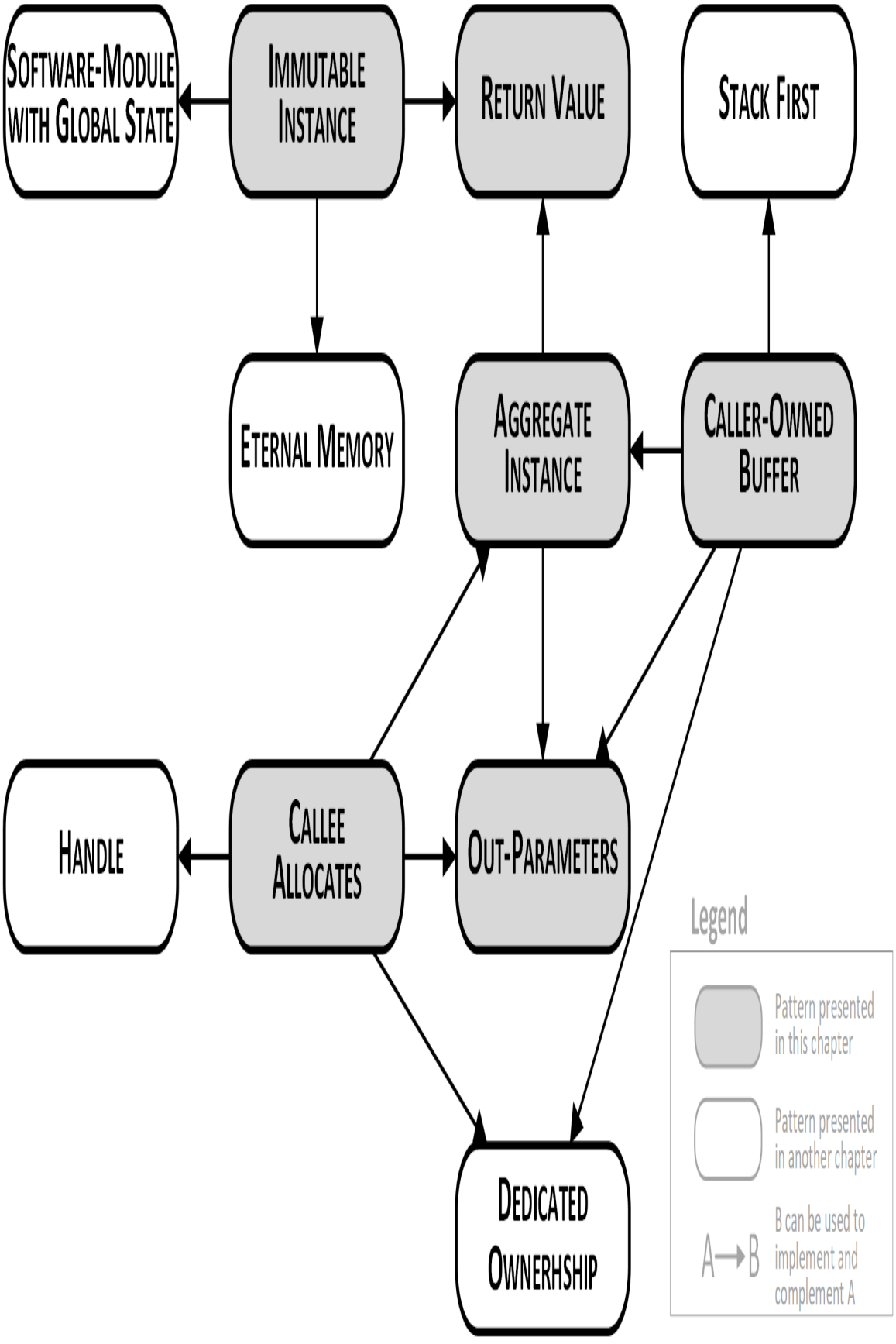

Software-Module with Global State
Immutable Instance
Return Value
Stack First
Eternal Memory
Aggregate Instance
Caller-Owned Buffer
Handle
Callee Allocates
Out-Parameters
Dedicated Ownerhship
Legend
Pattern presented in this chapter
Pattern presented in another chapter
A → B
B can be used to implement and complement A

Figure 4-1. Overview of patterns on returning information

Table 4-1. Patterns on return information form

a
t
i
o
n

	Pattern Name	Summary
	Return Value	The function parts you want to split are not independent from one another. As usual in procedural programming, some part delivers a result that is then needed by some other part. The function parts that you want to split need to share some data. Therefore, simply use the one C mechanism intended to retrieve information about the result of a function call: the Return Value. The mechanism to return data in C copies the function result and provides the caller access to this copy.
	Out-Parameters	C only supports returning a single type from a function call and that makes it complicated to return multiple pieces of information. Therefore, return all the data with one single function call by emulating by-reference arguments with pointers.
	Aggregate Instance	C only supports returning a single type from a function call and that makes it complicated to return multiple pieces of information. Therefore, put all data that is related together into a newly defined type. Define this Aggregate Instance to contain all the related data that you want to share. Define it in the interface of your component to let the caller directly access all the data stored in the instance.
	Immutable Instance	You want to provide information held in large pieces of immutable data from your component to a caller. Therefore, have an instance (for example, a `struct`) containing the data to share in static memory. Provide this data to users who want to access it and make sure that they cannot modify it.
	Caller-Owned Buffer	You want to provide complex or large data of known size to the caller and that data is not immutable - it changes at runtime. Therefore, require the caller to

	provide a buffer and its size to the function that returns the complex, large data. In the function implementation, copy the required data into the buffer if the buffer size is large enough.
Callee Allocates	You want to provide complex or large data of unknown size to the caller, and that data is not immutable (it changes at runtime). Therefore, allocate a buffer with the required size inside the function that provides the complex, large data. Copy the required data into the buffer and return a pointer to that buffer.

Running Example

You want to implement the functionality to display diagnostic information of an Ethernet driver to the user. First, you simply add this functionality directly into the file with the Ethernet driver implementation and you directly access the variables that contain the required information:

```
void ethShow()
{
  printf("%i packets received\n", driver.internal_data.rec);
  printf("%i packets sent\n", driver.internal_data.snd);
}
```

Later on, you realize that the functionality to display diagnostic information for your Ethernet driver will quite likely grow, so you decide to put it into a separate implementation file in order to keep your code clean. Now you need some simple way to transport the information from your Ethernet driver component to your diagnostics component.

One solution would be to use global variables to transport this information, but if you use global variables, then the effort to split the implementation file was useless. You split the files, because you want to show that these code parts are not tightly coupled - with global variables you would bring that tight coupling back in.

A much better and very simple solution is the following: let your Ethernet component have getter-functions that provide the desired information as Return Value.

Return Value

Context

You want to split your code into separate functions, as having everything in one function and in one implementation file is bad practice, because it gets difficult to read and to debug the code.

Problem

The function parts you want to split are not independent from one another. As usual in procedural programming, some part delivers a result that is then needed by some other part. The function parts that you want to split need to share some data.

You want to have a mechanism for sharing data that makes your code easy to understand. You want to make it explicit in your code that data is shared between functions and you want to make sure that functions don't communicate over side-channels not clearly visible in the code. Thus using global variables to return information to a caller is not a good solution for you, because global variables can be accessed and modified from any other part of the code and because it is not clear from the function signature, which exact global variable is used for returning data.

Also, global variables have the drawback that they could be used to store state information, which could lead to different results for identical function calls. Aside from that, code using global variables for returning information would not be reentrant and it would not be safe to use in a multi-threaded environment.

Solution

Simply use the one C mechanism intended to retrieve information about the result of a function call: the Return Value. The mechanism to return data in C copies the function result and provides the caller access to this copy.

Figure 4-2 and the following code show how to implement the Return Value.

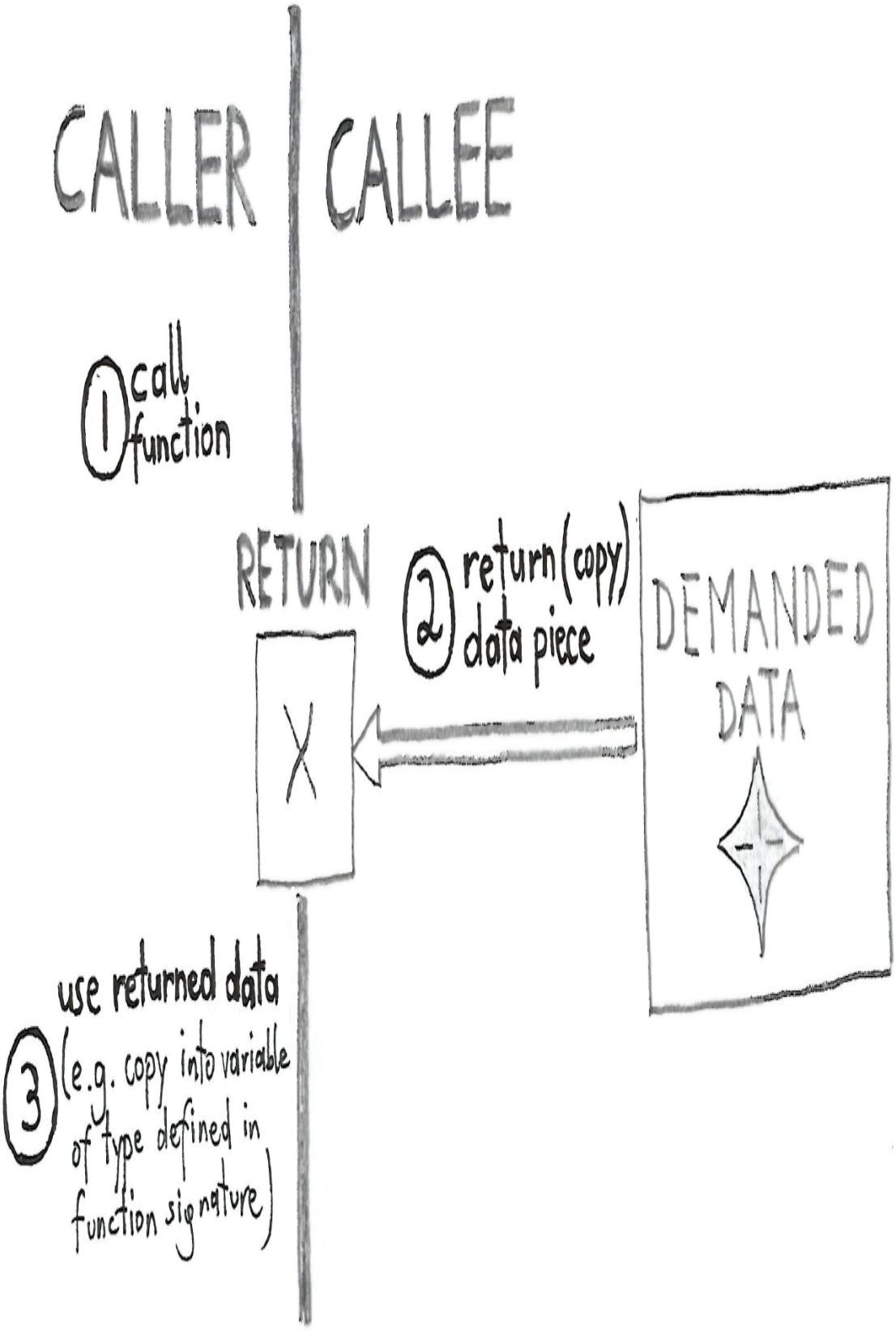

CALLER
CALLEE
1 call function
RETURN
2 return (copy) data piece
DEMANDED DATA
X
use returned data
3 (e.g. copy into variable of type defined in function signature)

Figure 4-2. Return value

Caller's code

```
int my_data = getData();
/* use my_data */
```

Callee's code

```
int getData()
{
  int requested_data;
  /* .... */
  return requested_data;
}
```

Consequences

A Return Value allows the caller to retrieve a copy of the function result. No other code apart from the function implementation can modify this value and, as it is a copy, this value is solely used by the calling function. So compared to using global variables, it is more clearly defined which code influences the data retrieved from the function call.

Also, by not using global variables and using the copy of the function result instead, the function can be reentrant and it can safely be used in a multi-threaded environment.

However, for built-in C types, a function can return only a single object of the type specified in the function signature. It is not possible to define a function with multiple return types. You cannot, for example, have a function that returns three different `int` objects. If you want to return more information than contained in just one simple, scalar C type, then you have to use an Aggregate Instance or Out-Parameters.

Also, if you want to return data from an array, then the Return Value is not what you want, because it does not copy the content of the array, but only the pointer to the array and then the caller might end up with a pointer to

data that ran out of scope. For returning arrays, you have to use other mechanisms like a Caller-Owned Buffer or like when the Callee Allocates.

Remember that whenever the simple Return Value mechanism is sufficient, then you should always take this most simple option to return data and you should not go for more powerful, but also more complex ways like Out-Parameters, Aggregate Instance, Caller-Owned Buffer or Callee Allocates.

Known Uses

- Every C program uses this pattern. For example, every C program has a `main` function that already provides a return value to its caller (such as the operating system).

Applied to Running Example

Well… applying Return Value was quite obvious and simple. Now you have a new diagnostic component in an implementation file separate from the Ethernet driver and this component obtains the diagnostic information from the Ethernet driver as shown in the following code:

Ethernet driver API

```
/* Returns the number of total received packets*/
int ethernetDriverGetTotalReceivedPackets();

/* Returns the number of total sent packets*/
int ethernetDriverGetTotalSentPackets();
```

Caller's code

```
void ethShow()
{
  int received_packets = ethernetDriverGetTotalReceivedPackets();
  int sent_packets = ethernetDriverGetTotalSentPackets();
  printf("%i packets received\n", received_packets);
  printf("%i packets sent\n", sent_packets);
}
```

This code is very easy to read and if you want to add additional information, you can simply add additional functions to obtain this information. Now that is exactly what you want to do next. You want to show more information about the sent packets. You want to show the user how many packets were successfully sent and how many failed. Your first attempt is to write the following code:

```
void ethShow()
{
  int received_packets = ethernetDriverGetTotalReceivedPackets();
  int total_sent_packets = ethernetDriverGetTotalSentPackets();
  int successfully_sent_packets =
ethernetDriverGetSuccesscullySentPackets();
  int failed_sent_packets = ethernetDriverGetFailedPackets();
  printf("%i packets received\n", received_packets);
  printf("%i packets sent\n", total_sent_packets);
  printf("%i packets successfully sent\n",
successfully_sent_packets);
  printf("%i packets failed to send\n", failed_sent_packets);
}
```

With this code, after some time, you realize that sometimes, different from what you expected, `successfully_sent_packets` plus `failed_sent_packets` results in a number higher than `total_sent_packets`. That is, because your Ethernet driver runs in a separate thread and between your function calls to obtain the information, the Ethernet driver continues working and updates its packet information. So, if for example the Ethernet driver successfully sends a packet between your `ethernetDriverGetTotalSentPackets` call and `ethernetDriverGetSuccesscullySentPackets`, then the information that you show to the user is not consistent.

A possible solution would be to make sure that the Ethernet driver is not working while you call the functions to obtain the packet information. You could, for example, use a Mutex or a Semaphore to make sure of that, but for such as simple task like obtaining packet statistics, you'd expect that you are not the one who has to cope with that.

As a much easier alternative you can return multiple pieces of information from one function call by using Out-Parameters.

Out-Parameters

Context

You want to provide data that represents related pieces of information from your component to a caller and these pieces of information change during runtime.

Problem

C only supports returning a single type from a function call and that makes it complicated to return multiple pieces of information.

Using global variables to transport the data representing your pieces of information is not a good solution, because code using global variables for returning information would not be reentrant and it would not be safe to use in a multi-threaded environment. Aside from that, global variables can be accessed and modified from any other part of the code and when using global variables, it is not clear from the function signature, which exact global variables are used for returning the data. Thus global variables would make your code hard to understand and maintain. Also using the Return Values of multiple functions is not a good option, because the data you want to return is related, so splitting it across multiple function calls makes the code less readable.

As the pieces of data are related, the caller wants to retrieve a consistent snapshot of all this data. That becomes an issue when using multiple Return Values as soon as in a multithreaded environment the data can change at runtime. In that case, you would have to make sure that between the caller's multiple function calls, the data does not change. But you cannot know whether the caller already finished reading all the data or whether there will be another piece of information that the caller wants to retrieve with another

function call. Because of that, you cannot make sure that the data is not modified between the caller's function calls. When using multiple functions to provide related, consistent information, then you don't know the timespan during which the data must not change. Thus, with this approach, you cannot guarantee the caller to retrieve a consistent snapshot of the information.

Having multiple functions with Return Values also might not be a good solution, if a lot of preparation work is required for calculating the related pieces of data. If, for example you want to return the home- and the mobile-telephone-number for a specified person from a phone book and you'd have separate functions to retrieve the numbers, you'd have to separately search through the phone book for the entry of this person for each of the function calls. That requires unnecessary computation time and resources.

Solution

Return all the data with one single function call by emulating by-reference arguments with pointers.

C does not support returning multiple types using the Return Value and C does not natively support by-reference arguments, but by-reference arguments can be emulated as shown in Figure 4-3 and the following code.

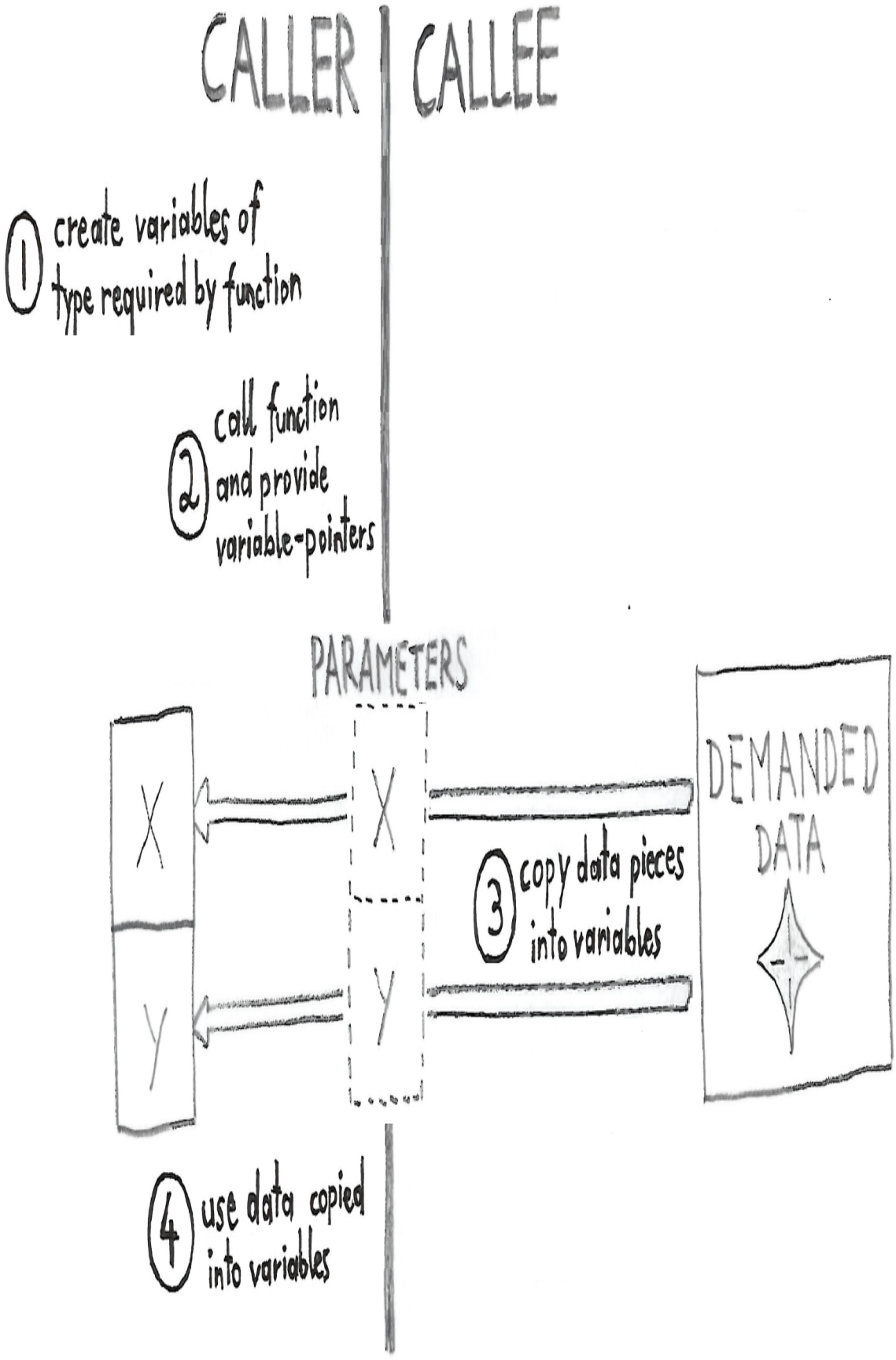

CALLER
CALLEE
1 create variables of type required by function
2 call function and provide variable-pointers
PARAMETERS
X
Y
X
Y
3 copy data pieces into variables
DEMANDED DATA
4 use data copied into variables

Figure 4-3. Out-Parameters

Caller's code

```
int x,y;
getData(&x,&y);
/* use x,y */
```

Callee's code

```
void getData(int* x, int* y)
{
  *x = 42;
  *y = 78;
}
```

Have one single function with many pointer arguments. In the function implementation, de-reference the pointers and copy the data you want to return to the caller into the instance pointed to. Make sure in the function implementation, that the data does not change while copying. That can be achieved by mutual exclusion.

MULTI-THREADED ENVIRONMENTS

In modern systems it is very common that you work in a multi-threaded environment. That makes things a lot more difficult, because you have to implement your functions in a way that they can safely be called by multiple threads in arbitrary order or even at the same time.

That requires your functions to be reentrant, which means that it still properly works if the function is interrupted at any time and continued later on. When working on shared resources such as global variables, you have to make sure to protect accessing these resources from simultaneous access by other threads. That can be done with synchronization primitives such as Mutex or Semaphores.

This book does not focus on such synchronization primitives or on approaches how to use them, but the book *Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems* by Bruce P. Douglass (Addison-Wesley, 2002) does and provides C patterns on concurrency and resource management.

Consequences

Now all data that represents related pieces of information are returned in one single function call and can be kept consistent (for example, by copying data protected by Mutex or Semaphores). The function is reentrant and can safely be used in a multi-threaded environment.

For each additionally required data item, an additional pointer is passed to the function. That has the drawback, that if you want to return a lot of data, the function's parameter list becomes longer and longer. Having many parameters for one function is a bad code smell as it makes the code unreadable. That is why only rarely multiple Out-Parameters are used for a function and instead, to clean up the code, related pieces of information are returned with an Aggregate Instance.

Also, for each piece of data, the caller has to pass a pointer to the function. That means that for each piece of data, an additional pointer has to be put onto the stack. If the caller's stack memory is very limited, that might become an issue.

Out-Parameters have the disadvantage, that when only looking at the function signature, they cannot clearly be identified as Out-Parameters. From the function signature, callers can only guess whenever they see a pointer, that that might be an Out-Parameter. But such a pointer parameter could also be an input for the function. Thus, it has to be clearly described in the API documentation which parameters are for input and which are for output.

For simple, scalar C types the caller can simply pass the pointer to a variable as a function argument. For the function implementation all the information to interpret the pointer is specified, because of the specified pointer type. To return data with complex types, like arrays, either a Caller-Owned Buffer has to be provided, or the Callee Allocates and additional information about that data, like its size, has to be communicated.

Known Uses

- The Windows `RegQueryInfoKey` function returns information about a registry key via the function's Out-Parameters. The caller provides `unsigned long` pointers and the the functions writes, amongst other pieces of information, the number of subkeys and the size of the key's value into the `unsigned long` variables being pointed to.
- Apple's Cocoa API for C programs uses an additional `NSError` parameter, to store errors occurring during the function calls.
- The function `userAuthenticate` of the real-time operating system VxWorks uses Return Values to return information, whether a provided password is correct for a provided login name. Additionally the function takes an Out-Parameter to return the user ID associated with the provided login name.

Applied to Running Example

By applying Out-Parameters you'll get the following code:

Ethernet driver API

```
/* Returns driver status information via out-parameters.
   total_sent_packets   --> number of packets tried to send
(successful and failed)
   success_sent_packets --> number of packets successfully sent
   failed_sent_packets  --> number of packets failed to send */
void ethernetDriverGetStatistics(int* total_sent_packets, int*
success_sent_packets,
                                 int* failed_sent_packets); ❶
```

❶ To retrieve information about sent packets, you only have one single function call to the Ethernet driver and the Ethernet driver can make sure that the data delivered within this call is consistent.

Caller's code

```
void ethShow()
{
  int total_sent_packets, success_sent_packets,
failed_sent_packets;
  ethernetDriverGetStatistics(&total_sent_packets,
&success_sent_packets,
                              &failed_sent_packets);
  printf("%i packets sent\n", total_sent_packets);
  printf("%i packets successfully sent\n", success_sent_packets);
  printf("%i packets failed to send\n", failed_sent_packets);


  int received_packets = ethernetDriverGetTotalReceivedPackets();
  printf("%i packets received\n", received_packets);
}
```

You consider also retrieving the `received_packets` in one and the same function call with the sent packets, but you realize that the one function call becomes more and more complicated. Having the one function call with three Out-Parameters is already complicated to write and to read. When calling the functions, the parameters could easily be mixed up. Adding a fourth parameter wouldn't make the code better.

To make the code more readable, an Aggregate Instance can be used.

Aggregate Instance

Context

You want to provide data that represents related pieces of information from your component to a caller and these pieces of information change during runtime.

Problem

C only supports returning a single type from a function call and that makes it complicated to return multiple pieces of information.

Using global variables to transport the data representing your pieces of information is not a good solution, because code using global variables for returning information would not be reentrant and it would not be safe to use in a multi-threaded environment. Aside from that, global variables can be accessed and modified from any other part of the code and when using global variables, it is not clear from the function signature, which exact global variables are used for returning the data. Thus global variables would make your code hard to understand and maintain. Also using the Return Values of multiple functions is not a good option, because the data you want to return is related, so splitting it across multiple function calls makes the code less readable.

Having one single function with many Out-Parameters also is not a good idea, because if you have many such Out-Parameters, it gets easy to mix them up and your code becomes unreadable. Also, you want to show that the parameters are closely related and you might even need the same set of parameters be provided to or returned by other functions. When explicitly doing that with function parameters, you'd have to modify each such function in case additional parameters are added later on.

As the pieces of data are related, the caller wants to retrieve a consistent snapshot of all this data. That becomes an issue when using multiple Return Values as soon as in a multithreaded environment the data can change at runtime. In that case, you would have to make sure that between the caller's multiple function calls, the data does not change, but you cannot know whether the caller already finished reading all the data or whether there will be another piece of information that the caller wants to retrieve with another function call. Because of that, you cannot make sure that the data is not modified between the caller's function calls. When using multiple functions to provide related information, then you don't know the timespan during which the data must not change. Thus, with this approach, you cannot guarantee the caller to retrieve a consistent snapshot of the information.

Having multiple functions with Return Values also might not be a good solution, if a lot of preparation work is required for calculating the related pieces of data. If, for example you want to return the home- and the mobile-

telephone-number for a specified person from a phone book and you'd have separate functions to retrieve the numbers, you'd have to separately search through the phone book for the entry of this person for each of the function calls. That requires unnecessary computation time and resources.

Solution

Put all data that is related together into a newly defined type. Define this Aggregate Instance to contain all the related data that you want to share. Define it in the interface of your component to let the caller directly access all the data stored in the instance.

To implement that, define a `struct` in your header file and define all types to be returned from the called function as members of this `struct`. In the function implementation, copy the data to be returned into the `struct` members as shown in Figure 4-4. Make sure in the function implementation, that the data does not change while copying. That can be achieved by mutual exclusion via Mutex or Semaphores.

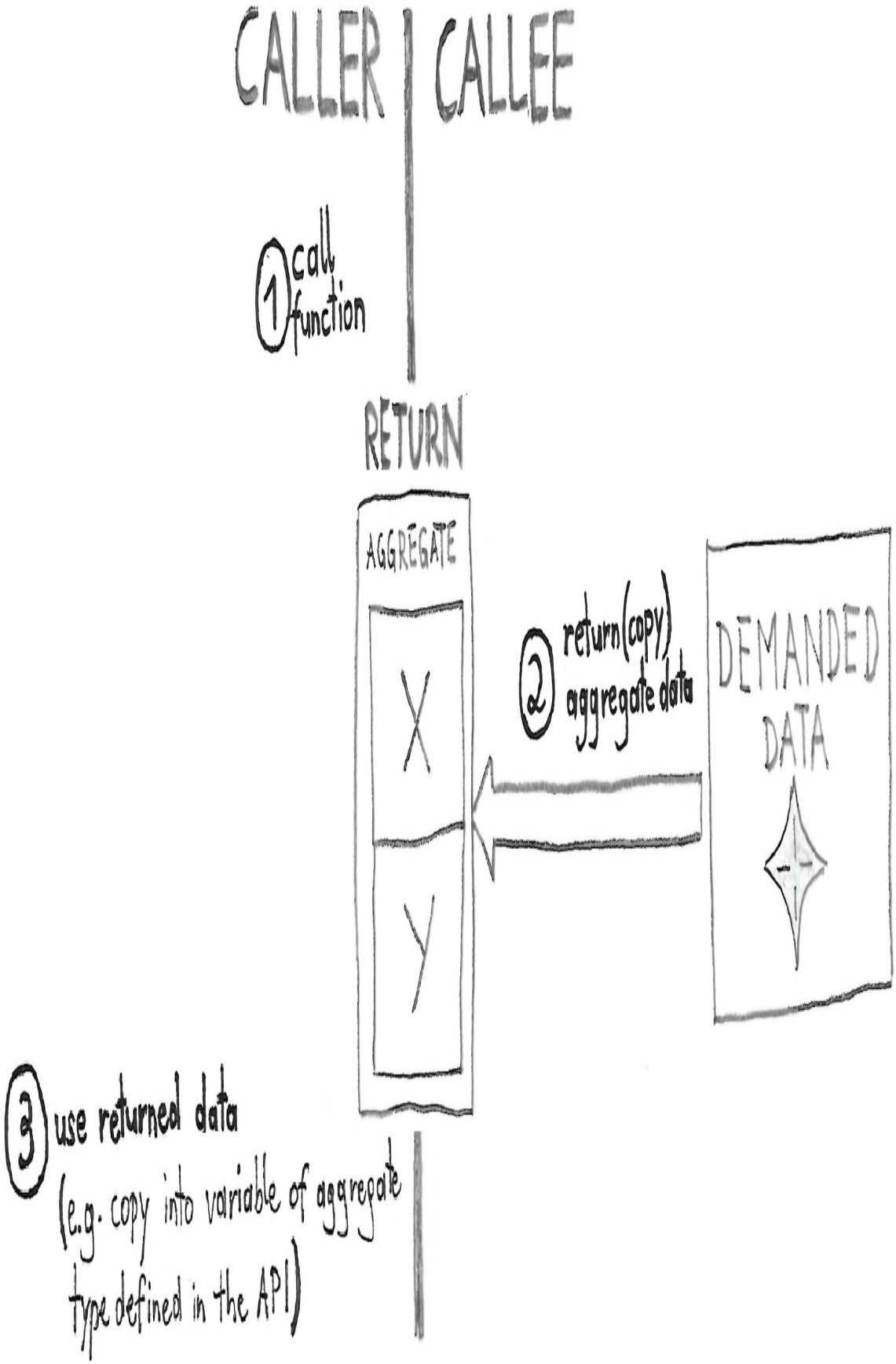

CALLER
CALLEE
1 call function
RETURN
AGGREGATE
X
Y
2 return(copy) aggregate data
DEMANDED DATA
3 use returned data
(e.g. copy into variable of aggregate type defined in the API)

Figure 4-4. Aggregate Instance

To actually return the `struct` to the caller, there are two main options:

- Pass the whole `struct` as Return Value. C allows not only built-in types to be passed as Return Value of functions, but also user-defined types such as a `struct` can be passed.
- Pass a pointer to the `struct` using an Out-Parameter. However, when only passing pointers, the issue of who provides and owns the memory being pointed to comes up. That issue is addressed in Caller-Owned Buffer and Callee Allocates. Alternatively to passing a pointer and letting the caller directly access the Aggregate Instance, you could consider hiding the `struct` from the caller by using a Handle.

The following code shows the variant with passing the whole `struct`:

Caller's code

```
struct AggregateInstance my_instance;
my_instance = getData();
/* use my_instance.x
   use my_instance.y, ... */
```

Callee's code

```
struct AggregateInstance
{
  int x;
  int y;
};

struct AggregateInstance getData()
{
  struct AggregateInstance inst;
  /* fill inst.x and inst.y */
  return inst; ❶
}
```

❶

When returning, the content of `inst` is copied (even though it is a `struct`), and the caller can access the copied content even after `inst` runs out of scope.

Consequences

Now the caller can retrieve multiple data that represents related pieces of information via the Aggregate Instance with one single function call. The function is reentrant and can safely be used in a multi-threaded environment.

That provides the caller with a consistent snapshot of the related pieces of information and it makes the caller's code clean, because the caller does not have to call multiple functions or one function with many Out-Parameters.

When passing data between functions without pointers by using Return Values, all this data is put on the stack. When passing one `struct` to 10 nested functions, then this `struct` is on the stack 10 times. In some cases that is no problem, but in some other cases it is - especially if the `struct` is too large and you don't want to waste stack memory by copying the whole `struct` onto the stack every time. Because of that, quite often instead of directly passing or returning a `struct`, a pointer to that `struct` is passed or returned.

When passing pointers to the `struct`, or if the `struct` contains pointers, you have to keep in mind that C does not perform the work of doing a deep copy for you. C only copies the pointer values and does not copy the instances they point to. That might not be what you want, so you have to keep in mind that as soon as pointers come into play, you have to deal with providing and cleaning up the memory being pointed to. This issue is addressed in Caller-Owned Buffer and Callee Allocates.

Known Uses

- The article *Patterns of Argument Passing* by Uwe Zdun (dsg.tuwien.ac.at/Staff/zdun/publications/arguments.ps) describes this

pattern including C++ examples as Context Object and the book *Refactoring: Improving the Design of Existing Code* by Martin Fowler (Addison-Wesley, 1999) describes it as Parameter Object.

- The code of the game NetHack stores monster-attributes in Aggregate Instances and provides a function for retrieving this information.
- The implementation of the text editor `sam` copies `structs` when passing them to functions and when returning them from functions in order to keep the code simpler.

Applied to Running Example

With the Aggregate Instance, you'll get the following code:

Ethernet driver API

```
struct EthernetDriverStat{
  int received_packets;           /* Number of received packets
*/
  int total_sent_packets;         /* Number of sent packets
(successfuly and failed */
  int successfully_sent_packets;  /* Number of successfully sent
packets */
  int failed_sent_packets;        /* Number of packets failed to
send */
};

/* Returns statistics information of the Ethernet driver */
struct EthernetDriverStat ethernetDriverGetStatistics();
```

Caller's code

```
void ethShow()
{
  struct EthernetDriverStat eth_stat =
ethernetDriverGetStatistics();
  printf("%i packets received\n", eth_stat.received_packets);
  printf("%i packets sent\n", eth_stat.total_sent_packets);
  printf("%i packets successfully sent\n",
eth_stat.successfully_sent_packets);
```

```
  printf("%i packets failed to send\n",
eth_stat.failed_sent_packets);
}
```

Now you have one single call to the Ethernet driver and the Ethernet driver can make sure that the data delivered within this call is consistent. Also, your code looks cleaned up, because the data that belongs together now is collected in a single `struct`.

Next, you want to show some more information about the Ethernet driver to your user. You want to show the user to which Ethernet interface the packet statistics information belongs to and thus you want to show the driver name including a textual description of the driver. Both is contained in a string stored in the Ethernet driver component. The string is quite long and you don't exactly know how long the string is. Luckily the string does not change during runtime, so you can access an Immutable Instance.

Immutable Instance

Context

Your component contains a lot of data and another component wants to access this data.

Problem

You want to provide information held in large pieces of immutable data from your component to a caller.

Copying the data for each and every caller would be a waste of memory, so providing all the data by returning an Aggregate Instance or by copying all the data into Out-Parameters is not an option due to stack memory limitations.

Usually, simply returning a pointer to such data is tricky. You'd have the problem that with a pointer such data can be modified and as soon as

multiple callers read and write the same data, you have to come up with mechanisms to ensure that the data you want to access is consistent and up-to-date. Luckily you are in the situation, that the data you want to provide to the caller is fixed at compile time or at boot time and does not change at runtime.

Solution

Have an instance (for example, a `struct`) containing the data to share in static memory. Provide this data to users who want to access it and make sure that they cannot modify it.

Write the data to be contained in the instance at compile-time or at boot-time and do not change it at runtime anymore. You can either directly write the data hardcoded in your program or you can initialize it at program startup (see Software-Module with Global State for initialization variants and Eternal Memory for storage variants). As shown in Figure 4-5, even if multiple callers (and multiple threads) access the instance at the same time, they don't have to care about each other, because the instance does not change and is thus always in a consistent state and contains the required information.

Implement a function which returns a pointer to the data. Alternatively, you could even directly make the variable containing the data global and put it into your API, because the data does not change at runtime anyway. But still, the getter function is better, because compared to global variables, it makes writing unit-tests easier and in case of future behavior changes of your code (if your data is not immutable anymore), you'd not have to change your interface.

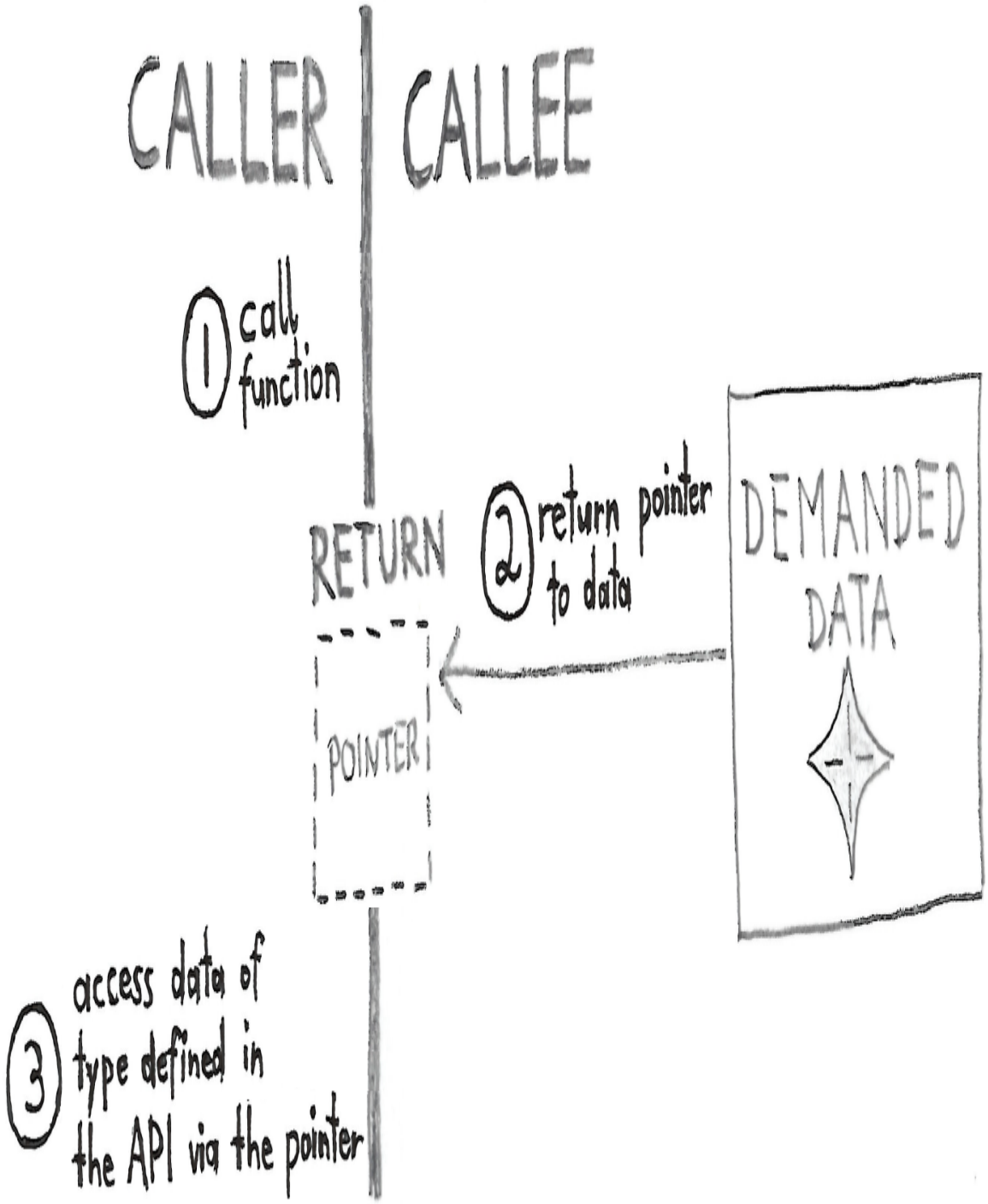


Figure 4-5. Immutable Instance

To make sure that the caller does not modify the data, when returning a pointer to the data, make the data being pointed to `const` as shown in the following code:

Caller's code

```
const struct ImmutableInstance* my_instance;
my_instance = getData(); ❶
/* use my_instance->x,
   use my_instance->y, ... */
```

The caller obtains a reference, but doesn't get ownership of the memory.

❶

Callee API

```
struct ImmutableInstance
{
  int x;
  int y;
};
```

Callee Implementation

```
static struct ImmutableInstance inst = {12, 42};
const struct ImmutableInstance* getData()
{
   return &inst;
}
```

Consequences

The caller can call one simple function to get access to even complex or large data and does not have to care about where this data is stored. The caller does not have to provide buffers where this data can be stored in, does not have to cleanup memory, and does not have to care about the lifetime of the data - it simply always exists.

The caller can read all data via the retrieved pointer. The simple function for retrieving the pointer is reentrant and can safely be used in multi-threaded environments. Also the data can safely be accessed in multi-threaded environments, because it does not change at runtime and multiple threads that only read the data are no problem.

However, the data cannot be changed at runtime without taking further measures. If it is required that the caller can change the data, then something like copy-on-write can be implemented. If the data in general can change at runtime, then an Immutable Instance isn't an option and instead, for sharing complex and large data, a Caller-Owned Buffer has to be used or the Callee Allocates.

Known Uses

- Kevlin Henney describes in his article *Patterns in Java: Patterns of Value* (*http://www.curbralan.com/*) the similar Immutable Object pattern in detail and provides C++ code examples.
- The code of the game NetHack stores immutable monster-attributes in an Immutable Instance and provides a function for retrieving this information.

Applied to Running Example

Usually, returning a pointer to access data stored within a component is tricky, because if multiple callers access (and maybe write) this data, then a plain pointer isn't the solution for you, because you never know whether the pointer you have is still valid and whether the data contained in this pointer is consistent. But in this case we are lucky, because we have an Immutable Instance. The driver name and description are both information that is determined at compile-time and that does not change afterwards. Thus, we can simply retrieve a constant pointer to this data:

Ethernet driver API

```
struct EthernetDriverInfo{
  char name[64];
  char description[1024];
};

/* Returns the driver name and description */
const struct EthernetDriverInfo* ethernetDriverGetInfo();
```

Caller's code

```
void ethShow()
{
  struct EthernetDriverStat eth_stat =
ethernetDriverGetStatistics();
  printf("%i packets received\n", eth_stat.received_packets);
  printf("%i packets sent\n", eth_stat.total_sent_packets);
  printf("%i packets successfully sent\n",
eth_stat.successfully_sent_packets);
  printf("%i packets failed to send\n",
eth_stat.failed_sent_packets);

  const struct EthernetDriverInfo* eth_info =
ethernetDriverGetInfo();
  printf("Driver name: %s\n", eth_info->name);
  printf("Driver description: %s\n", eth_info->description);
}
```

As a next step, additionally to the name and description of the Ethernet interface, you also want to show the user the currently configured IP address and subnet mask. The addresses are stored as a string in the Ethernet driver. Both addresses are information that might change during runtime, so you cannot simply return a pointer to an Immutable Instance.

While it would be possible to have the Ethernet driver pack these strings into an Aggregate Instance and simply return this instance (arrays in a `struct` are copied when returning the `struct`), such a solution is rather uncommon for large amounts of data, because it consumes a lot of stack memory. Usually, instead, pointers are used.

Using pointers is the exact solution you are looking for - use a Caller-Owned Buffer.

Caller-Owned Buffer

Context

You have large data that you want to share between different components.

Problem

You want to provide complex or large data of known size to the caller and that data is not immutable - it changes at runtime.

As the data changes at runtime (maybe because you provide the callers with functions to write the data), you cannot simply provide the caller with a pointer to static data (as it is the case with an Immutable Instance). You cannot do that, because if you simply provide the callers with such a pointer, you'd run into the problem, that the data one caller reads might be inconsistent (partially overwritten), because, in a multi-threaded environment, another caller might simultaneously write that data.

Simply copying all the data into an Aggregate Instance and passing it via the Return Value to the caller is not an option, because, as the data is large, it cannot be passed via the stack, which only has very limited memory.

When instead only returning a pointer to the Aggregate Instance, there would be no problem with stack memory limitations anymore, but you have to keep in mind that C does not do the work of performing a deep copy for you. C only returns the pointer. You have to make sure that the data (stored in an Aggregate Instance or in an array) being pointed to is still valid after the function call. For example, you cannot store the data in auto-variables within your function and provide a pointer to these variables, because after the function call, the variables run out of scope.

Now the question where the data should be stored arises. It has to be clarified whether the caller or the callee should provide the required memory and it has to be clarified whether the caller or the callee is then responsible for managing and cleaning up the memory.

Solution

Require the caller to provide a buffer and its size to the function that returns the complex, large data. In the function implementation, copy

the required data into the buffer if the buffer size is large enough.

Make sure that the data does not change while copying. That can be achieved by mutual exclusion via Mutex or Semaphores. The caller then has a snapshot of the data in the buffer, is the sole owner of this snapshot, and thus can consistently access this snapshot even if the original data changes in the meantime.

The caller can provide the buffer and the size each as a separate function parameter, or the caller can pack the buffer and the size into an Aggregate Instance and pass a pointer to the Aggregate Instance to the function.

As the caller has to provide the buffer and the size to the function, the caller has to know the size beforehand. To let the caller know of which size the buffer has to be, the information of the required size has to be present in the API. That can be implemented by defining the size as a macro or by defining a `struct` containing a buffer of the required size in the API.

Figure 4-6 and the following code show the concept of a Caller-Owned Buffer.

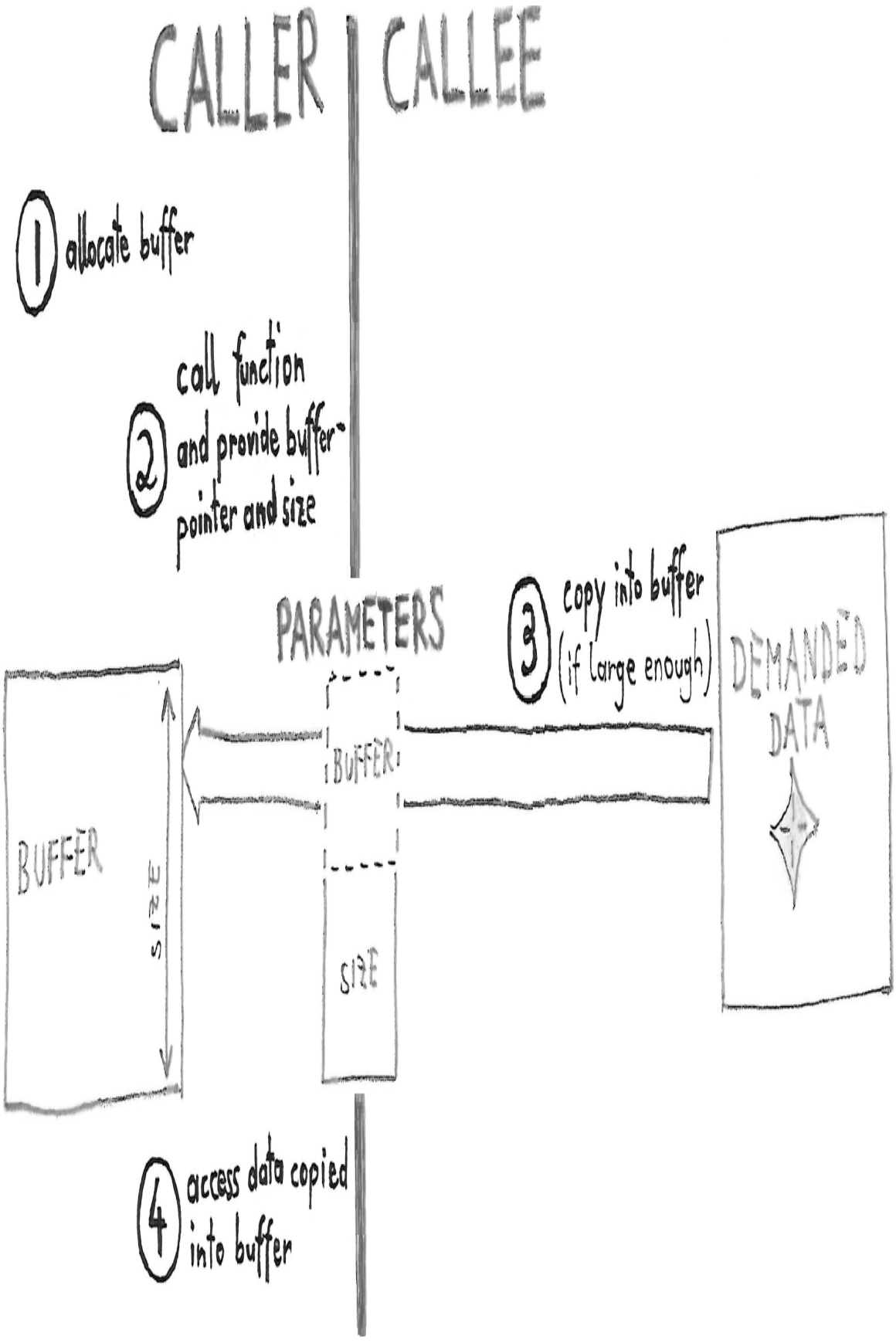

CALLER
CALLEE
1 allocate buffer
call function
2 and provide buffer-
pointer and size
PARAMETERS
3 copy into buffer
(if large enough)
DEMANDED
DATA
BUFFER
BUFFER
SIZE
SIZE
4 access data copied
into buffer

Figure 4-6. Caller-Owned Buffer

Caller's code

```
struct Buffer buffer;

getData(&buffer);
/* use buffer.data */
```

Callee's API

```
#define BUFFER_SIZE 256
struct Buffer
{
  char data[BUFFER_SIZE];
};

void getData(struct Buffer* buffer);
```

Callee's implementation

```
void getData(struct Buffer* buffer)
{
  memcpy(buffer->data, some_data, BUFFER_SIZE);
}
```

Consequences

The complex, large data can be consistently provided to the caller with one single function call. The function is reentrant and can safely be used in a multi-threaded environment. Also the caller can safely access the data in multi-threaded environments, because the caller is the sole owner of the buffer.

The caller provides a buffer of the expected size and can even decide the kind of memory for that buffer. The caller can put the buffer on the stack (see Stack First) and benefit from the advantage that stack memory will be cleaned up after the variable runs out of scope. Alternatively the caller can

put the memory on the heap to determine the lifetime of the variable or to not waste stack memory. Also, the calling function might only have a reference to a buffer obtained by its calling function. In this case this buffer can simply be passed on and there is no need to have multiple buffers.

The time-intensive operation of allocating and freeing memory is not performed during the function call. The caller can determine when these operations take place and thus the function call becomes quicker and more deterministic.

From the API it is absolutely clear that the caller has Dedicated Ownership of the buffer. The caller has to provide the buffer and the caller has to clean it up afterwards. If the caller allocated the buffer, then the callero is the one responsible for freeing it afterwards.

The caller has to know the size of the buffer beforehand and knowing this size, the function can safely operate in the buffer. But in some cases the caller might not know the exact size required and it would be better if instead the Callee Allocates.

Known Uses

- The NetHack code uses this pattern to provide the information about a savegame to the component that then actually stores the game on the disk.
- The B&R Automation Runtime operating system uses this pattern for a function to retrieve the IP address.
- The C stdlib function `fgets` reads input from a stream and stores it into a provided buffer.

Applied to Running Example

You now provide a Caller-Owned Buffer to the Ethernet driver function and the function copies its data into this buffer. You have to know beforehand how large the buffer has to be. In case of obtaining the IP address string that

is no problem, because the string has a fixed size. So you can simply put the buffer for the IP address on the stack and provide this stack variable to the Ethernet driver. Alternatively it would have been possible to allocate the buffer on the heap, but in this case that is not required, because the size of the IP address is known and because the size of the data is small enough to fit on the stack:

Ethernet driver API

```
struct IpAddress{
  char address[16];
  char subnet[16];
};

/* Stores the IP information into 'ip', which has to be provided
by the caller*/
void ethernetDriverGetIp(struct IpAddress* ip);
```

Caller's code

```
void ethShow()
{
  struct EthernetDriverStat eth_stat =
ethernetDriverGetStatistics();
  printf("%i packets received\n", eth_stat.received_packets);
  printf("%i packets sent\n", eth_stat.total_sent_packets);
  printf("%i packets successfully sent\n",
eth_stat.successfully_sent_packets);
  printf("%i packets failed to send\n",
eth_stat.failed_sent_packets);

  const struct EthernetDriverInfo* eth_info =
ethernetDriverGetInfo();
  printf("Driver name: %s\n", eth_info->name);
  printf("Driver description: %s\n", eth_info->description);

  struct IpAddress ip;
  ethernetDriverGetIp(&ip);
  printf("IP address: %s\n", ip.address);
}
```

Next, you want to extend your diagnostic component to also print a dump of the last received packet. This now is a piece of information that is too large to put it on the stack and because Ethernet packets have variable size, you cannot know beforehand how large the buffer for the packet has to be, so Caller-Owned Buffer isn't an option for you.

You could, of course simply have functions `EthernetDriverGetPacketSize()` and `EthernetDriverGetPacket(buffer)`, but here again you'd have the problem that you'd have to call two functions and between the two function calls the Ethernet driver could receive another packet, which would make your data inconsistent. Also this solution is not very elegant, because you'd have to call two different functions to achieve one purpose.

Instead, it is much easier, if the Callee Allocates.

Callee Allocates

Context

You have large data that you want to share between different components.

Problem

You want to provide complex or large data of unknown size to the caller, and that data is not immutable (it changes at runtime).

The data changes at runtime (maybe because you provide the callers with functions to write the data), so you cannot simply provide the caller with a pointer to static data (as it is the case with an Immutable Instance). You cannot do that, because if you simply provide the callers with such a pointer, you'd run into the problem, that the data one caller reads might be inconsistent (partially overwritten), because, in a multi-threaded environment, another caller might simultaneously write that data.

Simply copying all the data into an Aggregate Instance and passing it via the Return Value to the caller is not an option, because, with the Return Value you can only pass data of known size and because, as the data is large, it cannot be passed via the stack, which only has very limited memory.

When instead only returning a pointer to the Aggregate Instance, there would be no problem with stack memory limitations anymore, but you have to keep in mind that C does not do the work of performing a deep copy for you. C only returns the pointer. You have to make sure that the data (stored in an Aggregate Instance or in an array) being pointed to is still valid after the function call. For example, you cannot store the data in auto-variables within your function and provide a pointer to these variables, because after the function call, the variables run out of scope and are being cleaned up.

Now the problem arises, where the data should be stored. It has to be clarified whether the caller or the callee should provide the required memory and it has to be clarified whether the caller or the callee is then responsible for managing and cleaning up the memory.

The amount of data you want to provide is not fixed at compile time. For example, you want to return a string of beforehand unknown size. That makes using a Caller-Owned Buffer impractical, because the caller does not know the size of the buffer beforehand. The caller could beforehand ask for the required buffer size (for example, with a `getRequiredBufferSize()` function), but that also is impractical, because in order to retrieve one piece of data, the caller would have to make multiple function calls. Also, maybe the data you want to provide could change between those function calls and then the caller again would provide a buffer of the wrong size.

Solution

Allocate a buffer with the required size inside the function that provides the complex, large data. Copy the required data into the buffer and return a pointer to that buffer.

Provide the pointer to the buffer and its size to the caller as Out-Parameters. After the function call, the caller can operate on the buffer, knows its size and has the sole ownership of the buffer. The caller determines its lifetime, and thus is responsible for cleaning it up as shown in Figure 4-7 and the following code.

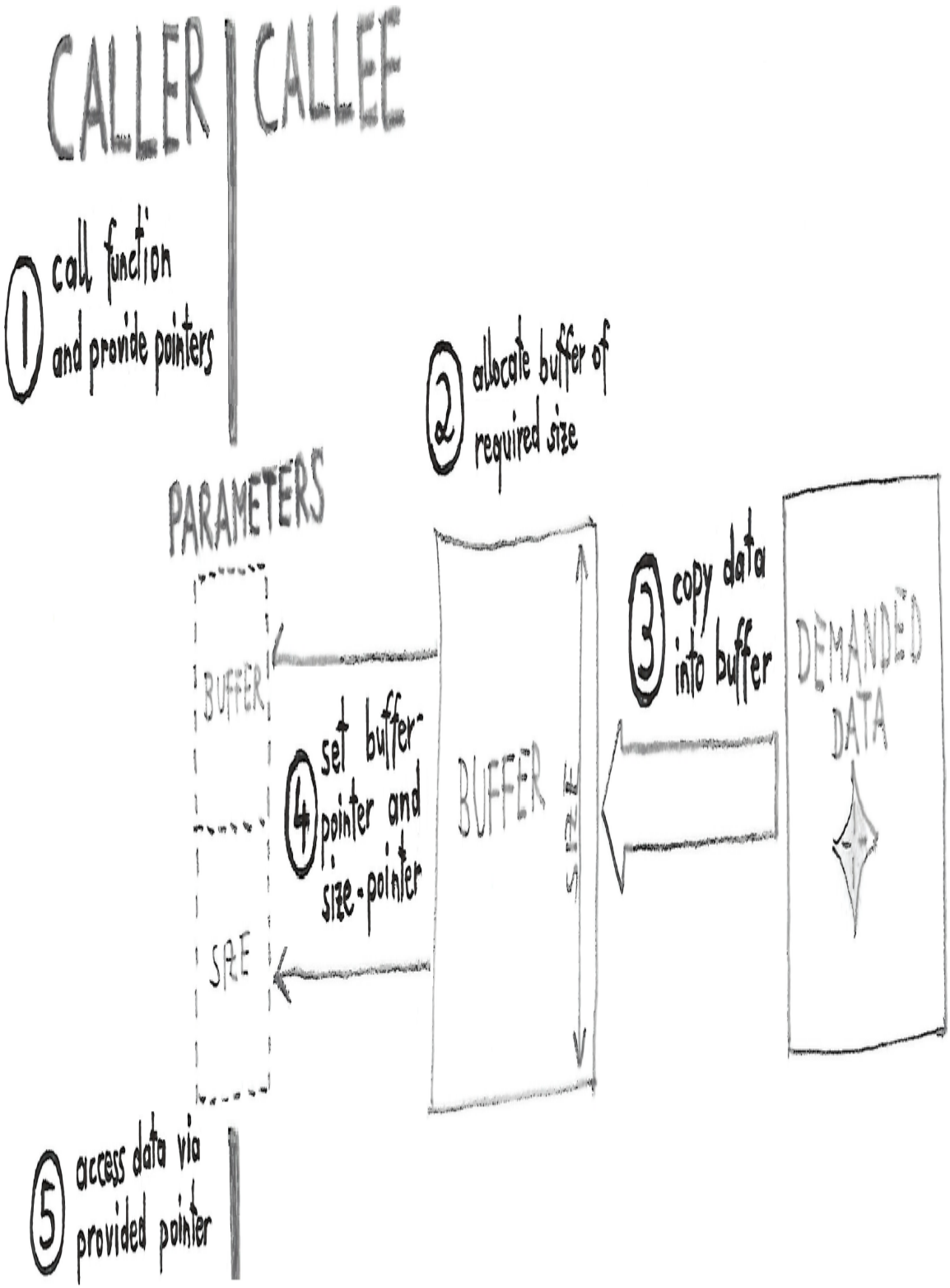


Figure 4-7. Callee Allocates

Caller's code

```
char* buffer;
int size;
getData(&buffer, &size);
/* use buffer */
free(buffer);
```

Callee's code

```
void getData(char** buffer, int* size)
{
  *size = data_size;
  *buffer = malloc(data_size);
  /* write data to buffer */ ❶
}
```

❶ When copying the data into that buffer, make sure that it does not change in the meantime. That can be achieved by mutual exclusion via Mutex or Semaphores.

Alternatively, the pointer to the buffer and the size can be put into an Aggregate Instance provided as Return Value. To make it more clear for the caller that there is a pointer in the Aggregate Instance that has to be freed, the API can provide an additional function for cleaning it up. When also providing a function to clean up, the API already looks very similar to an API with a Handle, which would bring the additional benefit of flexibility while maintaining API compatibility.

No matter whether the called function provides the buffer via an Aggregate Instance or via Out-Parameters, it has to be made clear to the caller, that the caller owns the buffer is responsible for freeing it. That Dedicated Ownership has to be well documented in the API.

Consequences

The caller can retrieve the buffer of beforehand unknown size with one single function call. The function is reentrant, can safely be used in multi-threaded environments, and provides the caller with consistent information about the buffer and its size. Knowing the size, the caller can safely operate

on the data. For example, the caller can even handle unterminated strings transported via such buffers.

The caller has ownership of the buffer, determines its lifetime, and is responsible for freeing it (just like it would be the case with a Handle). From looking at the interface, it has to be made very clear that the caller has to do that. One possibility to make that clear, is to document it in the API. Another approach is to have an explicit cleanup-function to make it more obvious that something has to be cleaned up. Such a cleanup function has the additional advantage that the same component that allocates the memory also frees it. That is important, if the two involved components are compiled with different compilers or if they run on different platforms - in such cases the functions for allocating and freeing memory could differ between the components, which makes it mandatory that the same component that allocates also frees.

The caller cannot determine which kind of memory should be used for the buffer - that would have been possible with a Caller-Owned Buffer. Now the caller must use the provided kind of memory that is allocated inside the function call.

Allocating takes time, which means that compared to Caller-Owned Buffer, the function call becomes slower and less deterministic.

Known Uses

- The `malloc` function does exactly that. It allocates some memory and provides it to the caller.
- The `strdup` function takes a string as input and allocates the duplicated string and returns it.
- The `getifaddrs` Linux function provides information about configured IP addresses. The data holding this information is stored in a buffer allocated by the function.
- The NetHack code uses this pattern to retrieve buffers.

Applied to Running Example

The following final code of your diagnostic component retrieves the packet data in a buffer that the Callee Allocates:

Ethernet driver API

```
struct Packet
{
  char data[1500]; /* maximum 1500 byte per packet */
  int size;        /* actual size of data in the packet */
};

/* Returns a pointer to a packet that has to be freed by the
caller */
struct Packet* ethernetDriverGetPacket();
```

Caller's code

```
void ethShow()
{
  struct EthernetDriverStat eth_stat =
ethernetDriverGetStatistics();
  printf("%i packets received\n", eth_stat.received_packets);
  printf("%i packets sent\n", eth_stat.total_sent_packets);
  printf("%i packets successfully sent\n",
eth_stat.successfully_sent_packets);
  printf("%i packets failed to send\n",
eth_stat.failed_sent_packets);

  const struct EthernetDriverInfo* eth_info =
ethernetDriverGetInfo();
  printf("Driver name: %s\n", eth_info->name);
  printf("Driver description: %s\n", eth_info->description);

  struct IpAddress ip;
  ethernetDriverGetIp(&ip);
  printf("IP address: %s\n", ip.address);

  struct Packet* packet = ethernetDriverGetPacket();
  printf("Packet Dump:");
  fwrite(packet->data, 1, packet->size, stdout);
  free(packet);
}
```

With this final version of the diagnostic component we can see all the presented ways of how to retrieve information from another function. Mixing all these ways in one piece of code might not be what you actually want to do, because it gets a bit confusing to have one piece of data on the stack and to have another piece of data on the heap. As soon as you allocate buffers, you don't want to mix different approaches, so using Caller-Owned Buffer and Callee Allocates in one single function might not be what you want to do. Instead, pick the one approach that suits all your needs and stick to that within one function or within one component. That makes your code more uniform and easier to understand.

However, if you just have to obtain a single piece of data from another component and if you have the choice to use the easier alternatives to retrieve data (the patterns presented earlier in this chapter), then always do that to keep your code simple. For example, if you have the possibility to put buffers on the stack, then do that, because it saves you the effort to free the buffer.

Summary

This chapter showed different ways of how to return data from functions and on how to handle buffers in C. The simplest way is to use Return Value to return a single piece of data, but if multiple pieces of related data have to be returned, then instead Out-Parameters or, even better, Aggregate Instance have to be used. If the data to be returned does not change at runtime, Immutable Instance can be used. When returning data in a buffer, Caller-Owned Buffer can be used if the size of the buffer is known beforehand and Callee Allocates can be used if the size is unknown beforehand.

With the patterns from this chapter, a C programmer has some basic tools and guidance on how to transport data between functions and on how to cope with returning, allocating, and freeing buffers.

Outlook

The next chapter covers how larger programs are organized into software-modules and how lifetime and ownership of data is handled by these software-modules. These patterns give an overview of the building blocks that are used to construct larger pieces of C code.

Chapter 5. Data Lifetime and Ownership

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 5th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

If we have a look at procedural programming languages like C, there are no native object-oriented mechanisms and that makes life to some extent harder, because much design guidance is tailored for object-oriented software (like the Gang of Four design patterns).

This chapter presents patterns on how to structure your C program with object-like elements. For these objects-like elements, the patterns put special focus on who is responsible for creating and destroying them - in other words they put special focus on lifetime and ownership. That topic is especially important for C, because C has no automatic destructor and no garbage collection mechanism and thus special attention has to be put on cleanup of resources.

However, what is an "object-like element" and what is the meaning of it for C? The term object is well defined for object-oriented programming languages, but for non-object-oriented programming languages it is not clear what the term object means. For C, a simple definition for objects is the following:

"An object is a named region of storage."

—Kernighan and Ritchie

Usually such an object describes a related set of data that has an identity and properties and that is used to store representations of things found in the real world. In object-oriented programming an object additionally has the capability of polymorphism and inheritance. The objects-like elements described throughout this book do not address polymorphism or inheritance and therefore we'll not use the term object anymore. Instead, we'll consider such an object-like element simply as an instance of a data structure and will furthermore call it *"instance"*.

Such instances do not stand by themselves, but instead they usually come with related pieces of code that makes it possible to operate on the instances. This code is usually put together into a set of header files for its interface and a set of implementation files for its implementation. Throughout this chapter, the sum of all this related code that often defines, similar to object-oriented classes, the operations that can be performed on an instance, will be called *"software-module"*.

When programming C, the described instances of data are usually implemented as abstract data types (for example, by having an instance of a `struct` with functions accessing the `struct` members). An example for such an instance is the C stdlib `FILE` `struct` that stores information like the file pointer or the position in the file. The corresponding software-module is the *stdio.h* API and its implementation of functions like `fopen` and `fclose`, which provide access to the `FILE` instances.

Figure 5-1 shows an overview of the patterns presented in this chapter and their relationships and Table 5-1 provides a summary of the patterns.

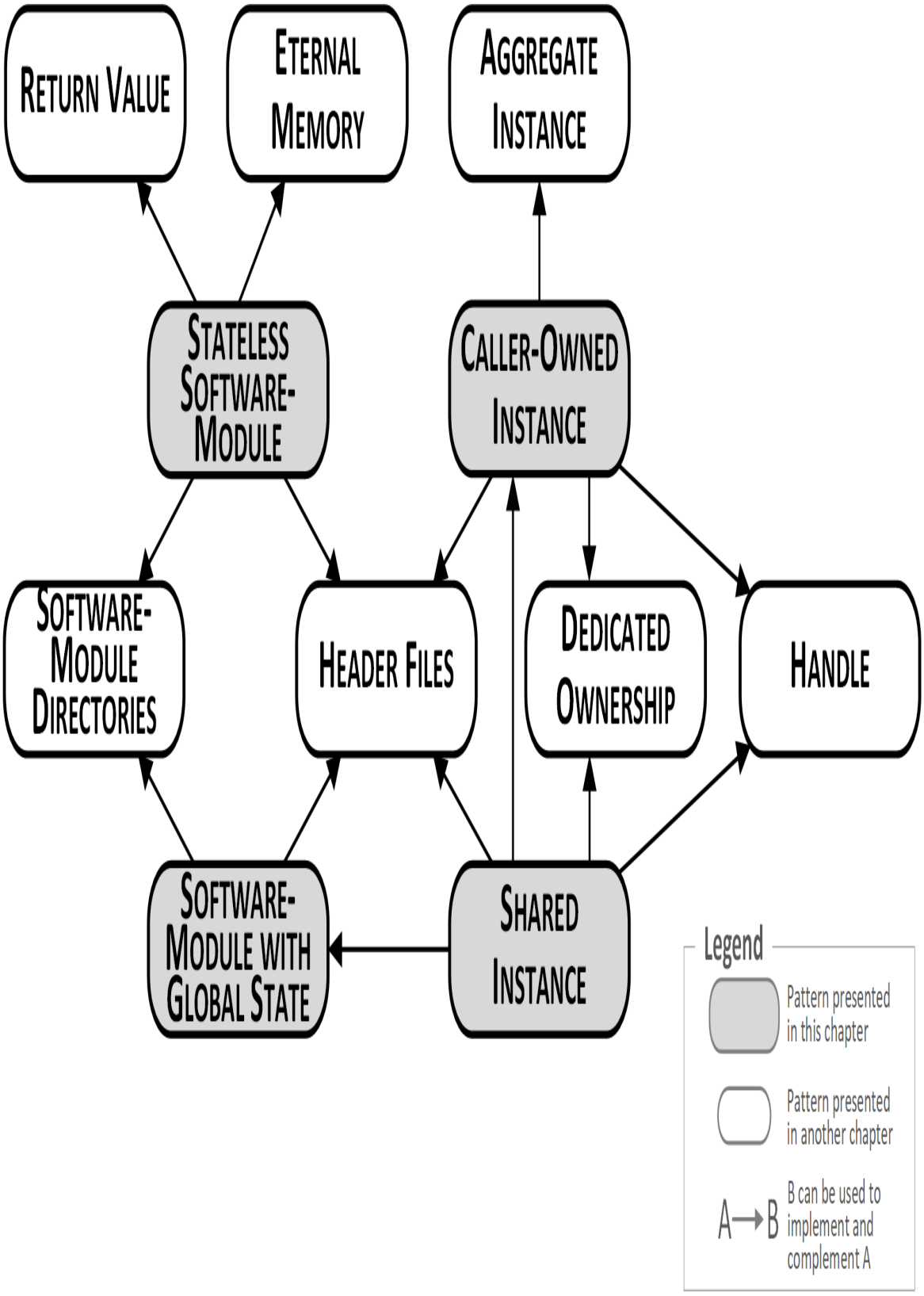


Figure 5-1. Overview of patterns on lifetime and ownership

Table 5-1. Patterns on life etimeand downers

h
i
p

	Pattern Name	Summary
	Stateless Software-Module	You want to provide logically related functionality to your caller and you and make that functionality for the caller as easy as possible to use. Therefore, keep your functions simple and don't build up state information in your implementation. Put all related functions into one header file and provide the caller this interface to your software-module.
	Software-Module with Global State	You want to structure your logically related code that requires common state information and you want to make that functionality for the caller as easy as possible to use. Therefore, have one global instance to let your related functions share common resources. Put all functions that operate on that instance into one header file and provide the caller this interface to your software-module.
	Caller-Owned Instance	You want to provide multiple callers access to functionality with functions that depend on one another and the interaction of the caller with your functions builds up state information. Therefore, require the caller to pass an instance, which is used to store resource and state information, along to your functions. Provide explicit functions to create and destroy these instances, so that the caller can determine their lifetime.
	Shared Instance	You want to provide multiple callers access to functionality with functions that depend on one another and the interaction of the caller with your functions builds up state information, which your callers want to share. Therefore, require the caller to pass an instance, which is used to store resource and state information, along to your functions. Use the same instance for multiple callers and keep the ownership of that instance in your software-module.

Running Example

You want to implement a device driver for your Ethernet network interface card. The Ethernet network interface card is installed on the operating system

your software runs on, so you can use the POSIX socket functions to send and receive network data.

You want to build some abstraction for your user, because you want to provide an easier way to send and receive data compared to socket functions and because you want to add some additional features to your Ethernet driver. Thus you want to implement something that encapsulates all the socket details.

To achieve that, start with a simple Stateless Software-Module.

Stateless Software-Module

Context

You want to provide functions with related functionality to a caller. The functions don't operate on common data shared between the functions and they don't require preparation of resources, like memory that has to be initialized previous to the function call.

Problem

You want to provide logically related functionality to your caller and you and make that functionality for the caller as easy as possible to use.

You want to make it simple for the caller to access your functionality. The caller should not have to deal with initialization and cleanup aspects of the provided functions, and the caller should not be confronted with implementation details.

You don't necessarily require the functions to be very flexible regarding future changes while maintaining backwards compatibility - instead the functions should only provide a simple to use abstraction for accessing the implemented functionality.

For organizing the header and implementation files you have many options and going through and evaluating each of these options becomes quite some effort it you have to do that for each and every functionality that you implement.

Solution

Keep your functions simple and don't build up state information in your implementation. Put all related functions into one header file and provide the caller this interface to your software-module.

No communication and no sharing of internal or external state information takes place between the functions and also no storing of state information takes place between function calls. That means the functions calculate a result or perform an action that does not depend on other function calls in the API and does not depend on previous function calls. The only communication that takes place is between the caller and the one called function at a time (for example, in the form of Return Values).

If a function requires any resources, such as heap memory for example, then the resources have to be handled transparently for the caller. They have to be acquired, implicitly initialized before they are used, and released within the function call. That makes it possible to call the functions completely independent from one another.

Still, the functions are related and because of that they are put together into one API. Being related means that the function are usually applied together by a caller (interface seggregation principle) and that if they change, they change for the same reason (common closure principle). These principles are described in the book *Clean Architecture* by Robert C. Martin (Prentice Hall, 2018).

Put the declarations of the related functions into one Header File and put the implementations of the functions into one or more implementation files, but into the same Software-Module Directory. The functions are related, because they logically belong together, but they do not share a common state or influence each others state, so there is no need to share information between the functions via global variables or to encapsulate this information by passing instances between the functions. That's why even each single function implementation could be put into a separate implementation file.

The following code shows and example for a simple Stateless Software-Module:

Caller's code

```
int result = sum(10, 20);
```

API

```
/* Returns the sum of the two parameters */
int sum(int summand1, int summand2);
```

Implementation

```
int sum(int summand1, int summand2)
{
  /* calculate result only depending on parameters and
     not requiring any state information */
  return summand1 + summand2;
}
```

The caller calls `sum` and retrieves a copy of the function result. When calling the function twice with same input parameters, the function would deliver the exact same result as no state information is maintained in the Stateless Software-Module and as in this special case, also no other function that holds state information is called.

Figure 5-2 shows on overview of the Stateless Software-Module.

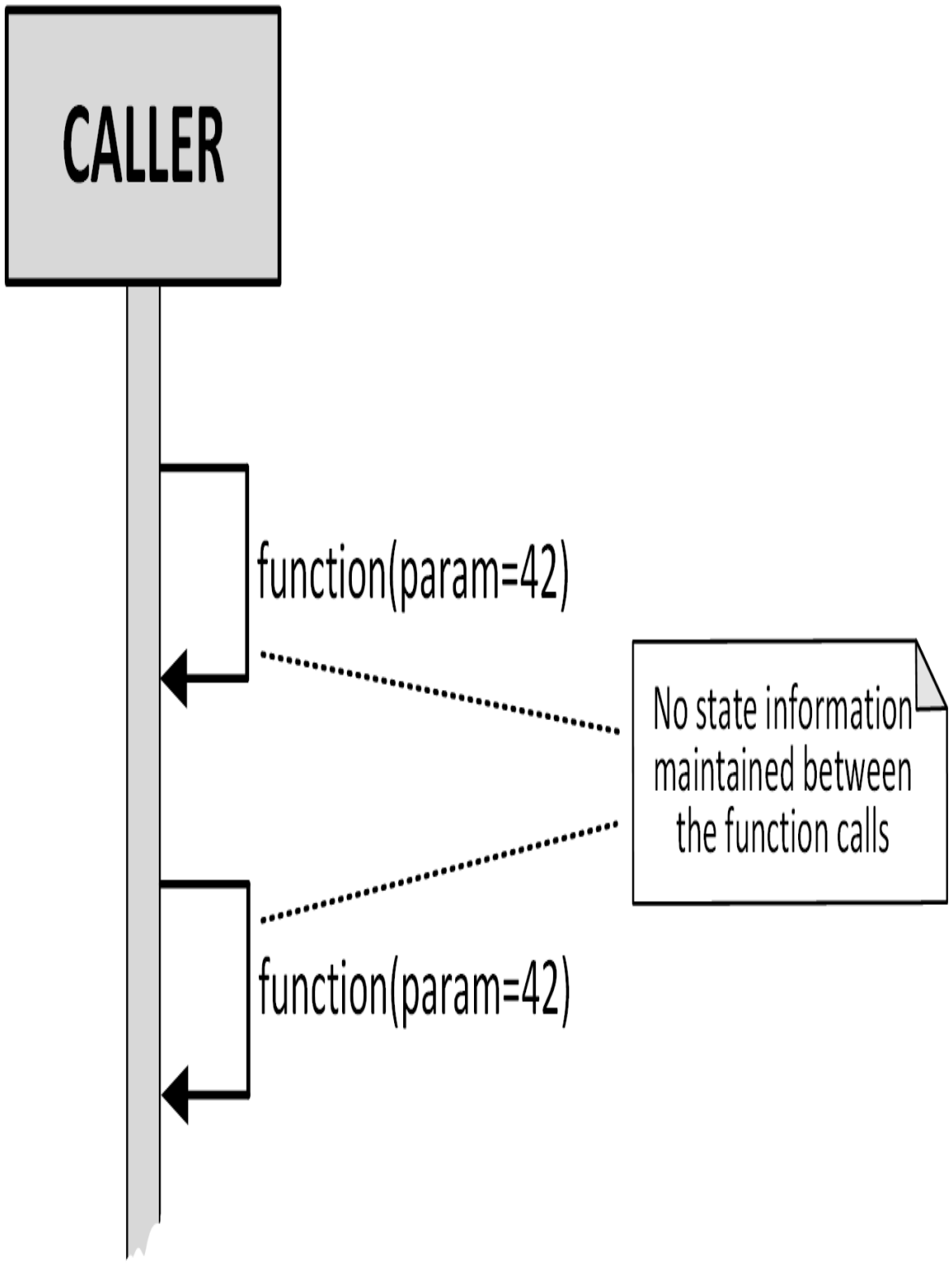


Figure 5-2. Stateless software-module

Consequences

You have a very simple interface and the caller does not have to cope with initializing or cleaning up anything for your software-module. The caller can simply call one of the functions independently of previous function calls and independently of other parts of the program, like for example other threads that concurrently access the software-module. Having no state information makes it much easier to understand what a function does.

The caller does not have to cope with questions about ownership, because there is nothing to own - the functions have no state. The resources required by the function are allocated and cleaned up within the function call and are thus transparent to the caller.

But not all functionality can be provided with such a simple interface. If the functions within an API share state information or data (for example, one has to allocate resources required by another), then a different approach, like a Software-Module with Global State or a Caller-Owned Instance has to be taken in order to share this information.

Known Uses

These types of related functions gathered into one API are found each time that the function within the API do not require shared information or state information.

- The `sin` and `cos` functions from *math.h* are provided in the same header file and they calculate their results solely based on the function input. They do not maintain state information and each call with the same input produces the same output.
- The *string.h* functions `strcpy` or `strcat` do not depend on one another. They don't share information, but they belong together and are thus part of a single API.
- The Windows header file *VersionHelpers.h* provides information about which Microsoft Windows version is currently running. Functions like `IsWindows7OrGreater` or `IsWindowsServer` provide related

information, but still the functions share no information and are independent from one another.

- The Linux header file *parser.h* comes with functions like `match_int` or `match_hex`. These functions try to parse an integer or a hexadecimal value from a substring. The functions are independent from one another, but still they belong together into the same API.
- The source code of the NetHack game also has many applications of this pattern. For example, the *vision.h* header file includes functions to calculate, whether the player is able to see specific other items on the game map. The functions `couldsee(x,y)` and `cansee(x,y)` calculate whether the player has clear line of sight to the other item and whether the player additionally faces that other item. Both functions are independent from one another and don't share state information.
- The Header Files present a variant of this pattern with more focus on API flexibility.
- The pattern called Per-Request Instance from the book *Remoting Patterns* by Markus Voelter et al. (Wiley, 2007) describes that a server in a distributed object middleware should activate a new servant for each invocation and that it should, after the servant handles the request, return the result and deactivate the servant. Such a call to a server maintains no state information and is similar to calls in Stateless Software-Modules, but with the difference that Stateless Software-Modules don't cope with remote entities.

Applied to Running Example

Your first device driver looks as shown in the following code:

API

```
void sendByte(char data, char* destination_ip);
char receiveByte();
```

Implementation

```
void sendByte(char data, char* destination_ip)
{
  /* open socket to destination_ip, send data via this socket and
close the socket */
}

char receiveByte()
{
  /* open socket for receiving data, wait some time and return
received data */
}
```

The user of your Ethernet driver does not have to cope with implementation details like how to access sockets and can simply use the provided API. Both of the functions in this API can be called at any time independently from one another and the caller can obtain data provided by the functions without having to cope with ownership and with freeing resources. Using this API is very simple, but also very limited.

Next, you want to provide additional functionality for your driver. You want to make it possible for the user to see whether the Ethernet communication works fine and thus, you want to provide statistics showing the number of sent or received packets. With a simple Stateless Software-Module, you cannot achieve that, because you have no retained memory for storing state information from one function call to another.

To achieve that, you need a Software-Module with Global State.

Software-Module with Global State

Context

You want to provide functions with related functionality to a caller. The functions do operate on common data shared between them and they might require preparation of resources, like memory that has to be initialized previous to using your functionality, but the functions do not require any caller-dependent state information.

Problem

You want to provide logically related functionality that requires common state information to your caller and you and make that functionality for the caller as easy as possible to use.

You want to make it simple for the caller to access your functionality. The caller should not have to deal with initialization and cleanup aspects of the functions, and the caller should not be confronted with implementation details. The caller should not necessarily realize that the functions access common data.

You don't necessarily require the functions to be very flexible regarding future changes while maintaining backwards compatibility - instead the functions should only provide a simple to use abstraction for accessing the implemented functionality.

Solution

Have one global instance to let your related function implementations share common resources. Put all these functions into one header file and provide the caller this interface to your software-module.

Put the function declaration in one Header File and put all the implementations for your software-module into one single implementation file in a Software-Module Directory. In this implementation file, have a global instance (a file-global static `struct` or several file-global static variables - see Eternal Memory) that holds the common shared resources that should be available for your function implementations. Your function implementations can then access these shared resources similar to private variables in object-oriented programming languages.

The initialization and lifetime of the resources is transparently managed in the software-module and is independent from the lifetime of its callers. If the resources have to be initialized, then you can initialize them at startup time or you can use lazy acquisition to initialize the resources right before they are needed.

The caller does not necessarily realize that the functions operate on common resources. Within your software-module, the access to these file-global resources might have to be protected by synchronization primitives such as a

Mutex to make it possible to have multiple callers from different threads. Make this synchronization within your function implementation, so that the caller does not have to deal with synchronization aspects.

The following code shows and example for a simple Software-Module with Global State:

Caller's code

```
int result;
result = addNext(10);
result = addNext(20);
```

API

```
/* Adds the parameter 'value' to the values accumulated
   with previous calls of this function. */
int addNext(int value);
```

Implementation

```
static int sum = 0;

int addNext(int value)
{
  /* calculation of the result depending on the parameter
     and on state information from previous function calls */
  sum = sum + value;
  return sum;
}
```

The caller calls `function` and retrieves a copy of the result. When calling the function twice with same input parameters, the function might deliver different results, because the function maintains state information.

Figure 5-3 shows on overview of the Software-Module with Global State.

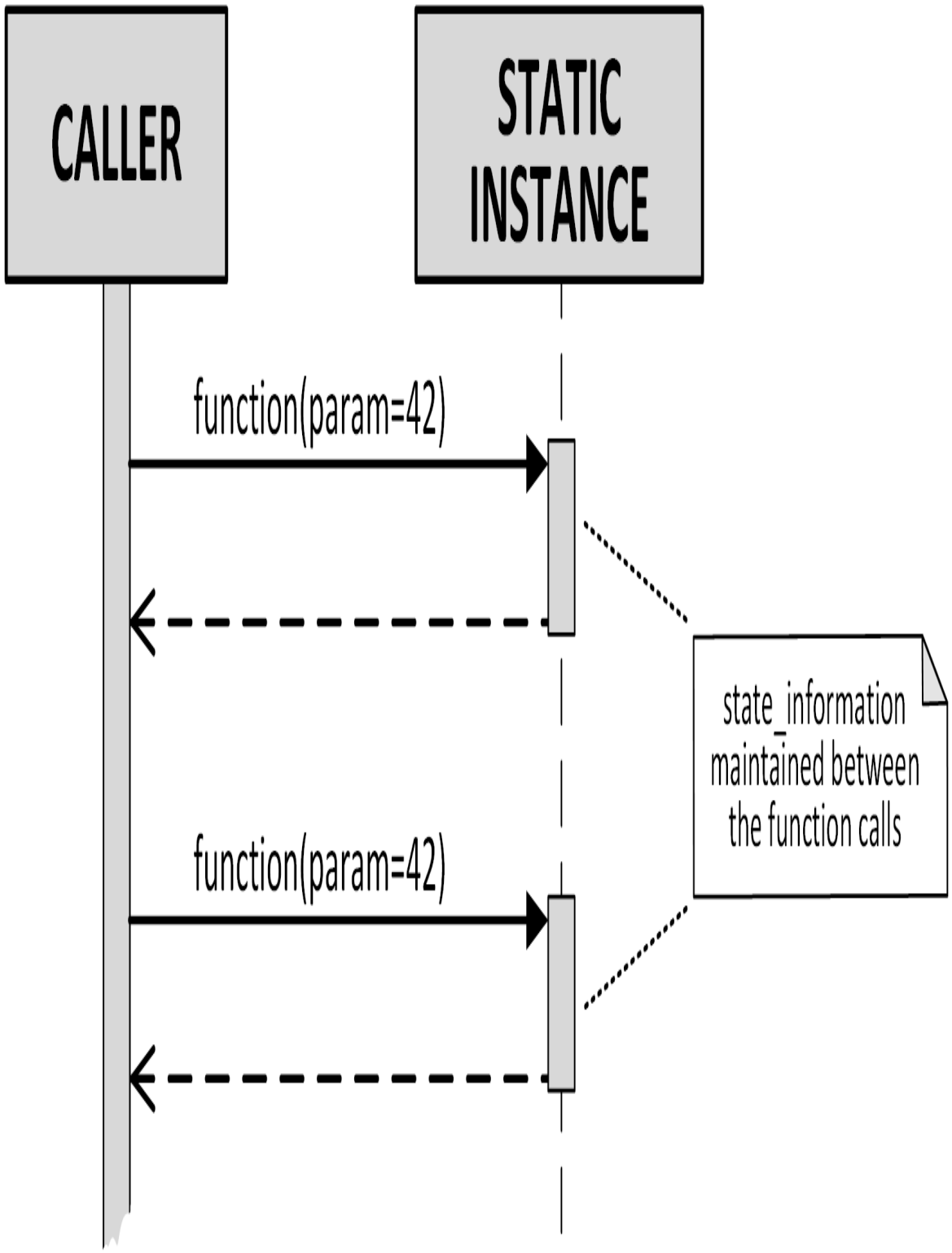


Figure 5-3. Software-Module with Global State

Consequences

Even though your functions share information or resources, they are available to the caller without requiring the caller to pass parameters containing this shared information and the caller is not responsible for allocating and cleaning up resources. To achieve this sharing of information in your software-module, you implemented the C-version of a Singleton. Beware of the Singleton - literature documents many disadvantages of this pattern and often it is rather called an anti-pattern. Still, in C such Software-Modules with Global State are wide-spread, because it is quite easy to write the keyword static before a variable and as soon as you do that, you have your Singleton. In some cases that is ok. If your implementation files are short, having file-global variables is quite similar to having private variables in object-oriented programming and if your functions do not require state information or do not operate in multi-threaded environments, then you might be just fine. However if multi-threading and state information becomes an issue and if your implementation file becomes longer and longer, then you are in trouble and the Software-Module with Global State is not a good solution anymore.

If your Software-Module with Global State requires initialization, then you either have to initialize it during some initialization phase like at system startup, or you have to use lazy acquisition to initialize short before the first use of resources, which has the drawback, that the duration for your function calls vary, because at the first call additional initialization code is implicitly called. In any case, the resource acquisition is performed transparently to the caller. The resources are owned by your software-module and thus the caller is not burdened with ownership of resources and does not have to explicitly acquire or release the resources.

However, not all functionality can be provided with such a simple interface. If the functions within an API share caller-specific state information, then a different approach, like a Caller-Owned Instance has to be taken.

Known Uses

- The *string.h* function `strtok` splits a string into tokens. Each time the function is called, the next token for the string is delivered. In order to

have the state information about which token to deliver next, the function uses static variables.

- With a Trusted Platform Module (TPM) one can accumulate hash values of loaded software. The corresponding function in the TPM-Emulator v0.7 code uses static variables to store this accumulated hash value.
- The `math` library uses a state for its random number generation. Each call of `rand` calculates a new pseudo random number based on the number calculated with the previous `rand` call. Initially `srand` has to be called in order to set the seed (the initial static information) for the pseudo random number generator called with `rand`.
- An Immutable Instance can be seen as part of a Software-Module with Global State with the special case, that the instance is not modified at runtime.
- The source code of the NetHack game stores information about items (swords, shields) in a static list defined at compile-time and provides functions to access this shared information.
- The pattern called Static Instance from the book *Remoting Patterns* by Markus Voelter et al. (Wiley, 2007) suggests to provide remote objects with lifetime decoupled from the lifetime of the caller. The remote objects can, for example, be initialized at startup time and then be provided to a caller when requested. Software-Module with Global State presents the same idea of having static data, but it is not meant for having multiple instances for different callers.

Applied to Running Example

Now you have the following code for your Ethernet driver:

API

```
void sendByte(char data, char* destination_ip);
char receiveByte();
int getNumberOfSentBytes();
int getNumberOfReceivedBytes();
```

Implementation

```
static int number_of_sent_packets = 0;
static int number_of_received_packets = 0;

void sendByte(char data, char* destination_ip)
{
  number_of_sent_packets++;
  /* socket stuff */
}

char receiveByte()
{
  number_of_received_packets++;
  /* socket stuff */
}

int getNumberOfSentBytes()
{
  return number_of_sent_packets;
}

int getNumberOfReceivedBytes()
{
  return number_of_received_packets;
}
```

The API looks very similar an API of a Stateless Software-Module, but behind this API now lies functionality to retain information between the function calls and the counters for sent and received bytes need that. As long as there is just one user (one thread) who uses this API, everything is just fine, but if there are multiple threads, then with static variables you always run into the problem, that race conditions occur if you don't look after that by implementing mutual exclusion for the access to the static variables.

Alright - now you want the Ethernet driver to be more efficient and you want to send more data. You could simply call your `sendByte` function very often to do that, but in your Ethernet driver implementation that means that for each `sendByte` call, you establish a socket connection, send the data and close the socket connection again. Establishing and closing the socket connection takes most of the communication time.

That is quite inefficient and you'd rather want to open your socket connection once, then send all the data by calling your `sendByte` function several times, and then close the socket connection. But now your `sendByte` function requires some preparation and some teardown phase and this state cannot be stored in a Software-Module with Global State, because as soon as you have more than one caller (one thread), you'd run into the problem that multiple callers want to simultaneously send data - maybe even to different destinations.

To achieve that, provide each of these callers with a Caller-Owned Instance.

Caller-Owned Instance

Context

You want to provide functions with related functionality to a caller. The functions do operate on common data shared between the functions, they might require preparation of resources, like memory that has to be initialized previous to using your functionality, and they share caller-specific state information amongst each other.

Problem

You want to provide multiple callers access to functionality with functions that depend on one another and the interaction of the caller with your functions builds up state information.

Maybe one function has to be called before another, because it influences some state stored in your software-module that is then needed by the other function. That can be achieved with a Software-Module with Global State, but that only works as long as there is just one single caller. In a multi-threaded environment with multiple callers, you cannot have one central software-module holding all caller-dependent state information.

Still, you want to hide implementation details from the caller and you want to make it as simple as possible for the caller to access your functionality. It has to be clearly defined whether the caller is responsible for allocating and cleaning up resources.

Solution

Require the caller to pass an instance, which is used to store resource and state information, along to your functions. Provide explicit functions to create and destroy these instances, so that the caller can determine their lifetime.

To implement such an instance that can be accessed from multiple functions, pass a `struct` pointer along with all functions that require sharing resources or state information. The functions can now use the `struct` members similar to private variables in object-oriented languages to store and read resource and state information.

The `struct` can be declared in the API to let the caller conveniently directly access its members. Alternatively, the `struct` can be declared in the implementation and only a pointer to the `struct` can be declared in the API (as suggested by Handle). The caller does not know the `struct` members (they are like private variables) and can only operate with functions on the `struct`.

As the instance has to be manipulated by multiple of your functions and as you do not know when the caller finished calling functions, the lifetime of the instance has to be determined by the caller. Therefore, Dedicate Ownership to the caller and provide explicit functions for creating and destroying the instance. The caller has an aggregate relationship to the instance.

> **AGGREGATION VS. ASSOCIATION**
>
> If an instance is semantically related to another instance, then those instances are associated. A stronger type of association is an aggregation, where one instance has ownership of the other.

The following code shows and example for a simple Caller-Owned Instance:

Caller's code

```
struct INSTANCE* inst;
inst = createInstance();
operateOnInstance(inst);
```

```
/* access inst->x or inst->y */
destroyInstance(inst);
```

API

```
struct INSTANCE
{
  int x;
  int y;
};

/* Creates an instance which is required for working
   with the function 'operateOnInstance' */
struct INSTANCE* createInstance();

/* Operates on the data stored in the instance */
void operateOnInstance(struct INSTANCE* inst);

/* Cleans up an instance created with 'createInstance' */
void destroyInstance(struct INSTANCE* inst);
```

Implementation

```
struct INSTANCE* createInstance()
{
  struct INSTANCE* inst;
  inst = malloc(sizeof(struct INSTANCE));
  return inst;
}

void operateOnInstance(struct INSTANCE* inst)
{
  /* work with inst->x and inst->y */
}

void destroyInstance(struct INSTANCE* inst)
{
  free(inst);
}
```

The function `operateOnInstance` works on resources created with the previous function call `createInstance`. The resource or state information between the two function calls is transported by the caller who has to provide

the `INSTANCE` for each function call and who also has to clean up all the resources by calling `destroyInstance`.

Figure 5-4 shows on overview of the Caller-Owned Instance.

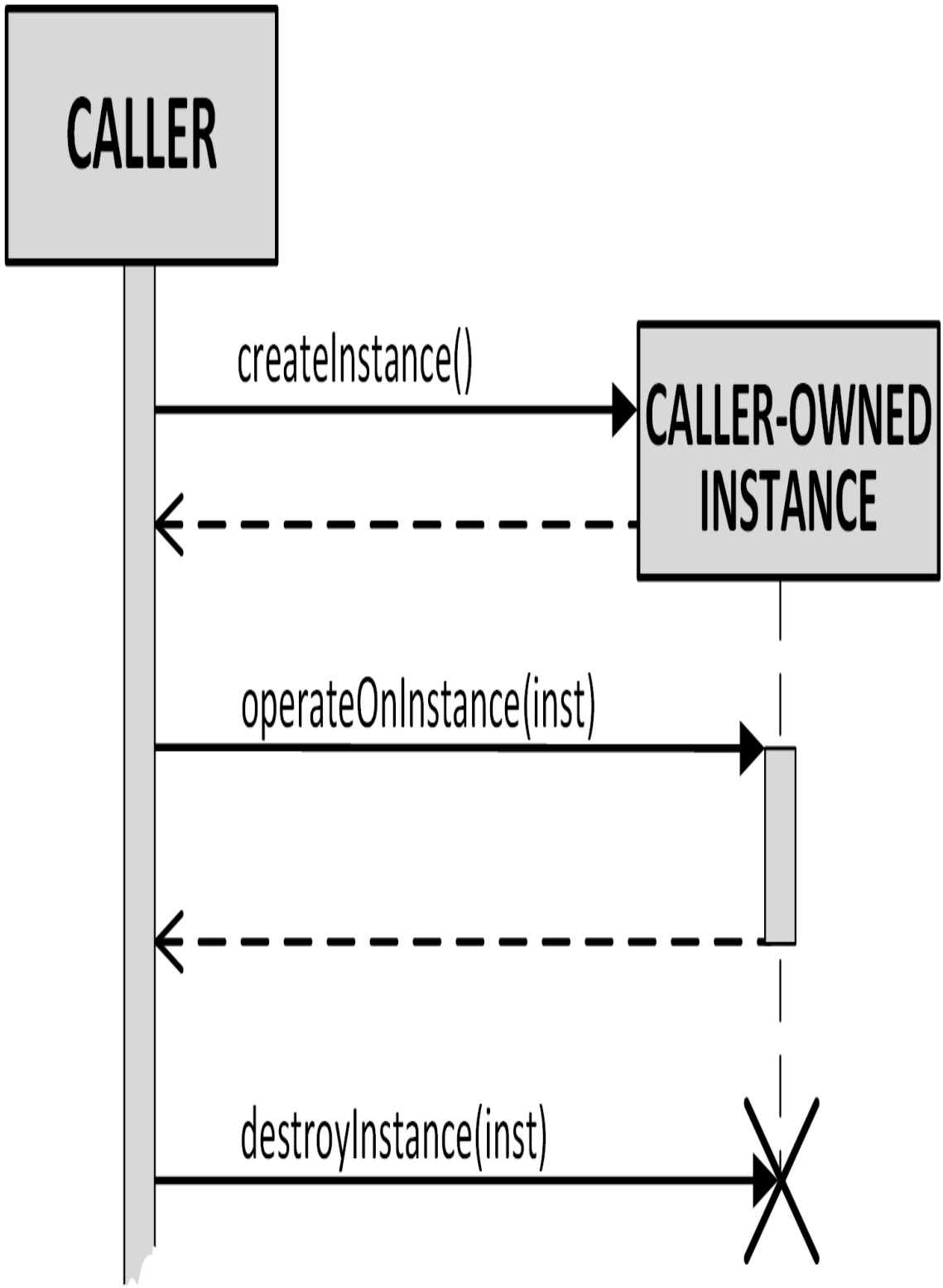


Figure 5-4. Caller-Owned Instance

Consequences

The functions in your API are more powerful now, because they can share state information and they can operate on shared data while still being available for multiple callers (multiple threads). Each created Caller-Owned Instance has its own private variables and even if more than one such Caller-Owned Instance is created (for example, by multiple callers in a multi-threaded environment) it is no problem.

However, to achieve that, your API becomes more complicated. You have to make explicit `create()` and `destroy()` calls for managing the instance's lifetime, because C does not support constructors and destructors. This makes handling with instances much more difficult, because the caller obtains ownership and is responsible for correctly explicitly cleaning up the instance. As this has to be done manually with the `destroy()` call and not via an automatic destructor like in object-oriented programing languages, this is a common pitfall for memory leaks. This issue is addressed by Object-Based Error Handling, which suggests that also the caller should have a dedicated cleanup function to make this task more explicit.

Also, compared to a Stateless Software-Module, calling each of the functions becomes a bit more complicated. Each function takes an additional parameter referencing the instance and the functions cannot be called in arbitrary order - the caller has to know which one has to be called first and that is made explicit through the function signatures.

Known Uses

- An example for the use of a Caller-Owned Instance is the doubly linked list provided with the `glibc` library. The caller creates a list with `g_list_alloc` and can then insert items into this list with `g_list_insert`. When finished working with the list, the caller is responsible to clean it up with `g_list_free`.
- This pattern is described in an article on how to write modular C programs (*http://metamodulaire.com/Computing/modular-c.pdf*). The article states the importance of identifying abstract data types in the application, which can be manipulated or accessed with functions.

- The Windows API to create menus in the menu bar has a function to create a menu instance (`CreateMenu`), it has functions to operate on menus (like `InsertMenuItem`), and a function to destroy the menu instance (`DestroyMenu`). All these functions have one parameter to pass the Handle to the menu instance.
- Apache's software-module to handle HTTP requests provides a function to create all required request information (`ap_sub_req_lookup_uri`), to process it (`ap_run_sub_req`), and to destroy it (`ap_destroy_sub_req`). These functions take a `struct` pointer to the request instance in order to share request information.
- The source code of the NetHack game uses a `struct` instance to represent monsters and provides functions to create and destroy a monster. Also, the NetHack code provides functions to obtian information from monsters (`is_starting_pet`, `is_vampshifter`).
- The pattern called Client-Dependent Instance from the book *Remoting Patterns* by Markus Voelter et al. (Wiley, 2007) suggests for distributed object middlewares to provide remote objects whose lifetime is controlled by the clients. The server creates new instances for clients and the client can then work with these instances, pass this instance along, or destroy them.

Applied to Running Example

Now you have the following code for your Ethernet driver:

API

```
typedef struct Sender* SENDER;
SENDER createSender(char* destination_ip);
void sendByte(SENDER s, char data);
void destroySender(SENDER s);
```

Implementation

```
struct Sender
{
  char destination_ip[16];
  int socket;
}

SENDER createSender(char* destination_ip)
{
  SENDER s = malloc(sizeof(struct Sender));
  /* create socket to destination_ip and store it in SENDER s*/
  return s;
}

void sendByte(SENDER s, char data)
{
  number_of_sent_packets++;
  /* send data via socket stored in SENDER s */
}

void destroySender(SENDER s)
{
  /* close socket stored in SENDER s */
  free(s);
}
```

A caller can first create a sender, then send all the data, and then destroy the sender. Thus, the caller can make sure that the socket connection does not have to be established again for each `sendByte()` call. The caller has ownership of the created sender, has full control over how long the sender lives, and is responsible for cleaning it up:

Caller's code

```
SENDER s = createSender("192.168.0.1");
char* dataToSend = "Hello World!";
char* pointer = dataToSend;
while(*pointer != '\0')
{
  sendByte(s, *pointer);
  pointer++;
}
destroySender(s);
```

Next, let's assume that you are not the only user of this API. There might be multiple threads using your API. As long as one thread creates a sender for

sending to IP address X and another thread creates a sender for sending to Y, we are just fine and the Ethernet driver creates independent sockets for both threads.

However, let's say the two threads want to send data to the same recipient. Now the Ethernet driver is in trouble, because on one specific port, it can only open one socket per destination IP. A solution to that problem would be to not allow two different threads to send to the same destination. The second thread creating the sender could simply receive an error, but it is also possible to allow both threads sending data using the same sender.

To achieve that, simply construct a Shared Instance.

Shared Instance

Context

You want to provide functions with related functionality to a caller. The functions do operate on shared common data, they might require preparation of resources, like memory that has to be initialized previous to using your functionality. There are multiple contexts in which the functionality can be called and these contexts are shared between the callers.

Problem

You want to provide multiple callers access to functionality with functions that depend on one another and the interaction of the caller with your functions builds up state information, which your callers want to share.

Storing the state information in a Software-Module with Global State is not an option, because there are multiple callers who want to build up different state information. Storing the state information per caller in a Caller-Owned Instance is not an option, because either some of your callers want to access and operate on one and the same instance, or because you don't want to create new instances for every caller in order to keep resource costs low.

Still, you want to hide implementation details from the caller and you want to make it as simple as possible for the caller to access your functionality. It has to

be clearly defined whether the caller is responsible for allocating and cleaning up resources.

Solution

Require the caller to pass an instance, which is used to store resource and state information, along to your functions. Use the same instance for multiple callers and keep the ownership of that instance in your software-module.

Just like with the Caller-Owned Instance, provide a `struct` pointer or a Handle that the caller then passes along the function calls. When creating the instance, the caller now additionally has to provide an identifier (for example, a unique name) to specify the kind of instance to create. With this identifier you can know, whether such an instance already exists. If it exists, you don't create a new instance, but instead return the `struct` pointer or Handle to the instance that you already created and returned to other callers before. In any case, it makes no difference to the caller whether the retrieved instance is new or was already created before by a different caller.

To know whether an instance already exists, you have to hold a list of already created instances in your software-module. That can be done by implementing a Software-Module with Global State to hold the list. Additionally to whether an instance was already created or not, you can store the information of who currently accesses which instances or at least how many callers currently access an instance. This additional information is required, because when everybody is finished accessing it, it is your duty to clean the instance up, because you are the one who has Dedicated Ownership of it.

You also have to check, whether your functions can be called simultaneously by different callers on one and the same instance. In some easier cases, there might be no things whose access has to be mutually excluded by different callers, because information is only read and in such cases an Immutable Instance, which does not allow the caller changing the instance, could be implemented. But in other cases, in your functions you have to implement mutual exclusion for resources shared through the instance.

The following code shows and example for a simple Shared Instance:

Caller1's code

```
struct INSTANCE* inst = openInstance(42);
/* operate on the same instance as caller2 */
operateOnInstance(inst);
closeInstance(inst);
```

Caller2's code

```
struct INSTANCE* inst = openInstance(42);
/* operate on the same instance as caller1 */
operateOnInstance(inst);
closeInstance(inst);
```

API

```
struct INSTANCE
{
  int x;
  int y;
};


/* Retrieve an instance identified by the parameter 'id'. That
instance is
   created if no instance of that 'id' was yet retrieved from any
other caller. */
struct INSTANCE* openInstance(int id);

/* Operates on the data stored in the instance. */
void operateOnInstance(struct INSTANCE* inst);

/* Releases an instance which was retrieved with 'openInstance'.
   If all callers release an instance, it gets destroyed. */
void closeInstance(struct INSTANCE* inst);
```

Implementation

```
struct INSTANCELIST
{
  struct INSTANCE* inst;
```

```
  int count;
};

static struct INSTANCELIST list[MAX_INSTANCES];

struct INSTANCE* openInstance(int id)
{
  if(list[id].count == 0)
  {
    list[id].inst =  malloc(sizeof(struct INSTANCE));
  }
  list[id].count++;
  return list[id].inst;
}

void operateOnInstance(struct INSTANCE* inst)
{
  /* work with inst->x and inst->y */
}

static int getInstanceId(struct INSTANCE* inst)
{
  int i;
  for(i=0; i<MAX_INSTANCES; i++)
  {
    if(inst == list[i].inst)
    {
      break;
    }
  }
  return i;
}

void closeInstance(struct INSTANCE* inst)
{
  int id = getInstanceId(inst);
  list[id].count--;
  if(list[id].count == 0)
  {
    free(inst);
  }
}
```

The caller retrieves an `INSTANCE` by calling `openInstance`. The `INSTANCE` might be created by this function call, or it might already have been created by a previous function call and might also be used by another caller. The caller can then pass the `INSTANCE` along to the

`operateOnInstance` function calls, to provide this function with the required resource or state information from the `INSTANCE`. When finished, the caller has to call `closeInstance`, so that the resources can be cleaned up, if no other caller operates on the `INSTANCE` anymore.

Figure 5-5 shows on overview of the Shared Instance.

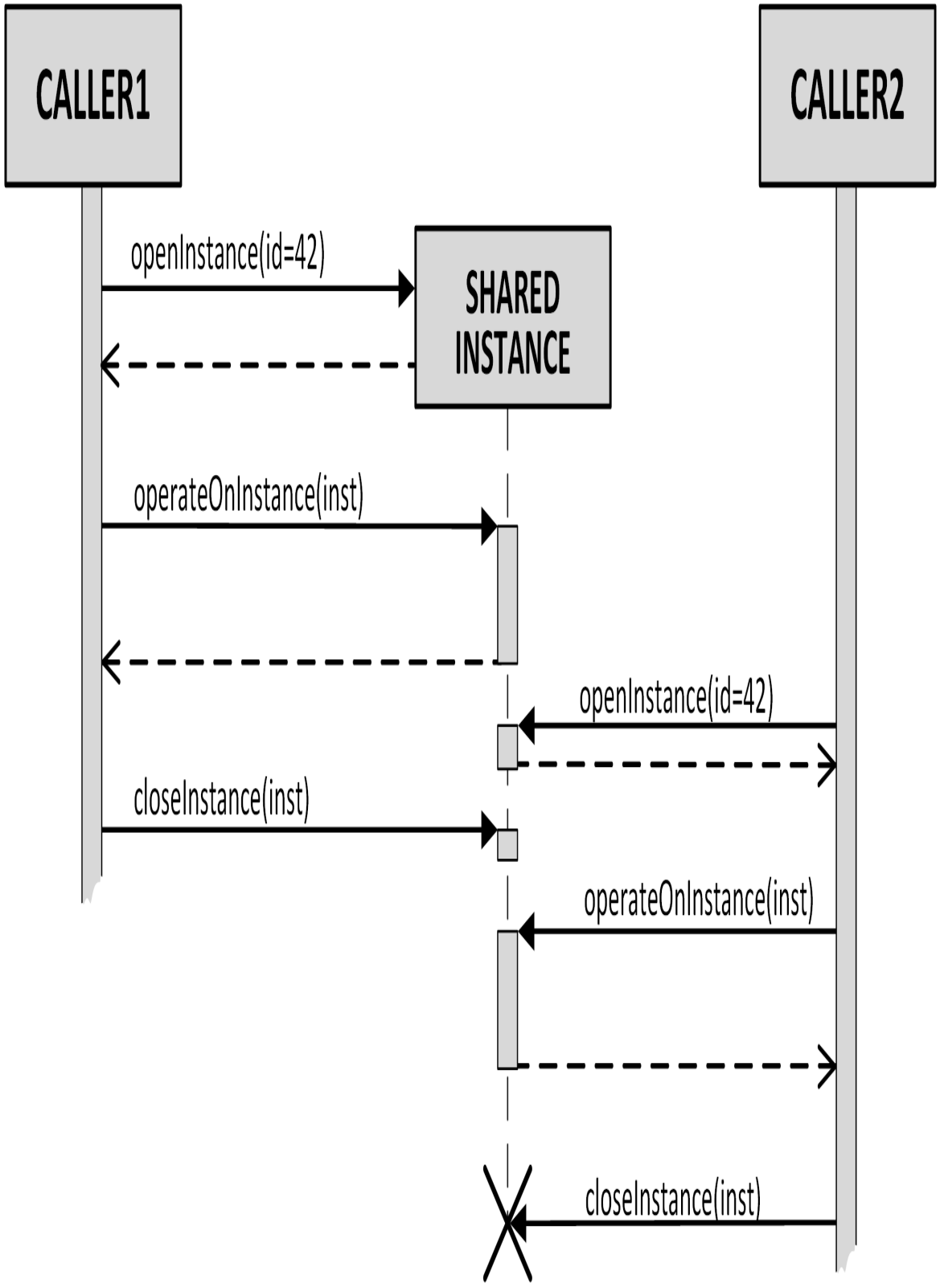


Figure 5-5. Shared Instance

Consequences

Multiple callers now have simultaneous access to one single instance. This quite often implies that you have to cope with mutual exclusion within your implementation in order to not burden the user with such issues. This implies that the duration for a function call varies, because the caller never knows whether another caller currently uses the same resources and whether another caller blocks them.

Your software-module and not the caller has ownership of the instance and your software-module is responsible for cleaning up resources. Still the caller is responsible for releasing the resources so that your software-module knows when to clean everything up - like for the Caller-Owned Instance, this a pitfall for memory leaks.

As the software-module has ownership of the instances, the software-module can also clean up the instances without requiring the callers to initiate that. For example, if the software-module receives a shutdown signal from the operating system, it has the possibility to clean up all instances, because it has ownership of all the instances.

Known Uses

- An example for the use of a Shared Instance are the *stdio.h* file-functions. A file can be opened by multiple callers via the function `fopen`. The caller retrieves a handle to the file and can the read from or write to the file (`fread`, `fprintf`). The file is a shared resource. For example, there is one global cursor position in the file for all callers. When a caller finished operating on the file, it has to be closed with `fclose`.
- This pattern and its implementation details for object-oriented programming languages is presented in the article *C++ Patterns: Reference Accounting* by Kevlin Henney (*https://hillside.net/europlop/HillsideEurope/Papers/EuroPLoP2001/2001_Henney_ReferenceAccounting.pdf*) as Counting Handle. It describes how a shared object on the heap can be accessed and how its lifetime can transparently be handled.

- The Windows registry can be accessed simultaneously by multiple threads with the function `RegCreateKey` (that opens the key, if it already exists). The function delivers a Handle that can be used by other functions to operate on the registry key. When the registry operations are finished the `RegCloseKey` function has to be called by everybody who opened the key.
- The Windows functionality to access Mutex (`CreateMutex`) can be used to access a shared resource (the Mutex) from multiple threads. With the Mutex, interprocess synchronization can be implemented. When finished working with the Mutex, each caller has to close it by using the function `CloseHandle`.
- The B&R Automation Runtime operating system allows multiple callers to access device drivers simultaneously. A caller uses the function `DmDeviceOpen` to select one of the available devices. The device driver framework checks whether the selected driver is available and then provides a Handle to the caller, also if other callers currently already operate on this driver. The callers can then simultaneously interact with the driver (send or read data, interact via IO-controls, …) and after this interaction they tell the device driver framework that they are finished by calling `DmDeviceClose`.

Applied to Running Example

The driver now implements the following functions:

API

```
typedef struct Sender* SENDER;
SENDER openSender(char* destination_ip);
void closeSender(SENDER s);
```

Implementation

```
SENDER openSender(char* destination_ip)
{
  SENDER s;
```

```
  if(isInSenderList(destination_ip))
  {
    s = getSenderFromList(destination_ip);
  }
  else
  {
    s = createSender(destination_ip);
  }
  increaseNumberOfCallers(s);
  return s;
}

void sendByte(SENDER s, char data)
{
  lock(); /* mutual exclusion for threads */
  number_of_sent_packets++;
  unlock();
  /* send data via socket stored in SENDER s */
}

void closeSender(SENDER s)
{
  decreaseNumberOfCallers(s);
  if(numberOfCallers(s) == 0)
  {
    /* close socket stored in SENDER s */
    free(s);
  }
}
```

The API of the running example did not change a lot - instead of having create/destroy-functions, your driver now provides open/close-functions. By calling such a function, the caller retrieves the Handle for the sender and indicates the driver, that this caller is now operating a sender, but the driver does not necessarily create this sender at that point in time. That might already have happened by an earlier call to the driver (maybe performed by a different thread). Also a close-call might not actually destroy the sender. The ownership of this sender remains in the driver implementation, which can decide when to destroy the senders (for example, when all callers close the sender, or if some termination signal is received).

The fact that you now have a Shared Instance instead of a Caller-Owned Instance is mostly transparent to the caller. But the driver implementation changed - it has to remember whether a specific sender was already created and

has to provide this shared instance instead of creating a new one. When opening a sender, the caller does not know, whether this sender will be newly created or whether and existing sender is retrieved. Depending on that, the duration of the function call might vary.

The presented running driver example showed different kinds of ownership and data lifetime in one single example. We saw, how a simple Ethernet driver evolved by adding functionality. First, a Stateless Software-Module was sufficient, because the driver did not require any state information. Next, such state information was required and it was realized by having a Software-Module with Global State in the driver. Then, the need for more performant send-functions and for multiple callers for these send-functions came up and was first implemented by the Caller-Owned Instance and in a next step by the Shared Instance.

Summary

The patterns in this chapter showed different ways on how to structure your C programs and on how long different instances in your program live. Table 5-2 shows an overview of the patterns and compares their consequences.

Table 5-2. Comparing Patterns on Lifetime

a
n
d
O
w
n
e
r
s
h
i
p

	Stateless Software-Module	Software-Module with Global State	Caller-Owned Instance	Shared Instance
Resource sharing between functions	Not possible	Single set of resources	Set of resources per instance (= per caller)	Set of resources per instance (shared by multiple callers)
Resource ownership	Nothing to own	The software-module owns the static data	The caller owns the instance	The software-module owns instances and provides references
Resource lifetime	No resources live longer than a function call	Static data lives forever in the software-module	Instances live until callers destroy them	Instances live until the software-module destroys them
Resource initialization	Nothing to initialize	At compile time or at startup	By the caller when creating an instance	By the software-module when the first caller opens an instance

With these patterns, a C programmer has some basic guidance about the design options for organizing programs into software-modules and about the design options regarding ownership and lifetime when constructing instances.

Further Reading

The patterns in this chapter cover the topic of how to provide access to instances and on who has ownership of these instances. A very similar topic is covered by a subset of the patterns from the book *Remoting Patterns* by Markus Voelter et al. (Wiley, 2007). The book “Remoting Patterns” presents patterns for building distributed object middleware and three of these patterns focus on lifetime and ownership of objects created by remote servers. Compared to that, the patterns presented in this chapter focus on a different context. They are not patterns for remote systems, but for local procedural programs. They focus on C programming, but can also be used for other procedural programming languages. Still, some of the underlying ideas in the patterns are very similar to the *Remoting Patterns*.

Outlook

The next chapter shows different kinds of interfaces for software-modules with special focus on how to make that interface flexible. The patterns elaboarate on the tradeoff between simplicity and flexibility.

Chapter 6. Flexible APIs

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 6th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

Designing interfaces with the right level of flexibility and the right level of abstraction is one of the most important things when writing software, because interfaces represent a contract that often cannot be changed anymore once the system is in operation. Because of that it is important to put stable things into the interface and to abstract implementation details, which should have the flexibility to change at a later point in time.

While for object-oriented programming languages there exists much guidance on how to design interfaces (for example in the form of design patterns), there is just very few guidance of this kind for procedural programming languages like C. There are the SOLID design principles which tell you how in general to design good software. However, for the C programming language, more detailed design guidance on how to design interfaces is hard to find and that's where the patterns from chapter come in.

SOLID

The SOLID principles tell us how to implement good flexible and maintainable software.

- Single responsibility principle: The code has one single responsibility and one single reason to be changed in future.
- Open-closed principle: Code should be open for behavior changes without requiring changes to the existing code.
- Liskow substitution principle: Codes that implement the same interface should be interchangable for the caller.
- Interface segregation principle: Interfaces should be slim and tailored for the caller's needs.
- Dependency inversion principle: High-level modules should be independent from low-level modules.

An article by James Grenning (*https://www.renaissancesoftware.net/files/articles/ESC-204Paper_Grenning-v1r0.pdf*) gives you more details on how to implement the SOLID principles in C.

Figure 6-1 shows the four patterns, which are presented in this chapter, as well as related patterns and Table 6-1 contains a short description of the four patterns. Keep in mind that not all of the patterns should always be applied in all possible contexts. Generally it is advisable to design a system not more complex than it has to be. This means that some of the presented patterns should only be applied if the gained flexibility is already required or will likely be required in future by your API. If it is not likely to be required, then the pattern should perhaps not be applied to keep the API as simple as possible.

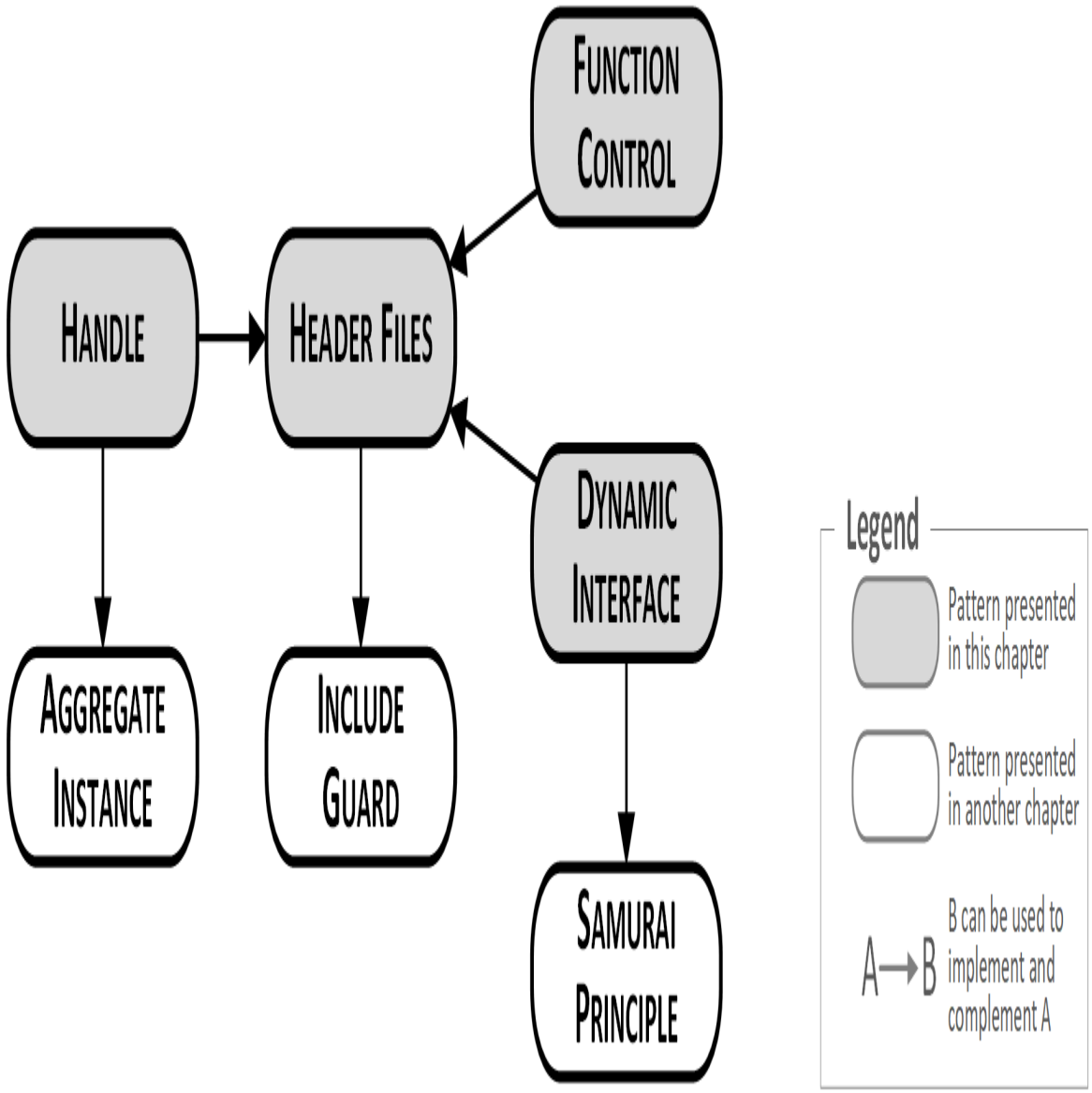


Figure 6-1. Overview of the patterns on flexible APIs

Table 6-1. Patterns for flexible AP

I
s

	Pattern Name	Pattern Description
	Header Files	You want some functionality that you implement to be accessible for code from other implementation files, but you want to hide your implementation details from the caller. Therefore, provide function declarations in your API for any functionality you want to provide to your user. Hide any internal functions, internal data, and your function definitions (the implementations) in your implementation file and don't provide this implementation file to the user.
	Handle	You have to share state information or operate on shared resources in your function implementations, but you don't want your caller to see or even access all that state information and shared resources. Therefore, have a function to create the context on which the caller operates and return an abstract pointer to internal data for that context. Require the caller to pass that pointer to all your functions which can then use the internal data to store state information and resources.
	Dynamic Interface	It should be possible to call implementations with slightly deviating behaviors, but it should not be necessary to duplicate any code, not even the control logic implementation and interface declaration. Therefore, define a common interface for the deviating functionalities in your API and require the caller to provide a callback function for that functionality which you then call in your function implementation.
	Function Control	You want to call implementations with slightly deviating behaviors, but you don't want to duplicate any code, not even the control logic implementation or the interface declaration. Therefore, apply data-based abstraction. Add a parameter to your function that passes meta-information about the function call and that specifies the actual functionality to be performed.

Running Example

You want to implement a device driver for your Ethernet network interface card. The firmware of this card provides several registers with which you can send or receive data and with which you can configure the card.

You want to build some abstraction of these hardware details and you want to make sure that a user of your API does not have to care if you change some parts of your implementation. To achieve that, you build an API consisting of Header Files.

Header Files

Context

You write a larger piece of software in C. You split that software up into several functions and you implement these functions in several files, because you want to make your program modular and easy to maintain.

Problem

You want some functionality that you implement to be accessible for code from other implementation files, but you want to hide your implementation details from the caller.

Unlike, for example, with many object-oriented languages, C does not provide any built-in support for defining APIs, for abstracting functionality, and for enforcing that the caller can only access this abstraction. C only provides a mechanism to include files into other files.

The caller of your code could use that mechanism to simply include your implementation file. But then the caller could access all internal data in that file, like variables or functions with file scope that you only intend to use internally. Once the caller uses this internal functionality, it might not be easy to change that later on, so the code becomes tightly coupled in places where you might not want that. If the caller includes the implementation

file, it might even happen that the names of the internal variables and functions clash with names used by the caller.

Solution

Provide function declarations in your API for any functionality you want to provide to your user. Hide any internal functions, internal data, and your function definitions (the implementations) in your implementation file and don't provide this implementation file to the user.

In C it is a common convention that somebody who uses functions of your software, only uses functions defined in your header file (*.h file) and does not use other functions in your implementation (in your *.c files). In some cases this abstraction can partly be enforced (for example, someone cannot use a `static` function from another file), but the C language does not support such enforcements to full extent and therefore the convention to not access other implementation files is even more important than the enforcement mechanisms.

Within the header file, make sure to include all artifacts that your functions in the header file need. For example, include other header files for data types needed in the API or include `#defines` needed by the API. It should not be neccessary for your caller to include other header files in order to be able to use the functionality from your header file.

Clearly document the behavior of your API in the header file. The user should not be required to have a look at the implementation in order to understand how the functions provided in the API work.

The following code shows an example for a Header File:

API (h-file)

```
/* Sorts the numbers of the 'array' in ascending order.
   'length' defines the number of elements in the 'array'. */
void sort(int* array, int length);
```

Implementation (c-file)

```
void sort(int* array, int length)
{
  /* here goes the implementation*/
}
```

Consequences

You have a very clear separation between the things relevant for your caller (the *.h file) and the implementation details that the caller does not have to care about (the *.c file). Thus, you abstracted some functionality for the caller.

Having many header files will influence your build times. On the one hand it enables you to split your implementations into separate files and your toolchain will be able to have an incremental build which only rebuilds files that changed. On the other hand, a complete rebuild will have slightly increased build times compared to having all the code in one single file, because for the build all the different files have to be opened and read.

If you discover that your functions require more interaction between one another or that they have to be called in different contexts that require different internal state information, then you have to think about how to realized that with your API. A Handle can help in such cases.

The caller of your functions now relies on the abstraction and might rely on the fact that the behavior of these functions does not change. The API might have to be kept stable. For adding new functionality, it is always an option to add new functions to the API. But in some cases you might want to extend existing functions and to be able to cope with such future changes, you have to consider how to make your functions flexible while keeping them stable. Handles, Dynamic Interfaces or Function Controls can help in such cases.

Known Uses

- Pretty much every C program that is larger than some simple “Hello World” program contains header files.
- The Pimpl Idiom describes to hide private implementation details and to not put them into the header file. You can find a description of that idiom in the Portland Pattern Repository.

Applied to Running Example

Your first device driver API looks like the following:

```
void sendByte(char byte);
char receiveByte();
void setIpAddress(char* ip);
void setMacAddress(char* mac);
```

The user of your API does not have to cope with implementation details like how you access Ethernet registers and you are free to change these details without affecting the user.

Now your requirements for your driver change. Your system has a second, identical Ethernet network interface card and it should be possible to operate both of them. Two straight-forward options to achieve that would be:

- You copy your code and have one piece of code for each network interface card. In the copied code, you only modify the address of the exact interface to be accessed. However, such code duplication is never a good idea and makes maintenance of your code much more difficult.
- You add a parameter to address the network interface card (for example, a device name string) to each function. But quite likely more than just one parameter will have to be shared between the functions and passing each of them to every function makes the usage of your API cumbersome.

A better idea to support multiple Ethernet network interface cards is to introduce Handles to your API.

Handle

Context

You want to provide a set of functions to your caller and these functions operate on shared resources or they share state information.

Problem

You have to share state information or operate on shared resources in your function implementations, but you don't want your caller to see or even access all that state information and shared resources.

That state information and shared resources should remain invisible to your caller, because later on you might want to change it or add to it without requiring any changes to your caller's code.

In object-oriented programming languages, such data on which functions can operate is something realized by class member variables. These class member variables can be made private if the caller should not be able to access them. However, C does not natively support classes and private member variables.

Simply having a Software-Module with Global State holding static global variables in your implementation file for storing shared data between your functions is not an option to you, because it should be possible to call your functions in multiple contexts. The function calls for each of your callers should be able to build up its own state information. And eventhough that information should remain invisible to your callers, you need a way to identify which information belongs to which specific caller and how to access that information in your your function implementations.

Solution

Have a function to create the context on which the caller operates and return an abstract pointer to internal data for that context. Require the caller to pass that pointer to all your functions which can then use the internal data to store state information and resources.

Your functions know how to interpret this abstract pointer, which is an opaque data type als called Handle. However, the data structure that you point to should not be part of the API. The API only provides the functionality to relay some hidden data to the functions.

The Handle can be implemented as a pointer to an Aggregate Instance like a `struct`. The `struct` should contain all required state information or other variables - it usually holds similar variables as you would declare as member variables for objects in object-oriented programming. The `struct` should be hidden in your implementation. The API only contains the definition of a pointer to the `struct` as shown in the following code:

API

```
typedef struct SORT_STRUCT* SORT_HANDLE;

SORT_HANDLE prepareSort(int* array, int length);
void sort(SORT_HANDLE context);
```

Implementation

```
struct SORT_STRUCT
{
  int* array;
  int length;
  int sortOrder;
  /* other parameters*/
};

SORT_HANDLE prepareSort(int* array, int length)
{
  struct SORT_STRUCT* context = malloc(sizeof(struct
SORT_STRUCT));
```

```
  context->array = array;
  context->length = length;

  /* fill context with required data or state information */

  return context;
}

void sort(SORT_HANDLE context)
{
  /* operate on context data */
}
```

Have one function in your API for creating a handle. That function returns the handle to the caller. The caller can then call other functions of your API that require the handle. In most cases, you also need a function to delete the handle by cleaning up all the allocated resources.

Consequences

You can now share state information and resources between your functions without requiring the caller to care about it and without giving the caller the opportunity to make the code depend on these internals.

Multiple instances of data are supported. You can call the function that creates the Handle multiple times to obtain multiple contexts and then you can work with these contexts independently from one another.

In case your functions that operate on the Handle are changed at a later point in time and in case they have to share different or additional data, the members of the `struct` can simply be changed without requiring any changes to the caller's code.

The declaration of your functions explicitly show that they are tightly coupled, because they all require the Handle. That makes it on the one hand easy to see which functions should go into the same Header File and it makes it on the other hand very easy for the caller to spot which functions should be applied together.

With the Handle, you now require the caller to provide one additional parameter to all function calls and each additional parameter makes the code harder to read.

Known Uses

- The C standard library contains the definition of `FILE` in *stdio.h*. This `FILE` is in most implementations defined as a pointer to a `struct` and the `struct` is not part of the header file. The `FILE` handle is created by the function `fopen` and several other functions can then be called for an opened file (`fwrite`, `fread`, …).
- The `struct AES_KEY` in the OpenSSL code is used to exchange the context between several functions related to AES encryption (`AES_set_decrypt_key`, `AES_set_encrypt_key`). The `struct` and its members are not hidden in the implementation, but instead it is part of the header file, because some parts of other OpenSSL code require to know the size of the `struct`.
- The code for the logging functionality of the subversion project operates on a Handle. The `struct logger_t` is defined in the implementation file of the logging functionality and a pointer to this `struct` is defined in the corresponding header file.
- This pattern is described in the book *C Interfaces and Implementations* by David R. Hanson (Addison-Wesley, 1996) as Opaque Pointer Type and in the book *Patterns in C* by Adam Tornhill (Leanpub, 2014) as “First Class Abstract Data Type Pattern”.

Applied to Running Example

You now support as many Ethernet interface cards as you want. Each created instance of your driver creates its own data-context that is then passed to the functions via the Handle. Now you have the following code for your device driver API:

```
/* the INTERNAL_DRIVER_STRUCT contains data shared by the
functions
   (like the way how to select  the interface card the driver is
responsible for) */
typedef struct INTERNAL_DRIVER_STRUCT* DRIVER_HANDLE;

/* 'initArg' contains information for the implementation to
identify
   the exact interface for the driver instance */
DRIVER_HANDLE driverCreate(void* initArg);
void driverDestroy(DRIVER_HANDLE h);
void sendByte(DRIVER_HANDLE h, char byte);
char receiveByte(DRIVER_HANDLE h);
void setIpAddress(DRIVER_HANDLE h, char* ip);
void setMacAddress(DRIVER_HANDLE h, char* mac);
```

Again, your requirements changed. Now you have to support multiple different Ethernet network interface cards, for example, from different vendors. The cards provide similar functionality, but they differ in the details how the registers have to be accessed and thus different implementations for the drivers are needed. Two straight-forward options to support that would be:

- You have two separate driver APIs. This approach has the drawback that for the user it is cumbersome to build mechanisms for selecting the driver at runtime. Also, having two separate APIs duplicates code, because the two device drivers at least share a common control flow (for example, for creating or destroying the driver).
- You add functions like `sendByteDriverA` and `sendByteDriverB` to your API. However, usually you want your API to be rather minimal, because having all driver functions in one single API can be confusing for the API user. Also, the user's code depends on all function signatures included via your API and if code depends on something, that something should be rather minimal (as stated by the interface segregation SOLID principle).

A better idea to support different Ethernet network interface cards is to provide a Dynamic Interface.

Dynamic Interface

Context

You or your caller want to implement multiple functionalities that follow a similar control logic, but that deviate in their behavior.

Problem

It should be possible to call implementations with slightly deviating behaviors, but it should not be necessary to duplicate any code, not even the control logic implementation and interface declaration.

You want to be able to later on add additional implementation behaviors to the declared interface, without requiring callers who use the up to then existing implementation behaviors to change anything in their code.

Maybe you do not only want to provide differing behaviors to your caller without duplicating your own code, but you also want to provide the callers a mechanism to bring in their own implementation behaviors.

Solution

Define a common interface for the deviating functionalities in your API and require the caller to provide a callback function for that functionality which you then call in your function implementation.

To implement such an interface in C, define function signatures in your API. The caller then implements functions according to these signatures and attaches them via function pointers. They can either be attached and stored permanently inside your software-module or they can be attached with each function call as shown in the following code:

API

```
/* The compare function should return true if x<y, else it should
return false */
typedef bool (*COMPARE_FP)(int x, int y);
```

```
void sort(COMPARE_FP compare, int* array, int length);
```

Implementation

```
void sort(COMPARE_FP compare, int* array, int length)
{
  int i, j;
  for(i=0; i<length; i++)
  {
    for(j=i; j<length; j++)
    {
      /* call provided user function */
      if(compare(array[i], array[j]))
      {
        swap(&array[i], &array[j]);
      }
    }
  }
}
```

Make sure to clearly document next to the definition of the function signature, what behavior the function implementations should have. Also, document the behavior in case no such function implementation is attached to your function call. Maybe then you'd abort the program (Samurai Principle) or maybe you'd provide some default functionalty as fallback.

Consequences

The caller can use different implementations and still there is no code duplication. Not even the control logic, the interface, or the interface documentation is duplicated.

Implementations can be added by the caller at a later point in time without changing the API. This means that the role of the API designer and the implementation provider can be completely separated.

In you code, you now execute the caller's code. Thus, you must trust the caller who must exactly know what the function has to do. In case of bugs

in your caller's code, it might still happen that first your code will be suspected, because after all the faulty behavior occurs in the context of your code.

Using function pointers implies that you have a platform-specific and programming-language-specific interface. You can just use this pattern if the caller's code is also written in C. You cannot add marshaling functionality to this interface and provide it to a caller who is for example writing applications with Java code.

Known Uses

- This pattern and a variant are described as in the article *SOLID in C* (*https://www.renaissancesoftware.net/files/articles/ESC-204Paper_Grenning-v1r0.pdf*) as "Dynamic Interface" and "Per-Type Dynamic Interface".
- The presented solution is a C-version of the Strategy design pattern. You can find alternative C implementations of that pattern in the books *Patterns in C* by Adam Tornhill (Leanpub, 2014) and *C Interfaces and Implementations* by David R. Hanson (Addison-Wesley, 1996).
- Device driver frameworks often use function pointers where the driver inserts its function at startup. The device drivers in the Linux kernel usually work that way.
- The function `svn_sort__hash` of the source code of the subversion project sorts a list according to some key value. The function takes the function pointer `comparison_func` as an parameter. The `comparison_func` has to return information, which of two provided key values is greater than the other.
- The OpenSSL function `OPENSSL_LH_new` creates a hash table. The caller has to provide a function pointer to a hash function which is used as a callback when operating on the hash table.

- The wireshark code contains the function pointer `proto_tree_foreach_func` that is provided as a function parameter when traversing tree structures. The function pointer is used to decide which actions to perform on the tree elements.

Applied to Running Example

Your driver API now supports multiple different Ethernet network interface cards. The specific drivers for these network interface cards have to implement the send and receive functions and provide them in a separate header file. The API user can then include and attach these specific send and receive functions to the API.

You have the benefit that users of your API can bring in their own driver implementation. Thus, you as the API designer are independent from the provider of the driver implementation and integrating new drivers does not require any API changes which means it does not require any work from you as the API designer. All that is possible with the following API:

```
typedef struct INTERNAL_DRIVER_STRUCT* DRIVER_HANDLE;

typedef void (*DriverSend_FP)(char byte);       /* this is the
*/
typedef char (*DriverReceive_FP)();             /* interface
definition */

struct DriverFunctions
{
  DriverSend_FP fpSend;
  DriverReceive_FP fpReceive;
};

DRIVER_HANDLE driverCreate(void* initArg, struct DriverFunctions
f);
void driverDestroy(DRIVER_HANDLE h);
void sendByte(DRIVER_HANDLE h, char byte);      /* internally
calls fpSend    */
char receiveByte(DRIVER_HANDLE h);              /* internally
calls fpReceive */
void setIpAddress(DRIVER_HANDLE h, char* ip);
void setMacAddress(DRIVER_HANDLE h, char* mac);
```

Again the requirements changed. Now you don't just have to support Ethernet network interface cards, but also other interface cards (like USB interface cards). From the view of the API, these interfaces have some similar functionalities (the send and receive data functions), but they also have some completely different functionalities (for example, a USB interface has no IP address to set, but might require other configurations).

A straight-forward solution for that would be:

- You provide two different APIs for the different driver types. But this would duplicate code for the send/receive and create/destroy functions.

A good solution to support different kinds of device drivers in one single abstract API is to introduce Function Control.

Function Control

Context

You want to implement multiple functionalities that follow a similar control logic, but that deviate in their behavior.

Problem

You want to call implementations with slightly deviating behaviors, but you don't want to duplicate any code, not even the control logic implementation or the interface declaration.

The caller should be able to use specific existing behaviors that you implemented. It should even be possible for you to later on add new behaviors without touching the already existing implementations and without requiring changes to the existing caller's code.

Having a Dynamic Interface is not an option for you, because you do not want to offer the callers the flexibility of attaching their own implementation. That might be, because the interface should be easier to

use for the caller or maybe, because you cannot easily attach the implementations of your caller, which is the case if your caller for example uses another programming language to access your functionality.

Solution

Apply data-based abstraction. Add a parameter to your function that passes meta-information about the function call and that specifies the actual functionality to be performed.

Compared to a Dynamic Interface, you do not require the caller to provide the implementation, but instead the caller selects from existing implementations.

To implement this pattern, you can add an additional parameter (for example, an `enum` or `#define` integer value) to a function. In the implementation, the parameter is then evaluated and depending on the value of the parameter, different implementations are called:

API

```
#define QUICK_SORT 1
#define MERGE_SORT 2
#define RADIX_SORT 3

void sort(int algo, int* array, int length);
```

Implementation

```
void sort(int algo, int* array, int length)
{
  switch(algo)
  {
    case QUICK_SORT: ❶
      quicksort(array, length);
    break;
    case MERGE_SORT:
      mergesort(array, length);
    break;
    case RADIX_SORT:
```

```
        radixsort(array, length);
      break;
    }
  }
```

❶ When adding new functionality at a later point in time, you can simply add a new `enum` or `#define` value and select the corresponding new implementation.

Consequences

The caller can use different implementations and still there is no code duplication. Not even the control logic, the interface or the interface documentation is duplicated.

It is easy to add new functionality at a later point in time. Existing implementations do not have to be touched to do that and the existing caller's code is not affected by the change.

Compared to Dynamic Interface, this pattern can easier be used to select functionalities across different programs or platforms (for example, remote procedure calls), because no program-specific pointers are passed via the API.

When providing to select different implementation behaviors in one function, you might be tempted to pack multiple functionalities that do not closely belong together into a single function. This violates the single responsibility SOLID principle.

Known Uses

- Device drivers often use Function Control to pass specific functionalities that do not fit into common init/read/write functions. For device drivers this pattern is commonly known as I/O-Control. That concept is described in the book *Making Embedded Systems - Design Patterns for Great Software* by Elecia White (O'Reilly Media, 2011)

- Some Linux syscalls were extended to have flags that extend the syscalls' functionality depending on the value of the flag without breaking old code.
- In general, the concept of data-driven APIs is described in the book *API Design for C++* by Martin Reddy (Morgan Kaufmann, 2011).
- The OpenSSL code uses the function `CTerr` to log errors. This function takes an `enum` parameter to specify how and where the error should be logged.
- The POSIX socket function `ioctl` takes a numeric parameter `cmd` that determines which actual action will be performed on a socket. The allowed values for the parameter are defined and documented in a header file and since the first release of that header file, many additional values and thus function behaviors were added.
- The function `svn_fs_ioctl` of the subversion project performs some file-system-specific input or output operations. The function takes the `struct svn_fs_ioctl_code_t` as a parameter. This `struct` contains a numeric value that determines which kind of operation should be performed.

Applied to Running Example

The following code shows the final version of your device driver API:

Driver.h

```
typedef struct INTERNAL_DRIVER_STRUCT* DRIVER_HANDLE;
typedef void (*DriverSend_FP)(char byte);
typedef char (*DriverReceive_FP)();
typedef void (*DriverIOCTL_FP)(int ioctl, void* context);

struct DriverFunctions
{
  DriverSend_FP fpSend;
  DriverReceive_FP fpReceive;
  DriverIOCTL_FP fpIOCTL;
};
```

```
DRIVER_HANDLE driverCreate(void* initArg, struct DriverFunctions
f);
void driverDestroy(DRIVER_HANDLE h);
void sendByte(DRIVER_HANDLE h, char byte);
char receiveByte(DRIVER_HANDLE h);
void driverIOCTL(DRIVER_HANDLE h, int ioctl, void* context);
/* the parameter "context" is required to pass information like
the
   value of the IP address to configure to the implementation */
```

EthIOCTL.h

```
#define SET_IP_ADDRESS  1
#define SET_MAC_ADDRESS 2
```

UsbIOCTL.h

```
#define SET_USB_PROTOCOL_TYPE   3
```

Usera who want to use the Ethernet- or USB-specific functions (for example, the application actually sending or receiving data via the interface) have to know which driver type they operate on in order to call the right IO-control and additionally have to include the *EthIOCTL.h* or *UsbIOCTL.h* files.

Figure 6-2 shows the include-relationships of the source code files of this final version of our device driver API. Note that the *EthApplication.c* code does not depend on USB-specific things. If, for example, an additional USB-IOCTL is added, the *EthApplication.c* code does not even need to be re-compiled, because none of the files it depends on is changed.

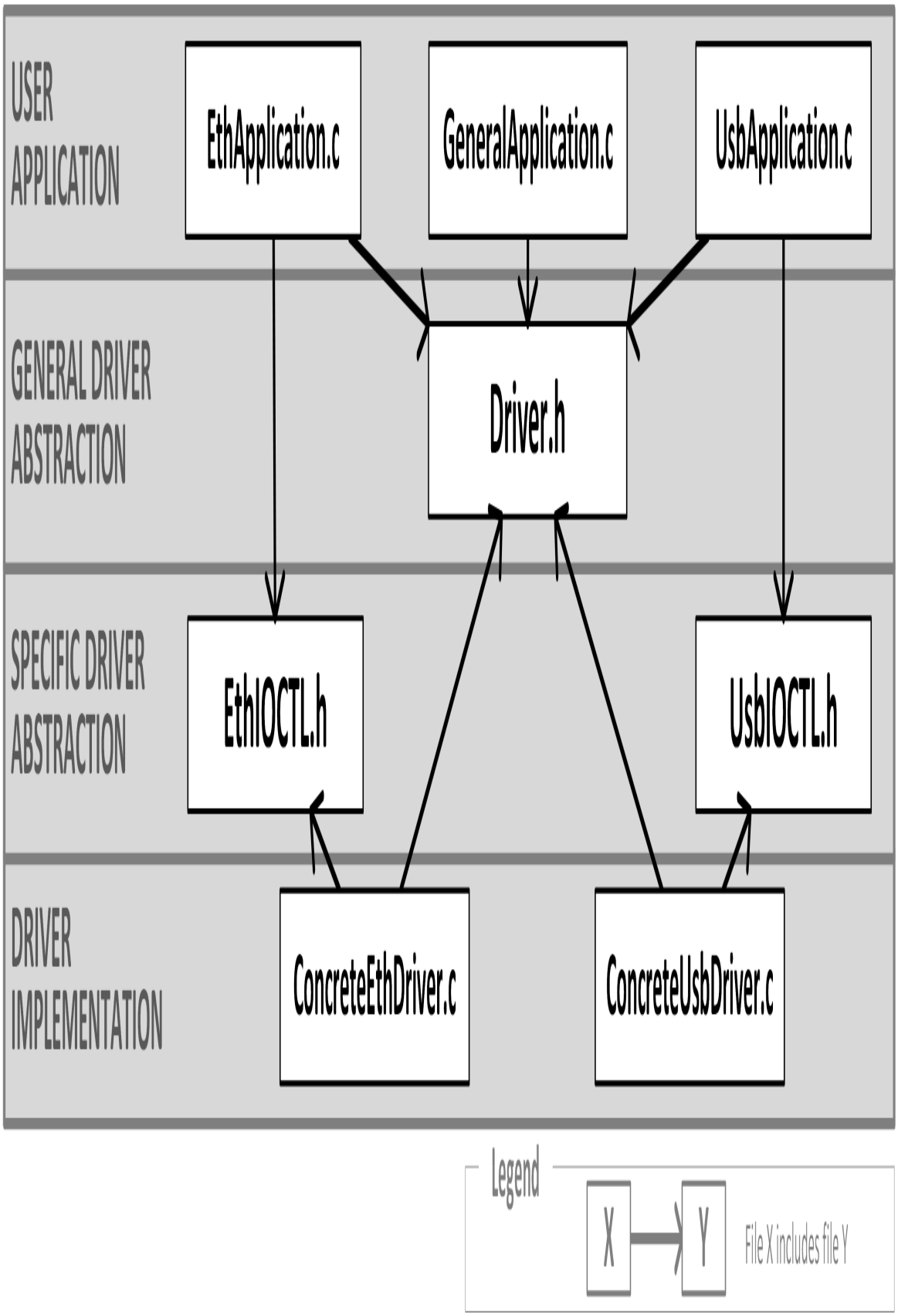

USER APPLICATION
EthApplication.c
GeneralApplication.c
UsbApplication.c
GENERAL DRIVER ABSTRACTION
Driver.h
SPECIFIC DRIVER ABSTRACTION
EthIOCTL.h
UsbIOCTL.h
DRIVER IMPLEMENTATION
ConcreteEthDriver.c
ConcreteUsbDriver.c
Legend
X
Y
File X includes file Y

Figure 6-2. File relationships for function control

Keep in mind that from all the code snippets presented in this chapter, this last, most flexible code snippet of the device drivers might not always be what you are looking for. You buy increased flexibility with complexity of your interface and while you have to make your code as flexible as required, you should also always try to keep it as simple as possible.

Summary

This chapter presented four API patterns for C and showed their application in a running example on how to design a device driver. Header Files tells you the basic concept of hiding implementation details in c-files while providing a well-defined interface in your h-files. The pattern Handle is about the well known concept of passing opaque data types between functions to share state information. Dynamic Interface makes it possible to not duplicate program logic by allowing the injection of caller-specific code via a callback function and Function Control uses an additional function parameter that specifies the actual action that should be performed by the function call. These patterns showed basic C design options to make an interface more flexible by introducing abstractions.

Further Reading

- The article *SOLID in C* (*https://www.renaissancesoftware.net/files/articles/ESC-204Paper_Grenning-v1r0.pdf*), covers the five SOLID design principles in general and presents four ways to implement flexibility for C interfaces. What makes this article unique is that it is the only article that covers the topic of interfaces specifically for C and that also includes detailed code snippets.
- The book “Patterns in C” presents several patterns including C code snippets. The patterns include C-versions of GoF patterns like Strategy

or Observer as well as C-specific patterns and idioms. The book does not explicitly focus on interfaces, but some of the patterns describe interactions on an interface level.

- The book *API Design for C++* by Martin Reddy (Morgan Kaufmann, 2011) covers design principles for interfaces, object-oriented interface patterns with C++ examples, and quality issues with interfaces like testing and documentation. The book addresses C++ design, but some parts of the book are also relevant for C.
- The book *C Interfaces and Implementations* by David R. Hanson (Addison-Wesley, 1996) presents interface design including C code for specific components implemented in C.

Outlook

The next chapter goes into detail on how to find the right level of abstraction and the right interface for one very specific kind of application: it describes how to design and implement iterators.

Chapter 7. Flexible Iterator Interfaces

> **A NOTE FOR EARLY RELEASE READERS**
>
> With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.
>
> This will be the 7th chapter of the final book. Please note that the GitHub repo will be made active later on.
>
> If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

Iterating over a set of elements is a very common operation in any program. Some programming languages provide native constructs to iterate over elements and for object-oriented progamming languages there exists guidance in the form of design patterns on how to implement generic iteration functionality. However, there is just very few guidance of this kind for procedural programming languages like C.

The verb "iterate" means to do the same thing multiple times. For programming, the verb "iterate" usually means to run the same program code on multiple data elements. Such an operation is often required and that's why it is natively supported in C for arrays as shown in the following code:

```
for (i=0; i<MAX_ARRAY_SIZE; i++)
{
  doSomethingWith(my_array[i]);
}
```

If you want to iterate over a different data structure, like a red-black tree for example, then you have to implement your own iteration function and you might equip this function with data-structure-specific iteration options, like whether to traverse the tree depth-first or breadth-first. There is literature available on how to implement such specific data structures and on how the iteration interfaces for these data structures look like. If you use such a data-structure-specific interface for iteration, then in case your underlying data structure changes, you'd have to adapt your iteration function and all your code that calls this function. In some cases that is just fine and even required, because you want to perform some special kind of iteration specific for the underlying data structure - perhaps to optimize the performance of your code.

In some other cases, if you have to provide an iteration interface across component boundaries, having such an abstraction which leaks implementation details isn't an option, because it might require interface changes in future. For example, if you sell your customers a component providing iteration functions and your customers write code using these functions, then quite likely your customers expect their code to work without any modification if you provide them with a newer version of your component that maybe uses a different data structure. In that case you'd even accept putting some extra effort into your implementation to make sure that the interface to the customers stays compatible so that they do not have to change (or maybe not even recompile) their code.

That is where we start in this chapter. I'll show you three patterns on how you, as the iterator implementer, can provide stable iterator interfaces to the user (the customer). The patterns do not describe the specific kinds of iterators for specific kinds of data structures. Instead, the patterns assume that within your implementation you already have functions to retrieve the elements from your underlying data structure and the patterns show the options you have to abstract these functions in order to provide a stable iteration interface.

Figure 7-1 shows an overview of the patterns presented in this chapter and their relationships and Table 7-1 provides a summary of the patterns.

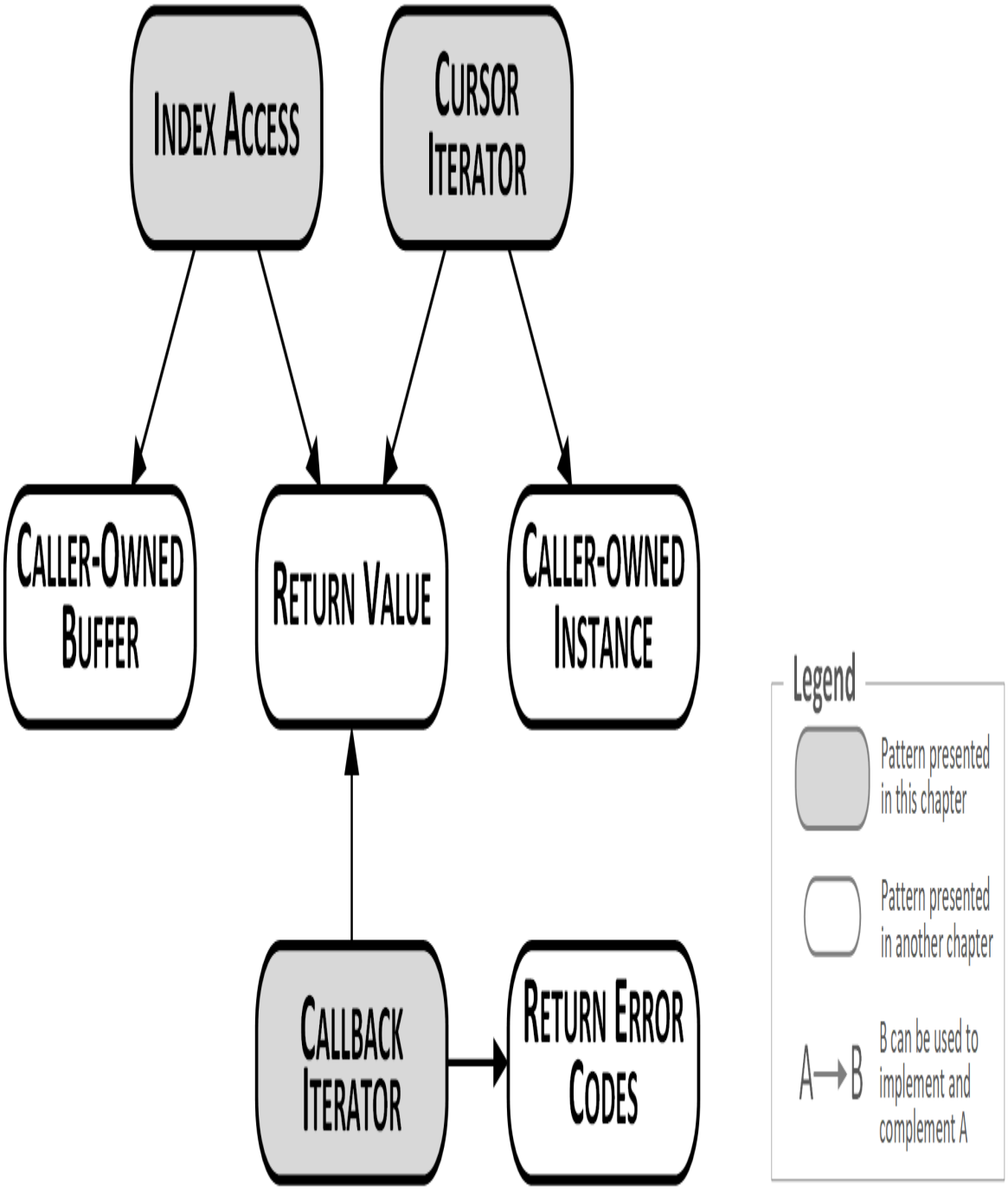


Figure 7-1. Overview of patterns for iterator interfaces

Table 7-1. Patterns for iterator in

t
e
r
f
a
c
e
s

	Pattern Name	Summary
	Index Access	You want to make it possible for the user to iterate elements in your data structure in a convenient way, and it should be possible to change internals of the data structure without resulting in changes to the user's code. Therefore, provide a function that takes an index to address the element in your underlying data structure and return the content of this element. The user calls this function in a loop to iterate over all elements.
	Cursor Iterator	You want to provide an iteration interface to your user which is robust in case the elements change during the iteration and which enables you to change the underlying data structure at a later point in time without requiring any changes to the user's code. Therefore, create an iterator instance that points to an element in the underlying data structure. An iteration function takes this iterator instance as argument, retrieves the element the iterator currently points to, and modifies the iteration instance to point to the next element. The user then iteratively calls this function to retrieve one element at a time.
	Callback Iterator	You want to provide a robust iteration interface which does not even require the user to implement a loop in the code for iterating over all elements and and which enables you to change the underlying data structure at a later point in time without requiring any changes to the user's code. Therefore, use your existing data-structure-specific operations to iterate over all your elements within your implementation and call some provided user-function on each element during this iteration. This

user-function gets the element content as a parameter and can then perform its operations on this element. The user just calls one function to trigger the iteration and the whole iteration takes place inside your ismplementation.

Running Example

You implemented an access control component for your application and as part of this access control component you implemented an underlying data structure where you have a function to randomly access any of the elements. More specifically, in the following code you have a struct array that holds account information like login names and passwords:

```
struct ACCOUNT
{
  char loginname[MAX_NAME_LENGTH];
  char password[MAX_PWD_LENGTH];
};
struct ACCOUNT accountData[MAX_USERS];
```

The next code shows how users can access this struct to read specific information like the login names:

```
void accessData()
{
  char* loginname;

  loginname = accountData[0].loginname;
  /* do something with loginname */

  loginname = accountData[1].loginname;
  /* do something with loginname */
}
```

Of course, you could simply not care about abstracting access to your data structure and let other programmers directly retrieve a pointer to this struct to loop over the struct elements and to access any information in the struct.

But that would be a very bad idea, because there might be information in your data structure that you do not want to provide to the client. If you have to keep your interface to the client stable over time, you won't be able to remove information you once revealed to the client, because your client might use that information and you don't want to break the client's code.

To avoid this problem, a much better idea is to let the user only access the required information. A simple solution for that is to provide Index Access.

Index Access

Context

You have a set of elements stored in a data structure that can be randomly accessed. For example, you have an array or a database with functions to randomly retrieve single elements. A user wants to iterate these elements.

Problem

You want to make it possible for the user to iterate elements in your data structure in a convenient way, and it should be possible to change internals of the data structure without resulting in changes to the user's code.

The user might be somebody who writes code that is not versioned and released with your code-base, so you have to make sure that future versions of your implementation also work with the user code written against the current version of your code. Thus, the user should not be able to access any internal implementation details, such as the underlying data structure you use to hold your elements, because you might want to change that at a later point in time.

Solution

Provide a function that takes an index to address the element in your underlying data structure and return the content of this element. The user calls this function in a loop to iterate over all elements as shown in Figure 7-2.

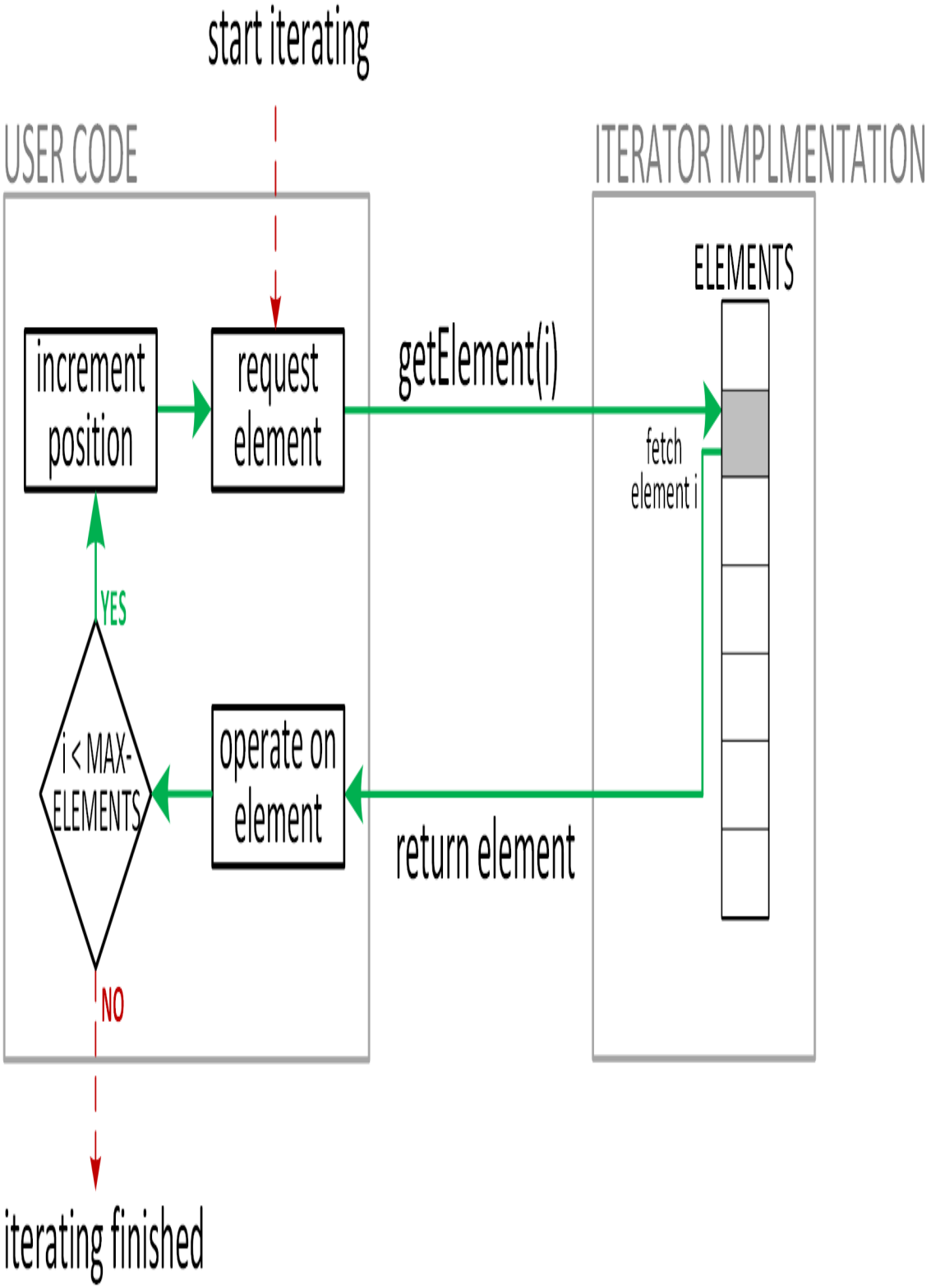

start iterating
USER CODE
ITERATOR IMPLMENTATION
ELEMENTS
increment position
request element
getElement(i)
fetch element i
YES
i < MAX-ELEMENTS
operate on element
return element
NO
iterating finished

Figure 7-2. Index accessed iteration

The equivalent to that approach would be that in an array, the user would simply use an index to retrieve the value of one array element or to iterate over all elements. But when having a function that takes such an index, also more complex underlying data structures are possible to iterate without requiring the user to know that.

In order to achieve that, only provide the users the data they are interested in and do not reveal all elements of your underlying data structure. For example, do not return a pointer to the whole struct element, but only return a pointer to the struct member the user is interested in:

Caller's code

```
void* element;

element = getElement(1);
/* operate on element 1 */

element = getElement(2);
/* operate on element 2 */
```

Iterator API

```
#define MAX_ELEMENTS 42

/* Retrieve one single element identified by the provided 'index'
*/
void* getElement(int index);
```

Consequences

The users can retrieve the elements by using the index to conveniently loop over the elements in their code. They do not have to cope with the internal data structure from which this data was gathered. If something in the implementation changes (for example, the retrieved struct member is renamed), the users need not even recompile their code.

Other changes to the underlying data structure might turn out to be more difficult. If for example, the underlying data structure changes from an array (randomly accessible) to a linked list (sequentially accessible), then you'd have to iterate the list each time until you get to the requested index. That would not be efficient at all and to make sure to also allow such changes in the underlying data structure, it would be better to use a Cursor Iterator or Callback Iterator instead.

If the user just retrieves basic data types that can be transported as Return Value of a C function, then the user implicitly retrieves a copy of this element and if the corresponding element in the underlying data structure changes in the meantime, then this would not affect the user. But if the user retrieves a more complex data type (like a string), then compared to simply providing direct access to the underlying data structure, you have with Index Access the advantage that you can copy the current data element in a thread-safe way and provide it to the user, for example with a Caller-Owned Buffer. If you are not operating in a multi-threaded environment, you could for complex data types simply return a pointer.

Quite often when accessing a set of elements, the user wants to iterate over all elements. If somebody else adds or removes an element in the underlying data in the meantime, then the user's understanding of the index to access the elements might become invalid and the user might unintentionally retrieve an element twice during the iteration. A straight forward solution to that would be to simply copy all elements the user is interested in into an array and provide this exclusive array to the user who can then conveniently loop over this array. But such a solution requires copying many elements. A much more convenient solution where the user does not have to worry about changes of the underlying data order during iteration is to provide a Callback Iterator instead.

Known Uses

- James Noble describes the External Iterator pattern in his article *Iterators and encapsulation*

(*https://dl.acm.org/doi/10.5555/832260.833174*). This is an object-oriented version of the concept described in this pattern.

- The book *Data Structures and Problem Solving Using Java* by Mark Allen Weiss (Addison Wesley, 2006) describes this approach and calls it access with an array-like interface.
- The function `service_response_time_get_column_name` of the Wireshark code returns the name of columns for a statistics table. The name to be returned is addressed with an index parameter provided by the user. The column names cannot change at runtime and therefore even in multi-threaded environments this way of accessing the data or iterating over column names is safe.
- The Subversion project contains code that is used to build up a table of strings. These strings can be accessed with the function `svn_fs_x__string_table_get`. This function takes an index as parameter that is used to address the string to be retrieved. The retrieved string is copied into a provided buffer.
- The OpenSSL function `TXT_DB_get_by_index` retrieves a string selected with an index from a text database and stores it into a provided buffer.

Applied to Running Example

Now you have a clean abstraction for reading the login names:

```
char* getLoginName(int index)
{
  return accountData[index].loginname;
}
```

Users do not have to cope with accessing the underlying `struct` array. That has the advantage that the access to the required data is easier for the users and that they cannot use any information that is not intended for them. For example, they cannot access sub-elements of your struct that you might

want to change in future and that can only be changed if nobody accesses this data because you do not want to break the users' code.

Somebody using this interface, like somebody who wants to write a function that checks whether there is any login-name starting with the letter "X", writes the following code:

```
bool anyoneWithX()
{
  int i;
  for(i=0; i<MAX_USERS; i++)
  {
    char* loginName = getLoginName(i);
    if(loginName[0] == 'X')
    {
      return true;
    }
  }
  return false;
}
```

You are very happy with your implementation until the data structure that you use to store the login names changes, because you need a more convenient way to insert and delete account data which is quite difficult when storing the data in a plain array. Now the login names are not stored in a single plain array anymore, but instead they are now stored in an underlying data structure offering you an operation to get from one element to the next one, but not offering you an operation to randomly access elements. More specifically you have a linked list that can be accessed as shown in the following code:

```
struct ACCOUNT_NODE
{
  char loginname[MAX_NAME_LENGTH];
  char password[MAX_PWD_LENGTH];
  struct ACCOUNT_NODE* next;
};

struct ACCOUNT_NODE* accountList;

struct ACCOUNT_NODE* getFirst()
```

```
{
  return accountList;
}

struct ACCOUNT_NODE* getNext(struct ACCOUNT_NODE* current)
{
  return current->next;
}

void accessData()
{
  struct ACCOUNT_NODE*  account = getFirst();
  char* loginname = account->loginname;
  account = getNext(account);
  loginname = account->loginname;
  ...
}
```

That makes the situation difficult with your current interface. Your current interface provides one randomly index-accessed login-name at a time. To further support that, you'd have to emulate the index by calling the `getNext` function and count until you reach the indexed element. That is quite inefficient. All that hassle is just necessary because you designed the interface in a way that turned out to be not very clever.

To make things easier, provide a Cursor Iterator to access the login names.

Cursor Iterator

Context

You have a set of elements stored in a data structue that can be accessed randomly or sequentially. For example, you have an array, a linked list, a hash map, or a tree data structure. A user wants to iterate these elements.

Problem

You want to provide an iteration interface to your user which is robust in case the elements change during the iteration and which enables you

to change the underlying data structure at a later point in time without requiring any changes to the user's code.

The user might be somebody who writes code that is not versioned and released with your code-base, so you have to make sure that future versions of your implementation also work with the user code written against the current version of your code. Thus, the user should not be able to access any internal implementation details, such as the underlying data structure you use to hold your elements, because you might want to change that at a later point in time.

Aside from that, when operating in multi-threaded environments, you want to provide the user a robust and clearly defined behavior if the element's content changes while the user iterates over them. Even for complex data like strings the user should not have to worry about other threads changing that data while the user wants to read it.

You don't care very much if you have to take some extra implementation effort to achieve all this, because many users will use your code and if you can take implementation effort away from the user by implementing it in your code, then the overall effort will be decreased.

Solution

Create an iterator instance that points to an element in the underlying data structure. An iteration function takes this iterator instance as argument, retrieves the element the iterator currently points to, and modifies the iteration instance to point to the next element. The user then iteratively calls this function to retrieve one element at a time as shown in Figure 7-3.

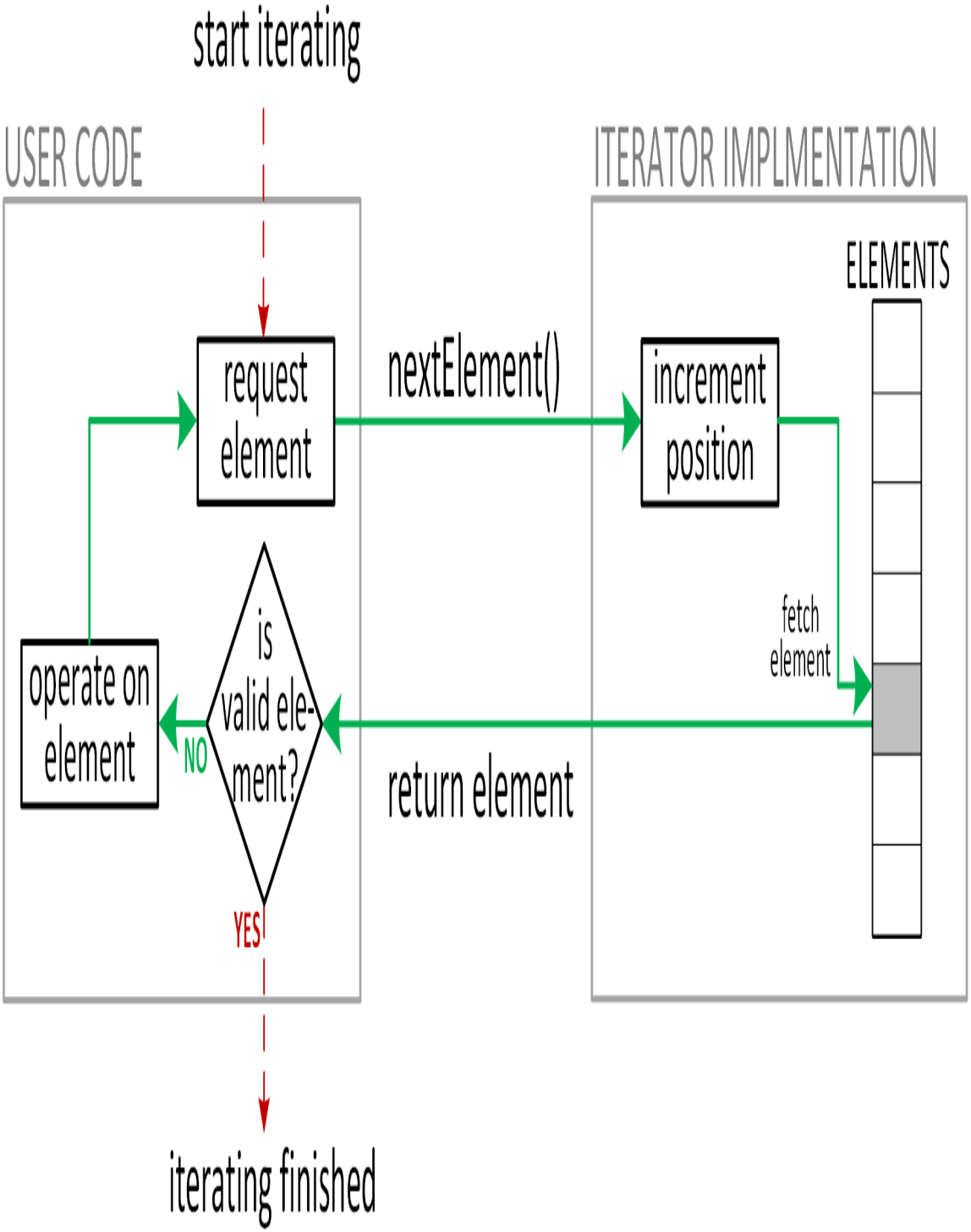


Figure 7-3. Iteration with a cursor iterator

The iterator interface requires two functions to create and destroy the iterator instance and one function to perform the actual iteration and to retrieve the current element. Having explicit create/destroy functions makes it possible to have an instance where you store your internal iteration data (position, data of the current element). The user then has to pass this instance to all your iteration function calls as shown in the following code:

Caller's code

```
void* element;
ITERATOR* it = createIterator();

while(element = getNext(it))
{
  /* operate on element */
}

destroyIterator(it);
```

Iterator API

```
/* Creates an iterator and moves it to the first element */
ITERATOR* createIterator();

/* Returns the element currently pointed to and sets the iterator
to the
   next element. Returns NULL if the element does not exist. */
void* getNext(ITERATOR* iterator);

/* Cleans up an iterator created with the function
createIterator() */
void destroyIterator(ITERATOR* iterator),
```

If you do not want the user to be able to access this internal data, then you can hide it and provide the user a Handle instead. That makes it possible that even changes to this internal data of the iteration instance do not affect the user.

When retrieving the current element, basic data types can directly be provided as Return Value. Complex data types can either be returned as a

reference or can be copied into the iterator instance. Copying it into the iterator instance, brings the advantage that the data is consistent, even if the data in the underlying data structure changes in the meantime (for example, because it is being modified by someone else in a multi-threaded environment).

Consequences

The user can iterate the data by simply calling the `getNext` method as long as valid elements are retrieved. The user does not have to cope with the internal data structure from which this data was gathered and does not have to care about an element index or about the maximum number of elements. But not being able to index the elements also means that the user cannot randomly access the elements (which could be done with Index Access).

Even if the underlying data structure changes, for example, from a linked list to a random accessible data structure like an array, then that change can be hidden in the iterator implementation and the user need not change or recompile code.

No matter which kind of data the user retrieves - simple or complex data types - the user need not be afraid that the retrieved element becomes invalid in case the underlying element is changed or removed in the meantime. To make that possible, the user now has to explicitly call functions to create and destroy the iterator instance. Compared to Index Access more function calls are necessary.

Quite often when accessing a set of elements, the user wants to iterate over all elements. If somebody else adds an element to the underlying data in the meantime, then the user might miss this element during the iteration. If that is a problem for you and you want to make sure that the elements do not change at all during the iteration, then it is easier to use a Callback Iterator.

Known Uses

- James Noble describes in his article *Iterators and Encapsulation* (*https://dl.acm.org/doi/10.5555/832260.833174*) an object-oriented version of this iterator as the Magic Cookie pattern.
- The Article *Interruptable Iterators* by Jed Liu et al. (*https://dl.acm.org/doi/10.1145/1111320.1111063*) describes the presented concapt as “Cursor Object”.
- This kind of iteration is used for file access. For example, the `getline` C function iterates over the lines in a file and the iteratior position is stored in the `FILE` pointer.
- The OpenSSL code provides the function `ENGINE_get_first` and `ENGINE_get_next` to iterate a list of encryption engines. Each of these calls takes the pointer to an `ENGINE struct` as a parameter. This `struct` stores the current position in the iteration.
- The Wireshark code contains the function `proto_get_first_protocol` and the function `proto_get_next_protocol`. These functions make it possible for a user to iterate over a list of network protocols. The functions take a `void` pointer as out-parameter to store and pass along state information.
- The code of the Subversion project for generating diffs between files contains the function `datasource_get_next_token`. This function is to be called in a loop in order to get the next diff token from a provided datasource object that stores the iteration position.

Applied to Running Example

You now have the following function to retrieve the login names:

```
struct ITERATOR
{
  char buffer[MAX_NAME_LENGTH];
  struct ACCOUNT_NODE* element;
};
```

```
struct ITERATOR* createIterator()
{
  struct ITERATOR* iterator = malloc(sizeof(struct ITERATOR));
  iterator->element = getFirst();
  return iterator;
}

char* getNextLoginName(struct ITERATOR* iterator)
{
  if(iterator->element != NULL)
  {
    strcpy(iterator->buffer, iterator->element->loginname);
    iterator->element = getNext(iterator->element);
    return iterator->buffer;
  }
  else
  {
    return NULL;
  }
}

void destroyIterator(struct ITERATOR* iterator)
{
  free(iterator);
}
```

The following code shows how this interface is used:

```
bool anyoneWithX()
{
  char* loginName;
  struct ITERATOR* iterator = createIterator();
  while(loginName = getNextLoginName(iterator)) ❶
  {
    if(loginName[0] == 'X')
    {
      destroyIterator(iterator); ❷
      return true;
    }
  }
  destroyIterator(iterator); ❷
  return false;
}
```

❶

The application does not have to cope with the index and with the maximum number of elements anymore

❷ The required cleanup-code for destroying the iterator in this case leads to code duplication.

Next, you don't just want to implement the `anyoneWithX` function, but you also want to implement an additional function that, for example, tells you how many login names start with the letter "Y". You could simply copy the code and modify the body of the `while` loop and count the occurrence of "Y", but with this approach you'll end up with duplicated code, because both of your functions will contain the same code for creating and destroying the iterator and for performing the loop operation. To avoid this code duplication, you can use a Callback Iterator instead.

Callback Iterator

Context

You have a set of elements stored in a data structure that can be accessed randomly or sequentially. For example, you have an array, a linked list, a hash map, or a tree data structure. A user wants to iterate these elements.

Problem

You want to provide a robust iteration interface which does not even require the user to implement a loop in the code for iterating over all elements and and which enables you to change the underlying data structure at a later point in time without requiring any changes to the user's code.

The user might be somebody who writes code that is not versioned and released with your code-base, so you have to make sure that future versions of your implementation also work with the user code written against the current version of your code. Thus, the user should not be able to access any internal implementation details, such as the underlying data structure

you use to hold your elements, because you might want to change that at a later point in time.

Aside from that, when operating in multi-threaded environments, you want to provide the user a robust and clearly defined behavior if the element's content changes while the user iterates over them. Even for complex data like strings the user should not have to worry about other threads changing that data while the user wants to read it. Also, you want to make sure that the user iterates over each element exactly once. That should hold, even if other threads try to create new elements or to delete existing elements during the iteration.

You don't care very much if you have to take some extra implementation effort to achieve all this, because many users will use your code and if you can take implementation effort away from the user by implementing it in your code, then the overall effort will be decreased.

You want to make access to your elements as easy as possible. In particular, the user shouldn't have to cope with iteration details like mappings between index and element or like the number of available elements. Also the user shouldn't have to implement loop code, because that would lead to duplications in the user code, so Index Access or a Cursor Iterator isn't an option for you.

Solution

Use your existing data-structure-specific operations to iterate over all your elements within your implementation and call some provided user-function on each element during this iteration. This user-function gets the element content as a parameter and can then perform its operations on this element. The user just calls one function to trigger the iteration and the whole iteration takes place inside your implementation as shown in Figure 7-4.

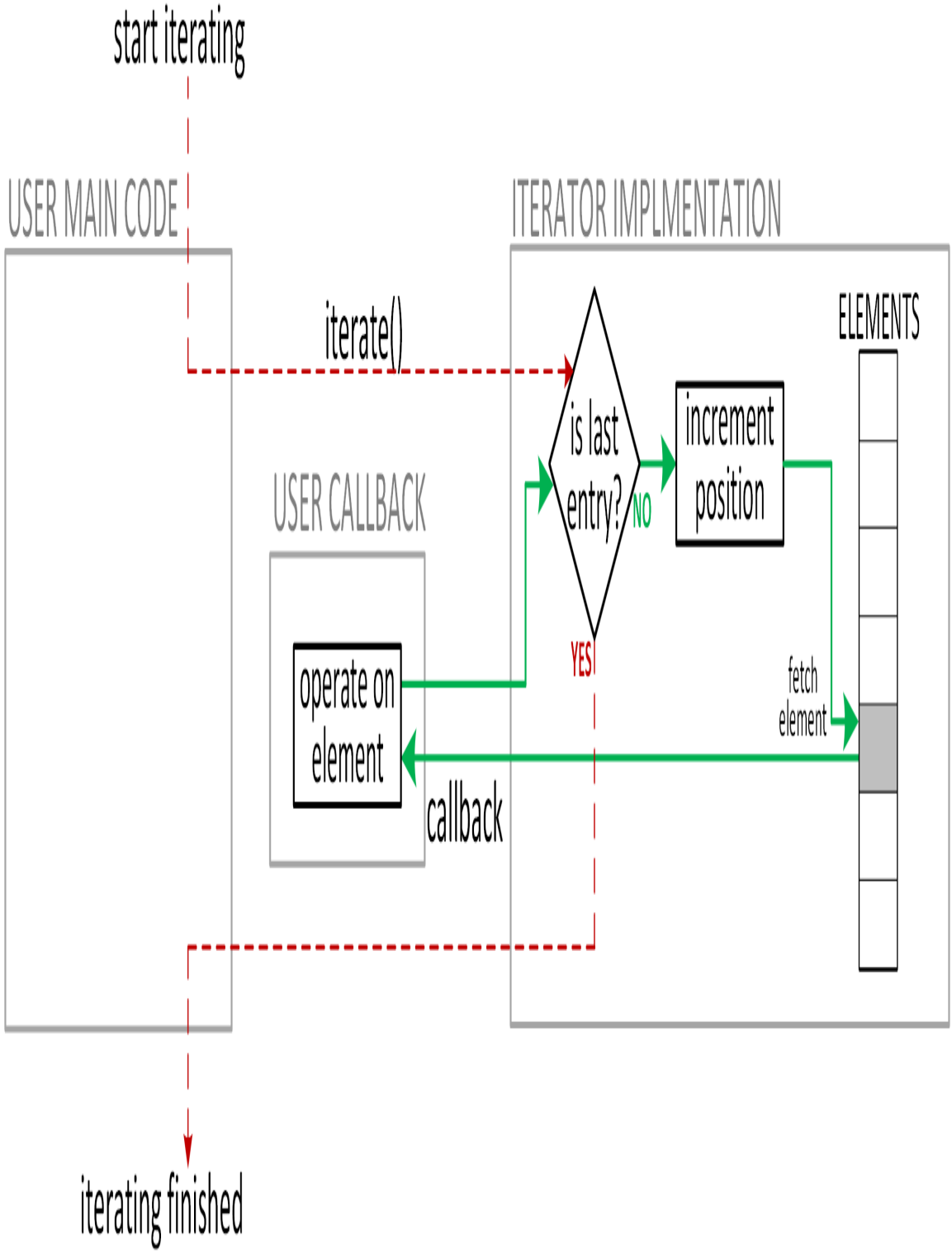


Figure 7-4. Iteration with a callback iterator

To realize that, you have to declare a function pointer in your interface. The declared function takes an element that should be iterated over as parameter. The user implements such a function and passes it to your iteration function. Within your implementation you iterate over all elements and you'll call the user's function for each element with the current element as parameter.

You can add an additional `void*` parameter to your iteration function and to the function pointer declaration. In the implementation of your iteration function, you simply pass that parameter to the user's function. That makes it possible for the user to pass some context information to the function:

Caller's code

```
void myCallback(void* element, void* arg)
{
  /* operate on element */
}

void doIteration()
{
  iterate(myCallback, NULL);
}
```

Iterator API

```
/* Callback for the iteration to be implemented by the caller. */
typedef void (*FP_CALLBACK)(void* element, void* arg);

/* Iterates over all elements and calls callback(element, arg)
   on each element. */
void iterate(FP_CALLBACK callback, void* arg);
```

Sometimes the user does not want to iterate over all elements, but wants to find one specific element. To make that use case more efficient, you can add a break condition to your iteration function. For example, you can declare the function pointer for the user function that operates on the elements of return type `bool` and if the user function returns the Return

Value `true` you stop the iteration. Then the user can signal as soon as the desired element is found and it saves the time it would take for iterating all the rest of the elements.

When implementing the iteration function for multi-threaded environments, then make sure to cover the case if during the iteration the current element is changed, new elements are added, or elements are deleted by other threads. In case of such changes you could Return Error Codes to the user who currently iterates or alternatively you could prevent such changes during an iteration by locking write access to the elements in the meantime.

Because the implementation can ensure that the data is not changed during the iteration, it is not necessary to copy the elements on which the user operates. The user simply retrieves a pointer to this data and works with the original data.

Consequences

The user code for iterating over all elements is now just one single line of code. All the implementation details like an element index and the maximum number of elements are hidden inside the iterator implementation and the user does not even has to implement a loop to iterate over the elements. Also the user does not have to create or destroy an iterator instance and does not have to cope with the internal data structure from which the elements are gathered. Even if you change the type of underlying data structure in your implementation, the user need not even recompile the code.

If the underlying elements change during an iteration, then the iterator implementation can react accordingly, which ensures the user to iterates over a consistent set of data while not having to cope with locking functionality in the user-code. All this is possible, because the control flow does not jump between the user-code and the iterator-code. The control flow stays inside the iterator implementation and thus the iterator implementation can detect if elements are changed during the iteration and can react accordingly.

The user can iterate over all elements, but the iteration loop is implemented inside the iterator implementation, so the user cannot randomly access elements like with Index Access.

In the callback your implementation runs user-code on each element. To some extent this means that you have to trust the user that the code does the right thing. For example, if your iterator implementation locks all elements during the iteration, then you expect the user-code to quickly do something with the retrieved element and to not do any time-consuming operations, because during this iteration, all other calls accessing this data will be locked.

Using callbacks implies that you have a platform-specific and programming-language-specific interface, because you call the code implemented by your caller and you can just do that if that code uses the same calling conventions (the same way of providing function parameters and returning data). That means, for implementing an iterator in C, you can just use this pattern if the user code is also written in C. You cannot provide a C Callback Iterator, for example, to a user writing code with Java (which could with some effort be done with any of the other iterator patterns).

When reading the code, the program flow with callbacks is more difficult to follow. For example, compared to having a simple `while` loop directly in the code, it might be more difficult to find out that the program iterates over elements when only seeing one single line of user code with a callback parameter. Thus, it is very important to give the iteration function a name that makes very clear that this function performs an iteration.

Known Uses

- James Noble describes in his article *Iterators and encapsulation* (*https://dl.acm.org/doi/10.5555/832260.833174*) an object-oriented version of this iterator as the Internal Iterator pattern.
- The function `svn_iter_apr_hash` of the Subersion project iterates over all elements in a hash table that is provided to the function

as a parameter. For each element of the hash table, a function pointer, which has to be provided by the caller, is called and if that call returns `SVN_ERR_ITER_BREAK`, the iteration is stopped.

- The OpenSSL function `ossl_provider_forall_loaded` iterates over a set of OpenSSL provider objects. The function takes a function pointer as a parameter and that function pointer is called for each provider object. A `void*` parameter can be provided to the iteration call and this parameter is then provided for each call in the iteration so that the users can pass their own context.
- The Wireshark function `conversation_table_iterate_tables` iterates through a list of "conversation" objects. Each such object stores information about sniffed network data. The function takes a function pointer and a `void*` as parameters. For each conversation object, the function pointer is called with the `void*` as context.

Applied to Running Example

You now provide the following function for accessing the login names:

```
typedef void (*FP_CALLBACK)(char* loginName, void* arg);

void iterateLoginNames(FP_CALLBACK callback, void* arg)
{
  struct ACCOUNT_NODE* account = getFirst(accountList);
  while(account != NULL)
  {
    callback(account->loginname, arg);
    account = getNext(account);
  }
}
```

The following code shows how to use this interface:

```
void findX(char* loginName, void* arg)
{
  bool* found = (bool*) arg;
```

```
  if(loginName[0] == 'X')
  {
    *found = true;
  }
}

void countY(char* loginName, void* arg)
{
  int* count = (int*) arg;
  if(loginName[0] == 'Y')
  {
    (*count)++;
  }
}

bool anyoneWithX()
{
  bool found=false;
  iterateLoginNames(findX, &found); ❶
  return found;
}

int numberOfUsersWithY()
{
  int count=0;
  iterateLoginNames(countY, &count); ❶
  return count;
}
```

❶ The application does not contain any explicit loop statement anymore.

As a possible enhancement, the callback function could have a return value that determines whether the iteration is continued or whether the iteration is stopped. With such a return value, the iteration could for example be stopped once the `findX` function iterates over the first user starting with "X".

Summary

This chapter showed you three different ways to implement interfaces that provide iteration functionality. Table 7-2 gives an overview of the three patterns and compares their consequences.

Table 7-2. Comparison of the iterator to

r

p
a
t
t
e
r
n
s

	Index Access	Cursor Iterator	Callback Iterator
Element Access	allows random access	only sequential access	only sequential access
Data structure changes	underlying data structure can only easily be changed to another random-acess data structure	underlying data structure can easily be changed	underlying data structure can easily be changed
Info leaked through interface	amount of elements; usage of a random access data structure	iterator position (user can stop and continue the iteration at a later point in time)	-
Code duplication	loop in user-code; index increment in user code	loop in user-code	-
Robustness	difficult to implement robust iteration behavior	difficult to implement robust iteration behavior	easy to implement robust iteration behavior, because control flow stays within the iteration code and insert/delete/modify operations can simply be locked during the iteration (but would block other iterations for that time)
Platforms	interface can be	interface can be	can only be used with the same

	used across different languages and platforms	used across different languages and platforms	language and platform (with the same calling convention) as the implementation

Further Reading

- The most closely related work regarding iterators in C is an online version of university class notes by James Aspnes (*http://www.cs.yale.edu/homes/aspnes/pinewiki/C(2f)Iterators.html*). The class notes describe different C iterator designs, discuss their advantages and disadvantages, and provide source code examples.
- There is more guidance on iterators for other programming languages, but many of the concepts can also be applied to C. For example, the article *Interators and encapsulation* by James Noble (*https://dl.acm.org/doi/10.5555/832260.833174*) describes 8 patterns on how do design object-oriented iterators, the book *Data Structures and Problem Solving Using Java* by Mark Allen Weiss (Addison Wesley, 2006) describes different iterator designs for Java, and the book *Higher-Order Perl* by Mark Jason Dominus (Powell's Books, 2005) describes different iterator designs for Perl.
- The article *Loop Patterns* by Owen Astrachan and Eugene Wallingford (*https://users.cs.duke.edu/~ola/patterns/plopd/loops.html*) contains patterns that describe best practices for implementing loops and that include C++ and Java code snippets. Most of the ideas are also relevant for C.
- The book *C Interfaces and Implementations* by David R. Hanson (Addison-Wesley, 1996) describes C implementations and their interfaces for several common data structures like linked lists or hash tables. These interfaces of course also contain functions how to traverse these data structures.

Outlook

The next chapter focuses on how to organize the code files in large programs. Once the interfaces are defined and build the base for constructing modular programs, their file organization has to be tackled for implementing modular, large scale programs.

Chapter 8. Organizing Files in Modular Programs

> **A NOTE FOR EARLY RELEASE READERS**
>
> With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.
>
> This will be the 8th chapter of the final book. Please note that the GitHub repo will be made active later on.
>
> If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

Any programmer who implements a larger piece of software and who wants to make that software maintainable comes across the question of how to make the software modular. The most important part of that question that is related to dependencies between software-modules is answered, for example, by the SOLID design principles described in the book *Clean Code: A Handbook of Agile Software Craftsmanship* by Robert C. Martin (Prentice Hall, 2008) or by the design patterns described in the book *Design Patterns: Elements of Reusable Object-Oriented Software* by the "Gang of Four" (Prentice Hall, 1997).

However, making software modular also raises the question of how to organize the source files in a way that allows someone to make the software modular. That question is not yet answered very well and that results in bad file structures in codebases. It is difficult to make such codebases modular later on, because you don't know which files you should separate into

different software-modules or into different codebases. Also, as a programmer it is difficult to find the files containing APIs that you are supposed to use and thus you might bring in dependencies to APIs that you are not supposed to use. In particular for C that is an issue, because C does not support any mechanism to mark APIs for internal use only and to restrict access to them.

For some other programming languages, there are such mechanisms and there is advice on how to structure files. For example, the Java programming language comes with the concept of “packages”. For these packages, Java provides a default way for the developer to organize the classes and thus the files within the package. For other programming languages, like for C, there is no such advice on how to structure files and developers have to come up with their own approach of how to structure the header files containing the C function declarations and the implementation files containing the C function definitions.

This chapter shows how to tackle this problem by providing guidance for C programmers on how to structure implementation files and in particular on how to structure header files (APIs) in order to allow developing large modular C programs.

Figure 8-1 shows an overview of the patterns presented in this chapter and Table 8-1 provieds a short description of these patterns.

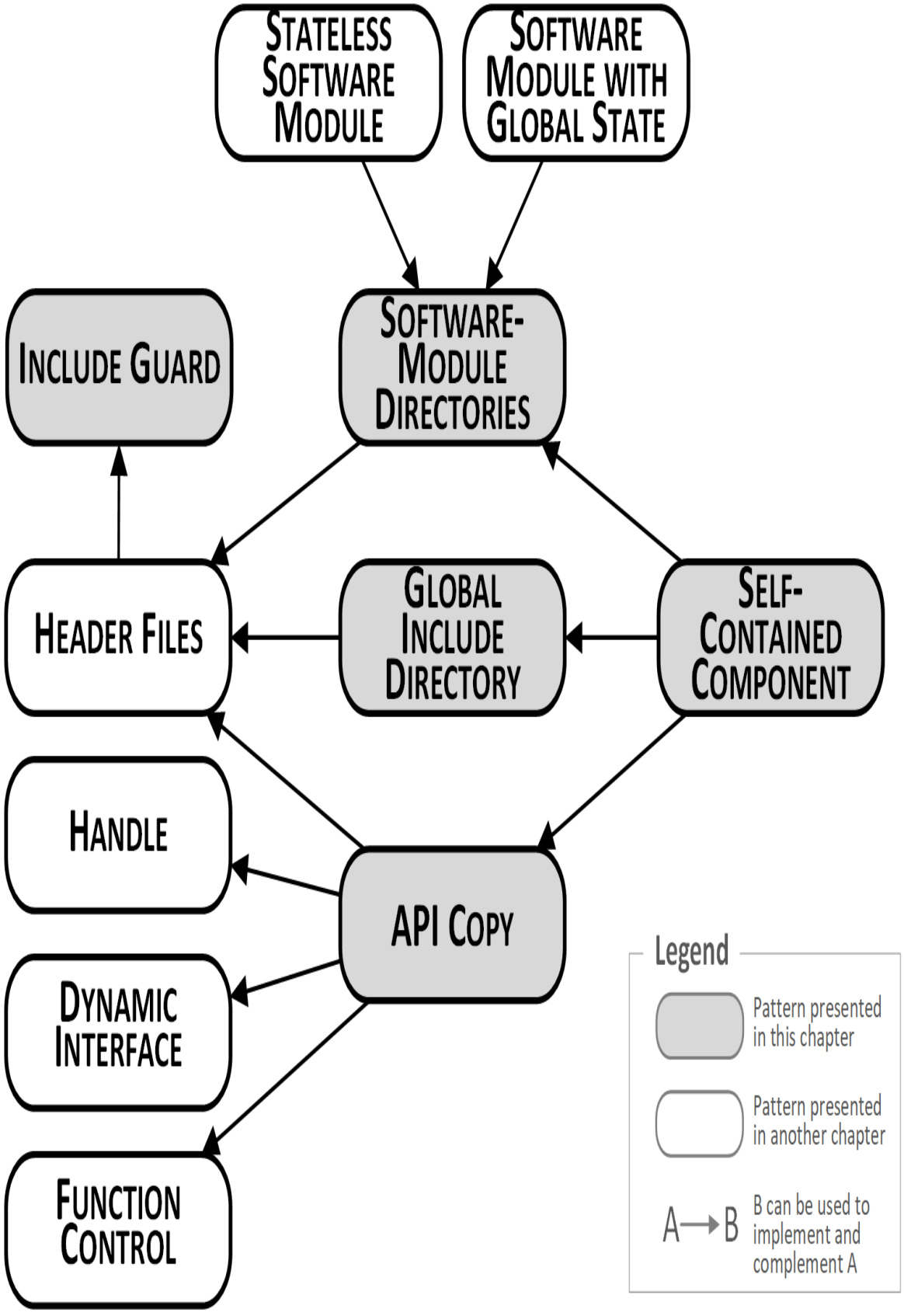

Stateless Software Module
Software Module with Global State
Include Guard
Software-Module Directories
Header Files
Global Include Directory
Self-Contained Component
Handle
API Copy
Dynamic Interface
Function Control
Legend
Pattern presented in this chapter
Pattern presented in another chapter
A → B
B can be used to implement and complement A

Figure 8-1. Overview of patterns on how to organize your code files

Table 8-1. Patterns on how to organize y

o
u
r
c
o
d
e
f
i
l
e
s

	Pattern Name	Summary
	Header Files	You want some functionality that you implement to be accessible for code from other implementation files, but you want to hide your implementation details from the caller. Therefore, provide function declarations in your API for any functionality you want to provide to your user. Hide any internal functions, internal data, and your function definitions (the implementations) in your implementation file and don't provide this implementation file to the user.
	Include Guard	It's easy to include a header file multiple times, but including one and the same header file leads to compile errors if types or certain macros are part of it, because during compilation they get redefined. Therefore, protect the content of your header files against multiple inclusion so that the developer using the header files does not have to care whether it is included multiple times. Use an interlocked `#ifdef` statement or a `#pragma once` statement to achieve that.
	Software-Module Directories	Splitting code into different files increases the number of files in your codebase. Having all files in one single directory makes it difficult to keep an overview of all the files, in particular for large codebases. Therefore, put header files and implementation files that belong to a tightly coupled functionality into one directory. Name

		that directory after the functionality that is provided via the header files.
	Global Include Directory	To include files from other software-modules, you have to use relative paths like *../othersoftwaremodule/file.h*. You have to know the exact location of the other header file. Therefore, have one global directory in your codebase that contains all software-module APIs. Add this directory to the global include paths in your toolchain.
	Self-Contained Components	From the directory structure it is not possible to see the dependencies in the code. Any software-module can simply include the header files from any other software-module, so it's impossible to check dependencies in the code via the compiler. Therefore, identify software-modules that contain similar functionality and that should be deployed together. Put these software-modules into a common directory and have a designated subdirectory for their header files that are relevant for the caller.
	API Copy	You want to develop, version, and deploy the parts of your codebase independently from one another. However, to do that, you need clearly defined interfaces between the code parts and to be able to separate that code into different repositories. Therefore, to use the functionality of another component, copy its API. Build that other component separately and copy the build artifacts and its public header files. Put these files into a directory inside your component and configure that directory as a global include path.

Running Example

Imagine you want to implement a piece of software that prints the hash value for some file content. You start with the following code for a simple hash function:

main.c

```
#include <stdio.h>
```

```
static unsigned int adler32hash(const char* buffer, int length)
{
  unsigned int s1=1;
  unsigned int s2=0;
  int i=0;

  for(i=0; i<length; i++)
  {
     s1=(s1+buffer[i]) % 65521;
     s2=(s1+s2) % 65521;
  }
  return (s2<<16) | s1;
}

int main(int argc, char* argv[])
{
  char* buffer = "Some Text";
  unsigned int hash = adler32hash(buffer, 100);
  printf("Hash value: %u", hash);
  return 0;
}
```

The preceding code simply prints the hash output of some fixed string to the console output. Next, you want to extend that code. You want to read the content of a file and print the hash of the file content. You could simply add all this code to the *main.c* file, but that would make the file very long and it would make the code more unmaintainable the more it grows.

Instead, it is much better to have separate implementation files and access their functionality with Header Files. Well… using Header Files was quite obvious. You now have the following code for reading the content of some file and printing the hash of the file content. To make it easier to see which parts of the code changed, the implementations that did not change are skipped:

main.c

```
#include <stdio.h>
#include <stdlib.h>
#include "hash.h"
#include "filereader.h"

int main(int argc, char* argv[])
```

```
{
  char* buffer = malloc(100);
  getFileContent(buffer, 100);
  unsigned int hash = adler32hash(buffer, 100);
  printf("Hash value: %u", hash);
  return 0;
}
```

hash.h

```
/* Returns the hash value of the provided "buffer" of size
"length".
   The hash is calculated according to the Adler32 algorithm. */
unsigned int adler32hash(const char* buffer, int length);
```

hash.c

```
#include "hash.h"

unsigned int adler32hash(const char* buffer,  int length)
{
  /* no changes here */
}
```

filereader.h

```
/* Reads the content of a file and stores it in the  provided
"buffer"
   if is is long enough according to its provided "length" */
void getFileContent(char* buffer, int length);
```

filereader.c

```
#include <stdio.h>
#include "filereader.h"

void getFileContent(char* buffer, int length)
```

```
{
  FILE* file = fopen("SomeFile", "rb");
  fread(buffer, length, 1, file);
}
```

With organizing the code in separate files, the code became more modular, because dependencies in the code can now be made explicit as all related functionality is now put into the same file. Your files of the codebase are currently all simply stored in the same directory as shown in Figure 8-2

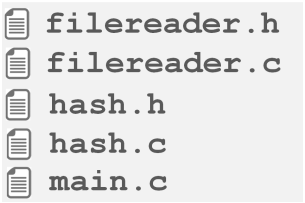


Figure 8-2. File overview

Now that you have separate header files, you can include these header files in your implementation files. However, you'd soon end up with the problem that you get a build error if the header files are included multiple times. To help out with this issue, you can install Include Guards.

Include Guard

Context

You split your implementation into multiple files. Inside the implementation you include Header Files to get forward declarations of other code that you want to call or use.

Problem

It's easy to include a header file multiple times, but including one and the same header file leads to compile errors if types or certain macros are part of it, because during compilation they get redefined.

In C, during compilation, the `#include` directive lets the C preprocessor simply fully copy the included file into your compilation unit. If, for example, a `struct` is defined in the header file and that header file is included multiple times, then that `struct` definition is copied multiple times and is present multiple times in the compilation unit which then leads to a compile error.

To avoid that, you could try to not include files more than once. However, when including a header file, you usually don't have the overview of whether inside that header file other additional header files are included. Thus, it happens very easily that files are included multiple times.

Solution

Protect the content of your header files against multiple inclusion so that the developer using the header files does not have to care whether it is included multiple times. Use an interlocked `#ifdef` statement or a `#pragma once` statement to achieve that.

The following code shows how to use the Include Guard:

somecode.h

```
#ifndef SOMECODE_H
#define SOMECODE_H
 /* put the content of your headerfile here */
#endif
```

othercode.h

```
#pragma once
 /* put the content of your headerfile here */
```

During the build procedure, the interlocked `#ifdef` statement or the `#pragma once` statement protect the content of the header file against being compiled multiple times in a compilation unit.

The `#pragma once` statement is not defined in the C standard, but it is supported by most C preprocessors. Still you have to consider that you could have a problem with this statement when switching to a different toolchain with a different C preprocessor.

While the interlocked `#ifdef` statement works with all C preprocessors, it brings the difficulty that you have to use a unique name for the defined macro. Usually, a name scheme that relates to the name of the header file is taken, but that could lead to outdated names if you rename a file and forget to change the Include Guard. Also, you could run into problems when using 3rd party code, because the names of your Include Guards might collide. An alternative to avoid these problems is to not use the name of the header file, but instead use some other unique name like the current timestamp or a UUID.

Consequences

As a developer who includes header files, you now don't have to care whether that file might be included multiple times. That makes life a lot easier, because especially when having nested `#include` statements, it is difficult to exactly know which files are already included.

You have to either take the non-standard `#pragma once` statement or you have to come up with a unique naming scheme for your interlocked `#ifdef` statement. While file names work as unique names most of the times, there could still be problems with similar names in 3rd party code that you use. Also, there could be inconsistent names of the `#define` statements when renaming your own files, but some IDEs help out here. They already create an Include Guard when creating a new header file or adapt the name of the `#define` when renaming the header file.

The interlocked `#ifdef` statements prevent from compilation errors when having a file included multiple times, but they don't prevent from opening and copying the included file multiple times into the compilation unit. That is an unnecessary part of the compilation time and could be optimized. One approach to optimize that would be to have an additional Include Guard

around each of your `#include` statements, but that makes including the files more cumbersome. Also, for most modern compilers, that is not necessary, because they optimize that by themselves (e.g. by caching the header file content or by remembering which files are already included).

Known Uses

- Pretty much every C code that consists of more than one file applies this pattern.
- The book *Large-Scale C++ Software Design* by John Lakos (Addison Wesley, 1996) describes performance optimization of Include Guards by having an additional guard around each `#include` statement.
- The portland pattern repository describes the Include Guard pattern and also describes a pattern to optimize compilation time by having an additional guard around each `#include` statement.

Applied to Running Example

The Include Guard makes in the following code sure that even if a header file is included multiple times, no build error occurs:

hash.h

```
#ifndef HASH_H
#define HASH_H
/* Returns the hash value of the provided "buffer" of size
"length".
   The hash is calculated according to the Adler32 algorithm. */
unsigned int adler32hash(const char* buffer, int length);
#endif
```

filereader.h

```
#ifndef FILEREADER_H
#define FILEREADER_H
/* Reads the content of a file and stores it in the provided
```

```
   "buffer"
      if is is long enough according to its provided "length" */
   void getFileContent(char* buffer, int length);
   #endif
```

As a next feature of your code, you want to additionally print the hash value calculated by another kind of hash function. Simply adding another *hash.c* file for the other hash function is not possible, because file names have to be unique. It would be an option to give another name to the new file. However, even if you do that, you are still not happy with the situation, because more and more files are now in one single directory and that makes it difficult to get an overview of the files and to see which files are related. To improve the situation you could have Software-Module Directories.

Software-Module Directories

Context

You split your source code into different implementation files and you use Header Files to use functionality from other implementation files. More and more files are being added to your codebase.

Problem

Splitting code into different files increases the number of files in your codebase. Having all files in one single directory makes it difficult to keep an overview of all the files, in particular for large codebases.

Putting the files into different directories raises the question which files you want to put into which directory. It should be easy to find files that belong together and it should be easy to know where to put files if later on additional files have to be added.

Solution

Put header files and implementation files that belong to a tightly coupled functionality into one directory. Name that directory after the functionality that is provided via the header files.

The directory and its content is furthermore called a “software-module”. Quite often, such a software-module contains all code that provides operations on an instance addressed with Handles. In that case, the software-module is the non-object-oriented equivalent to an object-oriented class. Having all files for a software-module in one directory is the equivalent to having all files for a class in one directory.

The software-module could contain one single header file and one single implementation file or multiple such files. The main criteria for putting the files into one directory is the high cohesion between the files within the directory and low coupling to other Software-Module Directories.

When having several header files to only be used inside the software-module and having several header files to be used from outside the software-module, then name the files in a way to make clear which header files are not to be used from outside the software-module (e.g. by giving them the postfix “internal” like shown in Figure 8-3 and the following code).

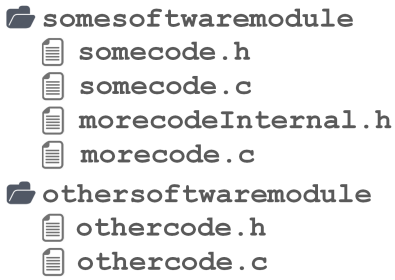


Figure 8-3. File overview

somecode.c

```
#include "somecode.h"
#include "morecode.h"
#include "../othersoftwaremodule/othercode.h"
...
```

morecode.c

```
#include "morecode.h"
...
```

othercode.c

```
#include "othercode.h"
...
```

The preceding code excerpt shows how the files are being included. The code excerpt does not show the implementation. Note that files from the same software-module can easily be included while it is necessary to know the path to other software-modules when including its header files.

When having your files distributes across different directories, you have to make sure that your toolchain is configured in a way to compile all these files. Maybe your IDE automatically compiles all files in subdirectories of your codebase, but it could also be the case that you have to adapt build settings or to manipulate Makefiles to compile the files from the new directories.

CONFIGURING INCLUDE DIRECTORIES AND FILES TO COMPILE

Modern C programming IDEs usually provide a care-free environment where the C programmer can focus on programming and does not necessarily have to get in touch with the build procedure. These IDEs provide build settings where you can easily configure which directories contain the implementation files to be built and which directories contain your include files. That allows the C programmer to focus on programming and not on writing Makefiles and compiler commands. In this chapter we assume having such an IDE and we don’t focus on Makefiles and their syntax.

Consequences

Splitting code files into different directories makes it possible to have the same file names in different directories. That comes quite handy when using 3rd party code, because otherwise its file names might clash with the file names in your own codebase.

However, having similar file names is nothing you should aim for. In particular for header files it is advisable to have unique file names in order to make sure that the file that will be included does not depend on the search order of your include paths. To make file names unique you can use a short and unique prefix for all files of your software-module.

Putting all files that are related to a software-module into one single directory makes it easier to find files that are related, because you simply have to know the name of the software-module. The number of files inside a software-module is usually low enough to be able to quickly spot files in that directory.

Known Uses

- The GIT source code structures some of its code in directories and then other code includes these headers by using relative paths. E.g. *kwset.c* includes *compat/obstack.h*.
- The Netdata real-time performance monitoring and visualization system organizes its code files into directories like *database* or *registry*, which contain a handful of files each. To include files from another directory, relative include paths are used.
- The network mapper Nmap organizes its software-modules into directories like *ncat* or *ndiff*. Header files from other software-modules are included using relative paths.

Applied to Running Example

The code pretty much stayed the same. Only a new header file and a new implementation file for the new hash function was added. The location of the files changed, as you can see by the include paths. In addition to putting

the files into separate directories, also their names were changed to make the file names unique:

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include "adler/adlerhash.h"
#include "bernstein/bernsteinhash.h"
#include "filereader/filereader.h"

int main(int argc, char* argv[])
{
  char* buffer = malloc(100);
  getFileContent(buffer, 100);

  unsigned int hash = adler32hash(buffer, 100);
  printf("Adler32 hash value: %u", hash);

  unsigned int hash = bernsteinHash(buffer, 100);
  printf("Bernstein hash value: %u", hash);

  return 0;
}
```

bernstein/bernsteinhash.h

```c
#ifndef BERNSTEINHASH_H
#define BERNSTEINHASH_H
/* Returns the hash value of the provided "buffer" of size
"length".
   The hash is calculated according to the D.J. Bernstein
algorithm. */
unsigned int bernsteinHash(const char* buffer, int length);
#endif
```

bernstein/bernsteinhash.c

```c
#include "bernsteinhash.h"

unsigned int bernsteinHash(const char* buffer, int length)
```

```
{
  unsigned int hash = 5381;
  int i;
  for(i=0; i<length; i++)
  {
    hash = 33 * hash ^ buffer[i];
  }
  return hash;
}
```

Splitting the code files into separate directories is very common. It makes it easier to find a file and it makes it possible to have files with similar file names. Still, instead of having similar file names it might even be better to have unique file names, for example by having a unique file name prefix per software-module. Anyways, without these prefixes, you'll end up with the directory structure and file names as shown in Figure 8-4.



Figure 8-4. File overview

All files that belong together are now in the same directory. The files are well structured into directories and the header files from other directories can be accessed with relative paths.

However, relative paths bring the problem that if you want to rename one of the directories, you also have to touch other source files to fix their include paths. This is a dependency that you don't want to have and that you can get rid of by having a Global Include Directory.

Global Include Directory

Context

You have Header Files and you have structured your code into Software-Module Directories.

Problem

To include files from other software-modules, you have to use relative paths like *../othersoftwaremodule/file.h*. You have to know the exact location of the other header file.

If the path to the other header file changes, you have to change your code that includes that header file. If, for example, the other software-module is being renamed, you have to change your code. So you have a dependency on the name and location of the other software-module.

As a developer, you want to clearly see which header files belong to the API of a software-module that you are supposed to use and which header files are just internal header files that nobody outside the software-module should use.

Solution

Have one global directory in your codebase that contains all software-module APIs. Add this directory to the global include paths in your toolchain.

Leave all implementation files and all header files that are only used by one software-module in the directory of this software-module. If a header file is used by other code as well, then put it in the global directory, which is commonly named */include* as shown in Figure 8-5 and in the following code.

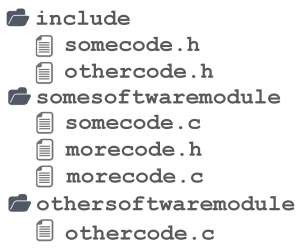


Figure 8-5. File overview

Configured global include path:
/include

somecode.c

```
#include <somecode.h>
#include <othercode.h>
#include "morecode.h"
...
```

morecode.c

```
#include "morecode.h"
...
```

othercode.c

```
#include <othercode.h>
...
```

The preceding code excerpt shows how the files are being included. Note that there are no more relative paths. To make more clear in this code which files are included from the global include path, all these files are included with angle brackets in the `#include` statement.

#INCLUDE SYNTAX

For all of the included files, the syntax with the quotation marks could be used as well (`#include "stdio.h"`). These include files would by most C preprocessors be looked up by relative path first, would not be found there and then they would be looked up in the global directories configured on your system and by the toolchain. In C usually the included files from outside of your codebase use the syntax with the angle brackets (`#include <stdio.h>`). But that syntax could as well be used for files in your own codebase if they are not included by a relative path.

The global include path has to be configured in the build settings of your toolchain or if you manually write Makefiles and compiler commands, you have to add that include path there.

If the number of header files in this directory grows large or if there are very specific header files which are only used by few software-modules, you should consider splitting your codebase into Self-Contained Components.

Consequences

It is very clear which header files are supposed to be used by other software-modules and which header files are internal and are supposed to be used within the software-module only.

Now there is no more need to use relative directories in order to include files from other software-modules. But the code from other software-modules now is not inside one single directory anymore and is instead split over your codebase.

Putting all APIs into one single directory might lead to many files inside this directory, which makes it difficult to find files that belong together. You have be careful to not end up with having all your header files of the whole codebase in that one single include directory. That would mitigate the benefits to having Software-Module Directories. And what would you do in case software-module A is the only one who needs the interfaces of software-module B? With the proposed solution you'd put the interfaces of sotware-module B into the Global Include Directory. However, if nobody else needs these interfaces, then you might not want these interfaces to be available for everybody else in your code base. To avoid that problem, use Self-Contained Components.

Known Uses

- The OpenSSL code has an */include* directory that contains all header files that are used in multiple software-modules.

- The code of the game NetHack has all its header files in the directory */include*. The implementations are not organized into software-modules, but instead the are all in one single */src* directory.
- The OpenZFS code for Linux contains one global directory called *lude* that contains all header files. This directory is configured as include path in the Makefiles that are in the directories of the implementation files.

Applied to Running Example

The thing that changed in your code base is the location of the header files. You moved them to a Global Include Directory which you configured in your toolchain. Now you can simply include the files without searching through relative file paths. Note that because of that, now angle brackets instead of quotation marks are used for the `#include` statements:

main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <adlerhash.h>
#include <bernsteinhash.h>
#include <filereader.h>

int main(int argc, char* argv[])
{
  char* buffer = malloc(100);
  getFileContent(buffer, 100);

  unsigned int hash = adler32hash(buffer, 100);
  printf("Adler32 hash value: %u", hash);

  hash = bernsteinHash(buffer, 100);
  printf("Bernstein hash value: %u", hash);

  return 0;
}
```

In you code you now have the file organization and the global include path */include* configured in your toolchain as shown in Figure 8-6.

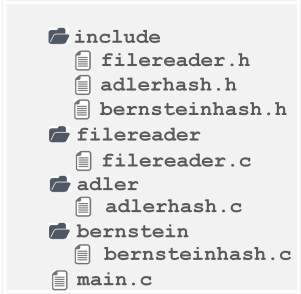


Figure 8-6. File overview

Now, even if you rename of the directories, you do not have to touch the implementation files. So you decoupled the implementations a bit more.

Next you want to extend the code. You want to use the hash functions not only to hash the content of the files, but you also want to use the hash function in another application context. You want to calculate a pseudo-random number based on the hash function. You want to make it possible to develop the two applications, which both use the hash functions, independently from one another, maybe even by independent development teams.

Having to share one single global include directory with another development team is not an option as you don’t want to mix the code files between the different teams. You want to separate the two applications as far as possible from one another and to do that, organize them as Self-Contained Components.

Self-Contained Component

Context

You have Software-Module Directories and maybe you have a Global Include Directory. The number of software-modules keeps growing and your code becomes larger.

Problem

From the directory structure it is not possible to see the dependencies in the code. Any software-module can simply include the header files from any other software-module, so it's impossible to check dependencies in the code via the compiler.

Including header files can be done by using relative paths, which means that any software-module can include the header files from any other software-module.

It gets difficult to keep an overview of the software-modules as their number grows. Just like before having Software-Module Directories where you had too many files in one single directory, now you have too many Software-Module Directories.

Like with the dependencies, it is also not possible to see the code responsibility from the code structure. If multiple development teams work on the code, you might want to define who is responsible for which software-module.

Solution

Identify software-modules that contain similar functionality and that should be deployed together. Put these software-modules into a common directory and have a designated subdirectory for their header files that are relevant for the caller.

Furthermore, such a group of software-modules including all its header files will be called "component". Compared to software-modules, a component is usually something bigger and something that could be deployed independently from the rest of the codebase.

When grouping the software-modules, simply check, which part of your code could be independently deployed from the rest of the codebase. Check which part of the code is developed by separate teams and thus might be developed in a way to only have loose coupling to the rest of the codebase. Such software-module groups are candidates for components.

If you have one single Global Include Directory, simple move all header files from your component from that directory and put them inside the designated directory in your component (for example *myComponentlude*). Developers who use the component, can simply add this path to their global include paths in their toolchain or can modify the Makefile and compiler command accordingly.

You can use the toolchain to check whether the code in one of the components only uses functionality that it is allowed to use. For example, if you have a component that abstracts the operating system, you might want all other code to use that abstraction and to not use operating-system-specific functions. You can configure your toolchain to set the include paths to the operating-system-specific functions only for your component that abstracts the operating system. For all other code, only the directory with the interface of you operating-system abstraction is configured as include path. Then an unexperienced developer who does not know that there is an operating system abstraction and directly tries to use the operating-system-specific functions would have to use the relative include path to these function declarations to get the code compiling (and that hopefully discourages the developer from doing that).

Figure 8-7 and the following code shows the file structure and the include file paths.

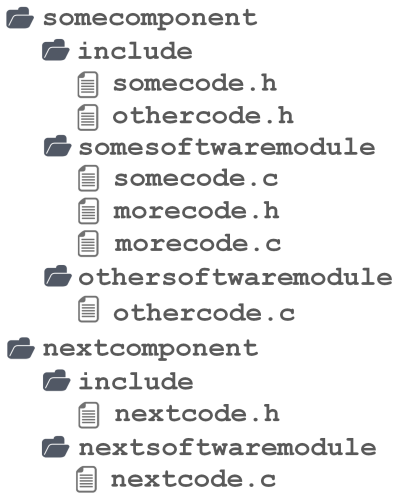


Figure 8-7. File overview

Configured global include path:
/someComponent/include
/nextComponentlude

somecode.c

```
#include <somecode.h>
#include <othercode.h>
#include "morecode.h"
...
```

morecode.c

```
#include "morecode.h"
...
```

othercode.c

```
#include <othercode.h>
...
```

nextcode.c

```
#include <nextcode.h>
#include <othercode.h> // use API of other component
...
```

Consequences

The software-modules are well organized and it is easier to find software-modules that belong together. If the components are well split, then it should also be clear to which component which kind of new code should be added.

Having everything that belongs together in one single directory makes it easier to configure specific things for that component in the toolchain. For

example, you can have stricter compiler warnings for new components that you create in your codebase and you can automatically check code dependencies between components.

When developing the code in multiple teams, component directories make it easier to set the responsibilities between the teams, because usually these components have very low coupling between each other and even the functionality for the overall product might not depend on one another. It is easier to split responsibilities on a component level, compared to a software-module level.

Known Uses

- The GCC code has separate components with their own directories gathering its header files. For example */libffi/include* or *libcpp/include*.
- The operating system RIOT organizes its drivers into well-separated directories. For example the directories */drivers/xbee* or */drivers/soft_spi* each contain an *include* subdirectory that contains all interfaces for that software-module.
- The Radare reverse engineering framework has well-separated components each with its own *include* directory that contains all its interfaces.

Applied to Running Example

You added the implementation of pseudo-random numbers that uses one of the hash functions. Apart from that you isolated three different parts of your code:

- The hash functions
- The hash calculation of a file content
- The pseudo-random number calculation

All three parts of the code are now well separated and could easily be developed by different teams or could even be deployed independently from one another:

main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <adlerhash.h>
#include <bernsteinhash.h>
#include <filereader.h>
#include <pseudorandom.h>

int main(int argc, char* argv[])
{
  char* buffer = malloc(100);
  getFileContent(buffer, 100);

  unsigned int hash = adler32hash(buffer, 100);
  printf("Adler32 hash value: %u", hash);

  hash = bernsteinHash(buffer, 100);
  printf("Bernstein hash value: %u", hash);

  unsigned int random = getRandomNumber(50);
  printf("Random value: %u", random);

  return 0;
}
```

rand/include/pseudorandom.h

```
#ifndef PSEUDORANDOM_H
#define PSEUDORANDOM_H
/* Returns a pseudo random number lower than the
   provided maximum number (parameter `max')*/
unsigned int getRandomNumber(int max);
#endif
```

rand/pseudorandom.c

```
#include <pseudorandom.h>
#include <adlerhash.h>

unsigned int getRandomNumber(int max)
{
  char* seed = "seed-text";
  unsigned int random = adler32hash(seed, 10);
  return random % max;
}
```

Your code now has the following directory structure. Note how each part of the code files is well separated from the other ones. For example, all code related to hashes is in one directory and for a developer using these functions, it is easy to spot where to find the API to these functions, which are all in the *inc* directory as shown in Figure 8-8.

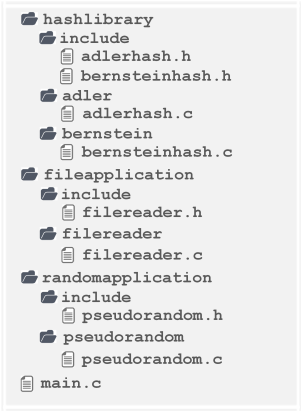


Figure 8-8. File overview

For this code, the following global include directories are configured in the toolchain:
/hashlibrary/include
/filehandlinglibrary/include
/mathlibrary/include

Now the code is well separated into different directories, but there are still dependencies that you could remove. Have a look at the include paths. You have one single codebase and all include path are used for all that code. However, for example, for the code of the hash functions, there is no need to have the file handling include path.

Also, you compile all code and simply link all the objects into one executable file. However, you might want to split that code and independently deploy it. You might want to have one application that prints the hash output and one application that prints the pseudo-random number. Those two applications should be independently developed, but both should use the same hash function code, which you do not want to duplicate.

To decouple the applications and to have a defined way how to access the functionality from other parts without having to share private information like include paths to those parts, you should have an API Copy.

API Copy

Context

You have a large codebase with different teams developing it. In the codebase the functionality is abstracted via Header Files that are organized in Software-Module Directories. Best case is that you even have well organized Self-Contained Components.

Problem

You want to develop, version, and deploy the parts of your codebase independently from one another. However, to do that, you need clearly defined interfaces between the code parts and to be able to separate that code into different repositories.

If you have Self-Contained Components then you are nearly there. The components have well defined interfaces and all code for those components is already in separate directories, so they could easily be checked in into separate repositories.

But there is still a directory structure dependency between the components: the configured include path. That path still includes the full path to the code of the other component and, for example, if the name of that component

changes, you have to change the configured include path. That is a dependency you do not want to have.

Solution

To use the functionality of another component, copy its API. Build that other component separately and copy the build artifacts and its public header files. Put these files into a directory inside your component and configure that directory as a global include path.

Copying code seems like something you might think of as a bad idea. In general it is, but here you only copy the interface of another component. With the header files you copy its function declarations, so there are no multiple implementations because of this. Think about what you do when you install a 3rd party library: you also have a copy of its interfaces to access its functionality.

In addition to the copied header files you have to provide other build artefacts for the for using it during the build of your component. You could version and deploy the other component as a separate library which you'd have to link to your component. Figure 8-9 and the following code shows the overview of the involved files .

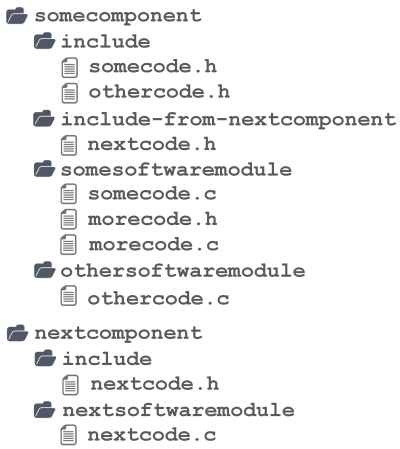


Figure 8-9. File overview

Configured global include path for `someComponent`:
/include

/include-from-nextComponent

somecode.c

```
#include <somecode.h>
#include <othercode.h>
#include "morecode.h"
...
```

morecode.c

```
#include "morecode.h"
...
```

othercode.c

```
#include <othercode.h>
...
```

Configured global include path for `nextComponent`:
/include

nextcode.c

```
#include <nextcode.h>
...
```

Note that the preceding code is now split into two different code blocks. It is now possible to split the code and put it into separate repositories, or in other words: to have separate codebases. There are no more dependencies regarding the directory structure between the components. However, now you are in the situation that different versions of the components have to ensure that their interfaces stay compatible even if their implementations

change. Depending on your deployment strategy, you have to define which kind of interface compatibility (API compatible or ABI compatible) you want to provide. To still keep your interfaces flexible while being compatible, you can use Handles, Dynamic Interfaces, or Function Controls.

INTERFACE COMPATIBILITY

The *Application Programming Interface* (API) stays compatible if there is no need to change anything in the caller's code. You break API compatibility if you, for example, add another parameter to an existing function or if you change to the type of the return value or the parameters.

The *Application Binary Interface* (ABI) stays compatible if there is not even the need to recompile the caller's code. You break the ABI if you, for example, change the platform for which you compile your code or of you update your compiler to a newer version which has a different function calling convention compared to previous compiler versions.

Consequences

Now there are no more dependencies regarding the directory structure between the components. It is possible to rename one of the components without having to change the include directives of the code from other components (or as you can call them now: other codebases).

Now the code can be checked in into different repositories and there is absolutely no need to know the path to other components in order to include their header files. To get to the header files of another component you copy it. So initially you have to know where to get the header files and build artifacts from. Maybe the other component provides some kind of setup installer or maybe it just provides a versioned list of all required files.

You need an agreement that the interfaces of the components stay compatible in order to use the main benefit from the split codebases: independent development and versioning. The requirement for compatible interfaces restricts the development of components providing such

interfaces, because once such an interface can be used by others, it cannot be changed anymore.

You buy the flexibility of separte codebases with the additional complexity of having to cope with API compatibility requirements and with more complexity in the build procedure (copying header files, linking the other component, versioning the interfaces).

> **VERSION NUMBERS**
>
> The way you version your interfaces should tell whether a new version brings incompatible changes. Commonly, *semantic versioning* (*https://semver.org*) is used to indicate in the version number whether there are major changes. With semantic versioning you have a three-digit version number for your interface (for example 1.0.7) and only a change in the first number means an incompatible change.

Known Uses

- Wireshark copies the APIs of the independently deployed Kazlib to use its exception emulation functionality.
- The B&R Visual Components software accesses functionality form the underlying Automation Runtime operating system. The Visual Components software is independently deployed and versioned from Automation Runtime. To access the Automation Runtime functionality, its public header files are copied into the Visual Components codebase.
- The Education First company develops digital learning products. In their C code, they copy include files into a global include directory when building the software in order to decouple the components in their codebase.

Applied to Running Example

Now the different parts of the code are well separated. The hash implementation has a well-defined interface to the code for printing file

hashes and to the code for generating pseudo-random numbers. Additionally these parts of the code are well separated into directories. Even the APIs of other components are copied, so that all code that has to be accessed by one of the components is in its own directory. The code for each of the components could even be stored in its own repository and could be deployed and versioned independently from the other components.

The implementations did not change at all. Only the APIs of other components were copied and the include paths for the codebases changed. The hashing code is now even isolated from the main application. The hashing code is treated as an independently deployed component and is simply linked to the rest of the application. Example 8-1 shows the code of your main application which is now separated from the hash library.

Example 8-1. Code of the main application

main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <adlerhash.h>
#include <bernsteinhash.h>
#include <filereader.h>
#include <pseudorandom.h>

int main(int argc, char* argv[])
{
  char* buffer = malloc(100);
  getFileContent(buffer, 100);

  unsigned int hash = adler32hash(buffer, 100);
  printf("Adler32 hash value: %u\n", hash);

  hash = bernsteinHash(buffer, 100);
  printf("Bernstein hash value: %u\n", hash);

  unsigned int random = getRandomNumber(50);
  printf("Random value: %u\n", random);

  return 0;
}
```

randomapplication/include/pseudorandom.h

```
#ifndef PSEUDORANDOM_H
#define PSEUDORANDOM_H
/* Returns a pseudo random number lower than the provided maximum
number (parameter `max') */
unsigned int getRandomNumber(int max);
#endif
```

randomapplication/pseudorandom/pseudorandom.c

```
#include <pseudorandom.h>
#include <adlerhash.h>

unsigned int getRandomNumber(int max)
{
  char* seed = "seed-text";
  unsigned int random = adler32hash(seed, 10);
  return random % max;
}
```

fileapplication/include/filereader.h

```
#ifndef FILEREADER_H
#define FILEREADER_H
/* Reads the content of a file and stores it in the provided
"buffer"
   if is is long enough according to its provided "length" */
void getFileContent(char* buffer, int length);
#endif
```

fileapplication/filereader/filereader.c

```
#include <stdio.h>
#include "filereader.h"

void getFileContent(char* buffer, int length)
{
  FILE* file = fopen("SomeFile", "rb");
  fread(buffer, length, 1, file);
}
```

This code has the directory structure and include path showin in Figure 8-10 and the following code example. Note that no source code regarding the hash implementation is part of this codebase anymore. The hash functionality is accessed by including the copied header files and then the provided .a file has to be linked to the code in the build process.



Figure 8-10. File overview

Configured include paths:
/hashlibrary
/fileapplication/include
/randomapplication/include

Example 8-2 for the hash implementation is now managed in its own repository. Every time the code changes, a new version of the hash library can be shipped. That means that the object file compiled for that library has to be copied into the other code and as long as the API of the hash library does not change, there is nothing more to do.

Example 8-2. Code of the hash library

inc/adlerhash.h

```
#ifndef ADLERHASH_H
#define ADLERHASH_H
/* Returns the hash value of the provided "buffer" of size
"length".
   The hash is calculated according to the Adler32 algorithm. */
unsigned int adler32hash(const char* buffer, int length);
#endif
```

adler/adlerhash.c

```
#include "adlerhash.h"

unsigned int adler32hash(const char* buffer, int length)
{
  unsigned int s1=1;
```

```
  unsigned int s2=0;
  int i=0;

  for(i=0; i<length; i++)
  {
     s1=(s1+buffer[i]) % 65521;
     s2=(s1+s2) % 65521;
  }
  return (s2<<16) | s1;
}
```

inc/bernsteinhash.h

```
#ifndef BERSTEINHASH_H
#define BERNSTEINHASH_H
/* Returns the hash value of the provided "buffer" of size
"length".
   The hash is calculated according to the D.J. Bernstein
algorithm. */
unsigned int bernsteinHash(const char* buffer, int length);
#endif
```

bernstein/bernsteinhash.c

```
#include "bernsteinhash.h"

unsigned int bernsteinHash(const char* buffer, int length)
{
    unsigned int hash = 5381;
    int i;
    for(i=0; i<length; i++)
    {
        hash = 33 * hash ^ buffer[i];
    }
    return hash;
}
```

This code has the following directory structure and include path shown in <<fig_dir11>. Note that source code regarding the file handling or the pseudo-random number calculation is not part of this codebase anymore. This codebase here is generic and could be used in other contexts as well.

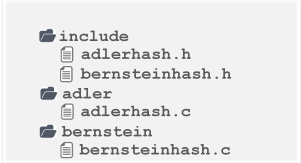


Figure 8-11. File overview

Configured include paths:
/include

Starting from a simple hash application, we now ended up with this code, which allows to develop and deploy the hash code separately from its application. Going one step further, even the two applications could be split into separate parts with can be separately deployed.

Organizing the directory structure as proposed in this example is by far not the most important issue in order to make the code modular. There are many more important issues that are not explicitly addressed in this chapter and in this running example, like code dependencies, which can very well be addressed by applying the SOLID principles.

However, once the dependencies are set in a way that the code can be modular, the directory structure as shown in this example makes it easier to split the ownership of the code and to version and deploy the code independently from other parts of the code.

Summary

This chapter presented pattern on how structure source and header files in order to build large modular C programs.

The Header Files pattern suggests to declare functions in a header file and to include this header file in other implementations. Software-Module Directories suggests to put all files for a software-module into one directory and Global Include Directory suggests to have all header files that are used by multiple software-modules in one global directory. Instead, for larger programs, Self-Contained Component suggests to have one global header file directory per component and in order to decouple these components,

API Copy suggests to copy the header files and build artifacts that are used from other components.

The presented patterns to some extent build on one another. The later patterns in this chapter can more easily be applied if the former ones were already applied. After applying all of the patterns to your codebase, the codebase reaches a high level of flexibility for developing and deploying parts of it separately, but that flexibility is not always needed and it does not come for free: with each of these patterns you add complexity to your codebase. In particular for very small codebases it will not be required to deploy parts of it separately, so quite likely it will not be necessary to apply API Copy, but it might even be sufficient to simply stop after applying Header Files and Include Guard. So do not blindly apply all of the patterns. Instead, only apply them if you face the problems described in the patterns and if solving these problems is worth the additional complexity.

With these patterns as part of the programming vocabulary, a C programmer has a toolbox and has step-by-step guidance on how to build modular C programs and how to organize its files. For experienced programmers, some of the patterns might look like obvious solutions. That is good. One of the tasks of patterns is to educate people to do the right thing and once people know how to do the right thing, the patterns are not necessary anymore, because people then intuitively do as suggested by the patterns.

Outlook

The next chapter covers an aspect that concerns many large-scale programs: handling multi-platform code. The chapter presents patterns on how to implement code in a way that makes it easier to have one single codebase for multiple processor architectures or multiple operating systems.

Chapter 9. Escape #ifdef Hell

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 9th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

C is widespread, in particular with systems where high performance or hardware-near programming is required. With hardware-near programming comes the necessity to cope with hardware variants. Aside from hardware variants, some systems have to cope with supporting multiple operating systems. A commonly used approach to address these issues is to use `#ifdef` statements of the C preprocessor in order to distinguish hardware or operating system variants in the code. The C preprocessor comes with this power, but with this power also comes the responsibility to use it in a well structured way.

However, that is where the weakness of the C preprocessor with its `#ifdef` statements shows up. The C-preprocessor does not support any methods to enforce rules regarding its usage. That is a pity, because it can very easily be abused. It is very easy to add another hardware variant or another optional feature in the code by adding yet another `#ifdef`. Also, `#ifdef` statements can easily be abused to add quick bug-fixes that only affect a single variant. That makes the code for different variants more

diverse and leads to code that more and more has to be fixed for each of the variants separately.

Using `#ifdef` statements in such an unstructured and ad-hoc way is the certain way to hell. The code becomes unreadable and unmaintainable and that is definitely something each developer wants to avoid. This chapter presents approaches to escape from such a hell, or even better: a way to avoid getting into that hell.

This chapter gives detailed guidance on how to implement variants, like operating system variants or hardware variants, in C code and the chapter gives detailed guidance on how to use and how to get rid of `#ifdef` statements.

This chapter presents five patterns on how to cope with code variants and on how to organize or even get rid of `#ifdef` statements. The patterns can be seen as a step-by-step introduction into organizing such code or maybe as a step-by-step guide on how to refactor unstructured `#ifdef` code.

Figure 9-1 shows the way out of the `#ifdef` hell and Table 9-1 provides a short summary of the patterns presented in this chapter.

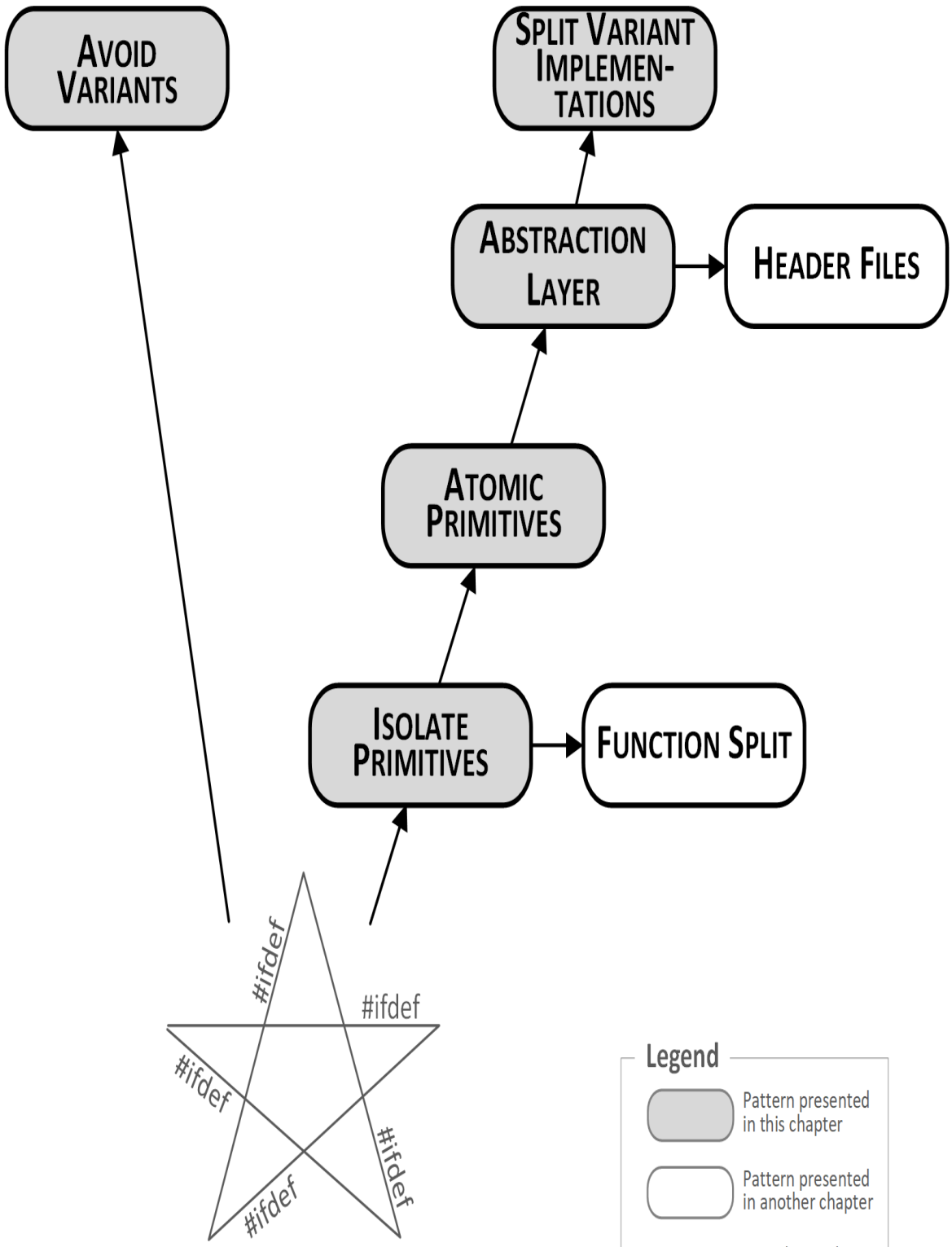

Avoid Variants
Split Variant Implemen-tations
Abstraction Layer
Header Files
Atomic Primitives
Isolate Primitives
Function Split
#ifdef
#ifdef
#ifdef
#ifdef
#ifdef
Your starting point:
The #ifdef hell
Legend
Pattern presented in this chapter
Pattern presented in another chapter
A → B
B can be used to implement and complement A

Figure 9-1. The way out of the #ifdef hell

Table 9-1. Patterns from the chapter on

large-scale code

	Pattern Name	Summary
	Avoid Variants	Using different functions for each platform makes the code harder to read and harder to write. The programmer is required to initially understand, to correctly use, and to test these multiple functions in order to achieve one single functionality across multiple platforms. Therefore, use standardized functions, which are available on all platforms. If there are no standardized functions, consider not implementing the functionality.
	Isolate Primitives	Having code variants organized with `#ifdef` statements makes the code unreadable. It is very difficult to follow the program flow, because it is implemented multiple times for multiple platforms. Therefore, isolate your code variants. In your implementation file, put the code handling the variants into separate functions and call these functions from your main program logic, which then only contains platform independent code.
	Atomic Primitives	The function that contains the variants and is called by the main program is still hard to comprehend, because all the complex `#ifdef` code was simply put into this function in order to get rid of it in the main program.

		Therefore, make your primitives atomic. Only handle exactly one kind of variant per function. If you handle multiple kinds of variants, for example, operating system variants and hardware variants, then have separate functions for that.
	Abstraction Layer	You want to use the functionality which handles platform variants at several places in your code base, but you do not want to duplicate the code of that functionality. Therefore, provide an API for each functionality that requires platform-specific code. Define only platform independent functions in the header file and put all platform-specific `#ifdef` code into the implementation file. The caller of your functions only includes your header file and does not have to include any platform-specific files.
	Split Variant Implementations	The platform-specific implementations still contain `#if def` statements to distinguish between code variants. That makes it difficult to see and to select which part of the code should be built for which platform. Therefore, put each variant implementation into a separate implementation file and select per file what you want to compile for which platform.

Running Example

You want to implement the functionality to write some text into a file to be stored in a newly created directory that, depending on a configuration flag, is either created in the user- or home-directory. To make things more complicated, your code should run on Windows systems as well as on Linux systems.

Your first attempt is to have one single implementation file that contains all the code for all configurations and operating systems. To do that, the file contains many `#ifdef` statements to distinguish between the code variants:

```
#include <string.h>
#include <stdio.h>
```

```c
#include <stdlib.h>
#ifdef __unix__
  #include <sys/stat.h>
  #include <fcntl.h>
  #include <unistd.h>
#elif defined _WIN32
  #include <windows.h>
#endif

int main()
{
  char dirname[50];
  char filename[60];
  char* my_data = "Write this data to the file";
  #ifdef __unix__
    #ifdef STORE_IN_HOME_DIR
      sprintf(dirname, "%s%s", getenv("HOME"), "/newdir/");
      sprintf(filename, "%s%s", dirname, "newfile");
    #elif defined STORE_IN_CWD
      strcpy(dirname, "newdir");
      strcpy(filename, "newdir/newfile");
    #endif
    mkdir(dirname,S_IRWXU);
    int fd = open (filename, O_RDWR | O_CREAT, 0666);
    write(fd, my_data, strlen(my_data));
    close(fd);
  #elif defined _WIN32
    #ifdef STORE_IN_HOME_DIR
      sprintf(dirname, "%s%s%s", getenv("HOMEDRIVE"),
getenv("HOMEPATH"),
              "\\newdir\\");
      sprintf(filename, "%s%s", dirname, "newfile");
    #elif defined STORE_IN_CWD
      strcpy(dirname, "newdir");
      strcpy(filename, "newdir\\newfile");
    #endif
    CreateDirectory (dirname, NULL);
    HANDLE hFile = CreateFile(filename, GENERIC_WRITE, 0, NULL,
                              CREATE_NEW, FILE_ATTRIBUTE_NORMAL,
NULL);
    WriteFile(hFile, my_data, strlen(my_data), NULL, NULL);
    CloseHandle(hFile);
  #endif
  return 0;
}
```

This code is chaos. The program logic is completely duplicated. This is not operating system independent code, but instead it is only two different operating-system-specific implementations put into one single file. Especially the orthogonal code variants of different operating systems and different places for creating the directory make the code very ugly as they lead to nested `#ifdef` statements, which are very hard to understand. When reading the code, you always have to jump between the lines. You have to skip the code from other `#ifdef` branches, in order to follow the program logic. Such duplicated program logic invites programmers to fix errors or to add new features only in the code variant that they currently work on. That makes the code pieces and the behavior for the variants drift apart, which makes the code hard to maintain.

Where to start? How to clean this mess up? As a first step, if possible, you can use standardized functions in order to Avoid Variants.

Avoid Variants

Context

You write portable code that should be used on multiple operating system platforms or on multiple hardware platforms. Some of the functions you call in your code are available on one platform, but are not available in exactly the same syntax and semantic on another platform. Because of that, you implement code variants - one for each platform. Now you have different pieces of code for your different platforms and you distinguish between the variants with `#ifdef` statements in your code.

Problem

Using different functions for each platform makes the code harder to read and harder to write. The programmer is required to initially understand, to correctly use, and to test these multiple functions in order to achieve one single functionality across multiple platforms.

Quite often it is the aim that you implement functionality that should behave exactly the same on all platforms, but when using platform-dependent functions, that aim is more difficult to achieve and might require writing additional code, because not only the syntax, but also the semantics of the functions might slightly differ between the platforms.

Using multiple functions for multiple platforms make the code not only more difficult to write, but also more difficult to read and to understand. Distinguishing between the different functions with `#ifdef` statements makes the code longer and requires the reader to jump across lines when trying to find out what the code does for one single `#ifdef` branch.

Solution

Use standardized functions, which are available on all platforms. If there are no standardized functions, consider not implementing the functionality.

Good examples for standardized functions that you can use are the C Standard Library functions and the POSIX functions. If possible, these functions should be used instead of more specific functions that are only available on one of the platforms as shown in the following code:

Caller's code

```
#include <standardizedApi.h>

int main()
{
  /* just one single function instead of multiple via
     ifdef distinguished functions is called */
  somePosixFunction();
  return 0;
}
```

Standardized API

```
/* this function is available on all operating systems
   that adhere to the POSIX standard */
somePosixFunction();
```

Not at all implementing the functionality only because there is no standardized function will not always be an option, but if there are only platform-dependent functions available for the functionality you want to achieve, then seriously consider whether providing that functionality in your product is worth it. Maybe maintaining different code for different platforms is not worth the coding or testing effort.

However, in some cases you do have to provide functionality in your product even if there are no standardized functions available. That means that you have to use different functions across different platforms or maybe even have to implement features on one platform, which are alredy available on another. To do that in a structured way, Isolate Primitives for your code variants and hide them behind an Abstraction Layer.

To avoid variants, for example, use C Library file access functions like `fopen` instead of using operating-system-specific functions like Linux' `open` or Windows' `CreateFile` functions. Another example are the C Library time functions. Avoid using operating-system-specific time functions like Windows' `GetLocalTime` and Linux' `localtime_r`, but instead use the standardized `localtime` function from *time.h*.

Consequences

The code is simple to write and to read, because one single piece of code can be used for multiple platforms. The programmer does not have to understand different functions for different platforms when writing the code and the programmer does not have to jump between `#ifdef` branches when reading the code.

As there is the same piece of code on all platforms, the problem does not occur that on different platforms functions for similar functionality might slightly differ in their behavior. You just use one single function that is

standardized and that behaves the same on each platform that adheres to the standard.

The standardized function might not be the most efficient and the most high-performance way to achieve the required functionality on each of the platforms. Some platforms might provide other platform-specific functions that, for example, use specialized hardware on that platform to achieve higher performance. Such hardware-specific advantages might not be used by the standardized functions.

Known Uses

- The code of the editor VIM uses the operating system independent functions `fopen`, `fwrite`, `fread`, and `fclose` to access files.
- The OpenSSL code writes the current local time to its log messages. To do that, it converts the current UTC time to local time using the operating system independent function `localtime`.
- The OpenSSL function `BIO_lookup_ex` looks up the node and service to connect to. This function is compiled on Windows and Linux and uses the operating system independent function `htons` to convert a value to network byte order.

Applied to Running Example

For your functionality to access files, you are in the lucky position where there are operating system independent functions available and you now have the following code:

```
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#ifdef __unix__
  #include <sys/stat.h>
#elif defined _WIN32
  #include <windows.h>
#endif
```

```
int main()
{
  char dirname[50];
  char filename[60];
  char* my_data = "Write this data to the file";
  #ifdef __unix__
    #ifdef STORE_IN_HOME_DIR
      sprintf(dirname, "%s%s", getenv("HOME"), "/newdir/");
      sprintf(filename, "%s%s", dirname, "newfile");
    #elif defined STORE_IN_CWD
      strcpy(dirname, "newdir");
      strcpy(filename, "newdir/newfile");
    #endif
    mkdir(dirname,S_IRWXU);
  #elif defined _WIN32
    #ifdef STORE_IN_HOME_DIR
      sprintf(dirname, "%s%s%s", getenv("HOMEDRIVE"),
getenv("HOMEPATH"),
              "\\newdir\\");
      sprintf(filename, "%s%s", dirname, "newfile");
    #elif defined STORE_IN_CWD
      strcpy(dirname, "newdir");
      strcpy(filename, "newdir\\newfile");
    #endif
    CreateDirectory(dirname, NULL);
  #endif
  FILE* f = fopen(filename, "w+"); ❶
  fwrite(my_data, 1, strlen(my_data), f);
  fclose(f);
  return 0;
}
```

❶ The functions `fopen`, `fwrite`, and `fclose` are part of the C Library and are available on Windows as well as on Linux.

The standardized file-related function calls in the preceding code made things a lot simplare already. Instead of having the separate file access calls for Windows and for Linux, you now have have one common code for that. Having that common code has the major advantage that you can be sure that the calls have the same behavior for both operating systems and there is no danger that two different implementations run apart after bug-fixes or added features.

Still, your code is dominated by `#ifdefs` and due to that, The code is very difficult to read. Therefore, make sure that your main program logic does not get obfuscated by code variants. Isolate Primitives containing the code variants from the main program logic.

Isolate Primitives

Context

Your code calls platform-specific functions. You have different pieces of code for different platforms and you distinguish between the code variants with `#ifdef` statements. You cannot simply Avoid Variants, because there are no standardized functions available that provide the feature you need in a uniform way on all your platforms.

Problem

Having code variants organized with `#ifdef` statements makes the code unreadable. It is very difficult to follow the program flow, because it is implemented multiple times for multiple platforms.

When trying to understand the code, you usually just focus on one platform and then you have to jump between the lines in the code in order to only scan through the code variant you are interested in.

The `#ifdef` statements also make the code difficult to maintain. Such statements invite programmers to only fix the code for the one platform they are interested in and to not touch any other code, because of the danger of breaking it. But only fixing a bug or only introducing a new feature for one single platforms means that the behavior of the code on the different platforms drifts apart. The alternative, to fix such a bug on all platforms in different ways, requires testing the code on all platforms.

Testing code with many code variants is difficult. Each new kind of `#ifdef` statement doubles the testing effort as all possible combinations

have to be tested. Even worse, each such statement doubles the number of binaries that can be built and have to be tested. That brings in a logistic problem, because build times increase and the number of binaries provided to the test department and to the customer increase.

Solution

Isolate your code variants. In your implementation file, put the code handling the variants into separate functions and call these functions from your main program logic, which then only contains platform independent code.

Each of your functions should either only contain program logic or it should only cope with handling variants. None of your functions should do both. So either there is no `#ifdef` statement at all in a function, or there are just `#ifdef` statements with one single platform dependent or feature dependent function call per `#ifdef` branch as shown in the following code:

```
static void handlePlatformVariants()
{
  #ifdef PLATFORM_A
    /* call function of platform A */
  #elif defined PLATFORM_B
    /* call function of platform B */
  #endif
}

int main()
{
  /* program logic goes here */
  handlePlatformVariants();
  /* program logic continues */
}
```

When only having one single function call per `#ifdef` branch, then finding a good abstraction granularity for the functions handling the variants should not be a problem. Usually the granularity is exactly at the

level of the available platform-specific or feature-specific functions to be wrapped.

If the functions that handle the variants are still complicated and contain `#ifdef` cascades, then making sure to only have Atomic Variants helps.

Consequences

The main program logic is now easy to follow, because the code variants are separated from it. When reading the main code, it is not necessary anymore to jump between the lines in order to find out what the code does on one specific platform.

For finding out, what the code does on one specific platform, you have to look at the called function that implements this variant. Having such a function has the advantage that it can be called from other places in the file as well and thus code duplications can be avoided. If the functionality is also required in other implementation files, then an Abstraction Layer has to be implemented.

As no program logic should be introduced in the functions handling the variants, it is quite difficult to make a bug-fix that only affects a single platform and it is quite easy to pinpoint bugs that do not occur on all platforms, because it is now easy to pinpoint the places in the code where the behavior of the platforms differ.

Code duplication becomes less of an issue, because as the main program logic is well separated from the variant implementations, there is no temptation to duplicate the program logic anymore.

Known Uses

- The code of the editor VIM isolates the function `htonl2` that converts data to network byte order. The program logic of VIM uses `htonl2` that is defined as a macro in the implementation file. The macro is compiled differently depending on the platform endianness.

- The OpenSSL function `BIO_ADDR_make` copies socket information into an internal struct. The function uses `#ifdef` statements to handle operating-system-specific and feature-specific variants distinguishing between Linux/Windows and IPv4/IPv6. The function isolates these variants from the main program logic.
- The function `load_rcfile` of GNUplot reads data from an initialization file and isolated operating-system-specific file access operations from the rest of the code.

Applied to Running Example

Now that you Isolated Primitives, your main program logic is a lot easier to read without requiring the reader to jump between the lines only to keep the variants apart:

```
static void getDirectoryName(char* dirname)
{
  #ifdef __unix__
    #ifdef STORE_IN_HOME_DIR
      sprintf(dirname, "%s%s", getenv("HOME"), "/newdir/");
    #elif defined STORE_IN_CWD
      strcpy(dirname, "newdir/");
    #endif
  #elif defined _WIN32
    #ifdef STORE_IN_HOME_DIR
      sprintf(dirname, "%s%s%s", getenv("HOMEDRIVE"),
getenv("HOMEPATH"),
              "\\newdir\\");
    #elif defined STORE_IN_CWD
      strcpy(dirname, "newdir\\");
    #endif
  #endif
}

static void createNewDirectory(char* dirname)
{
  #ifdef __unix__
    mkdir(dirname,S_IRWXU);
  #elif defined _WIN32
    CreateDirectory (dirname, NULL);
  #endif
```

```
}

int main()
{
  char dirname[50];
  char filename[60];
  char* my_data = "Write this data to the file";
  getDirectoryName(dirname);
  createNewDirectory(dirname);
  sprintf(filename, "%s%s", dirname, "newfile");
  FILE* f = fopen(filename, "w+");
  fwrite(my_data, 1, strlen(my_data), f);
  fclose(f);
  return 0;
}
```

The code variants are now quite well isolated. The program logic of the `main` function is very easy to read and to understand as there are no variants anymore in this function. However, the new function `getDirectoryName` is still dominated by `#ifdefs` and is not easy to comprehend. Here it could help to only have Atomic Primitives.

Atomic Primitives

Context

You implemented variants in your code with `#ifdef` statements and you put these variants into separate functions in order to Isolate Primitives that handle these variants. The primitives separate the variants from the main program flow and that makes the main program well structured and easy to comprehend.

Problem

The function that contains the variants and is called by the main program is still hard to comprehend, because all the complex `#ifdef` code was simply put into this function in order to get rid of it in the main program.

Simply handling all kinds of variants in one function becomes difficult as soon as there are many different variants to handle. If, for example, a single function distinguishes with `#ifdef` statements between different hardware types and different operating systems, then adding an additional operating system variant becomes difficult, because it has to be added for all hardware variants. Each variant cannot be handled in one place anymore, but instead the effort multiplies with the number of different kinds of variants. That is a problem. It should be easy to add new variants at one place in the code.

Solution

Make your primitives atomic. Only handle exactly one kind of variant per function. If you handle multiple kinds of variants, for example, operating system variants and hardware variants, then have separate functions for that.

Let one of these functions call the other that already abstracts one kind of variant. If you abstract a platform-dependence and a feature-dependence, then let the feature-dependent function be the one calling the platform-dependent function, because usually you provide features across all platforms, so platform-dependent functions should be the most atomic ones as shown in the following code:

```
static void handleHardwareOfFeatureX()
{
  #ifdef HARDWARE_A
   /* call function for feature X on hardware A */
  #elif defined HARDWARE_B || defined HARDWARE_C
   /* call function for feature X on hardware B and C */
  #endif
}

static void handleHardwareOfFeatureY()
{
  #ifdef HARDWARE_A
   /* call function for feature Y on hardware A */
  #elif defined HARDWARE_B
   /* call function for feature Y on hardware B */
```

```
  #elif defined HARDWARE_C
   /* call function for feature Y on hardware C */
  #endif
}

void callFeature()
{
  #ifdef FEATURE_X
    handleHardwareOfFeatureX();
  #elif defined FEATURE_Y
    handleHardwareOfFeatureY();
  #endif
}
```

If there is a function that apparently has to provide some functionality across multiple kinds of variants and handles all these kind of variants, then the function scope might be wrong. Perhaps the function is too general or does more than one thing. Split the function.

Call atomic primitives in your main code containing the program logic. If you want to use the atomic primitives in other implementation files, then use an Abstraction Layer.

Consequences

Each function now only handles one single kind of variant. That makes each of the functions easy to understand as there are no more cascades of `#ifdef` statements. Each of the functions now only abstracts one kind of variant and does no more than exactly that one thing. So the functions do follow the single responsibility principle.

Having no `#ifdef` cascades makes it less tempting for programmers to simply handle one additional kind of variant in one single function, because starting an `#ifdef` cascade is less likely done compared to extending an existing cascade.

With separate functions, each kind of variant can easily be extended for an additional variant. To achieve that, only one single `#ifdef` branch has to be added in one single function and the functions which handle other kinds of variants do not have to be touched.

Known Uses

- The OpenSSL implementation file *threads_pthread.c* contains functions for thread handling. There are separate functions to abstract operating systems and separate functions to abstract whether pthreads are available at all.
- The code of SQLite contains functions to abstract operating-system-specific file access (for example the `fileStat` function). The code abstracts file access related compile time features with other, separate functions.
- The function `boot_jump_linux` calls another function that performs different boot actions depending on the CPU architecture that is handled via `#ifdef` statements in that function. Then the function `boot_jump_linux` calls another function that uses `#ifdef` statements to select which configured resources (USB, network, …) have to be cleaned up.

Applied to Running Example

With Atomic Primitives you now have the following code for your functions to determine the directory path:

```
static void getHomeDirectory(char* dirname)
{
  #ifdef __unix__
    sprintf(dirname, "%s%s", getenv("HOME"), "/newdir/");
  #elif defined _WIN32
    sprintf(dirname, "%s%s%s", getenv("HOMEDRIVE"),
getenv("HOMEPATH"),
            "\\newdir\\");
  #endif
}

static void getWorkingDirectory(char* dirname)
{
  #ifdef __unix__
    strcpy(dirname, "newdir/");
  #elif defined _WIN32
```

```
    strcpy(dirname, "newdir\\");
  #endif
}

static void getDirectoryName(char* dirname)
{
  #ifdef STORE_IN_HOME_DIR
    getHomeDirectory(dirname);
  #elif defined STORE_IN_CWD
    getWorkingDirectory(dirname);
  #endif
}
```

The code variants are now very well isolated. For obtaining the directory name, instead of having one complicated function with many `#ifdefs`, you now have several functions that only have one `#ifdef` each. That makes it a lot easier to understand the code, because now each of these function only performs one single thing instead of distinguishing between several kinds of variants with `#ifdef` cascades.

The functions are now very simple and easy to read, but your implementation file is still very long and the one single implementation file contains the main program logic as well as code to distinguish between variants. That makes parallel development or separate testing of the variant code next to impossible.

To improve that, best split the implementations up up by inserting an Abstraction Layer.

Abstraction Layer

Context

You have platform variants for which you distinguish with `#ifdef` statements in your code. Maybe you Isolated Primitives to separate the variants from the program logic and maybe you made sure that you have Atomic Primitives.

Problem

You want to use the functionality which handles platform variants at several places in your code base, but you do not want to duplicate the code of that functionality.

You don't want the caller at all having to cope with platform variants. In the caller code, it should not be necessary to know anything about the implementation details of the functionality for the different platforms. The caller should not have to use any `#ifdef` statements and the caller should not even have to include any platform-specific headerfiles.

You want to make it possible to work on the platform-specific code without requiring the caller of this code to care about that. Maybe you even want to have the platform-dependent code to be developed and tested by other programmers than those responsible for the platform-independent code.

If programmers of the platform-dependent code perform a bug-fix for one platform or if they add an additional platform, then this must not require changes to the caller's code.

Solution

Provide an API for each functionality that requires platform-specific code. Define only platform independent functions in the header file and put all platform-specific `#ifdef` code into the implementation file. The caller of your functions only includes your header file and does not have to include any platform-specific files.

Try to design the API for the abstraction layer to be stable, because changing the API later on requires changes in your caller's code and sometimes that is not possible. However, it is very difficult to design a stable API. For platform abstractions, you can have a look at different platforms, maybe even platforms that you don't yet support. When understanding how these platforms work and what the differences are, you can create an API to abstract features for these platforms. Then you can be

sure that, even when later on adding support for similar platforms, there is no need for changing the API.

Make sure to document the API well. Add comments to each function describing what the function does. Also describe on which platforms the functions are supported if that is not clearly defined elsewhere for your whole code-base.

The following code shows a simple Abstraction Layer:

caller.c

```
#include "someFeature.h"

int main()
{
  someFeature();
  return 0;
}
```

someFeature.h

```
/* Provides generic access to someFeature.
   Supported on platform A and platform B. */
void someFeature();
```

someFeature.c

```
void someFeature()
{
  #ifdef PLATFORM_A
    performFeaturePlatformA();
  #elif defined PLATFORM_B
    performFeaturePlatformB();
  #endif
}
```

Consequences

The abstracted features can be used from anywhere in the code and not only from one single implementation file. In other words, now you have distinct roles of caller and callee. The callee has to cope with platform variants and the caller can be platform independent.

On the up side, the caller does not have to cope with platform-specific code. The caller simply includes the provided headerfile and does not have to include any platform-specific headerfiles. However, on the down side, the caller might not be able to use platform-specific features anymore. Especially if the caller knows the platform-specific functions that are used below the abstraction layer and is used to these functions, then the caller might not be satisfied with using the abstracted functionality, which might be different to use and which might not provide as much functionality as the platform-specific functions.

The platform-specific part can now be developed and even be tested separately from the other code. This now is the first time that even though you support many platforms, like multiple hardware and multiple operating systems, the testing effort is still manageable, because now you can mock the hardware-specific part in order to write simple tests for the platform-independent code.

When building up such APIs for all platform-specific functions, then the sum of these functions and APIs is the platform abstraction layer for the code-base. When having such a platform abstraction layer, it is very clear which code is platform-dependent and which is platform independent. Such a platform abstraction layer also makes it clear, which parts of the code have to be touched in order to support an additional platform.

Known Uses

- A hardware abstraction is used for the Time Triggered Ethernet protocol described in the bachelor's thesis "Hardware-abstraction of an open source real-time Ethernet stack - Design, realisation and evaluation" by Flemming Bunzel (*https://core-researchgroup.de/bib/eigene/b-haose-13.pdf*). The hardware

abstraction layer contains functions for accessing interrupts and timers. The functions are marked as `inline` to not loose performance.

- The function `sock_addr_inet_pton` of the lighttpd web-server converts an IP address from text to binary form. The implementation distinguishes with `#ifdef` statements between code variants for IPv4 and IPv6. Callers of the API do not see this distinction.
- The function `getprogname` of the gzip data compression program returns the name of the invoking program. The way to obtain this name depends on the operating system and is distinguished via `#ifdef` statements in the implementation. The caller does not have to care on which operating system the function is called.

Applied to Running Example

Now you have a piece of code that might not anymore be considered ugly. Each of the functions only performs one single action and you hide implementation details about the variants behind APIs:

directoryNames.h

```
/* Copies the path to a new directory with name "newdir"
   located in the user's home directory into 'dirname'.
   Works on Linux and Windows. */
void getHomeDirectory(char* dirname);

/* Copies the path to a new directory with name "newdir"
   located in the current working directory into 'dirname'.
   Works on Linux and Windows. */
void getWorkingDirectory(char* dirname);
```

directoryNames.c

```
#include "directoryNames.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
void getHomeDirectory(char* dirname)
{
  #ifdef __unix__
    sprintf(dirname, "%s%s", getenv("HOME"), "/newdir/");
  #elif defined _WIN32
    sprintf(dirname, "%s%s%s", getenv("HOMEDRIVE"),
getenv("HOMEPATH"),
            "\\newdir\\");
  #endif
}

void getWorkingDirectory(char* dirname)
{
  #ifdef __unix__
    strcpy(dirname, "newdir/");
  #elif defined _WIN32
    strcpy(dirname, "newdir\\");
  #endif
}
```

directorySelection.h

```
/* Copies the path to a new directory with name "newdir" into
'dirname'.
   The directory is located in the user's home directory, if
STORE_IN_HOME_DIR
   is set or it is located in the current working directory, if
STORE_IN_CWD is set. */
void getDirectoryName(char* dirname);
```

directorySelection.c

```
#include "directorySelection.h"
#include "directoryNames.h"

void getDirectoryName(char* dirname)
{
  #ifdef STORE_IN_HOME_DIR
    getHomeDirectory(dirname);
  #elif defined STORE_IN_CWD
    getWorkingDirectory(dirname);
```

```
  #endif
}
```

directoryHandling.h

```
/* Creates a new directory of the provided name ('dirname').
   Works on Linux and Windows. */
void createNewDirectory(char* dirname);
```

directoryHandling.c

```
#include "directoryHandling.h"
#ifdef __unix__
  #include <sys/stat.h>
#elif defined _WIN32
  #include <windows.h>
#endif

void createNewDirectory(char* dirname)
{
  #ifdef __unix__
    mkdir(dirname,S_IRWXU);
  #elif defined _WIN32
    CreateDirectory (dirname, NULL);
  #endif
}
```

main.c

```
#include <stdio.h>
#include <string.h>
#include "directorySelection.h"
#include "directoryHandling.h"

int main()
{
  char dirname[50];
  char filename[60];
  char* my_data = "Write this data to the file";
  getDirectoryName(dirname);
```

```
    createNewDirectory(dirname);
    sprintf(filename, "%s%s", dirname, "newfile");
    FILE* f = fopen(filename, "w+");
    fwrite(my_data, 1, strlen(my_data), f);
    fclose(f);
    return 0;
}
```

Finally your file with the main program logic is completely independent from the operating system and not even operating-system-specific headerfiles are included here. Separating the implementation files with an Abstraction Layer on the one hand makes the files each by itself easier to comprehend and on the other hand makes it possible to reuse the functions in other parts of the code. Also development, maintenance, and testing can be split for the platform dependent and platform independent code.

When having many such Isolated Primitives behind an Abstraction Layer and organizing them according to the kind of variant that they abstract, then you'll end up with a hardware abstraction layer or with an operating system abstraction layer.

However, the implementations behind the APIs still contain `#ifdef` code for different variants. That has the disadvantage that these implementations have to be touched and grow if, for example, additional operating systems have to be supported. To avoid touching existing implementation files for adding another variant, you could could Split Variant Implementations.

Split Implementation Variants

Context

You have platform variants hidden behind an Abstraction Layer. In the platform-specific implementation you distinguish with `#ifdef` statements between the code variants.

Problem

The platform-specific implementations still contain `#ifdef` statements to distinguish between code variants. That makes it difficult to see and to select which part of the code should be built for which platform.

As code for different platforms is put into one single file, it is not possible to select the platform-specific code on a file-basis. However, that is the approach taken by tools such as Make, which are usually actually responsible to select via Makefiles which files should be compiled in order to come up with variants for different platforms.

When looking at the code from a high-level view, it is not possible to see which parts are platform-specific and which are not, but that would be very desirable when porting the code to another platform, to quickly see which code has to be touched.

The Open-Closed Principle says that to bring in new features (or to port to a new platform), it should not be necessary to touch existing code. The code should be open for such modifications, but having platform variants separated with `#ifdef` statements requires that existing implementations have to be touched for introducing a new platform, because simply another `#ifdef` branch has to be placed into an existing function.

Solution

Put each variant implementation into a separate implementation file and select per file what you want to compile for which platform.

Related functions of the same platform can sill be put into the same file. For example, there could be a file gathering all socket handling functions on Windows and one such file doing the same for Linux.

With separate files for each platform, `#ifdef` statements can still be used to determine which code is compiled on a specific platform. For example, a *fileHandlingWindows.c* file could still have on `#ifdef _WIN32` statement across the whole file:

someFeature.h

```
/* Provides generic access to someFeature. Supported on platform
A and platform B. */
  someFeature();
```

someFeatureWindows.c

```
#ifdef _WIN32
  someFeature()
  {
    performWindowsFeature();
  }
#endif
```

someFeatureLinux.c

```
#ifdef __unix__
  someFeature()
  {
    performLinuxFeature();
  }
#endif
```

Alternatively to using `#ifdef` statements across the while fome, other platform-independent mechanisms such as Make can be used to decide on a file-basis which code to compile on a specific platform.

With separate files for the platforms comes the question of where to put these files and how to name them:

- One option is to put platform-specific files per software-module next to each other and to name them in a way that makes it clear which platform they cover (for example *fileHandlingWindows.c*). That provides the advantage that the implementations of the software-modules are at the same place.
- Another option is to put all platform-specific files from the code-base into one directory and to have one subdirectory for each platform. That

provides the advantage that all files for one platform are at the same place.

Consequences

You now have the option to not have any `#ifdef` statements at all in the code, but to instead distinguish between the variants on file-basis with tools such as Make.

In each implementation file there is now just one code variant, so there is no need to jump between the lines when reading the code in order to only read the `#ifdef` branch you are looking for. It is much easier to read and understand the code.

When fixing a bug on one platform, no files for other platforms have to be touched. When porting to a new platform, only new files have to be added and no existing file or existing code has to be modified.

It is easy to spot which part of the code is platform-dependent and which code has to be added in order to port to a new platform. Either all platform-specific files are in one single directory, or the files are named in a way that makes clear they are platform-dependent.

However, with putting each variant into a separate file, you create many new files and the more files you have, the longer the compile-time for your code gets.

Known Uses

- The Simple Audio Library presented in the book *Write Portable Code: An Introduction to Developing Software for Multiple Platforms* by Brian Hook (No Starch Press, 2005) uses separate implementation files to provide access to threads and mutexes for Linux and OS-X. The implementation files use `#ifdef` statements to only compile the code on the corresponding operating system.

- The Multi-Processing-Module of the Apache web-server that is responsible for handling accesses to the web-server is implemented in separate implementation files for Windows and Linux. The implementation files use `#ifdef` statements to only compile the code on the corresponding operating system.
- The code of the uboot bootloader puts the source code for each hardware platform it supports into a separate directory. Each of these directories contains amongst others the file *cpu.c* that contains a function to reset the CPU. A Makefile decides which directory (and which *cpu.c* file) has to be compiled - there are no `#ifdef` statements in these files. The main program logic of uboot calls the function to reset the CPU and does not have to care about hardware platform details at that point.

Applied to Running Example

After Splitting Variant Implementations, you'll end up with the following final code for your functionality to create a directory and to write data to a file:

directoryNames.h

```
/* Copies the path to a new directory with name "newdir"
   located in the user's home directory into 'dirname'.
   Works on Linux and Windows. */
void getHomeDirectory(char* dirname);

/* Copies the path to a new directory with name "newdir"
   located in the current working directory into 'dirname'.
   Works on Linux and Windows. */
void getWorkingDirectory(char* dirname);
```

directoryNamesLinux.c

```
#ifdef __unix__
  #include "directoryNames.h"
  #include <string.h>
```

```
  #include <stdio.h>
  #include <stdlib.h>

  void getHomeDirectory(char* dirname)
  {
      sprintf(dirname, "%s%s", getenv("HOME"), "/newdir/");
  }

  void getWorkingDirectory(char* dirname)
  {
    strcpy(dirname, "newdir/");
  }
#endif
```

directoryNamesWindows.c

```
#ifdef _WIN32
  #include "directoryNames.h"
  #include <string.h>
  #include <stdio.h>
  #include <windows.h>

  void getHomeDirectory(char* dirname)
  {
    sprintf(dirname, "%s%s%s", getenv("HOMEDRIVE"),
getenv("HOMEPATH"),
            "\\newdir\\");
    }

  void getWorkingDirectory(char* dirname)
  {
    strcpy(dirname, "newdir\\");
  }
#endif
```

directorySelection.h

```
/* Copies the path to a new directory with name "newdir" into
'dirname'.
   The directory is located in the user's home directory, if
STORE_IN_HOME_DIR
   is set or it is located in the current working directory, if
```

```
  STORE_IN_CWD is set. */
  void getDirectoryName(char* dirname);
```

directorySelectionHomeDir.c

```
  #ifdef STORE_IN_HOME_DIR
    #include "directorySelection.h"
    #include "directoryNames.h"

    void getDirectoryName(char* dirname)
    {
      getHomeDirectory(dirname);
    }
  #endif
```

directorySelectionWorkingDir.c

```
  #ifdef STORE_IN_CWD
    #include "directorySelection.h"
    #include "directoryNames.h"

    void getDirectoryName(char* dirname)
    {
      return getWorkingDirectory(dirname);
    }
  #endif
```

directoryHandling.h

```
  /* Creates a new directory of the provided name ('dirname').
     Works on Linux and Windows. */
  void createNewDirectory(char* dirname);
```

directoryHandlingLinux.c

```
  #ifdef __unix__
    #include <sys/stat.h>
```

```
  void createNewDirectory(char* dirname)
  {
    mkdir(dirname,S_IRWXU);
  }
#endif
```

directoryHandlingWindows.c

```
#ifdef _WIN32
  #include <windows.h>

  void createNewDirectory(char* dirname)
  {
    CreateDirectory(dirname, NULL);
  }
#endif
```

main.c

```
#include "directorySelection.h"
#include "directoryHandling.h"
#include <string.h>
#include <stdio.h>

int main()
{
  char dirname[50];
  char filename[60];
  char* my_data = "Write this data to the file";
  getDirectoryName(dirname);
  createNewDirectory(dirname);
  sprintf(filename, "%s%s", dirname, "newfile");
  FILE* f = fopen(filename, "w+");
  fwrite(my_data, 1, strlen(my_data), f);
  fclose(f);
  return 0;
}
```

In the preceding code, there are still `#ifdef` statements present. Each of the implementations files has one huge `#ifdef` in order to decide whether

the whole code in the file should be compiled. Alternatively, the decision which files should be compiled could be put into a Makefile. The decision whether to use the home- or the working-directory could even be made a runtime decision. Both are ways to get rid of the `#ifdefs`. Both are ways to simply use another mechanism to chose between features, and deciding which mechanism to use is not so important. Much more important, as described throughout this chapter, is to isolate and abstract the variants.

While the code-files would look cleaner when using such other mechanisms to handle the variants, the complexity would simple be there, but it would simply be somewhere else. Putting the complexity into Makefiles can be a good idea, because the purpose of Makefiles is to decide which files to build. However, if for example for building the operating-system-specific code, instead of platform-independent Makefiles, a proprietary IDE for Windows and another IDE for Linux is used to decide which files to build, then simply using the solution with `#ifdef` statements in the code is much cleaner, because the configuration which files should be built for which operating system is only done once.

The final code of the running example showed very clearly, how code with operating-system-specific variants or other variants can be improved step by step. Compared to the first code example, this final piece of code is well readable and can easily be extended with additional features or can easily be ported to additional operating systems without requiring to touch any of the existing code.

Summary

This chapter presented pattern on how to handle variants, like hardware or operating system variants, in C code and on how to organize and get rid of `#ifdef` statements.

The Avoid Variants pattern suggests to use standardized functions instead of self-implemented variants. This pattern should be applied anytime it is applicable, because it resolved issues with code variants in one blow.

However, there is not always a standardized function available and in such cases, programmers have to implement their own function to abstract the variant. As a start, Isolate Primitives suggests to put variants into separate functions and Atomic Primitives suggests to only handle one kind of variant in such functions. Abstraction Layer takes the additional step to hide the implementations of the primitives behind an API and Split Variant Implementations suggests to put each variant into a separate implementation file.

With these patterns as part of the programming vocabulary, a C programmer has a toolbox and has step-by-step guidance on how to tackle C code variants in order to structure code and in order to escape from the `#ifdef` hell.

For experienced programmers, some of the patterns might look like obvious solutions. That is good. One of the tasks of patterns is to educate people to do the right thing and once people know how to do the right thing, the patterns are not necessary anymore, because people then intuitively do as suggested by the patterns.

Further Reading

- The book *Write Portable Code: An Introduction to Developing Software for Multiple Platforms* by Brian Hook (No Starch Press, 2005) describes how to write portable code in C. The book covers operating system variants and hardware variants, by for example giving advice for coping with byte-ordering, data-type-sizes, or line-separator tokens.
- The article *#ifdef Considered Harmful* (*http://www.literateprogramming.com/ifdefs.pdf*) is one of the first that sceptically discusses the use of `#ifdef` statements. The article elaborates on problems that arise when using them in an unstructured way and provides alternatives. *The article *Writing Portable Code* by Didier Malenfant

(*https://pontus.digipen.edu/~mmead/www/docs/WritingPortableCode.pdf*) describes how to structure portable code and which functionality should be put below and abstraction layer.

Outlook

Now you are equipped with some more patterns. Next, you'll learn how to apply these patterns as well as the patterns from the previous chapters. The next chapters cover larger code examples that show the application of all these patterns.

Part II. Pattern Stories

Telling stories is an inherent and natural way to convey information. In the world of patterns, it is sometimes difficult to see how the described patterns can be applied in a real-world context. To show an example of such pattern application, this second part of the book tells you stories of applying the C programming patterns from the first part of the book to implement larger programs. You'll learn how to build such programs bit by bit and you'll see how the patterns make your life easier by providing you with guidance on good design decisions.

Chapter 10. Implementing Logging Functionality

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 10th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

In the world of patterns, without stories, it is difficult to show how patterns are applied and in which context they should be applied. Of course that is described in general in the pattern text, but sometimes that is much easier to understand by looking at a concrete example. A pattern story tells such a concrete example of applying one or more patterns.

This chapter tells the story of applying the patterns from the previous chapters to a running example that was abstracted from an industrial-strength implementation of a logging system. To keep the example code easy to grasp, not all aspects of the original industrial-strength code are covered. For example, the code design does not focus on performance or testability aspects. Still, the example nicely shows how to build a logging system piece by piece by applying patterns.

The Pattern Story

Image you have a C program that is out in the field and that you have to maintain. If an error occurs, you get into your car, drive to the customer and debug the program there. That was just fine for you up until now, but the situation changed. Your customer moved to another city. The car ride now takes a few hours and right after the first trip to your customer you immediately realize that this situation is not at all satisfying.

You'd very much prefer to solve the problem right from your desk in your office to save much time and nerves. While in some cases that is already possible by remote debugging, in some other cases it isn't, because you need detailed data about the exact software states in which the error occurred and that is very hard to get via a remote connection - in particular in case of sporadic errors.

Thus, you need a way to get detailed debug information from your customer without requiring a lengthy car ride and without requiring a remote debugging session. Maybe by now you already guessed what the solution is. Your solution is to implement some logging functionality and to ask your customer in case of error to send you the log files containing the debug information. In other words, you want to implement the Log Errors pattern. Sounds simple, right? Well, just wait to see how many crucial design decisions you'll have to make…

File Organization

For a start, you organize the header- and implementation files that you expect to need. You already have a large codebase, so you want to clearly separate these files from the rest of your code. So how should you organize the files? Should you put all your logging related files into the same directory? Should you put all the header files of your code into a single directory?

To answer these questions, you look up patterns on organizing files from the previous chapters. You read through the problem statements of these

patterns and you trust in the knowledge provided in the described solutions. You end up with the following three patterns that nicely address your problems.

Software-Module Directory	Put header files and implementation files that belong to a tightly coupled functionality into one directory. Name that directory after the functionality that is provided via the header files.
Header Files	Provide function declarations in your API for any functionality you want to provide to your user. Hide any internal functions, internal data, and your function definitions (the implementations) in your implementation file and don't provide this implementation file to the user.
Global Include Directory	Have one global directory in your codebase that contains all software-module APIs. Add this directory to the global include paths in your toolchain.

You create a Software-Module Directory for your implementation files and you put the Header File of your logging software-module into the already existing Global Include Directory of your codebase. Having this header file in the Global Include Directory has the advantage that the callers of your code definitely know which header file they are supposed to use.

Thus, you end up with the file structure shown in Figure 10-1.

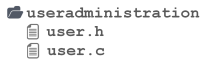


Figure 10-1. File structure

With that file structure you can put any implementation files that only concern your logging software-module into the *logger* directory and you can put the interface, which can be used from other parts of your program, into the *inc* directory.

Central Logging Function

As a start, you want to simply implement a central function for error logging that takes custom error texts, adds the current timestamp to the texts, and prints it to the output. The timestamp information will make it easier for you later on to analyze the error texts.

You put the function definition into the *logger.h* file. To protect your header file against multiple inclusion, you add an Include Guard. There is no need to store any information in that code or to initialize it, so you simply implement a Stateless Software-Module.

Include Guard	Protect the content of your header files against multiple inclusion so that the developer using the header files does not have to care whether it is included multiple times. Use an interlocked `#ifdef` statement or a `#pragma once` statement to achieve that.
Stateless Software-Module	Keep your functions simple and don't build up state information in your implementation. Put all related functions into one header file and provide the caller this interface to your software-module.

logger.h

```
#ifndef LOGGER_H
#define LOGGER_H
void logging(const char* text);
#endif
```

Caller's code

```
logging("Some text to log");
```

To implement the function in your *logger.h* file, you simply call a `printf` to write the timestamp and the text to `stdout`. But what if the caller of your function provides invalid logging input like a NULL pointer? Should you check for such invalid input and provide error information to the caller?

You decide otherwise. You adhere to the Samurai Principle according to which you should not return error information about programming errors.

Samurai Principle	Return from a function victorious or not at all. If there is a situation for that you know that an error cannot be handled, then abort the program.

So you simply forward the provided text to the `printf` function and in case of invalid input your program simply crashes, which makes it easy for the caller to find out programming errors regarding invalid input:

logger.c

```
void logging(const char* text)
{
  time_t mytime = time(NULL);
  printf("%s %s\n", ctime(&mytime), text);
}
```

And what if we call the function in context of a multi-threaded program? Can the string provided to the function be changed by other threads or is it necessary that the string remains unchanged until the logging function is finished? Well, you already saw the preceding code example: the caller has to provide the string and the caller is responsible that the string is valid until the function returns. So we have a Caller-Owned Buffer here. That behavior has to be documented in the function's interface.

Caller-Owned Buffer	Require the caller to provide a buffer and its size to the function that returns the complex, large data. In the function implementation, copy the required data into the buffer if the buffer size is large enough.

logger.h

```
/* Prints the current timestamp followed by the provided string
to stdout.
   The string must be valid until this function returns. */
void logging(const char* text);
```

Logging Source Filter

Now imagine that each and every software module calls the logging function in order to log some information. The output can become quite messy, in particular if you have a multi-threaded program.

To make it easier to get the information you are looking for, you want to make it possible to configure the code in a way that it only prints the logging information for configured software-modules. To achieve that, you add an additional parameter to your function, which identifies the current software module and you add a function to enable printing output for a software module. If that function is called, all future logging output for that software module will be printed:

logger.h

```
/* Prints the current timestamp followed by the provided string
to stdout.
   The string must be valid until this function returns. The
provided module
   identifies the software-module that calles this function. */
void logging(const char* module, const char* text);

/* Enables printing output for the provided module. */
bool enableModule(const char* module);
```

Caller's code

```
logging("MY-SOFTWARE-MODULE", "Some text to log");
```

How will you keep track about for which software-modules the logging information should be printed? Should you store that state information in a global variable, or is each global variable a code smell? Or should you, in order to avoid global variables, pass an additional parameter to all your

functions that stores this state information? Should the required memory be allocated throughout the whole lifetime of your program? So you are asking yourself the question how to implement Software-Module with Global State using Eternal Memory.

Software-Module with Global State	Have one global instance to let your related functions share common resources. Put all functions that operate on that instance into one header file and provide the caller this interface to your software-module.
Eternal Memory	Put your data into memory that is available throughout the whole lifetime of your program.

logger.c

```
#define MODULE_SIZE 20
#define LIST_SIZE 10
typedef struct
{
  char module[MODULE_SIZE];
}LIST;
static LIST list[LIST_SIZE];
```

The list in the preceding code example is filled by enabling software-modules with the following function:

logger.c

```
bool enableModule(const char* module)
{
  for(int i=0; i<LIST_SIZE; i++)
  {
    if(strcmp(list[i].module, "") == 0)
    {
      strcpy(list[i].module, module);
      return true;
    }
    if(strcmp(list[i].module, module) == 0)
    {
```

```
            return false;
        }
    }
    return false;
}
```

The preceding code adds the software-module name to the list if a slot in the list is empty and if that name is not already in the list. The caller simply sees through the Return Value whether an error occurred and does not see which of these errors occurred. So you don't Return Error Codes, but instead you only Return Relevant Errors, because there is no relevant scenario in which the caller could react differently on the described error situations. Of course, you also document this behavior in your function definition.

Return Value	Simply use the one C mechanism intended to retrieve information about the result of a function call: the Return Value. The mechanism to return data in C copies the function result and provides the caller access to this copy.
Return Error Codes	Use the Return Value of a function to transport error information. Return a value that represents a specific kind of error. You as the callee and the caller must have a mutual understanding of what the value means.
Return Relevant Errors	Only transport error information to the caller, if that information is relevant to the caller. Error information is only relevant to the caller if the caller can react to that information.

logger.h

```
/* Enables printing output for the provided module. Returns true
on success and
   false on error (no more modules can be enabled or module was
already enabled). */
bool enableModule(const char* module);
```

Conditional Logging

Now, with the activated software-modules in your list, you can conditionally log information depending on the activated modules as shown in the following code:

logger.c

```
void logging(const char* module, const char* text)
{
  time_t mytime = time(NULL);
  if(isInList(module))
  {
    printf("%s %s\n", ctime(&mytime), text);
  }
}
```

But how do you implement the `isInList` function? There are several ways to iterate through a list. You could have a Cursor Iterator that provides a `getNext` method as you know it from object oriented programming languages and that nicely abstracts the underlying data structure. But is that necessary here? After all, you only go through some array in your own software module. Because the iterated data is not carried across API boundaries that might have to be kept compatible, you can apply a much simpler solution here: Index Access, which simply directly uses an index to access the elements you want to iterate.

Index Access	Provide a function that takes an index to address the element in your underlying data structure and return the content of this element. The user calls this function in a loop to iterate over all elements.

logger.c

```
bool isInList(const char* module)
{
  for(int i=0; i<LIST_SIZE; i++)
  {
    if(strcmp(list[i].module, module) == 0)
```

```
      {
        return true;
      }
    }
    return false;
  }
```

Now all your code for software-module-specific logging is written. The code simply iterates the data structure by incrementing an index. Actually, the same kind of iteration was already used in your `enableModule` function.

Multiple Logging Destinations

Next you want to provide different destinations for your loggings. Until now, all output is logged to the `stdout`, but now you want your caller to be able to configure your code to directly log into a file. Such a configuration is usually done before the action to be logged is started. So first you need a function that allows to configure the logging destination for all future loggings:

logger.h

```
/* All future log messages will be logged to stdout */
void logToStdout();

/* All future log messages will be logged to a file */
void logToFile();
```

To implement this log destination selection, you could simply have some `if` or `switch` statements to call the right function depending on the configured logging destination, but that approach has the drawback that each time you add another logging destination, you'd have to touch that piece of code. That is not a good solution according to the Open-Closed-Principle. A much better solution is to implement a Dynamic Interface.

Dynamic Interface	Define a common interface for the deviating functionalities in your API and require the caller to provide a callback function for that functionality which you then call in your function implementation.

logger.c

```
typedef void (*logDestination)(const char*);
static logDestination fp = stdoutLogging;

void stdoutLogging(const char* buffer)
{
  printf("%s", buffer);
}

void logToStdout()
{
  fp = stdoutLogging;
}

void logToFile()
{
  fp = fileLogging;
}

#define BUFFER_SIZE 100
void logging(const char* module, const char* text)
{
  char buffer[BUFFER_SIZE];
  time_t mytime = time(NULL);
  if(isInList(module))
  {
    sprintf(buffer, "%s %s\n", ctime(&mytime), text);
    fp(buffer);
  }
}
```

Quite a lot changed in the existing code, but now additional log destinations can be added without any changes to the `logging` function. In the

preceding code, the `stdoutLogging` function is already implemented, but the `fileLogging` function is still missing.

File Logging

To log to a file, you could simply open and close the file each time you log a message, but that is not very efficient and if you want to log a lot of information, that approach takes awfully lot of time. So which alternative do you have? You could simply open the file once and then leave it open. But how do you know when to open the file? And when would you close it?

Looking at the patterns in this book, you cannot find one that solves your problem. However, you realize that this book is not the only source of information and you start looking for other patterns online. A quick Google search leads you to a pattern book on resource management in which you find the pattern that solves your problem: Lazy Acquisition. In the first call to your `fileLogging` function, you open the file once and then leave it open. You can store the file descriptor in Eternal Memory.

Lazy Acquisition	Implicitly initialize the object or data the first time it is used (see *Pattern-Oriented Software Architecture: Volume 3: Patterns for Resource Management* by Michael Kirchner and Prashant Jain (Wiley, 2004))
Eternal Memory	Put your data into memory that is available throughout the whole lifetime of your program.

logger.c

```
void fileLogging(const char* buffer)
{
  static int fd = 0;
  if(fd == 0)
  {
    open("log.txt", O_RDWR | O_CREAT, 0666);
  }
```

```
    write(fd, buffer, strlen(buffer));
}
```

To keep the code example simple, it does not target thread-safety. In order to be thread-safe, the code would have to protect the Lazy Acquisition with a mutex to make sure that the acquisition really only happens once.

What about closing the file? For some applications not closing the file is a valid option. Imagine that you want to log as long as your application is running and when you shut the application down, you rely on the operating system to clean up the file that you left open.

Cross-Platform Files

The code so far implements logging to a file on Linux systems, but you also want to use your code on Windows platforms, for which the current code won't yet work.

To support multiple platforms, you first consider to Avoid Variants so that you only have common code for all platforms. That would be possible for writing files by simply using the `fopen`, `fwrite`, and `fclose` functions, which are available on Linux as well as on Windows systems.

Avoid Variants	Use standardized functions, which are available on all platforms. If there are no standardized functions, consider to not implement the functionality.

However, you want to make your file logging code as efficient as possible and using the platform-specific functions for accessing files is more efficient. But how do you implement platform-specific code? Duplicating your codebase for having one full code version for Windows and one full code version for Linux is not an option, because future changes and maintanance of duplicated code can become a nightmare.

You decide to use `#ifdef` statements in your code to differentiate between the platforms. However, is that not some kind of code duplication as well? After all, when having huge `#ifdef` blocks in your code, all the program logic in these statements is duplicated. How can you avoid code duplication, while still supporting multiplce platforms?

Again the patterns show you the way. You first define platform-independent interfaces for the functionality that requires the platform-dependent functions. In other words, you define an Abstraction Layer.

Abstraction Layer	Provide an API for each functionality that requires platform-specific code. Define only platform independent functions in the header file and put all platform-specific `#ifdef` code into the implementation file. The caller of your functions only includes your header file and does not have to include any platform-specific files.

logger.c

```
void fileLogging(const char* buffer)
{
  void* fileDescriptor = initiallyOpenLogFile();
  writeLogFile(fileDescriptor, buffer);
}

/* Opens the logfile at the first call.
   Works on Linux and on Windows systems */
void* initiallyOpenLogFile()
{
  ...
}

/* Writes the provided buffer to the logfile.
   Works on Linux and on Windows systems */
void writeLogFile(void* fileDescriptor, const char* buffer)
{
  ...
}
```

Behind this Abstraction Layer you Isolate Primitives of your code variants. That means you don't use `#ifdef` statements across several functions, but you stick to one `#ifdef` for one function. However, should you have an `#ifdef` statement across the show function implementation, or just across the platform-specific part? The solution is to have both. You should have Atomic Primitives. The functions should be on a granularity so that they only contain platform-specific code. If they don't, then you can split these functions further up. That is the best way to keep platform-dependent code manageable.

Isolate Primitives	Isolate your code variants. In your implementation file, put the code handling the variants into separate functions and call these functions from your main program logic, which then only contains platform independent code.
Atomic Primitives	Make your primitives atomic. Only handle exactly one kind of variant per function. If you handle multiple kinds of variants, for example, operating system variants and hardware variants, then have separate functions for that.

The following code shows the implementations of your Atomic Primitives:

logger.c

```
void* initiallyOpenLogFile()
{
#ifdef __unix__
  static int fd = 0;
  if(fd == 0)
  {
    fd = open("log.txt", O_RDWR | O_CREAT, 0666);
  }
  return fd;
#elif defined _WIN32
  static HANDLE hFile = NULL;
  if(hFile == NULL)
  {
    hFile = CreateFile("log.txt", GENERIC_WRITE, 0, NULL,
                       CREATE_NEW, FILE_ATTRIBUTE_NORMAL, NULL);
  }
```

```
  return hFile;
#endif
}

void writeLogFile(void* fileDescriptor, const char* buffer)
{
#ifdef __unix__
  write((int)fileDescriptor, buffer, strlen(buffer));
#elif defined _WIN32
  WriteFile((HANDLE)fileDescriptor, buffer, strlen(buffer), NULL,
NULL);
#endif
}
```

The preceding code still looks ugly. But then again, any platform-dependent code does not look very nice. Is there anything else you can do to make that code easier to read and maintain? A possible approach to improve things is to Split Variant Implementations into separate files.

Split Variant Implementations	Put each variant implementation into a separate implementation file and select per file what you want to compile for which platform.

fileLinux.c

```
#ifdef __unix__
void* initiallyOpenLogFile()
{
  static int fd = 0;
  if(fd == 0)
  {
    fd = open("log.txt", O_RDWR | O_CREAT, 0666);
  }
  return fd;
}

void writeLogFile(void* fileDescriptor, const char* buffer)
{
  write((int)fileDescriptor, buffer, strlen(buffer));
```

```
}
#endif
```

fileWindows.c

```
#ifdef _WIN32
void* initiallyOpenLogFile()
{
  static HANDLE hFile = NULL;
  if(hFile == NULL)
  {
    hFile = CreateFile("log.txt", GENERIC_WRITE, 0, NULL,
                       CREATE_NEW, FILE_ATTRIBUTE_NORMAL, NULL);
  }
  return hFile;
}

void writeLogFile(void* fileDescriptor, const char* buffer)
{
  WriteFile((HANDLE)fileDescriptor, buffer, strlen(buffer), NULL,
NULL);
}
#endif
```

Both of the shown code files are a lot easier to read compared to the code where Linux and Windows code is mixed within a single function. Also, having different files has the advantage that instead of conditionally compiling the code on a platform via `#ifdef` statements, it would be possible to eliminate all `#ifdef` statements and to use Makefiles to select which files to compile.

Using the Logger

With these final changes to your logging functionality, you are happy for now. Your code can now log messages for configured software-modules to `stdout` or to cross-platform files. The following code shows how to use the logging functionality:

```
enableModule("MYMODULE");
logging("MYMODULE", "Log to stdout");
logToFile();
logging("MYMODULE", "Log to file");
logging("MYMODULE", "Log to file some more");
```

After being done with making all these coding decisions and after implementing it, you are very relieved. You take your hands of the keyboard and look at the code in admiration. You are astonished how some of your initial questions that seemed difficult to you were easily resolved by the patterns. The patterns took you the burden of making hundreds of decisions by yourself. Instead, you simply trusted the patterns and followed their guidance.

So what about the long car rides necessary to fix the bugs for your customer on-site? These car rides belong to the past. Now you simply get the debug information that you need via the log files. That makes your customer happy, because he gets quicker bug fixes and even more important, it makes your own life better. You can provide more professional software and you now have the time to get home from work earlier.

Summary

You constructed the code for this logging functionality step by step by applying the patterns in order to solve one problem after the other. At the start you had many questions on how to organize the files or on how to cope with error handling. The patterns showed you the way. They gave you guidance and made it easier to construct this piece of code. Also for somebody who did not write that code, it is with the help of the patterns easy to understand why it looks and behaves the way it does. Figure 10-2 shows an overview of the decisions that the patterns helped you to make.

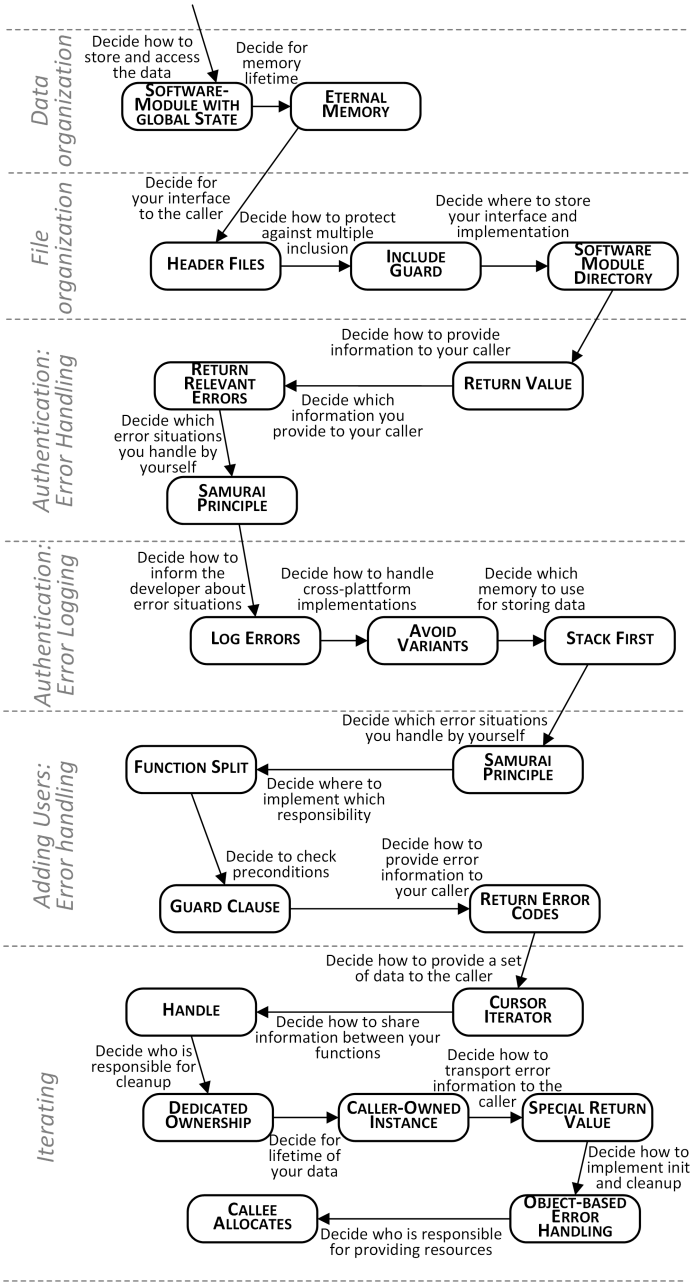


Figure 10-2. The patterns applied throughout this story

The story told in this chapter showed how to apply the patterns presented in Part I of this book. The story made it possible to show the value of the patterns by applying them to a concrete example and the story showed how to construct a larger industrial-strength software piece by piece with the help of patterns.

The next chapter will tell another story on how to apply the patterns to build another larger industrial-strength piece of code.

Chapter 11. Building a User Management System

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 11th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

Stories communicate information. Pattern stories communicate when and how patterns can be applied and which consequences arise. This chapter tells the story of applying the patterns from this book to a running example and illustrates with that example the benefits and the support for programmers when tackling design choices with the help of these patterns. The patterns are applied to a running example that was abstracted from an industrial-strength implementation of a user-management system.

The Pattern Story

Imagine you are fresh from university and started working for a software development company. Your boss hands you a product specification for a piece of software that has to store usernames and passwords and tells you to implement it. The software should provide functionality to check whether a

provided password for a user is correct and it should provide functionality to create, delete and view existing users.

You are eager to show your boss that you are a good programmer, but once you sit in front of your keyboard, you hesitate. So many questions pop up in your head. Should you write all code into one single file? At university you learned that that is a bad idea, but how many files are good? Which parts of the code will you put into the same files? Should you check the input parameters for each function? Should your functions return detailed error information? At university you learned how to build a software program that works, but you did not learn how to write good code that is maintainable. So what should you do? How to start?

Data Organization

To answer your questions, you simply have a look at patterns on C programming to get guidance on how to build good programs. You start with the part of your system that stores the usernames and passwords and now you ask yourself the question how to store the data in your program. Should you store it in global variables? Should you hold the data in local variables inside a function? Should you allocate dynamic memory?

To answer these questions, first you think about the exact problem that you want so solve in your application: you are not sure how to store the usename data. Currently, there is no need to make this data persistent, but instead you simply want to be able to build up and access this data at runtime. Also you don’t want the caller of your functions having to cope with explicit allocation and initialization of the data.

Next, you look for patterns that address your problem. You find the book chapter with C patterns on data lifetime and ownership, which address the issue of who is responsible of holding which data. You read through all the problem sections of these patterns and you find one pattern that matches your problem very well and that describes consequences which are acceptable for you. It is the Software-Module with Global State pattern, which suggests to have Eternal Memory in the form of global variables with

scope limited to the file in order for that data to be accessed from within that file.

Software-Module with Global State	Have one global instance to let your related functions share common resources. Put all functions that operate on that instance into one header file and provide the caller this interface to your software-module.
Eternal Memory	Put your data into memory that is available throughout the whole lifetime of your program.

```
#define MAX_SIZE 50
#define MAX_USERS 50

typedef struct
{
  char name[MAX_SIZE];
  char pwd[MAX_SIZE];
}USER;

static USER userList[MAX_USERS]; ❶
```

❶ The `userList` contains the data for your users. It is accessible within the implementation file and as it is kept in the static memory and thus there is no need to manually allocate it.

File Organization

Next, you want to define an interface for your caller. You want to make sure that it is easy for you to change your implementation later on without requiring the caller to change any code. Now you have the problem that you have to decide which part of your program should be defined in the interface and which part in your implementation file.

You solve that issue by having Header Files. You simply put as few things as possible (only those things that are relevant for the caller) into the interface (.h file). All the rest goes into your implementation files (.c files).

To protect against multiple inclusion of header files, you implement an Include Guard.

Header Files	Provide function declarations in your API for any functionality you want to provide to your user. Hide any internal functions, internal data, and your function definitions (the implementations) in your implementation file and don't provide this implementation file to the user.
Include Guard	Protect the content of your header files against multiple inclusion so that the developer using the header files does not have to care whether it is included multiple times. Use an interlocked `#ifdef` statement or a `#pragma once` statement to achieve that.

user.h

```
#ifndef USER_H
#define USER_H

#define MAX_SIZE 50

#endif
```

user.c

```
#include "user.h"

#define MAX_USERS 50

typedef struct
{
  char name[MAX_SIZE];
  char pwd[MAX_SIZE];
}USER;

static USER userList[MAX_USERS];
```

Now the caller can use the defined `MAX_SIZE` to know how long the strings provided to the software-module can be. By convention, the caller knows that everything in the .h file can be used, but that nothing of the .c file should be used.

Next, you want to make sure that your code files are well separated from the other code of your caller, because you want to avoid name clashes. You are not sure whether you should put all your files into one single directory or whether, for example, you should have all .h files in the whole codebase in one single directory in order to make it easier to include them

You decide to create a Software-Module Directory. You put all your files of your software-module, the interfaces and the implementations, into one directory.

Software Module Directories	Put header files and implementation files that belong to a tightly coupled functionality into one directory. Name that directory after the functionality that is provided via the header files.

With this directory structure as shown in Figure 11-1, it is now possible to easily spot all files that are related to your code and you don't have to be afraid that the names of your implementation files clash with other file names.

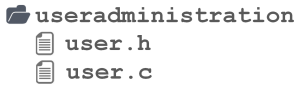


Figure 11-1. File structure

Authentication: Error Handling

Now it is time to implement the first functionality to access the data. You want to implement a function that checks whether a provided password is

correct for a provided user. First, you define the behavior of the function by declaring the function in the header file and by documenting its semantic.

You want the function to let the caller know whether the provided password is correct for a provided user and you decide to tell the caller by using the Return Value of the function, but which information should you return? Should you provide the caller with any error information that occurs?

You decide against that and you only Return Relevant Errors, because for any security-related functionality it is common to only provide the information that you must provide and no more. So you decide to not let the caller know exactly whether the provided user does not exist, or whether the provided password is wrong. Instead, you simply tell the caller whether authentication worked or whether it did not work.

Return Value	Simply use the one C mechanism intended to retrieve information about the result of a function call: the Return Value. The mechanism to return data in C copies the function result and provides the caller access to this copy.
Return Relevant Errors	Only transport error information to the caller, if that information is relevant to the caller. Error information is only relevant to the caller if the caller can react to that information.

user.h

```
/* Returns true if the provided username exists and
   if the provided password correct for that user. */
bool authenticateUser(char* username, char* pwd);
```

The preceding code defines very well which value is returned by the function, but it does not specify the behavior in case of invalid input. How should you cope with invalid input like NULL pointers? You could check against NULL pointers so that even in case of invalid input your code does not crash.

However, you decide to not do that. Instead, you simply require your user to provide valid input. According to the Samurai Principle, you document that in the header file and abort the program in case of invalid input.

Samurai Principle	Return from a function victorious or not at all. If there is a situation for that you know that an error cannot be handled, then abort the program.

user.h

```
/* Returns true if the provided username exists and
   if the provided password correct for that user,
   returns false otherwise. Undefined behavior in
   case of invalid input (NULL string) */
bool authenticateUser(char* username, char* pwd);
```

user.c

```
bool authenticateUser(char* username, char* pwd)
{
  for(int i=0; i<MAX_USERS; i++)
  {
    if(strcmp(username, userList[i].name) == 0 &&
       strcmp(pwd, userList[i].pwd) == 0)
    {
      return true;
    }
  }
  return false;
}
```

With the Samurai Principle, you take the burden from your caller to check for specific return values indicating that invalid input was provided. Invalid input is directly handed to the `strcmp` function, which then has undefined behavior and most likely the program crashes. That behavior is well documented in your header file.

At first glance, letting the program crash looks like a brutal solution, but with that behavior you make sure that calls with invalid parameter do not go unnoticed. Over the long term, this strategy makes the code more reliable. It does not let subtile bugs, like invalid parameters, manifest and show up somewhere else in the caller's code.

Authentication: Error Logging

Next, you want to keep track of callers who provide you with the wrong password. This information should be available for security audits later on, so you decide to Log Errors.

Log Errors	Use different channels to transport error information that is relevant for the calling code and error information that is relevant for the developer. For example, write debug error information into a log file and don't return the detailed debug error information to the caller.

You want this logging mechanism to be available on different platforms - for example on Linux as well as on Windows. That can make things difficult, because the different operating systems provide different functions for accessing files and multi-platform code is hard to implement and hard to maintain. So how can you implement your logging functionality as simple as possible? To implement that, you make sure to Avoid Variants and to use standardized functions, which are available on all platforms.

Avoid Variants	Use standardized functions, which are available on all platforms. If there are no standardized functions, consider to not implement the functionality.

You are lucky, because the C standard defines functions for accessing files and these functions can be used on Windows as well as on Linux systems. There would also be operating-system-specific functions for accessing files available and these functions might be more preformant or might provide you with operating-system-specific features. However, you don't need that, so you stay with the simple to use file access functions defined by the C standard.

To implement your logging functionality, you simply call the following function if the wrong password was provided:

user.c

```
static void logError(char* username, char* pwd)
{
  char logString[200];
  sprintf(logString, "Failed login. User:%s, Pwd:%s\n", username,
pwd);
  FILE* f = fopen("logfile", "a+"); ❶
  fwrite(logString, 1, strlen(logString), f);
  fclose(f);
}
```

❶ We use the platform independent functions `fopen`, `fwrite`, and `fclose`. Because of that this code works on Windows as well as on Linux platforms and there are no nasty `#ifdef` statements to handle the platform variants.

For storing the log information, the code uses Stack First, because the log message is small enough to fit on the stack and because that solution is the easiest for you as you don't have to cope with memory cleanup.

Stack First	Simply put your variables by default on the stack to profit from automatic cleanup of stack variables.

Adding Users: Error Handling

Looking at the whole code, you now have a function to check whether a password is correct for a username stored in your list, but your list of users is still empty. You need a way to fill your list of users, so now you implement a function that allows the caller to add new users.

You want to make sure that the usernames are unique and you want to let the caller know whether adding the new user worked or whether it did not work, because the username already exists or because there is no more space in your user list.

Now you have to decide how you want to inform the caller about these error situations. There are several ways to do that. You could use the Return Value to transport this information or alternatively you could set the `errno` variable. Also you have to ask yourself which kind of information you provide to your caller and which data type you use to achieve that.

You decide to inform the caller about error situations with Return Error Codes and again in case of invalid parameters you abort the program (Samurai Principle). You define the error codes in your interface to allow you and your caller to have a mutual understanding of what the codes mean and for different error situations, you return the corresponding error code. The caller can then distinguish in the code between the different error situations to react accordingly.

Return Error Codes	Use the Return Value of a function to transport error information. Return a value that represents a specific kind of error. You as the callee and the caller must have a mutual understanding of what the value means.

user.h

```
typedef enum{
  USER_SUCCESSFULLY_ADDED,
  USER_ALREADY_EXISTS,
```

```
  USER_ADMINISTRATION_FULL
}ERROR_CODE;

/* Adds a new user with the provided `username' and the provided
password `pwd'
   (must not be NULL). Returns USER_SUCCESSFULLY_ADDED on
success,
   USER_ALREADY_EXISTS if a user with the provided username
already exists
   and USER_ADMINISTRATION_FULL if no more users can be added. */
ERROR_CODE addUser(char* username, char* pwd);
```

As a next step, you want to implement the `addUser` function. You realize that there are different tasks to perform in order to do that. First you have to check, whether such a user already exists and next you have to actually add the user. To separate these tasks you perform a Function Split so that you split different tasks or responsibilities into different functions. First implement a function to check whether the user already exists.

Function Split	Split up the function. Take a part of a function that seems useful on its own, create a new function with that, and call that function.

user.c

```
static bool userExists(char* username)
{
  for(int i=0; i<MAX_USERS; i++)
  {
    if(strcmp(username, userList[i].name) == 0)
    {
      return true;
    }
  }
  return false;
}
```

This function can now be called inside the function that adds new users in order to only add new users if they don't yet exist. You ask yourself at which place in the code you should check for already existing users. Right at the beginning of the function? Or rather right before the place where you would add the user to the list? Which of these alternatives would make your function easier to read and maintain?

To solve that issue, you implement a Guard Clause and return immediately at the beginning of the function when realizing that the action to add a user cannot be performed because the user already exists. Having such check right at the beginning of the function makes it easier to follow the program flow.

Guard Clause	Check whether you have pre-conditions and immediately return from the function if these pre-conditions are not met.

user.c

```
ERROR_CODE addUser(char* username, char* pwd)
{
  if(userExists(username))
  {
    return USER_ALREADY_EXISTS;
  }

  for(int i=0; i<MAX_USERS; i++)
  {
    if(strcmp(userList[i].name, "") == 0)
    {
      strcpy(userList[i].name, username);
      strcpy(userList[i].pwd, pwd);
      return USER_SUCCESSFULLY_ADDED;
    }
  }

  return USER_ADMINISTRATION_FULL;
}
```

With the implemented code fragments so far, it is possible to fill your user administration with users and to check for these users whether a provided password is correct.

Iterating

Next, you want to provide some functionality to read out all usernames. You do that by implementing an iterator. You first consider to simply provide an interface that lets the caller access the `userList` array by index. But you soon realize that with such an interface you'd be in trouble if the underlying data structure changes (for example, to a linked list) or if the caller wants to access the array while another caller modifies the array.

To provide an iterator interface to the caller that solves the mentioned issues, you implement a Cursor Iterator, which uses a Handle to hide the underlying data structure from the caller.

Cursor Iterator	Create an iterator instance that points to an element in the underlying data structure. An iteration function takes this iterator instance as argument, retrieves the element the iterator currently points to, and modifies the iteration instance to point to the next element. The user then iteratively calls this function to retrieve one element at a time.
Handle	Have a function to create the context on which the caller operates and return an abstract pointer to internal data for that context. Require the caller to pass that pointer to all your functions which can then use the internal data to store state information and resources.

user.h

```
typedef struct ITERATOR* ITERATOR;

/* Create an iterator instance. Returns NULL on error. */
ITERATOR createIterator();

/* Retrieves the next element from an iterator instance. */
```

```
char* getNextElement(ITERATOR iterator);

/* Destroys an iterator instance. */
void destroyIterator(ITERATOR iterator);
```

The caller has full control about when to create and destroy the iterator. Thus, you have Dedicated Ownership with a Caller-Owned Instance. The caller can simply create the iterator Handle and use it to access the list of usernames. If creation fails, then the Special Return Value NULL indicates that. Having this Special Return Value instead of explicit error codes makes using the function easier, because no additional function parameters are needed to transport error information. When the caller is done with iterating, the caller can destroy the Handle.

Dedicated Ownership	Right at the time when you implement memory allocation: Clearly define where it's going to be cleaned up and who is going to do that.
Caller-Owned Instance	Require the caller to pass an instance, which is used to store resource and state information, along to your functions. Provide explicit functions to create and destroy these instances, so that the caller can determine their lifetime.
Special Return Value	Use the Return Value of your function to transport the data computed by the function. Reserve one or more special values to be returned if an error occurs.

As the interface provides the caller with explicit functions to create and destroy the iterator, it naturally leads to separate functions for intializing and cleaning up the resources for your iterator in the implementation. This Object-based Error Handling brings the advantage of nicely separated responsibilities in your functions, which makes them easier to extend later on if necessary. You can see this separation in the following code where all initialization code is in one function and all cleanup code is in another function.

Object-Based Error Handling	Put initialization and cleanup into separate functions similar to the concept of constructors and destructors in object-oriented programming.

user.c

```
struct ITERATOR
{
  int currentPosition;
  char currentElement[MAX_SIZE];
};

ITERATOR createIterator()
{
  ITERATOR iterator = (ITERATOR) calloc(sizeof(struct
ITERATOR),1);
  return iterator;
}

char* getNextElement(ITERATOR iterator)
{
  if(iterator->currentPosition < MAX_USERS)
  {
    strcpy(iterator->currentElement, userList[iterator-
>currentPosition].name);
    iterator->currentPosition++;
  }
  else
  {
    strcpy(iterator->currentElement, "");
  }
  return iterator->currentElement;
}

void destroyIterator(ITERATOR iterator)
{
  free(iterator);
}
```

When implementing the preceding code, at first you were not sure how to provide the username data to the caller. Should you simply provide the

caller with a pointer to that data? Or if you copy that data into a buffer, then should the caller allocate it, or should you do that?

You made the decision that the Callee Allocates the string buffer. That makes it possible for the caller to have full access to that string without having the possibility to change the data in the `userList` and without running into the problem of accessing data that might be changed by other callers at the same time.

Callee Allocates	Allocate a buffer with the required size inside the function that provides the complex, large data. Copy the required data into the buffer and return a pointer to that buffer.

Using the User Management System

You finally completed your user management code. The following code shows how to use that user maagement system:

```
char* element;
addUser("A", "pass");
addUser("B", "pass");
addUser("C", "pass");

ITERATOR it = createIterator();

while(true)
{
  element = getNextElement(it);
          if(strcmp(element, "") == 0)
          {
            break;
          }

  printf("User: %s ", element);
  printf("Authentication success? %d\n",
authenticateUser(element, "pass"));
}
```

```
destroyIterator(it);
```

After being done with making all these coding decisions and after implementing it, you are very relieved. You take your hands of the keyboard and look at the code in admiration. You are astonished how some of your initial questions that seemed difficult to you were easily resolved by the patterns. The patterns took you the burden of making hundreds of decisions by yourself. Instead, you simply trusted the patterns and followed their decisions.

Now you are ready to call your boss to proudly tell her that you completed the task of implementing the requested system for storing usernames and passwords. You have a good feeling about it, because you followed pattern-based design. You relied on documented solutions that are proven in use and you implemented these solutions to create someting bigger: your system for storing usernames and passwords.

Summary

You constructed the code in this chapter step by step by applying the patterns in order to solve one problem after the other. At the start you had many questions on how to organize the files or on how to cope with error handling. The patterns showed you the way. They gave you guidance and made it easier to construct this piece of code. Also for somebody who did not write that code, it is with the help of the patterns easy to understand why it looks and behaves the way it does. Throughout this chapter, you applied the patterns shown in Figure 11-2. In the figure you can see, how many decisions you had to make and how many decisions were guided by the patterns.

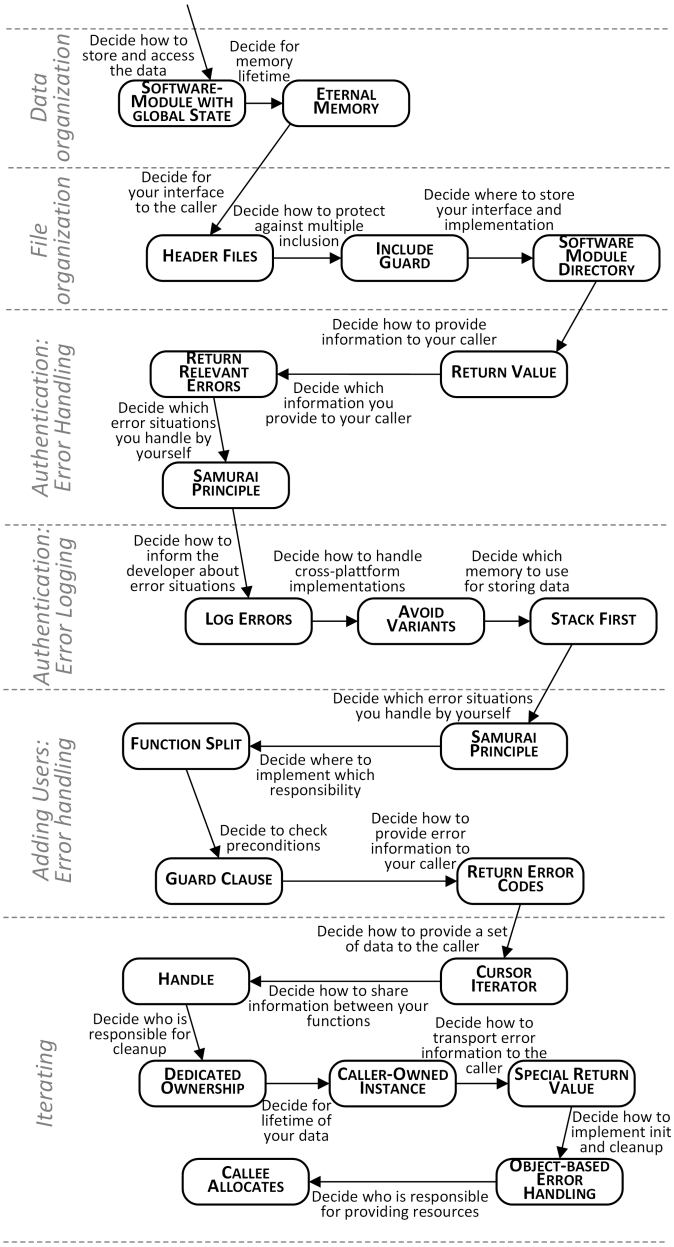


Figure 11-2. The patterns applied throughout this story

The story told in this chapter showed how to apply the patterns presented in Part I of this book. That story showed the value of the patterns by applying them to a concrete example and the story showed how to construct a larger industrial-strength software piece by piece with the help of patterns.

Chapter 12. Conclusion

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 12th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

What you've Learned

Now, after reading this book, you are familiar with several advanced C programming concepts and when looking at larger code examples you now know why the code looks the way it does. You now know the reasoning behind the design decisions made in that code. For example, in the presented Ethernet driver sample code you now understand why there is an explicit `driverCreate` method and you understand why there is a `DRIVER_HANDLE` that holds state information. The decisions of having that was simply derived from the guidance contained in the patterns.

The pattern stories from Part II showed you how C programmers can benefit from the application of the patterns from this book and they showed you how a code grows bit by bit through the application of patterns. Let these patterns also guide your way! Give it a try – when facing the next C programming problem, have a look at the problem sections of the patterns

and see whether one of them matches your problem. In that case, you are very lucky, because then you can benefit from the guidance contained in the pattern and you can start living your own pattern story.

Further Reading

I believe this book helps C programming novices to become advanced C programmers. There are also many other books out there that help you with that and that cover other aspects of C programming. Here are the books that particularly helped me improve my C programming skills:

- The book *Clean Code: A Handbook of Agile Software Craftsmanship* by Robert C. Martin (Prentice Hall, 2008) tells you the basic principles how to implement high quality code that lasts over time. The book is a good read for any programmer and covers topics like testing, documentation, code style, and others.
- The book *Test Driven Development for Embedded C: Building High Quality Embedded Software* by James W. Grenning (Pragmatic Bookshelf, 2011) tells you with a running example how to implement unit-tests with C in the context of hardware-near programs.
- The book *Expert C Programming* by Peter van der Linden (Prentice Hall, 1994) is an early book on advanced C programming guidance. It tells you how the C syntax works in detail and how to avoid common pitfalls. It also tells you about concepts like C memory management and it tells you how the linker works.
- Very related to my book is the book *Patterns in C* by Adam Tornhill (Leanpub, 2014). It also presents patterns and focuses on how to implement the Gang of Four design patterns with C.

Closing Remarks

Your situation improved. Compared to a C programmer fresh from university you now have advanced knowledge on which techniques to use in order to compose larger-scale and industrial-strength C code. You know how to perform error handling, even though you don’t have a mechanism like exceptions. You know how to implement flexible interfaces, even though you don’t have native abstraction mechanisms with C. You know how to structure your files and code, even though you don’t have classes or packages. Now you can perfectly well live with the fact that C does not provide some of the quite comfortable concepts that you get with modern programming languages.

About the Author

Christopher Preschern is a leading member of the design patterns community. He actively takes part in the organization of design pattern conferences and in initiatives to improve pattern writing. As a C programmer at the company ABB he gathered and documented hands-on knowledge on how to write industrial strength code. He lectured at the technical university of Graz courses on coding & quality and he holds a PhD in computer science.